Confidential line-by-line explanation of the vehicle14 MATLAB/CasADi package

This revised PDF was regenerated directly from the current uploaded code files. The previous PDF was checked against the code and this version corrects generic parser mistakes, especially function-name mislabeling. Sensitive method/tool terms and any exact identifiers containing them have been intentionally replaced by neutral aliases throughout this PDF. The line numbers still match the uploaded files.

Alias	Role in package	Lines checked	SHA-256 prefix
File A - Main MATLAB runner	Reads your run command, selects simulation mode/input source/road type, creates parameters and road data, calls the numerical solver, saves results, creates plots, and renders animation.	1153	b7a7c7f4f48b60f1
File B - Generated dynamics matrix function	Evaluates the generated mass matrix M, forcing vector F, and sensor vector S from q, u, parameters P, and external input vector IN. This file is generated and should not be edited by hand.	790	2b0cdb950d764b62
File C - User-editable outer model	Defines the driver signals, total drive/brake commands, powertrain and brake torque split, tyre/contact model, road-contact calculations, aero model, and assembly of IN(34). This is the main file to edit for vehicle-model experiments.	794	4cbb301b0fa005a3
File D - Export/regeneration helper script	Reference Python script that loads the source mechanical model, extracts matrix and sensor expressions, converts them to MATLAB/CasADi syntax, writes helper files, and packages the result.	928	de04bbb8a6eff4c4

Important data-flow summary

Step	Stage	Meaning
1	Run command	The runner reads optional tokens for mode, input-source selection, and road type. The default is simulation with the user-editable outer model on the 3D ribbon road.
2	Setup	The runner creates the callable CasADi function, parameter struct, option struct, road struct, and output filenames.
3	Steering-first pass	For the user-editable outer model, steering angle and steering rate are computed first because wheel geometry sensors depend on steering.
4	Sensor evaluation	The generated dynamics function returns sensor vector S: wheel-centre positions and velocities, wheel axes, carrier angular velocity, aero sensor positions, and body longitudinal speed.
5	Full input pass	The user-editable outer model converts controls, tyre/contact forces, wheel torques, and aero into IN(34).
6	Dynamics evaluation	The generated function returns M and F. The solver first tries M*xdot - F = 0; if needed it falls back to xdot = M\F.
7	Outputs	The runner writes a MAT-file, creates diagnostic plots, and renders animation. Output names include source and road type.

Beginner guide to the most important vectors

The state is x = [q; u]. The q part stores positions and angles. The u part stores body-frame speeds, angular rates, suspension speeds, and wheel speeds. The external input vector IN(34) is where steering, tyre/contact wrenches, wheel torques, and aero enter the generated dynamics equations.

Index	q name	Meaning
1	X	global X position
2	Y	global Y position
3	Z	global Z position
4	psi	yaw angle
5	theta	pitch angle
6	phi	roll angle
7	zFL	front-left suspension/wheel-centre heave coordinate
8	zFR	front-right suspension/wheel-centre heave coordinate
9	zRL	rear-left suspension/wheel-centre heave coordinate
10	zRR	rear-right suspension/wheel-centre heave coordinate
11	chiFL	front-left wheel spin angle
12	chiFR	front-right wheel spin angle
13	chiRL	rear-left wheel spin angle
14	chiRR	rear-right wheel spin angle

Index	u name	Meaning
1	vx	body-frame longitudinal velocity
2	vy	body-frame lateral velocity
3	vz	body-frame vertical velocity
4	p	body roll rate
5	q_pitch	body pitch rate
6	r	body yaw rate
7	dzFL	front-left suspension speed
8	dzFR	front-right suspension speed
9	dzRL	rear-left suspension speed
10	dzRR	rear-right suspension speed
11	omFL	front-left wheel angular speed
12	omFR	front-right wheel angular speed
13	omRL	rear-left wheel angular speed
14	omRR	rear-right wheel angular speed

IN indices	Names	Meaning
1	delta	steering angle used by wheel-geometry calculations
2	delta_dot	steering-rate input used by velocity-level wheel geometry
3-26	FB*/MB*	body-frame tyre/contact force and moment components at the four wheel centres
27-30	TFL,TFR,TRL,TRR	wheel torques applied to the four wheel spin equations
31-34	FdragF,FdownF,FdragR,FdownR	front/rear aero drag and downforce inputs

Practical editing guide

Task	Where	Reason
Change throttle/brake/steering	File C, function user_control_signals	Defines throttle, braking, steering angle, total drive torque, and total brake torque.
Change tyre model	File C, functions user_tyre_contact_wrench and tyre-force helpers	Computes compression, slip, normal load, Fx/Fy, and contact wrench.
Change brake/powertrain split	File C, function user_powertrain_brake_model	Converts total drive/brake commands into four wheel torques.
Change road selection	Run command or road-surface helper functions	Use flat or ribbon option; helper functions return surface point, tangent directions, and normal.
Change solver settings	File A, function vehicle14_demo_options	Controls duration, max step, tolerances, maneuver timing, and output names.
Change generated mechanical formulas	Source model plus exporter, then regenerate	Do not edit File B by hand; regenerate it if the mechanical equations change.

File A - Main MATLAB runner

Reads your run command, selects simulation mode/input source/road type, creates parameters and road data, calls the numerical solver, saves results, creates plots, and renders animation.

Lines checked against uploaded code: 1153. All code snippets below are sanitized aliases when required; line numbers still refer to the current uploaded file.

Line	Block/function	Purpose
1	run_vehicle14_matlab_casadi_demo	Helper function used by this file. Its line-by-line rows below describe the exact operations.
81	parse_run_mode_source_road	Parses flexible command arguments into run mode, input-model source, and road type.
126	apply_model_source_to_args	Stores the selected source and road type and creates output filenames that will not overwrite other runs.
135	check_casadi_available	Checks whether the CasADi MATLAB interface is visible and usable.
150	vehicle14_build_casadi_function	Builds the callable CasADi function that evaluates M, F, and S.
165	vehicle14_generated_dynamics_meta	Creates ordered names and index maps for states, speeds, parameters, inputs, and sensors.
314	vehicle14_params_default	Creates the default parameter structure used by the demo.
422	vehicle14_demo_options	Creates the default simulation options, road parameters, maneuver settings, and output names.
466	vehicle14_make_demo_road	Creates either the flat road or the 3D ribbon road data structure.
491	vehicle14_command_inputs	Computes the built-in original throttle, brake, steering, and steering-rate signals.
500	command_no_derivative	Computes built-in throttle, brake, and steering before finite-differencing steering rate.
552	smooth_step	Provides a smooth 0-to-1 transition to avoid discontinuous commands.
566	vehicle14_initial_state	Builds the initial 28-state vector and adjusts ride height for the requested drop test.
595	initial_suspension_coordinates	Sets initial suspension coordinates from zero-force or nominal assumptions.
606	resolved_preloads	Computes default or user-supplied static suspension preload forces.
618	min_contact_gap	Computes the minimum tyre-road gap for an initial state candidate.
637	oriented_tyre_contact_geometry_local	Helper function used by this file. Its line-by-line rows below describe the exact operations.
669	road_project_point_local	Helper function used by this file. Its line-by-line rows below describe the exact operations.
676	road_surface_at_local	Helper function used by this file. Its line-by-line rows below describe the exact operations.
683	centerline_quantities_local	Helper function used by this file. Its line-by-line rows below describe the exact operations.
700	normalize_num_local	Helper function used by this file. Its line-by-line rows below describe the exact operations.
707	vehicle14_core_numeric	Main numerical evaluator: computes inputs, calls the generated dynamics function, and returns M and F.
771	getfield_default_runner	Helper function used by this file. Its line-by-line rows below describe the exact operations.
778	vehicle14_component_inputs_numeric	Built-in original component model for contact, tyre, torque, and aero inputs.
839	contact_kinematics_from_pose	Computes contact-point velocity and compression rate from wheel pose and motion.
868	oriented_tyre_contact_geometry	Finds the tyre contact point and compression using wheel and road geometry.
905	ribbon_road_tyre_contact_to_body_wrench	Helper function used by this file. Its line-by-line rows below describe the exact operations.
938	road_height_normal_at_point	Evaluates the selected road surface and normal near a point.
943	body_point_state_num	Helper function used by this file. Its line-by-line rows below describe the exact operations.
948	body_rotation_from_q	Builds the yaw-pitch-roll body rotation matrix from q.
951	rot_x	Returns a 3-by-3 rotation matrix about the named axis.
956	rot_y	Returns a 3-by-3 rotation matrix about the named axis.
961	rot_z	Returns a 3-by-3 rotation matrix about the named axis.
966	smoothplus	Helper function used by this file. Its line-by-line rows below describe the exact operations.
969	smooth_max	Helper function used by this file. Its line-by-line rows below describe the exact operations.
973	smooth_abs	Helper function used by this file. Its line-by-line rows below describe the exact operations.
976	normalize_num	Normalizes vectors while protecting against tiny norms.
985	road_project_point	Projects a point to road coordinates and basis vectors.
999	projection_residual_given_s	Helper function used by this file. Its line-by-line rows below describe the exact operations.
1006	road_surface_at	Evaluates either flat or ribbon road height and normal.
1013	centerline_quantities	Computes the straight road centreline frame quantities.
1033	vehicle14_rhs_full_numeric	Returns xdot = M\F for the fallback explicit solver path.
1042	vehicle14_residual_full_numeric	Returns M*xdot - F for the preferred implicit solver path.
1051	vehicle14_save_plots_matlab	Creates diagnostic figures from a saved or current solution.
1076	render_vehicle14_matlab_animation	Renders the solved trajectory to MP4 and GIF.
1133	draw_chassis	Draws part of the vehicle or road for the animation.
1141	draw_wheel	Draws part of the vehicle or road for the animation.
1147	draw_local_road	Draws part of the vehicle or road for the animation.

Line-by-line explanation

Line	Sanitized code line	Explanation
1	`function run_vehicle14_matlab_casadi_demo(varargin)`	Defines the local MATLAB function `run_vehicle14_matlab_casadi_demo`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
2	`%RUN_VEHICLE14_MATLAB_CASADI_DEMO Integrate, plot, and render the 14-DOF MATLAB/CasADi demo.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
3	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
4	`% This version lets you choose which outer/component model supplies the generated dynamics input vector IN:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
5	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
6	`% run_vehicle14_matlab_casadi_demo`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
7	`% run_vehicle14_matlab_casadi_demo('outer')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
8	`% run_vehicle14_matlab_casadi_demo('original')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
9	`% run_vehicle14_matlab_casadi_demo('simulate','outer')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
10	`% run_vehicle14_matlab_casadi_demo('simulate','original')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
11	`% run_vehicle14_matlab_casadi_demo('plots-only','outer')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
12	`% run_vehicle14_matlab_casadi_demo('render-only','outer')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
13	`% run_vehicle14_matlab_casadi_demo('simulate','outer','flat')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
14	`% run_vehicle14_matlab_casadi_demo('simulate','outer','ribbon')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
15	`% run_vehicle14_matlab_casadi_demo('original','flat')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
16	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
17	`% Solver policy:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
18	`% 1) Try MATLAB ode15i on the implicit residual M(x)*xdot-F(x)=0 first.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
19	`% 2) If ode15i fails, fall back to ode15s on the explicit RHS xdot=M\F.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
20	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
21	`% The generated generated dynamics/CasADi file only supplies M, F, and sensors. The model_source`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
22	`% decides how tyre/contact/powertrain/brake/aero/control inputs are produced:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
23	`% 'outer' -> vehicle14_outer_models_user.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
24	`% 'original' -> built-in original demo component model in this runner`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
25		Blank line used to separate logical blocks and improve readability.
26	`% addpath('D:/Applications/casadi-windows-matlabR2016a-v3.6.5'); % edit if CasADi is not already on the MATLAB path`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
27		Blank line used to separate logical blocks and improve readability.
28	`[mode, model_source, road_type] = parse_run_mode_source_road(varargin{:});`	Executes this MATLAB statement in `run_vehicle14_matlab_casadi_demo`. It performs the operation shown by the sanitized code line.

Line	Sanitized code line	Explanation
29		Blank line used to separate logical blocks and improve readability.
30	`import casadi.*`	Imports a MATLAB package namespace so that functions/classes from that package can be called without full qualification.
31	`[ver_ok, msg] = check_casadi_available();`	Executes this MATLAB statement in `run_vehicle14_matlab_casadi_demo`. It performs the operation shown by the sanitized code line.
32	`if ~ver_ok, error('%s', msg); end`	Starts a conditional branch. The following block executes only when this logical test is true.
33		Blank line used to separate logical blocks and improve readability.
34	`[mf_fun, meta] = vehicle14_build_casadi_function();`	Executes this MATLAB statement in `run_vehicle14_matlab_casadi_demo`. It performs the operation shown by the sanitized code line.
35	`par = vehicle14_params_default();`	Assigns `par` from `vehicle14_params_default()`. This value is used by later computations in this function.
36	`args = vehicle14_demo_options();`	Assigns `args` from `vehicle14_demo_options()`. This value is used by later computations in this function.
37	`args = apply_model_source_to_args(args, model_source, road_type);`	Assigns `args` from `apply_model_source_to_args(args, model_source, road_type)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
38	`road = vehicle14_make_demo_road(args);`	Assigns `road` from `vehicle14_make_demo_road(args)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
39		Blank line used to separate logical blocks and improve readability.
40	`fprintf('vehicle14 MATLAB/CasADi: mode = %s, model_source = %s, road_type = %s\n', mode, args.model_source, args.road_type);`	Prints status information to the MATLAB command window so the user can see progress.
41		Blank line used to separate logical blocks and improve readability.
42	`if strcmpi(mode, 'render-only')`	Starts a conditional branch. The following block executes only when this logical test is true.
43	`render_vehicle14_matlab_animation(args.output_mat, args.animation_mp4, args.animation_gif);`	Executes this MATLAB statement in `run_vehicle14_matlab_casadi_demo`. It performs the operation shown by the sanitized code line.
44	`return;`	Returns immediately from the current function.
45	`elseif strcmpi(mode, 'plots-only')`	Starts an alternative conditional branch, tested only if previous branches were false.
46	`if ~exist(args.output_mat, 'file')`	Starts a conditional branch. The following block executes only when this logical test is true.

Line	Sanitized code line	Explanation
47	`error('Cannot run plots-only because %s does not exist yet. Run run_vehicle14_matlab_casadi_demo(''simulate'',''%s'') first.', args.output_mat, args.model_source);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
48	`end`	Ends the current function, conditional block, loop, or protected block.
49	`Sload = load(args.output_mat, 't', 'x', 'par', 'args', 'road', 'meta');`	Loads a saved MAT-file so plots or animation can be regenerated without rerunning the solver.
50	`vehicle14_save_plots_matlab(Sload.t, Sload.x, mf_fun, Sload.par, Sload.args, Sload.road, Sload.meta, Sload.args.plot_dir);`	Executes this MATLAB statement in `run_vehicle14_matlab_casadi_demo`. It performs the operation shown by the sanitized code line.
51	`return;`	Returns immediately from the current function.
52	`elseif ~strcmpi(mode, 'simulate')`	Starts an alternative conditional branch, tested only if previous branches were false.
53	`error('Unknown mode: %s. Use simulate, plots-only, or render-only.', mode);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
54	`end`	Ends the current function, conditional block, loop, or protected block.
55		Blank line used to separate logical blocks and improve readability.
56	`x0 = vehicle14_initial_state(mf_fun, par, args, road, meta);`	Assigns `x0` from `vehicle14_initial_state(mf_fun, par, args, road, meta)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
57	`xdot0 = vehicle14_rhs_full_numeric(0, x0, mf_fun, par, args, road, meta);`	Assigns `xdot0` from `vehicle14_rhs_full_numeric(0, x0, mf_fun, par, args, road, meta)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
58	`tspan = [0 args.t_final];`	Assigns `tspan` from `[0 args.t_final]`. This value is used by later computations in this function.
59	`opts = odeset('RelTol', args.rtol, 'AbsTol', args.atol, 'MaxStep', args.max_step);`	Builds MATLAB ODE solver options such as relative tolerance, absolute tolerance, and maximum step size.
60	`solver_used = 'ode15i';`	Calls MATLAB ode15i on the implicit residual M(x,input)*xdot - F(x,input) = 0. This is the preferred integration path.
61	`try`	Starts a protected block. If an error occurs, execution jumps to the matching catch block.
62	`fprintf('Trying implicit MATLAB solver ode15i on M(x)*xdot-F(x)=0...\n');`	Prints status information to the MATLAB command window so the user can see progress.
63	`[t,x] = ode15i(@(tt,xx,xxp) vehicle14_residual_full_numeric(tt,xx,xxp,mf_fun,par,args,road,meta), tspan, x0, xdot0, opts);`	Calls MATLAB ode15i on the implicit residual M(x,input)*xdot - F(x,input) = 0. This is the preferred integration path.
64	`catch ME`	Handles an error from the preceding protected block; used for solver or rendering fallback.
65	`warning('ode15i failed: %s\nFalling back to ode15s with explicit RHS M\\F.', ME.message);`	Issues a MATLAB warning while allowing execution to continue.
66	`solver_used = 'ode15s-explicit';`	Calls MATLAB ode15s on the explicit fallback right-hand side xdot = M\F.
67	`[t,x] = ode15s(@(tt,xx) vehicle14_rhs_full_numeric(tt,xx,mf_fun,par,args,road,meta), tspan, x0, opts);`	Calls MATLAB ode15s on the explicit fallback right-hand side xdot = M\F.
68	`end`	Ends the current function, conditional block, loop, or protected block.
69		Blank line used to separate logical blocks and improve readability.
70	`model_source_saved = args.model_source; %#ok<NASGU>`	Assigns `model_source_saved` from `args.model_source; %#ok<NASGU>`. This value is used by later computations in this function.
71	`save(args.output_mat, 't', 'x', 'par', 'args', 'road', 'meta', 'solver_used', 'model_source_saved', '-v7.3');`	Saves the solution trajectory and run metadata to a MAT-file.
72	`fprintf('Saved solution to %s using %s, model_source=%s, road_type=%s.\n', args.output_mat, solver_used, args.model_source, args.road_type);`	Prints status information to the MATLAB command window so the user can see progress.
73	`vehicle14_save_plots_matlab(t, x, mf_fun, par, args, road, meta, args.plot_dir);`	Executes this MATLAB statement in `run_vehicle14_matlab_casadi_demo`. It performs the operation shown by the sanitized code line.
74	`try`	Starts a protected block. If an error occurs, execution jumps to the matching catch block.
75	`render_vehicle14_matlab_animation(args.output_mat, args.animation_mp4, args.animation_gif);`	Executes this MATLAB statement in `run_vehicle14_matlab_casadi_demo`. It performs the operation shown by the sanitized code line.
76	`catch ME`	Handles an error from the preceding protected block; used for solver or rendering fallback.
77	`warning('Animation rendering failed: %s', ME.message);`	Issues a MATLAB warning while allowing execution to continue.
78	`end`	Ends the current function, conditional block, loop, or protected block.
79	`end`	Ends the current function, conditional block, loop, or protected block.
80		Blank line used to separate logical blocks and improve readability.
81	`function [mode, model_source, road_type] = parse_run_mode_source_road(varargin)`	Defines the local MATLAB function `parse_run_mode_source_road`. Parses flexible command arguments into run mode, input-model source, and road type.
82	`%PARSE_RUN_MODE_SOURCE_ROAD Flexible parser for mode/source/road options.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
83	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
84	`% Valid modes:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
85	`% simulate, plots-only, render-only`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
86	`% Valid model sources:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
87	`% outer, original`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
88	`% Valid road types:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
89	`% ribbon, 3d, 3d-ribbon, road-3d, flat`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
90	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
91	`% Examples:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
92	`% run_vehicle14_matlab_casadi_demo`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
93	`% run_vehicle14_matlab_casadi_demo('outer')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
94	`% run_vehicle14_matlab_casadi_demo('original','flat')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
95	`% run_vehicle14_matlab_casadi_demo('simulate','outer','flat')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
96	`% run_vehicle14_matlab_casadi_demo('plots-only','outer','ribbon')`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
97		Blank line used to separate logical blocks and improve readability.
98	`valid_modes = {'simulate','plots-only','render-only'};`	Assigns `valid_modes` from `{'simulate','plots-only','render-only'}`. This value is used by later computations in this function.
99	`valid_sources = {'outer','original'};`	Assigns `valid_sources` from `{'outer','original'}`. This value is used by later computations in this function.
100	`valid_ribbon_aliases = {'ribbon','3d','3d-ribbon','road-3d','demo-ribbon'};`	Assigns `valid_ribbon_aliases` from `{'ribbon','3d','3d-ribbon','road-3d','demo-ribbon'}`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
101	`valid_flat_aliases = {'flat','flat-road','plane'};`	Assigns `valid_flat_aliases` from `{'flat','flat-road','plane'}`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
102		Blank line used to separate logical blocks and improve readability.
103	`mode = 'simulate';`	Assigns `mode` from `'simulate'`. This value is used by later computations in this function.
104	`model_source = 'outer';`	Assigns `model_source` from `'outer'`. This value is used by later computations in this function.
105	`road_type = 'ribbon';`	Assigns `road_type` from `'ribbon'`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
106		Blank line used to separate logical blocks and improve readability.
107	`for ii = 1:nargin`	Starts a loop; the following block is repeated for the specified index range.
108	`if isempty(varargin{ii})`	Starts a conditional branch. The following block executes only when this logical test is true.
109	`continue;`	Executes this MATLAB statement in `parse_run_mode_source_road`. It performs the operation shown by the sanitized code line.
110	`end`	Ends the current function, conditional block, loop, or protected block.
111	`token = lower(char(varargin{ii}));`	Assigns `token` from `lower(char(varargin{ii}))`. This value is used by later computations in this function.
112	`if any(strcmp(token, valid_modes))`	Starts a conditional branch. The following block executes only when this logical test is true.

Line	Sanitized code line	Explanation
113	`mode = token;`	Assigns `mode` from `token`. This value is used by later computations in this function.
114	`elseif any(strcmp(token, valid_sources))`	Starts an alternative conditional branch, tested only if previous branches were false.
115	`model_source = token;`	Assigns `model_source` from `token`. This value is used by later computations in this function.
116	`elseif any(strcmp(token, valid_ribbon_aliases))`	Starts an alternative conditional branch, tested only if previous branches were false.
117	`road_type = 'ribbon';`	Assigns `road_type` from `'ribbon'`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
118	`elseif any(strcmp(token, valid_flat_aliases))`	Starts an alternative conditional branch, tested only if previous branches were false.
119	`road_type = 'flat';`	Assigns `road_type` from `'flat'`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
120	`else`	Starts the fallback branch for the current conditional block.
121	`error('Unknown argument: %s. Use mode={simulate,plots-only,render-only}, source={outer,original}, road={ribbon,flat}.', token);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
122	`end`	Ends the current function, conditional block, loop, or protected block.
123	`end`	Ends the current function, conditional block, loop, or protected block.
124	`end`	Ends the current function, conditional block, loop, or protected block.
125		Blank line used to separate logical blocks and improve readability.
126	`function args = apply_model_source_to_args(args, model_source, road_type)`	Defines the local MATLAB function `apply_model_source_to_args`. Stores the selected source and road type and creates output filenames that will not overwrite other runs.
127	`args.model_source = lower(char(model_source));`	Sets run option `args.model_source` to `lower(char(model_source))`. This controls solver, maneuver, road, output, or animation behavior.
128	`args.road_type = lower(char(road_type));`	Sets run option `args.road_type` to `lower(char(road_type))`. This controls solver, maneuver, road, output, or animation behavior.
129	`args.output_mat = sprintf('vehicle14_matlab_casadi_solution_%s_%s.mat', args.model_source, args.road_type);`	Sets run option `args.output_mat` to `sprintf('vehicle14_matlab_casadi_solution_%s_%s.mat', args.model_source, args.road_type)`. This controls solver, maneuver, road, output, or animation behavior.
130	`args.plot_dir = sprintf('vehicle14_matlab_casadi_plots_%s_%s', args.model_source, args.road_type);`	Sets run option `args.plot_dir` to `sprintf('vehicle14_matlab_casadi_plots_%s_%s', args.model_source, args.road_type)`. This controls solver, maneuver, road, output, or animation behavior.
131	`args.animation_mp4 = sprintf('vehicle14_matlab_casadi_animation_%s_%s.mp4', args.model_source, args.road_type);`	Sets run option `args.animation_mp4` to `sprintf('vehicle14_matlab_casadi_animation_%s_%s.mp4', args.model_source, args.road_type)`. This controls solver, maneuver, road, output, or animation behavior.
132	`args.animation_gif = sprintf('vehicle14_matlab_casadi_animation_%s_%s.gif', args.model_source, args.road_type);`	Sets run option `args.animation_gif` to `sprintf('vehicle14_matlab_casadi_animation_%s_%s.gif', args.model_source, args.road_type)`. This controls solver, maneuver, road, output, or animation behavior.
133	`end`	Ends the current function, conditional block, loop, or protected block.
134		Blank line used to separate logical blocks and improve readability.
135	`function [ok,msg] = check_casadi_available()`	Defines the local MATLAB function `check_casadi_available`. Checks whether the CasADi MATLAB interface is visible and usable.
136	`ok = true; msg = '';`	Assigns `ok` from `true; msg = ''`. This value is used by later computations in this function.
137	`try`	Starts a protected block. If an error occurs, execution jumps to the matching catch block.
138	`import casadi.*`	Imports a MATLAB package namespace so that functions/classes from that package can be called without full qualification.
139	`SX.sym('probe',1,1);`	Executes this MATLAB statement in `check_casadi_available`. It performs the operation shown by the sanitized code line.

Line	Sanitized code line	Explanation
140	`catch ME`	Handles an error from the preceding protected block; used for solver or rendering fallback.

Line	Sanitized code line	Explanation
141	`ok = false;`	Assigns `ok` from `false`. This value is used by later computations in this function.
142	`msg = sprintf('CasADi is not on the MATLAB path or failed to initialize: %s', ME.message);`	Assigns `msg` from `sprintf('CasADi is not on the MATLAB path or failed to initialize: %s', ME.message)`. This value is used by later computations in this function.
143	`end`	Ends the current function, conditional block, loop, or protected block.
144	`end`	Ends the current function, conditional block, loop, or protected block.
145		Blank line used to separate logical blocks and improve readability.
146		Blank line used to separate logical blocks and improve readability.
147	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
148	`% Local helper from original package file: vehicle14_build_casadi_function.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
149	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
150	`function [mf_fun, meta] = vehicle14_build_casadi_function()`	Defines the local MATLAB function `vehicle14_build_casadi_function`. Builds the callable CasADi function that evaluates M, F, and S.
151	`%VEHICLE14_BUILD_CASADI_FUNCTION Build the numeric CasADi Function used by MATLAB solvers.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
152	`import casadi.*`	Imports a MATLAB package namespace so that functions/classes from that package can be called without full qualification.
153	`meta = vehicle14_generated_dynamics_meta();`	Assigns `meta` from `vehicle14_generated_dynamics_meta()`. This value is used by later computations in this function.
154	`q = SX.sym('q', meta.nq, 1);`	Assigns `q` from `SX.sym('q', meta.nq, 1)`. This value is used by later computations in this function.
155	`u = SX.sym('u', meta.nu, 1);`	Assigns `u` from `SX.sym('u', meta.nu, 1)`. This value is used by later computations in this function.
156	`P = SX.sym('P', meta.np, 1);`	Assigns `P` from `SX.sym('P', meta.np, 1)`. This value is used by later computations in this function.
157	`IN = SX.sym('IN', meta.nin, 1);`	Assigns `IN` from `SX.sym('IN', meta.nin, 1)`. This value is used by later computations in this function.
158	`[M,F,S] = vehicle14_generated_dynamics_matrices_casadi(q,u,P,IN);`	Executes this MATLAB statement in `vehicle14_build_casadi_function`. It performs the operation shown by the sanitized code line.
159	`mf_fun = Function('vehicle14_mf', {q,u,P,IN}, {M,F,S}, {'q','u','P','IN'}, {'M','F','S'});`	Assigns `mf_fun` from `Function('vehicle14_mf', {q,u,P,IN}, {M,F,S}, {'q','u','P','IN'}, {'M','F','S'})`. This value is used by later computations in this function.
160	`end`	Ends the current function, conditional block, loop, or protected block.
161		Blank line used to separate logical blocks and improve readability.
162	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
163	`% Local helper from original package file: vehicle14_generated_dynamics_meta.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
164	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
165	`function meta = vehicle14_generated_dynamics_meta()`	Defines the local MATLAB function `vehicle14_generated_dynamics_meta`. Creates ordered names and index maps for states, speeds, parameters, inputs, and sensors.
166	`% Metadata matching vehicle14_generated_dynamics_matrices_casadi.m.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
167	`meta.q_names = {'X', 'Y', 'Z', 'psi', 'theta', 'phi', 'zFL', 'zFR', 'zRL', 'zRR', 'chiFL', 'chiFR', 'chiRL', 'chiRR'};`	Stores the ordered names for one vector used by the generated dynamics function; the order must match the generated formulas exactly.
168	`meta.u_names = {'vx', 'vy', 'vz', 'p', 'q_pitch', 'r', 'dzFL', 'dzFR', 'dzRL', 'dzRR', 'omFL', 'omFR', 'omRL', 'omRR'};`	Stores the ordered names for one vector used by the generated dynamics function; the order must match the generated formulas exactly.

Line	Sanitized code line	Explanation
169	`meta.p_names = {'mB', 'mW', 'Ixx', 'Iyy', 'Izz', 'Ixz', 'Iw', 'lf', 'lr', 'tf', 'tr', 'hc', 'rw', 'g', 'ksf', 'ksr', 'csf', 'csr', 'karbf', 'karbr', 'carbf', 'carbr', 'Fpre_FL', 'Fpre_FR', 'Fpre_RL', 'Fpre_RR', 'rack_m_per_rad', 'kcx1_f'...`	Stores the ordered names for one vector used by the generated dynamics function; the order must match the generated formulas exactly.
170	`meta.input_names = {'delta', 'delta_dot', 'FBxFL', 'FByFL', 'FBzFL', 'MBxFL', 'MByFL', 'MBzFL', 'FBxFR', 'FByFR', 'FBzFR', 'MBxFR', 'MByFR', 'MBzFR', 'FBxRL', 'FByRL', 'FBzRL', 'MBxRL', 'MByRL', 'MBzRL', 'FBxRR', 'FByRR', 'FBzRR', 'MBxRR...`	Stores the ordered names for one vector used by the generated dynamics function; the order must match the generated formulas exactly.
171	`meta.sensor_names = {'FL_Wc_x', 'FL_Wc_y', 'FL_Wc_z', 'FL_Wc_vx', 'FL_Wc_vy', 'FL_Wc_vz', 'FL_Vx_wc', 'FL_Vy_wc', 'FL_x_body', 'FL_y_body', 'FL_z_body', 'FL_toe', 'FL_camber', 'FL_pitch', 'FL_rack', 'FL_head_Nx', 'FL_head_Ny', 'FL_head_N...`	Stores the ordered names for one vector used by the generated dynamics function; the order must match the generated formulas exactly.
172	`meta.nq = 14; meta.nu = 14; meta.nx = 28;`	Assigns `meta.nq` from `14; meta.nu = 14; meta.nx = 28`. This value is used by later computations in this function.
173	`meta.np = 78; meta.nin = 34; meta.ns = 99;`	Assigns `meta.np` from `78; meta.nin = 34; meta.ns = 99`. This value is used by later computations in this function.
174	`meta.input_index = struct();`	Assigns `meta.input_index` from `struct()`. This value is used by later computations in this function.
175	`meta.input_index.delta = 1;`	Creates an index map for input `delta` so code can write/read IN(1) by name instead of remembering the number.
176	`meta.input_index.delta_dot = 2;`	Creates an index map for input `delta_dot` so code can write/read IN(2) by name instead of remembering the number.
177	`meta.input_index.FBxFL = 3;`	Creates an index map for input `FBxFL` so code can write/read IN(3) by name instead of remembering the number.
178	`meta.input_index.FByFL = 4;`	Creates an index map for input `FByFL` so code can write/read IN(4) by name instead of remembering the number.
179	`meta.input_index.FBzFL = 5;`	Creates an index map for input `FBzFL` so code can write/read IN(5) by name instead of remembering the number.
180	`meta.input_index.MBxFL = 6;`	Creates an index map for input `MBxFL` so code can write/read IN(6) by name instead of remembering the number.
181	`meta.input_index.MByFL = 7;`	Creates an index map for input `MByFL` so code can write/read IN(7) by name instead of remembering the number.
182	`meta.input_index.MBzFL = 8;`	Creates an index map for input `MBzFL` so code can write/read IN(8) by name instead of remembering the number.
183	`meta.input_index.FBxFR = 9;`	Creates an index map for input `FBxFR` so code can write/read IN(9) by name instead of remembering the number.
184	`meta.input_index.FByFR = 10;`	Creates an index map for input `FByFR` so code can write/read IN(10) by name instead of remembering the number.

Line	Sanitized code line	Explanation
185	`meta.input_index.FBzFR = 11;`	Creates an index map for input `FBzFR` so code can write/read IN(11) by name instead of remembering the number.
186	`meta.input_index.MBxFR = 12;`	Creates an index map for input `MBxFR` so code can write/read IN(12) by name instead of remembering the number.
187	`meta.input_index.MByFR = 13;`	Creates an index map for input `MByFR` so code can write/read IN(13) by name instead of remembering the number.
188	`meta.input_index.MBzFR = 14;`	Creates an index map for input `MBzFR` so code can write/read IN(14) by name instead of remembering the number.
189	`meta.input_index.FBxRL = 15;`	Creates an index map for input `FBxRL` so code can write/read IN(15) by name instead of remembering the number.
190	`meta.input_index.FByRL = 16;`	Creates an index map for input `FByRL` so code can write/read IN(16) by name instead of remembering the number.
191	`meta.input_index.FBzRL = 17;`	Creates an index map for input `FBzRL` so code can write/read IN(17) by name instead of remembering the number.
192	`meta.input_index.MBxRL = 18;`	Creates an index map for input `MBxRL` so code can write/read IN(18) by name instead of remembering the number.
193	`meta.input_index.MByRL = 19;`	Creates an index map for input `MByRL` so code can write/read IN(19) by name instead of remembering the number.
194	`meta.input_index.MBzRL = 20;`	Creates an index map for input `MBzRL` so code can write/read IN(20) by name instead of remembering the number.
195	`meta.input_index.FBxRR = 21;`	Creates an index map for input `FBxRR` so code can write/read IN(21) by name instead of remembering the number.
196	`meta.input_index.FByRR = 22;`	Creates an index map for input `FByRR` so code can write/read IN(22) by name instead of remembering the number.

Line	Sanitized code line	Explanation
197	`meta.input_index.FBzRR = 23;`	Creates an index map for input `FBzRR` so code can write/read IN(23) by name instead of remembering the number.
198	`meta.input_index.MBxRR = 24;`	Creates an index map for input `MBxRR` so code can write/read IN(24) by name instead of remembering the number.
199	`meta.input_index.MByRR = 25;`	Creates an index map for input `MByRR` so code can write/read IN(25) by name instead of remembering the number.
200	`meta.input_index.MBzRR = 26;`	Creates an index map for input `MBzRR` so code can write/read IN(26) by name instead of remembering the number.
201	`meta.input_index.TFL = 27;`	Creates an index map for input `TFL` so code can write/read IN(27) by name instead of remembering the number.
202	`meta.input_index.TFR = 28;`	Creates an index map for input `TFR` so code can write/read IN(28) by name instead of remembering the number.
203	`meta.input_index.TRL = 29;`	Creates an index map for input `TRL` so code can write/read IN(29) by name instead of remembering the number.
204	`meta.input_index.TRR = 30;`	Creates an index map for input `TRR` so code can write/read IN(30) by name instead of remembering the number.
205	`meta.input_index.FdragF = 31;`	Creates an index map for input `FdragF` so code can write/read IN(31) by name instead of remembering the number.
206	`meta.input_index.FdownF = 32;`	Creates an index map for input `FdownF` so code can write/read IN(32) by name instead of remembering the number.
207	`meta.input_index.FdragR = 33;`	Creates an index map for input `FdragR` so code can write/read IN(33) by name instead of remembering the number.
208	`meta.input_index.FdownR = 34;`	Creates an index map for input `FdownR` so code can write/read IN(34) by name instead of remembering the number.
209	`meta.sensor_index = struct();`	Assigns `meta.sensor_index` from `struct()`. This value is used by later computations in this function.
210	`meta.sensor_index.FL_Wc_x = 1;`	Creates an index map for sensor `FL_Wc_x` so code can read S(1) by name instead of remembering the number.
211	`meta.sensor_index.FL_Wc_y = 2;`	Creates an index map for sensor `FL_Wc_y` so code can read S(2) by name instead of remembering the number.
212	`meta.sensor_index.FL_Wc_z = 3;`	Creates an index map for sensor `FL_Wc_z` so code can read S(3) by name instead of remembering the number.
213	`meta.sensor_index.FL_Wc_vx = 4;`	Creates an index map for sensor `FL_Wc_vx` so code can read S(4) by name instead of remembering the number.
214	`meta.sensor_index.FL_Wc_vy = 5;`	Creates an index map for sensor `FL_Wc_vy` so code can read S(5) by name instead of remembering the number.
215	`meta.sensor_index.FL_Wc_vz = 6;`	Creates an index map for sensor `FL_Wc_vz` so code can read S(6) by name instead of remembering the number.
216	`meta.sensor_index.FL_Vx_wc = 7;`	Creates an index map for sensor `FL_Vx_wc` so code can read S(7) by name instead of remembering the number.
217	`meta.sensor_index.FL_Vy_wc = 8;`	Creates an index map for sensor `FL_Vy_wc` so code can read S(8) by name instead of remembering the number.
218	`meta.sensor_index.FL_x_body = 9;`	Creates an index map for sensor `FL_x_body` so code can read S(9) by name instead of remembering the number.
219	`meta.sensor_index.FL_y_body = 10;`	Creates an index map for sensor `FL_y_body` so code can read S(10) by name instead of remembering the number.
220	`meta.sensor_index.FL_z_body = 11;`	Creates an index map for sensor `FL_z_body` so code can read S(11) by name instead of remembering the number.
221	`meta.sensor_index.FL_toe = 12;`	Creates an index map for sensor `FL_toe` so code can read S(12) by name instead of remembering the number.
222	`meta.sensor_index.FL_camber = 13;`	Creates an index map for sensor `FL_camber` so code can read S(13) by name instead of remembering the number.
223	`meta.sensor_index.FL_pitch = 14;`	Creates an index map for sensor `FL_pitch` so code can read S(14) by name instead of remembering the number.
224	`meta.sensor_index.FL_rack = 15;`	Creates an index map for sensor `FL_rack` so code can read S(15) by name instead of remembering the number.

Line	Sanitized code line	Explanation
225	`meta.sensor_index.FL_head_Nx = 16;`	Creates an index map for sensor `FL_head_Nx` so code can read S(16) by name instead of remembering the number.
226	`meta.sensor_index.FL_head_Ny = 17;`	Creates an index map for sensor `FL_head_Ny` so code can read S(17) by name instead of remembering the number.
227	`meta.sensor_index.FL_head_Nz = 18;`	Creates an index map for sensor `FL_head_Nz` so code can read S(18) by name instead of remembering the number.
228	`meta.sensor_index.FL_spin_Nx = 19;`	Creates an index map for sensor `FL_spin_Nx` so code can read S(19) by name instead of remembering the number.
229	`meta.sensor_index.FL_spin_Ny = 20;`	Creates an index map for sensor `FL_spin_Ny` so code can read S(20) by name instead of remembering the number.
230	`meta.sensor_index.FL_spin_Nz = 21;`	Creates an index map for sensor `FL_spin_Nz` so code can read S(21) by name instead of remembering the number.
231	`meta.sensor_index.FL_carrier_wx = 22;`	Creates an index map for sensor `FL_carrier_wx` so code can read S(22) by name instead of remembering the number.
232	`meta.sensor_index.FL_carrier_wy = 23;`	Creates an index map for sensor `FL_carrier_wy` so code can read S(23) by name instead of remembering the number.
233	`meta.sensor_index.FL_carrier_wz = 24;`	Creates an index map for sensor `FL_carrier_wz` so code can read S(24) by name instead of remembering the number.
234	`meta.sensor_index.FR_Wc_x = 25;`	Creates an index map for sensor `FR_Wc_x` so code can read S(25) by name instead of remembering the number.
235	`meta.sensor_index.FR_Wc_y = 26;`	Creates an index map for sensor `FR_Wc_y` so code can read S(26) by name instead of remembering the number.
236	`meta.sensor_index.FR_Wc_z = 27;`	Creates an index map for sensor `FR_Wc_z` so code can read S(27) by name instead of remembering the number.

Line	Sanitized code line	Explanation
237	`meta.sensor_index.FR_Wc_vx = 28;`	Creates an index map for sensor `FR_Wc_vx` so code can read S(28) by name instead of remembering the number.
238	`meta.sensor_index.FR_Wc_vy = 29;`	Creates an index map for sensor `FR_Wc_vy` so code can read S(29) by name instead of remembering the number.
239	`meta.sensor_index.FR_Wc_vz = 30;`	Creates an index map for sensor `FR_Wc_vz` so code can read S(30) by name instead of remembering the number.
240	`meta.sensor_index.FR_Vx_wc = 31;`	Creates an index map for sensor `FR_Vx_wc` so code can read S(31) by name instead of remembering the number.
241	`meta.sensor_index.FR_Vy_wc = 32;`	Creates an index map for sensor `FR_Vy_wc` so code can read S(32) by name instead of remembering the number.
242	`meta.sensor_index.FR_x_body = 33;`	Creates an index map for sensor `FR_x_body` so code can read S(33) by name instead of remembering the number.
243	`meta.sensor_index.FR_y_body = 34;`	Creates an index map for sensor `FR_y_body` so code can read S(34) by name instead of remembering the number.
244	`meta.sensor_index.FR_z_body = 35;`	Creates an index map for sensor `FR_z_body` so code can read S(35) by name instead of remembering the number.
245	`meta.sensor_index.FR_toe = 36;`	Creates an index map for sensor `FR_toe` so code can read S(36) by name instead of remembering the number.
246	`meta.sensor_index.FR_camber = 37;`	Creates an index map for sensor `FR_camber` so code can read S(37) by name instead of remembering the number.
247	`meta.sensor_index.FR_pitch = 38;`	Creates an index map for sensor `FR_pitch` so code can read S(38) by name instead of remembering the number.
248	`meta.sensor_index.FR_rack = 39;`	Creates an index map for sensor `FR_rack` so code can read S(39) by name instead of remembering the number.
249	`meta.sensor_index.FR_head_Nx = 40;`	Creates an index map for sensor `FR_head_Nx` so code can read S(40) by name instead of remembering the number.
250	`meta.sensor_index.FR_head_Ny = 41;`	Creates an index map for sensor `FR_head_Ny` so code can read S(41) by name instead of remembering the number.
251	`meta.sensor_index.FR_head_Nz = 42;`	Creates an index map for sensor `FR_head_Nz` so code can read S(42) by name instead of remembering the number.
252	`meta.sensor_index.FR_spin_Nx = 43;`	Creates an index map for sensor `FR_spin_Nx` so code can read S(43) by name instead of remembering the number.

Line	Sanitized code line	Explanation
253	`meta.sensor_index.FR_spin_Ny = 44;`	Creates an index map for sensor `FR_spin_Ny` so code can read S(44) by name instead of remembering the number.
254	`meta.sensor_index.FR_spin_Nz = 45;`	Creates an index map for sensor `FR_spin_Nz` so code can read S(45) by name instead of remembering the number.
255	`meta.sensor_index.FR_carrier_wx = 46;`	Creates an index map for sensor `FR_carrier_wx` so code can read S(46) by name instead of remembering the number.
256	`meta.sensor_index.FR_carrier_wy = 47;`	Creates an index map for sensor `FR_carrier_wy` so code can read S(47) by name instead of remembering the number.
257	`meta.sensor_index.FR_carrier_wz = 48;`	Creates an index map for sensor `FR_carrier_wz` so code can read S(48) by name instead of remembering the number.
258	`meta.sensor_index.RL_Wc_x = 49;`	Creates an index map for sensor `RL_Wc_x` so code can read S(49) by name instead of remembering the number.
259	`meta.sensor_index.RL_Wc_y = 50;`	Creates an index map for sensor `RL_Wc_y` so code can read S(50) by name instead of remembering the number.
260	`meta.sensor_index.RL_Wc_z = 51;`	Creates an index map for sensor `RL_Wc_z` so code can read S(51) by name instead of remembering the number.
261	`meta.sensor_index.RL_Wc_vx = 52;`	Creates an index map for sensor `RL_Wc_vx` so code can read S(52) by name instead of remembering the number.
262	`meta.sensor_index.RL_Wc_vy = 53;`	Creates an index map for sensor `RL_Wc_vy` so code can read S(53) by name instead of remembering the number.
263	`meta.sensor_index.RL_Wc_vz = 54;`	Creates an index map for sensor `RL_Wc_vz` so code can read S(54) by name instead of remembering the number.
264	`meta.sensor_index.RL_Vx_wc = 55;`	Creates an index map for sensor `RL_Vx_wc` so code can read S(55) by name instead of remembering the number.
265	`meta.sensor_index.RL_Vy_wc = 56;`	Creates an index map for sensor `RL_Vy_wc` so code can read S(56) by name instead of remembering the number.
266	`meta.sensor_index.RL_x_body = 57;`	Creates an index map for sensor `RL_x_body` so code can read S(57) by name instead of remembering the number.
267	`meta.sensor_index.RL_y_body = 58;`	Creates an index map for sensor `RL_y_body` so code can read S(58) by name instead of remembering the number.
268	`meta.sensor_index.RL_z_body = 59;`	Creates an index map for sensor `RL_z_body` so code can read S(59) by name instead of remembering the number.
269	`meta.sensor_index.RL_toe = 60;`	Creates an index map for sensor `RL_toe` so code can read S(60) by name instead of remembering the number.
270	`meta.sensor_index.RL_camber = 61;`	Creates an index map for sensor `RL_camber` so code can read S(61) by name instead of remembering the number.
271	`meta.sensor_index.RL_pitch = 62;`	Creates an index map for sensor `RL_pitch` so code can read S(62) by name instead of remembering the number.
272	`meta.sensor_index.RL_rack = 63;`	Creates an index map for sensor `RL_rack` so code can read S(63) by name instead of remembering the number.
273	`meta.sensor_index.RL_head_Nx = 64;`	Creates an index map for sensor `RL_head_Nx` so code can read S(64) by name instead of remembering the number.
274	`meta.sensor_index.RL_head_Ny = 65;`	Creates an index map for sensor `RL_head_Ny` so code can read S(65) by name instead of remembering the number.
275	`meta.sensor_index.RL_head_Nz = 66;`	Creates an index map for sensor `RL_head_Nz` so code can read S(66) by name instead of remembering the number.
276	`meta.sensor_index.RL_spin_Nx = 67;`	Creates an index map for sensor `RL_spin_Nx` so code can read S(67) by name instead of remembering the number.
277	`meta.sensor_index.RL_spin_Ny = 68;`	Creates an index map for sensor `RL_spin_Ny` so code can read S(68) by name instead of remembering the number.
278	`meta.sensor_index.RL_spin_Nz = 69;`	Creates an index map for sensor `RL_spin_Nz` so code can read S(69) by name instead of remembering the number.
279	`meta.sensor_index.RL_carrier_wx = 70;`	Creates an index map for sensor `RL_carrier_wx` so code can read S(70) by name instead of remembering the number.
280	`meta.sensor_index.RL_carrier_wy = 71;`	Creates an index map for sensor `RL_carrier_wy` so code can read S(71) by name instead of remembering the number.

Line	Sanitized code line	Explanation
281	`meta.sensor_index.RL_carrier_wz = 72;`	Creates an index map for sensor `RL_carrier_wz` so code can read S(72) by name instead of remembering the number.
282	`meta.sensor_index.RR_Wc_x = 73;`	Creates an index map for sensor `RR_Wc_x` so code can read S(73) by name instead of remembering the number.
283	`meta.sensor_index.RR_Wc_y = 74;`	Creates an index map for sensor `RR_Wc_y` so code can read S(74) by name instead of remembering the number.
284	`meta.sensor_index.RR_Wc_z = 75;`	Creates an index map for sensor `RR_Wc_z` so code can read S(75) by name instead of remembering the number.
285	`meta.sensor_index.RR_Wc_vx = 76;`	Creates an index map for sensor `RR_Wc_vx` so code can read S(76) by name instead of remembering the number.
286	`meta.sensor_index.RR_Wc_vy = 77;`	Creates an index map for sensor `RR_Wc_vy` so code can read S(77) by name instead of remembering the number.
287	`meta.sensor_index.RR_Wc_vz = 78;`	Creates an index map for sensor `RR_Wc_vz` so code can read S(78) by name instead of remembering the number.
288	`meta.sensor_index.RR_Vx_wc = 79;`	Creates an index map for sensor `RR_Vx_wc` so code can read S(79) by name instead of remembering the number.

Line	Sanitized code line	Explanation
289	`meta.sensor_index.RR_Vy_wc = 80;`	Creates an index map for sensor `RR_Vy_wc` so code can read S(80) by name instead of remembering the number.
290	`meta.sensor_index.RR_x_body = 81;`	Creates an index map for sensor `RR_x_body` so code can read S(81) by name instead of remembering the number.
291	`meta.sensor_index.RR_y_body = 82;`	Creates an index map for sensor `RR_y_body` so code can read S(82) by name instead of remembering the number.
292	`meta.sensor_index.RR_z_body = 83;`	Creates an index map for sensor `RR_z_body` so code can read S(83) by name instead of remembering the number.
293	`meta.sensor_index.RR_toe = 84;`	Creates an index map for sensor `RR_toe` so code can read S(84) by name instead of remembering the number.
294	`meta.sensor_index.RR_camber = 85;`	Creates an index map for sensor `RR_camber` so code can read S(85) by name instead of remembering the number.
295	`meta.sensor_index.RR_pitch = 86;`	Creates an index map for sensor `RR_pitch` so code can read S(86) by name instead of remembering the number.
296	`meta.sensor_index.RR_rack = 87;`	Creates an index map for sensor `RR_rack` so code can read S(87) by name instead of remembering the number.
297	`meta.sensor_index.RR_head_Nx = 88;`	Creates an index map for sensor `RR_head_Nx` so code can read S(88) by name instead of remembering the number.
298	`meta.sensor_index.RR_head_Ny = 89;`	Creates an index map for sensor `RR_head_Ny` so code can read S(89) by name instead of remembering the number.
299	`meta.sensor_index.RR_head_Nz = 90;`	Creates an index map for sensor `RR_head_Nz` so code can read S(90) by name instead of remembering the number.
300	`meta.sensor_index.RR_spin_Nx = 91;`	Creates an index map for sensor `RR_spin_Nx` so code can read S(91) by name instead of remembering the number.
301	`meta.sensor_index.RR_spin_Ny = 92;`	Creates an index map for sensor `RR_spin_Ny` so code can read S(92) by name instead of remembering the number.
302	`meta.sensor_index.RR_spin_Nz = 93;`	Creates an index map for sensor `RR_spin_Nz` so code can read S(93) by name instead of remembering the number.
303	`meta.sensor_index.RR_carrier_wx = 94;`	Creates an index map for sensor `RR_carrier_wx` so code can read S(94) by name instead of remembering the number.
304	`meta.sensor_index.RR_carrier_wy = 95;`	Creates an index map for sensor `RR_carrier_wy` so code can read S(95) by name instead of remembering the number.
305	`meta.sensor_index.RR_carrier_wz = 96;`	Creates an index map for sensor `RR_carrier_wz` so code can read S(96) by name instead of remembering the number.
306	`meta.sensor_index.aero_front_z = 97;`	Creates an index map for sensor `aero_front_z` so code can read S(97) by name instead of remembering the number.
307	`meta.sensor_index.aero_rear_z = 98;`	Creates an index map for sensor `aero_rear_z` so code can read S(98) by name instead of remembering the number.
308	`meta.sensor_index.vx_body = 99;`	Creates an index map for sensor `vx_body` so code can read S(99) by name instead of remembering the number.

Line	Sanitized code line	Explanation
309	`end`	Ends the current function, conditional block, loop, or protected block.
310		Blank line used to separate logical blocks and improve readability.
311	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
312	`% Local helper from original package file: vehicle14_params_default.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
313	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
314	`function par = vehicle14_params_default()`	Defines the local MATLAB function `vehicle14_params_default`. Creates the default parameter structure used by the demo.
315	`% Default parameters copied from the uploaded Python VehicleParams dataclass.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
316	`par.mB = 1250.0;`	Sets default parameter `par.mB` to `1250.0`. This value is later placed in the parameter vector or used by outer models.
317	`par.mW = 42.0;`	Sets default parameter `par.mW` to `42.0`. This value is later placed in the parameter vector or used by outer models.
318	`par.Ixx = 650.0;`	Sets default parameter `par.Ixx` to `650.0`. This value is later placed in the parameter vector or used by outer models.
319	`par.Iyy = 1500.0;`	Sets default parameter `par.Iyy` to `1500.0`. This value is later placed in the parameter vector or used by outer models.
320	`par.Izz = 2200.0;`	Sets default parameter `par.Izz` to `2200.0`. This value is later placed in the parameter vector or used by outer models.
321	`par.Ixz = 0.0;`	Sets default parameter `par.Ixz` to `0.0`. This value is later placed in the parameter vector or used by outer models.
322	`par.Iw = 1.25;`	Sets default parameter `par.Iw` to `1.25`. This value is later placed in the parameter vector or used by outer models.
323	`par.lf = 1.35;`	Sets default parameter `par.lf` to `1.35`. This value is later placed in the parameter vector or used by outer models.
324	`par.lr = 1.35;`	Sets default parameter `par.lr` to `1.35`. This value is later placed in the parameter vector or used by outer models.
325	`par.tf = 1.6;`	Sets default parameter `par.tf` to `1.6`. This value is later placed in the parameter vector or used by outer models.
326	`par.tr = 1.6;`	Sets default parameter `par.tr` to `1.6`. This value is later placed in the parameter vector or used by outer models.
327	`par.hc = 0.55;`	Sets default parameter `par.hc` to `0.55`. This value is later placed in the parameter vector or used by outer models.
328	`par.rw = 0.31;`	Sets default parameter `par.rw` to `0.31`. This value is later placed in the parameter vector or used by outer models.
329	`par.g = 9.81;`	Sets default parameter `par.g` to `9.81`. This value is later placed in the parameter vector or used by outer models.
330	`par.ksf = 36000.0;`	Sets default parameter `par.ksf` to `36000.0`. This value is later placed in the parameter vector or used by outer models.
331	`par.ksr = 39000.0;`	Sets default parameter `par.ksr` to `39000.0`. This value is later placed in the parameter vector or used by outer models.
332	`par.csf = 3600.0;`	Sets default parameter `par.csf` to `3600.0`. This value is later placed in the parameter vector or used by outer models.
333	`par.csr = 3900.0;`	Sets default parameter `par.csr` to `3900.0`. This value is later placed in the parameter vector or used by outer models.
334	`par.karbf = 12000.0;`	Sets default parameter `par.karbf` to `12000.0`. This value is later placed in the parameter vector or used by outer models.
335	`par.karbr = 10000.0;`	Sets default parameter `par.karbr` to `10000.0`. This value is later placed in the parameter vector or used by outer models.
336	`par.carbf = 700.0;`	Sets default parameter `par.carbf` to `700.0`. This value is later placed in the parameter vector or used by outer models.

Line	Sanitized code line	Explanation
337	`par.carbr = 600.0;`	Sets default parameter `par.carbr` to `600.0`. This value is later placed in the parameter vector or used by outer models.
338	`par.Fpre_FL = NaN;`	Sets default parameter `par.Fpre_FL` to `NaN`. This value is later placed in the parameter vector or used by outer models.
339	`par.Fpre_FR = NaN;`	Sets default parameter `par.Fpre_FR` to `NaN`. This value is later placed in the parameter vector or used by outer models.
340	`par.Fpre_RL = NaN;`	Sets default parameter `par.Fpre_RL` to `NaN`. This value is later placed in the parameter vector or used by outer models.

Line	Sanitized code line	Explanation
341	`par.Fpre_RR = NaN;`	Sets default parameter `par.Fpre_RR` to `NaN`. This value is later placed in the parameter vector or used by outer models.
342	`par.rack_m_per_rad = 0.05;`	Sets default parameter `par.rack_m_per_rad` to `0.05`. This value is later placed in the parameter vector or used by outer models.
343	`par.kcx1_f = 0.0;`	Sets default parameter `par.kcx1_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
344	`par.kcx2_f = 0.0;`	Sets default parameter `par.kcx2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
345	`par.kcx1_r = 0.0;`	Sets default parameter `par.kcx1_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
346	`par.kcx2_r = 0.0;`	Sets default parameter `par.kcx2_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
347	`par.kcy1_f = 0.0;`	Sets default parameter `par.kcy1_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
348	`par.kcy2_f = 0.0;`	Sets default parameter `par.kcy2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
349	`par.kcy1_r = 0.0;`	Sets default parameter `par.kcy1_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
350	`par.kcy2_r = 0.0;`	Sets default parameter `par.kcy2_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
351	`par.kcz1_f = 1.0;`	Sets default parameter `par.kcz1_f` to `1.0`. This value is later placed in the parameter vector or used by outer models.
352	`par.kcz1_r = 1.0;`	Sets default parameter `par.kcz1_r` to `1.0`. This value is later placed in the parameter vector or used by outer models.
353	`par.kcz2_f = 0.0;`	Sets default parameter `par.kcz2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
354	`par.kcz2_r = 0.0;`	Sets default parameter `par.kcz2_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
355	`par.kcx_y_f = 0.0;`	Sets default parameter `par.kcx_y_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
356	`par.kcx_zy_f = 0.0;`	Sets default parameter `par.kcx_zy_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
357	`par.kcx_y2_f = 0.0;`	Sets default parameter `par.kcx_y2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
358	`par.kcy_y_f = 0.0;`	Sets default parameter `par.kcy_y_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
359	`par.kcy_zy_f = 0.0;`	Sets default parameter `par.kcy_zy_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
360	`par.kcy_y2_f = 0.0;`	Sets default parameter `par.kcy_y2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
361	`par.kcz_y_f = 0.0;`	Sets default parameter `par.kcz_y_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
362	`par.kcz_zy_f = 0.0;`	Sets default parameter `par.kcz_zy_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
363	`par.kcz_y2_f = 0.0;`	Sets default parameter `par.kcz_y2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
364	`par.toe0_f = 0.0;`	Sets default parameter `par.toe0_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.

Line	Sanitized code line	Explanation
365	`par.toe1_f = 0.0;`	Sets default parameter `par.toe1_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
366	`par.toe2_f = 0.0;`	Sets default parameter `par.toe2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
367	`par.toe0_r = 0.0;`	Sets default parameter `par.toe0_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
368	`par.toe1_r = 0.0;`	Sets default parameter `par.toe1_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
369	`par.toe2_r = 0.0;`	Sets default parameter `par.toe2_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
370	`par.toe_ysr_f = 20.0;`	Sets default parameter `par.toe_ysr_f` to `20.0`. This value is later placed in the parameter vector or used by outer models.
371	`par.toe_zysr_f = 0.0;`	Sets default parameter `par.toe_zysr_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
372	`par.toe_ysr2_f = 0.0;`	Sets default parameter `par.toe_ysr2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
373	`par.pitch0_f = 0.0;`	Sets default parameter `par.pitch0_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
374	`par.pitch1_f = 0.0;`	Sets default parameter `par.pitch1_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
375	`par.pitch2_f = 0.0;`	Sets default parameter `par.pitch2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
376	`par.pitch0_r = 0.0;`	Sets default parameter `par.pitch0_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
377	`par.pitch1_r = 0.0;`	Sets default parameter `par.pitch1_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
378	`par.pitch2_r = 0.0;`	Sets default parameter `par.pitch2_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
379	`par.pitch_ysr_f = 0.0;`	Sets default parameter `par.pitch_ysr_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
380	`par.pitch_zysr_f = 0.0;`	Sets default parameter `par.pitch_zysr_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
381	`par.pitch_ysr2_f = 0.0;`	Sets default parameter `par.pitch_ysr2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
382	`par.cam0_f = 0.0;`	Sets default parameter `par.cam0_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
383	`par.cam1_f = -0.06;`	Sets default parameter `par.cam1_f` to `-0.06`. This value is later placed in the parameter vector or used by outer models.
384	`par.cam2_f = 0.0;`	Sets default parameter `par.cam2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
385	`par.cam0_r = 0.0;`	Sets default parameter `par.cam0_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
386	`par.cam1_r = -0.04;`	Sets default parameter `par.cam1_r` to `-0.04`. This value is later placed in the parameter vector or used by outer models.
387	`par.cam2_r = 0.0;`	Sets default parameter `par.cam2_r` to `0.0`. This value is later placed in the parameter vector or used by outer models.
388	`par.cam_ysr_f = 0.0;`	Sets default parameter `par.cam_ysr_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
389	`par.cam_zysr_f = 0.0;`	Sets default parameter `par.cam_zysr_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
390	`par.cam_ysr2_f = 0.0;`	Sets default parameter `par.cam_ysr2_f` to `0.0`. This value is later placed in the parameter vector or used by outer models.
391	`par.kt = 220000.0;`	Sets default parameter `par.kt` to `220000.0`. This value is later placed in the parameter vector or used by outer models.
392	`par.ct = 1400.0;`	Sets default parameter `par.ct` to `1400.0`. This value is later placed in the parameter vector or used by outer models.

Line	Sanitized code line	Explanation
393	`par.normal_smooth_eps = 30.0;`	Sets default parameter `par.normal_smooth_eps` to `30.0`. This value is later placed in the parameter vector or used by outer models.
394	`par.mu = 1.15;`	Sets default parameter `par.mu` to `1.15`. This value is later placed in the parameter vector or used by outer models.
395	`par.Ck = 90000.0;`	Sets default parameter `par.Ck` to `90000.0`. This value is later placed in the parameter vector or used by outer models.
396	`par.Ca = 85000.0;`	Sets default parameter `par.Ca` to `85000.0`. This value is later placed in the parameter vector or used by outer models.
397	`par.v_eps = 1.0;`	Sets default parameter `par.v_eps` to `1.0`. This value is later placed in the parameter vector or used by outer models.
398	`par.T_drive_max = 1900.0;`	Sets default parameter `par.T_drive_max` to `1900.0`. This value is later placed in the parameter vector or used by outer models.
399	`par.T_brake_max = 4500.0;`	Sets default parameter `par.T_brake_max` to `4500.0`. This value is later placed in the parameter vector or used by outer models.
400	`par.brake_bias_front = 0.65;`	Sets default parameter `par.brake_bias_front` to `0.65`. This value is later placed in the parameter vector or used by outer models.
401	`par.steer_max = 0.12217304763960307;`	Sets default parameter `par.steer_max` to `0.12217304763960307`. This value is later placed in the parameter vector or used by outer models.
402	`par.diff_gain = 35.0;`	Sets default parameter `par.diff_gain` to `35.0`. This value is later placed in the parameter vector or used by outer models.
403	`par.diff_width = 15.0;`	Sets default parameter `par.diff_width` to `15.0`. This value is later placed in the parameter vector or used by outer models.
404	`par.rho = 1.225;`	Sets default parameter `par.rho` to `1.225`. This value is later placed in the parameter vector or used by outer models.
405	`par.CdA = 0.65;`	Sets default parameter `par.CdA` to `0.65`. This value is later placed in the parameter vector or used by outer models.
406	`par.ClA_front = 0.48;`	Sets default parameter `par.ClA_front` to `0.48`. This value is later placed in the parameter vector or used by outer models.
407	`par.ClA_rear = 0.72;`	Sets default parameter `par.ClA_rear` to `0.72`. This value is later placed in the parameter vector or used by outer models.
408	`par.aero_front_x = 1.25;`	Sets default parameter `par.aero_front_x` to `1.25`. This value is later placed in the parameter vector or used by outer models.
409	`par.aero_rear_x = -1.2;`	Sets default parameter `par.aero_rear_x` to `-1.2`. This value is later placed in the parameter vector or used by outer models.
410	`par.aero_z = 0.05;`	Sets default parameter `par.aero_z` to `0.05`. This value is later placed in the parameter vector or used by outer models.
411	`par.aero_h_ref = 0.08;`	Sets default parameter `par.aero_h_ref` to `0.08`. This value is later placed in the parameter vector or used by outer models.
412	`par.aero_h_sens_front = 1.0;`	Sets default parameter `par.aero_h_sens_front` to `1.0`. This value is later placed in the parameter vector or used by outer models.
413	`par.aero_h_sens_rear = 1.2;`	Sets default parameter `par.aero_h_sens_rear` to `1.2`. This value is later placed in the parameter vector or used by outer models.
414	`par.corner_names = {'FL','FR','RL','RR'};`	Sets default parameter `par.corner_names` to `{'FL','FR','RL','RR'}`. This value is later placed in the parameter vector or used by outer models.
415	`par.p_names = {'mB', 'mW', 'Ixx', 'Iyy', 'Izz', 'Ixz', 'Iw', 'lf', 'lr', 'tf', 'tr', 'hc', 'rw', 'g', 'ksf', 'ksr', 'csf', 'csr', 'karbf', 'karbr', 'carbf', 'carbr', 'Fpre_FL', 'Fpre_FR', 'Fpre_RL', 'Fpre_RR', 'rack_m_per_rad', 'kcx1_f',...`	Sets default parameter `par.p_names` to `{'mB', 'mW', 'Ixx', 'Iyy', 'Izz', 'Ixz', 'Iw', 'lf', 'lr', 'tf', 'tr', 'hc', 'rw', 'g', 'ksf', 'ksr', 'csf', 'csr', '...`. This value is later placed in the parameter vector or used by outer models.
416	`par.P = [1250.0; 42.0; 650.0; 1500.0; 2200.0; 0.0; 1.25; 1.35; 1.35; 1.6; 1.6; 0.55; 0.31; 9.81; 36000.0; 39000.0; 3600.0; 3900.0; 12000.0; 10000.0; 700.0; 600.0; 3065.625; 3065.625; 3065.625; 3065.625; 0.05; 0.0; 0.0; 0.0; 0.0; 0.0; 0.0...`	Sets default parameter `par.P` to `[1250.0; 42.0; 650.0; 1500.0; 2200.0; 0.0; 1.25; 1.35; 1.35; 1.6; 1.6; 0.55; 0.31; 9.81; 36000.0; 39000.0; 3600.0; 39...`. This value is later placed in the parameter vector or used by outer models.
417	`end`	Ends the current function, conditional block, loop, or protected block.
418		Blank line used to separate logical blocks and improve readability.
419	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
420	`% Local helper from original package file: vehicle14_demo_options.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
421	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
422	`function args = vehicle14_demo_options()`	Defines the local MATLAB function `vehicle14_demo_options`. Creates the default simulation options, road parameters, maneuver settings, and output names.
423	`%VEHICLE14_DEMO_OPTIONS Options matching the uploaded Python drop-then-maneuver demo.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
424	`args.drop_height = 0.25;`	Sets run option `args.drop_height` to `0.25`. This controls solver, maneuver, road, output, or animation behavior.
425	`args.settle_time = 2.0;`	Sets run option `args.settle_time` to `2.0`. This controls solver, maneuver, road, output, or animation behavior.
426	`args.maneuver_time = 16.0;`	Sets run option `args.maneuver_time` to `16.0`. This controls solver, maneuver, road, output, or animation behavior.
427	`args.t_final = args.settle_time + args.maneuver_time;`	Sets run option `args.t_final` to `args.settle_time + args.maneuver_time`. This controls solver, maneuver, road, output, or animation behavior.
428	`args.max_step = 0.006;`	Sets run option `args.max_step` to `0.006`. This controls solver, maneuver, road, output, or animation behavior.
429	`args.rtol = 1e-7;`	Sets run option `args.rtol` to `1e-7`. This controls solver, maneuver, road, output, or animation behavior.
430	`args.atol = 1e-9;`	Sets run option `args.atol` to `1e-9`. This controls solver, maneuver, road, output, or animation behavior.
431	`args.suspension_init = 'zero-force';`	Sets run option `args.suspension_init` to `'zero-force'`. This controls solver, maneuver, road, output, or animation behavior.
432	`args.initial_x = 0.0;`	Sets run option `args.initial_x` to `0.0`. This controls solver, maneuver, road, output, or animation behavior.
433	`args.initial_y = 0.0;`	Sets run option `args.initial_y` to `0.0`. This controls solver, maneuver, road, output, or animation behavior.
434	`args.initial_vx = 0.0;`	Sets run option `args.initial_vx` to `0.0`. This controls solver, maneuver, road, output, or animation behavior.
435	`args.initial_vz = 0.0;`	Sets run option `args.initial_vz` to `0.0`. This controls solver, maneuver, road, output, or animation behavior.
436	`args.initial_wheel_omega = 0.0;`	Sets run option `args.initial_wheel_omega` to `0.0`. This controls solver, maneuver, road, output, or animation behavior.
437	`args.maneuver_profile = 'aggressive-3d';`	Sets run option `args.maneuver_profile` to `'aggressive-3d'`. This controls solver, maneuver, road, output, or animation behavior.
438	`args.throttle = 0.65;`	Sets run option `args.throttle` to `0.65`. This controls solver, maneuver, road, output, or animation behavior.
439	`args.throttle_start = 0.2;`	Sets run option `args.throttle_start` to `0.2`. This controls solver, maneuver, road, output, or animation behavior.
440	`args.throttle_ramp = 0.45;`	Sets run option `args.throttle_ramp` to `0.45`. This controls solver, maneuver, road, output, or animation behavior.
441	`args.steer_deg = 7.0;`	Sets run option `args.steer_deg` to `7.0`. This controls solver, maneuver, road, output, or animation behavior.
442	`args.steer_start = 1.0;`	Sets run option `args.steer_start` to `1.0`. This controls solver, maneuver, road, output, or animation behavior.
443	`args.steer_ramp = 0.5;`	Sets run option `args.steer_ramp` to `0.5`. This controls solver, maneuver, road, output, or animation behavior.

Line	Sanitized code line	Explanation
444	`args.steer_freq = 0.55;`	Sets run option `args.steer_freq` to `0.55`. This controls solver, maneuver, road, output, or animation behavior.
445	`args.brake_start = 11.5;`	Sets run option `args.brake_start` to `11.5`. This controls solver, maneuver, road, output, or animation behavior.
446	`args.brake_level = 0.28;`	Sets run option `args.brake_level` to `0.28`. This controls solver, maneuver, road, output, or animation behavior.
447	`args.brake_ramp = 0.35;`	Sets run option `args.brake_ramp` to `0.35`. This controls solver, maneuver, road, output, or animation behavior.
448	`args.brake_hold = 1.15;`	Sets run option `args.brake_hold` to `1.15`. This controls solver, maneuver, road, output, or animation behavior.

Line	Sanitized code line	Explanation
449	`args.disable_aero = false;`	Sets run option `args.disable_aero` to `false`. This controls solver, maneuver, road, output, or animation behavior.
450	`args.road_type = 'ribbon'; % 'ribbon' for the 3D demo road, or 'flat' for a pure flat road`	Sets run option `args.road_type` to `'ribbon'; % 'ribbon' for the 3D demo road, or 'flat' for a pure flat road`. This controls solver, maneuver, road, output, or animation behavior.
451	`args.road_width = 12.0;`	Sets run option `args.road_width` to `12.0`. This controls solver, maneuver, road, output, or animation behavior.
452	`args.road_elev_amp = 1.0;`	Sets run option `args.road_elev_amp` to `1.0`. This controls solver, maneuver, road, output, or animation behavior.
453	`args.road_elev_wavelength = 80.0;`	Sets run option `args.road_elev_wavelength` to `80.0`. This controls solver, maneuver, road, output, or animation behavior.
454	`args.road_bank_deg = 2.0;`	Sets run option `args.road_bank_deg` to `2.0`. This controls solver, maneuver, road, output, or animation behavior.
455	`args.road_bank_wavelength = 60.0;`	Sets run option `args.road_bank_wavelength` to `60.0`. This controls solver, maneuver, road, output, or animation behavior.
456	`args.road_projection_iterations = 0;`	Sets run option `args.road_projection_iterations` to `0`. This controls solver, maneuver, road, output, or animation behavior.
457	`args.output_mat = 'vehicle14_matlab_casadi_solution.mat';`	Sets run option `args.output_mat` to `'vehicle14_matlab_casadi_solution.mat'`. This controls solver, maneuver, road, output, or animation behavior.
458	`args.plot_dir = 'vehicle14_matlab_casadi_plots';`	Sets run option `args.plot_dir` to `'vehicle14_matlab_casadi_plots'`. This controls solver, maneuver, road, output, or animation behavior.
459	`args.animation_mp4 = 'vehicle14_matlab_casadi_animation.mp4';`	Sets run option `args.animation_mp4` to `'vehicle14_matlab_casadi_animation.mp4'`. This controls solver, maneuver, road, output, or animation behavior.
460	`args.animation_gif = 'vehicle14_matlab_casadi_animation.gif';`	Sets run option `args.animation_gif` to `'vehicle14_matlab_casadi_animation.gif'`. This controls solver, maneuver, road, output, or animation behavior.
461	`end`	Ends the current function, conditional block, loop, or protected block.
462		Blank line used to separate logical blocks and improve readability.
463	`% ----------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
464	`% Local helper from original package file: vehicle14_make_demo_road.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
465	`% ----------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
466	`function road = vehicle14_make_demo_road(args)`	Defines the local MATLAB function `vehicle14_make_demo_road`. Creates either the flat road or the 3D ribbon road data structure.
467	`%VEHICLE14_MAKE_DEMO_ROAD Build either the 3D ribbon road or a pure flat road.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
468	`road.type = lower(getfield_default_runner(args, 'road_type', 'ribbon'));`	Sets road-field `road.type` from the selected road configuration.
469	`road.width = args.road_width;`	Sets road-field `road.width` from the selected road configuration.
470	`road.projection_iterations = args.road_projection_iterations;`	Sets road-field `road.projection_iterations` from the selected road configuration.
471	`road.projection_tol = 1e-11;`	Sets road-field `road.projection_tol` from the selected road configuration.
472		Blank line used to separate logical blocks and improve readability.
473	`if strcmp(road.type, 'flat')`	Starts a conditional branch. The following block executes only when this logical test is true.
474	`% Pure horizontal plane: z=0, no banking, no elevation variation.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
475	`road.elev_amp = 0.0;`	Sets road-field `road.elev_amp` from the selected road configuration.
476	`road.elev_wavelength = 1.0;`	Sets road-field `road.elev_wavelength` from the selected road configuration.

Line	Sanitized code line	Explanation
477	`road.bank_amp = 0.0;`	Sets road-field `road.bank_amp` from the selected road configuration.
478	`road.bank_wavelength = 1.0;`	Sets road-field `road.bank_wavelength` from the selected road configuration.
479	`road.projection_iterations = 0;`	Sets road-field `road.projection_iterations` from the selected road configuration.
480	`elseif strcmp(road.type, 'ribbon')`	Starts an alternative conditional branch, tested only if previous branches were false.
481	`% Original 3D straight ribbon demo road.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
482	`road.elev_amp = args.road_elev_amp;`	Sets road-field `road.elev_amp` from the selected road configuration.
483	`road.elev_wavelength = args.road_elev_wavelength;`	Sets road-field `road.elev_wavelength` from the selected road configuration.
484	`road.bank_amp = deg2rad(args.road_bank_deg);`	Sets road-field `road.bank_amp` from the selected road configuration.
485	`road.bank_wavelength = args.road_bank_wavelength;`	Sets road-field `road.bank_wavelength` from the selected road configuration.
486	`else`	Starts the fallback branch for the current conditional block.
487	`error('Unknown road.type: %s. Use ribbon or flat.', road.type);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
488	`end`	Ends the current function, conditional block, loop, or protected block.
489	`end`	Ends the current function, conditional block, loop, or protected block.
490		Blank line used to separate logical blocks and improve readability.
491	`function [throttle, brake, delta, delta_dot] = vehicle14_command_inputs(t_abs, par, args)`	Defines the local MATLAB function `vehicle14_command_inputs`. Computes the built-in original throttle, brake, steering, and steering-rate signals.
492	`%VEHICLE14_COMMAND_INPUTS Driver command profiles ported from the uploaded Python demo.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
493	`[throttle, brake, delta] = command_no_derivative(t_abs, par, args);`	Executes this MATLAB statement in `vehicle14_command_inputs`. It performs the operation shown by the sanitized code line.
494	`h = 1.0e-5;`	Assigns `h` from `1.0e-5`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
495	`[~,~,delta_p] = command_no_derivative(t_abs + h, par, args);`	Executes this MATLAB statement in `vehicle14_command_inputs`. It performs the operation shown by the sanitized code line.
496	`[~,~,delta_m] = command_no_derivative(t_abs - h, par, args);`	Executes this MATLAB statement in `vehicle14_command_inputs`. It performs the operation shown by the sanitized code line.
497	`delta_dot = (delta_p - delta_m)/(2*h);`	Assigns `delta_dot` from `(delta_p - delta_m)/(2*h)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
498	`end`	Ends the current function, conditional block, loop, or protected block.
499		Blank line used to separate logical blocks and improve readability.
500	`function [throttle, brake, delta] = command_no_derivative(t_abs, par, args)`	Defines the local MATLAB function `command_no_derivative`. Computes built-in throttle, brake, and steering before finite-differencing steering rate.
501	`if t_abs < args.settle_time`	Starts a conditional branch. The following block executes only when this logical test is true.
502	`throttle = 0; brake = 0; delta = 0; return;`	Assigns `throttle` from `0; brake = 0; delta = 0; return`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
503	`end`	Ends the current function, conditional block, loop, or protected block.
504	`tau = t_abs - args.settle_time;`	Assigns `tau` from `t_abs - args.settle_time`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
505	`profile = args.maneuver_profile;`	Assigns `profile` from `args.maneuver_profile`. This value is used by later computations in this function.
506	`if strcmp(profile, 'straight-launch')`	Starts a conditional branch. The following block executes only when this logical test is true.
507	`throttle = args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp);`	Assigns `throttle` from `args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
508	`brake = 0; delta = 0; return;`	Assigns `brake` from `0; delta = 0; return`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
509	`elseif strcmp(profile, 'creep-steer')`	Starts an alternative conditional branch, tested only if previous branches were false.
510	`throttle = args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp);`	Assigns `throttle` from `args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
511	`brake = 0;`	Assigns `brake` from `0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
512	`if args.brake_start >= 0 && tau >= args.brake_start`	Starts a conditional branch. The following block executes only when this logical test is true.
513	`bw = smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp);`	Assigns `bw` from `smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
514	`brake = args.brake_level*bw;`	Assigns `brake` from `args.brake_level*bw`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
515	`throttle = throttle*(1-bw);`	Assigns `throttle` from `throttle*(1-bw)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
516	`end`	Ends the current function, conditional block, loop, or protected block.
517	`steer_amp = deg2rad(args.steer_deg);`	Assigns `steer_amp` from `deg2rad(args.steer_deg)`. This value is used by later computations in this function.
518	`env = smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp);`	Assigns `env` from `smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp)`. This value is used by later computations in this function.
519	`delta = steer_amp*env*sin(2*pi*args.steer_freq*max(tau-args.steer_start,0));`	Assigns `delta` from `steer_amp*env*sin(2*pi*args.steer_freq*max(tau-args.steer_start,0))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
520	`return;`	Returns immediately from the current function.
521	`elseif strcmp(profile, 'source-demo')`	Starts an alternative conditional branch, tested only if previous branches were false.
522	`throttle = 0.17 * (1.0 - smooth_step(tau, 2.2, 2.7));`	Assigns `throttle` from `0.17 * (1.0 - smooth_step(tau, 2.2, 2.7))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
523	`brake = 0.0;`	Assigns `brake` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
524	`if 3.2 <= tau && tau <= 4.8`	Starts a conditional branch. The following block executes only when this logical test is true.
525	`brake = 0.20 * smooth_step(tau, 3.2, 3.5) * (1.0 - smooth_step(tau, 4.4, 4.8));`	Assigns `brake` from `0.20 * smooth_step(tau, 3.2, 3.5) * (1.0 - smooth_step(tau, 4.4, 4.8))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
526	`end`	Ends the current function, conditional block, loop, or protected block.
527	`envelope = smooth_step(tau, 0.8, 1.2) * (1.0 - smooth_step(tau, 5.4, 6.0));`	Assigns `envelope` from `smooth_step(tau, 0.8, 1.2) * (1.0 - smooth_step(tau, 5.4, 6.0))`. This value is used by later computations in this function.
528	`delta = par.steer_max * envelope * sin(2*pi*0.32*max(tau-1.0,0));`	Assigns `delta` from `par.steer_max * envelope * sin(2*pi*0.32*max(tau-1.0,0))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
529	`return;`	Returns immediately from the current function.
530	`elseif strcmp(profile, 'aggressive-3d')`	Starts an alternative conditional branch, tested only if previous branches were false.
531	`launch = smooth_step(tau, 0.10, 0.55);`	Assigns `launch` from `smooth_step(tau, 0.10, 0.55)`. This value is used by later computations in this function.
532	`throttle = args.throttle * launch;`	Assigns `throttle` from `args.throttle * launch`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.

Line	Sanitized code line	Explanation
533	`brake_window = 0;`	Assigns `brake_window` from `0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
534	`if args.brake_start >= 0`	Starts a conditional branch. The following block executes only when this logical test is true.
535	`brake_window = smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp) * ...`	Assigns `brake_window` from `smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp) * ...`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
536	`(1 - smooth_step(tau, args.brake_start + args.brake_hold, args.brake_start + args.brake_hold + args.brake_ramp));`	Executes this MATLAB statement in `command_no_derivative`. It performs the operation shown by the sanitized code line.
537	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
538	`brake = args.brake_level * brake_window;`	Assigns `brake` from `args.brake_level * brake_window`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
539	`throttle = throttle * (1 - 0.85*brake_window);`	Assigns `throttle` from `throttle * (1 - 0.85*brake_window)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
540	`steer_amp = deg2rad(args.steer_deg);`	Assigns `steer_amp` from `deg2rad(args.steer_deg)`. This value is used by later computations in this function.
541	`env = smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp) * ...`	Assigns `env` from `smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp) * ...`. This value is used by later computations in this function.
542	`(1 - smooth_step(tau, args.maneuver_time - 1.0, args.maneuver_time - 0.2));`	Executes this MATLAB statement in `command_no_derivative`. It performs the operation shown by the sanitized code line.
543	`tt = max(tau - args.steer_start, 0);`	Assigns `tt` from `max(tau - args.steer_start, 0)`. This value is used by later computations in this function.
544	`raw = 0.78*sin(2*pi*args.steer_freq*tt) + 0.28*sin(2*pi*1.75*args.steer_freq*tt + 0.45);`	Assigns `raw` from `0.78*sin(2*pi*args.steer_freq*tt) + 0.28*sin(2*pi*1.75*args.steer_freq*tt + 0.45)`. This value is used by later computations in this function.
545	`delta = steer_amp * env * min(max(raw, -1), 1);`	Assigns `delta` from `steer_amp * env * min(max(raw, -1), 1)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
546	`return;`	Returns immediately from the current function.
547	`else`	Starts the fallback branch for the current conditional block.
548	`error('Unknown maneuver_profile: %s', profile);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
549	`end`	Ends the current function, conditional block, loop, or protected block.
550	`end`	Ends the current function, conditional block, loop, or protected block.
551		Blank line used to separate logical blocks and improve readability.
552	`function y = smooth_step(t, t0, t1)`	Defines the local MATLAB function `smooth_step`. Provides a smooth 0-to-1 transition to avoid discontinuous commands.
553	`if t <= t0`	Starts a conditional branch. The following block executes only when this logical test is true.
554	`y = 0;`	Assigns `y` from `0`. This value is used by later computations in this function.
555	`elseif t >= t1`	Starts an alternative conditional branch, tested only if previous branches were false.
556	`y = 1;`	Assigns `y` from `1`. This value is used by later computations in this function.
557	`else`	Starts the fallback branch for the current conditional block.
558	`a = (t-t0)/(t1-t0);`	Assigns `a` from `(t-t0)/(t1-t0)`. This value is used by later computations in this function.
559	`y = a*a*(3 - 2*a);`	Assigns `y` from `a*a*(3 - 2*a)`. This value is used by later computations in this function.
560	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
561	`end`	Ends the current function, conditional block, loop, or protected block.
562		Blank line used to separate logical blocks and improve readability.
563	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
564	`% Local helper from original package file: vehicle14_initial_state.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
565	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
566	`function x0 = vehicle14_initial_state(mf_fun, par, args, road, meta)`	Defines the local MATLAB function `vehicle14_initial_state`. Builds the initial 28-state vector and adjusts ride height for the requested drop test.
567	`%VEHICLE14_INITIAL_STATE Sensor-based drop initial condition with requested tyre gap.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
568	`q0 = zeros(14,1);`	Assigns `q0` from `zeros(14,1)`. This value is used by later computations in this function.
569	`u0 = zeros(14,1);`	Assigns `u0` from `zeros(14,1)`. This value is used by later computations in this function.
570	`q0(1) = args.initial_x;`	Assigns `q0(1)` from `args.initial_x`. This value is used by later computations in this function.
571	`q0(2) = args.initial_y;`	Assigns `q0(2)` from `args.initial_y`. This value is used by later computations in this function.
572	`q0(7:10) = initial_suspension_coordinates(par, args.suspension_init);`	Executes this MATLAB statement in `vehicle14_initial_state`. It performs the operation shown by the sanitized code line.
573	`u0(1) = args.initial_vx;`	Assigns `u0(1)` from `args.initial_vx`. This value is used by later computations in this function.
574	`u0(3) = args.initial_vz;`	Assigns `u0(3)` from `args.initial_vz`. This value is used by later computations in this function.
575	`u0(11:14) = args.initial_wheel_omega;`	Executes this MATLAB statement in `vehicle14_initial_state`. It performs the operation shown by the sanitized code line.
576	`q0(3) = par.hc + args.drop_height;`	Assigns `q0(3)` from `par.hc + args.drop_height`. This value is used by later computations in this function.
577	`for k = 1:8`	Starts a loop; the following block is repeated for the specified index range.
578	`gap = min_contact_gap(q0, u0, mf_fun, par, road, meta);`	Assigns `gap` from `min_contact_gap(q0, u0, mf_fun, par, road, meta)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
579	`err = gap - args.drop_height;`	Assigns `err` from `gap - args.drop_height`. This value is used by later computations in this function.
580	`if abs(err) < 1e-10, break; end`	Starts a conditional branch. The following block executes only when this logical test is true.
581	`dZ = 1e-5;`	Assigns `dZ` from `1e-5`. This value is used by later computations in this function.
582	`qp = q0; qm = q0; qp(3) = qp(3) + dZ; qm(3) = qm(3) - dZ;`	Assigns `qp` from `q0; qm = q0; qp(3) = qp(3) + dZ; qm(3) = qm(3) - dZ`. This value is used by later computations in this function.
583	`gp = min_contact_gap(qp, u0, mf_fun, par, road, meta);`	Assigns `gp` from `min_contact_gap(qp, u0, mf_fun, par, road, meta)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
584	`gm = min_contact_gap(qm, u0, mf_fun, par, road, meta);`	Assigns `gm` from `min_contact_gap(qm, u0, mf_fun, par, road, meta)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
585	`dgap = (gp-gm)/(2*dZ);`	Assigns `dgap` from `(gp-gm)/(2*dZ)`. This value is used by later computations in this function.
586	`if ~isfinite(dgap) || abs(dgap) < 1e-9`	Starts a conditional branch. The following block executes only when this logical test is true.

Line	Sanitized code line	Explanation
587	`q0(3) = q0(3) - err;`	Assigns `q0(3)` from `q0(3) - err`. This value is used by later computations in this function.
588	`else`	Starts the fallback branch for the current conditional block.

Line	Sanitized code line	Explanation
589	`q0(3) = q0(3) - min(max(err/dgap, -0.5), 0.5);`	Assigns `q0(3)` from `q0(3) - min(max(err/dgap, -0.5), 0.5)`. This value is used by later computations in this function.
590	`end`	Ends the current function, conditional block, loop, or protected block.
591	`end`	Ends the current function, conditional block, loop, or protected block.
592	`x0 = [q0; u0];`	Assigns `x0` from `[q0; u0]`. This value is used by later computations in this function.
593	`end`	Ends the current function, conditional block, loop, or protected block.
594		Blank line used to separate logical blocks and improve readability.
595	`function z = initial_suspension_coordinates(par, mode)`	Defines the local MATLAB function `initial_suspension_coordinates`. Sets initial suspension coordinates from zero-force or nominal assumptions.
596	`Fpre = resolved_preloads(par);`	Assigns `Fpre` from `resolved_preloads(par)`. This value is used by later computations in this function.
597	`if strcmp(mode, 'zero-force')`	Starts a conditional branch. The following block executes only when this logical test is true.
598	`z = [-Fpre(1)/par.ksf; -Fpre(2)/par.ksf; -Fpre(3)/par.ksr; -Fpre(4)/par.ksr];`	Assigns `z` from `[-Fpre(1)/par.ksf; -Fpre(2)/par.ksf; -Fpre(3)/par.ksr; -Fpre(4)/par.ksr]`. This value is used by later computations in this function.
599	`elseif strcmp(mode, 'nominal') || strcmp(mode, 'static-source')`	Starts an alternative conditional branch, tested only if previous branches were false.
600	`z = zeros(4,1);`	Assigns `z` from `zeros(4,1)`. This value is used by later computations in this function.
601	`else`	Starts the fallback branch for the current conditional block.
602	`error('Unknown suspension_init: %s', mode);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
603	`end`	Ends the current function, conditional block, loop, or protected block.
604	`end`	Ends the current function, conditional block, loop, or protected block.
605		Blank line used to separate logical blocks and improve readability.
606	`function Fpre = resolved_preloads(par)`	Defines the local MATLAB function `resolved_preloads`. Computes default or user-supplied static suspension preload forces.
607	`total_w = par.mB*par.g;`	Assigns `total_w` from `par.mB*par.g`. This value is used by later computations in this function.
608	`front_axle = total_w*par.lr/(par.lf+par.lr);`	Assigns `front_axle` from `total_w*par.lr/(par.lf+par.lr)`. This value is used by later computations in this function.
609	`rear_axle = total_w*par.lf/(par.lf+par.lr);`	Assigns `rear_axle` from `total_w*par.lf/(par.lf+par.lr)`. This value is used by later computations in this function.
610	`defaults = [0.5*front_axle; 0.5*front_axle; 0.5*rear_axle; 0.5*rear_axle];`	Assigns `defaults` from `[0.5*front_axle; 0.5*front_axle; 0.5*rear_axle; 0.5*rear_axle]`. This value is used by later computations in this function.
611	`user = [par.Fpre_FL; par.Fpre_FR; par.Fpre_RL; par.Fpre_RR];`	Assigns `user` from `[par.Fpre_FL; par.Fpre_FR; par.Fpre_RL; par.Fpre_RR]`. This value is used by later computations in this function.
612	`Fpre = defaults;`	Assigns `Fpre` from `defaults`. This value is used by later computations in this function.
613	`for i=1:4`	Starts a loop; the following block is repeated for the specified index range.
614	`if ~isnan(user(i)), Fpre(i)=user(i); end`	Starts a conditional branch. The following block executes only when this logical test is true.
615	`end`	Ends the current function, conditional block, loop, or protected block.
616	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
617		Blank line used to separate logical blocks and improve readability.
618	`function gap = min_contact_gap(q0, u0, mf_fun, par, road, meta)`	Defines the local MATLAB function `min_contact_gap`. Computes the minimum tyre-road gap for an initial state candidate.
619	`IN0 = zeros(meta.nin,1);`	Assigns `IN0` from `zeros(meta.nin,1)`. This value is used by later computations in this function.
620	`[~,~,Sdm] = mf_fun(q0, u0, par.P, IN0);`	Executes this MATLAB statement in `min_contact_gap`. It performs the operation shown by the sanitized code line.
621	`S = full(Sdm);`	Assigns `S` from `full(Sdm)`. This value is used by later computations in this function.
622	`idx = meta.sensor_index;`	Assigns `idx` from `meta.sensor_index`. This value is used by later computations in this function.
623	`names = {'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This value is used by later computations in this function.
624	`gaps = zeros(4,1);`	Assigns `gaps` from `zeros(4,1)`. This value is used by later computations in this function.
625	`for i=1:4`	Starts a loop; the following block is repeated for the specified index range.
626	`nm = names{i};`	Assigns `nm` from `names{i}`. This value is used by later computations in this function.
627	`p_wc = [S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];`	Assigns `p_wc` from `[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))]`. This value is used by later computations in this function.
628	`head = [S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];`	Assigns `head` from `[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))]`. This value is used by later computations in this function.
629	`spin = [S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];`	Assigns `spin` from `[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))]`. This value is used by later computations in this function.
630	`geom = oriented_tyre_contact_geometry_local(par, road, p_wc, head, spin);`	Assigns `geom` from `oriented_tyre_contact_geometry_local(par, road, p_wc, head, spin)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
631	`gaps(i) = -geom.compression;`	Assigns `gaps(i)` from `-geom.compression`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
632	`end`	Ends the current function, conditional block, loop, or protected block.
633	`gap = min(gaps);`	Assigns `gap` from `min(gaps)`. This value is used by later computations in this function.
634	`end`	Ends the current function, conditional block, loop, or protected block.
635		Blank line used to separate logical blocks and improve readability.
636	`% Local minimal geometry copies needed before the main contact file is loaded.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
637	`function geom = oriented_tyre_contact_geometry_local(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`	Defines the local MATLAB function `oriented_tyre_contact_geometry_local`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
638	`[s,n,p_road_N,e_long_N,e_lat_N,n_road_N] = road_project_point_local(road, p_wc_N);`	Executes this MATLAB statement in `oriented_tyre_contact_geometry_local`. It performs the operation shown by the sanitized code line.
639	`wheel_heading_N = normalize_num_local(wheel_heading_N, e_long_N);`	Assigns `wheel_heading_N` from `normalize_num_local(wheel_heading_N, e_long_N)`. This value is used by later computations in this function.
640	`spin_axis_N = normalize_num_local(wheel_spin_axis_N, e_lat_N);`	Assigns `spin_axis_N` from `normalize_num_local(wheel_spin_axis_N, e_lat_N)`. This value is used by later computations in this function.
641	`t_long_raw = cross(spin_axis_N, n_road_N);`	Assigns `t_long_raw` from `cross(spin_axis_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
642	`if norm(t_long_raw) < 1e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
643	`t_long_raw = wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N;`	Assigns `t_long_raw` from `wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
644	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
645	`t_long_N = normalize_num_local(t_long_raw, e_long_N);`	Assigns `t_long_N` from `normalize_num_local(t_long_raw, e_long_N)`. This value is used by later computations in this function.
646	`if dot(t_long_N, wheel_heading_N) < 0, t_long_N = -t_long_N; end`	Starts a conditional branch. The following block executes only when this logical test is true.
647	`t_lat_N = normalize_num_local(cross(n_road_N,t_long_N), e_lat_N);`	Assigns `t_lat_N` from `normalize_num_local(cross(n_road_N,t_long_N), e_lat_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
648	`normal_in_radial_plane = n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N;`	Assigns `normal_in_radial_plane` from `n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
649	`gamma = norm(normal_in_radial_plane);`	Assigns `gamma` from `norm(normal_in_radial_plane)`. This value is used by later computations in this function.
650	`if gamma < 1e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
651	`gamma = 1.0; radial_down_N = -n_road_N;`	Assigns `gamma` from `1.0; radial_down_N = -n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
652	`else`	Starts the fallback branch for the current conditional block.
653	`radial_down_N = -normal_in_radial_plane/gamma;`	Assigns `radial_down_N` from `-normal_in_radial_plane/gamma`. This value is used by later computations in this function.
654	`end`	Ends the current function, conditional block, loop, or protected block.
655	`signed_center_distance = dot(p_wc_N - p_road_N, n_road_N);`	Assigns `signed_center_distance` from `dot(p_wc_N - p_road_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
656	`radial_depth_normal = par.rw*gamma;`	Assigns `radial_depth_normal` from `par.rw*gamma`. This value is used by later computations in this function.
657	`compression_normal = radial_depth_normal - signed_center_distance;`	Assigns `compression_normal` from `radial_depth_normal - signed_center_distance`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
658	`distance_center_to_road_along_radial = signed_center_distance/gamma;`	Assigns `distance_center_to_road_along_radial` from `signed_center_distance/gamma`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
659	`compression_radial = par.rw - distance_center_to_road_along_radial;`	Assigns `compression_radial` from `par.rw - distance_center_to_road_along_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
660	`r_wc_to_contact_N = distance_center_to_road_along_radial*radial_down_N;`	Assigns `r_wc_to_contact_N` from `distance_center_to_road_along_radial*radial_down_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
661	`p_contact_N = p_wc_N + r_wc_to_contact_N;`	Assigns `p_contact_N` from `p_wc_N + r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
662	`geom.s=s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N;`	Assigns `geom.s` from `s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
663	`geom.t_long_N=t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N;`	Assigns `geom.t_long_N` from `t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N`. This value is used by later computations in this function.
664	`geom.r_wc_to_contact_N=r_wc_to_contact_N; geom.p_contact_N=p_contact_N;`	Assigns `geom.r_wc_to_contact_N` from `r_wc_to_contact_N; geom.p_contact_N=p_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
665	`geom.compression=compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=compression_radial;`	Assigns `geom.compression` from `compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=compression_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
666	`geom.contact_gap_normal=-compression_normal; geom.contact_gap_radial=-compression_radial;`	Assigns `geom.contact_gap_normal` from `-compression_normal; geom.contact_gap_radial=-compression_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
667	`end`	Ends the current function, conditional block, loop, or protected block.
668		Blank line used to separate logical blocks and improve readability.
669	`function [s,n,p_road,e_long,e_lat,e_z] = road_project_point_local(road,p)`	Defines the local MATLAB function `road_project_point_local`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
670	`s = p(1);`	Assigns `s` from `p(1)`. This value is used by later computations in this function.
671	`[C,~,~,e_y,~,~] = centerline_quantities_local(road,s);`	Executes this MATLAB statement in `road_project_point_local`. It performs the operation shown by the sanitized code line.
672	`n = dot(p-C,e_y);`	Assigns `n` from `dot(p-C,e_y)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
673	`[p_road,e_long,e_lat,e_z] = road_surface_at_local(road,s,n);`	Executes this MATLAB statement in `road_project_point_local`. It performs the operation shown by the sanitized code line.
674	`end`	Ends the current function, conditional block, loop, or protected block.
675		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
676	`function [p_road,e_long,e_lat,e_z] = road_surface_at_local(road,s,n)`	Defines the local MATLAB function `road_surface_at_local`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
677	`[C,Cs,ex,ey,dey,ezc] = centerline_quantities_local(road,s);`	Executes this MATLAB statement in `road_surface_at_local`. It performs the operation shown by the sanitized code line.
678	`p_road = C + n*ey; ps = Cs + n*dey; pn = ey;`	Assigns `p_road` from `C + n*ey; ps = Cs + n*dey; pn = ey`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
679	`e_long = normalize_num_local(ps,ex); e_lat = normalize_num_local(pn,ey); e_z = normalize_num_local(cross(e_long,e_lat),ezc);`	Assigns `e_long` from `normalize_num_local(ps,ex); e_lat = normalize_num_local(pn,ey); e_z = normalize_num_local(cross(e_long,e_lat),ezc)`. This value is used by later computations in this function.
680	`if e_z(3)<0, e_z=-e_z; e_lat=-e_lat; end`	Starts a conditional branch. The following block executes only when this logical test is true.
681	`end`	Ends the current function, conditional block, loop, or protected block.
682		Blank line used to separate logical blocks and improve readability.
683	`function [C,Cs,ex,ey,dey,ezc] = centerline_quantities_local(road,s)`	Defines the local MATLAB function `centerline_quantities_local`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
684	`if isfield(road,'type') && strcmpi(road.type,'flat')`	Starts a conditional branch. The following block executes only when this logical test is true.
685	`C = [s;0;0];`	Assigns `C` from `[s;0;0]`. This value is used by later computations in this function.
686	`Cs = [1;0;0];`	Assigns `Cs` from `[1;0;0]`. This value is used by later computations in this function.
687	`ex = [1;0;0];`	Assigns `ex` from `[1;0;0]`. This value is used by later computations in this function.
688	`ey = [0;1;0];`	Assigns `ey` from `[0;1;0]`. This value is used by later computations in this function.
689	`dey = [0;0;0];`	Assigns `dey` from `[0;0;0]`. This value is used by later computations in this function.
690	`ezc = [0;0;1];`	Assigns `ezc` from `[0;0;1]`. This value is used by later computations in this function.
691	`return;`	Returns immediately from the current function.
692	`end`	Ends the current function, conditional block, loop, or protected block.
693	`ke=2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength;`	Assigns `ke` from `2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
694	`zc=road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s);`	Assigns `zc` from `road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
695	`bank=road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s);`	Assigns `bank` from `road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
696	`C=[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L];`	Assigns `C` from `[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L]`. This value is used by later computations in this function.
697	`dez=[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num_local(cross(ex,ey),[0;0;1]);`	Assigns `dez` from `[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num_local(cross(ex,ey),[...`. This value is used by later computations in this function.
698	`end`	Ends the current function, conditional block, loop, or protected block.
699		Blank line used to separate logical blocks and improve readability.
700	`function y = normalize_num_local(v,fallback)`	Defines the local MATLAB function `normalize_num_local`. Helper function used by this file. Its line-by-line rows below describe the exact operations.

Line	Sanitized code line	Explanation
701	`n=norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end`	Assigns `n` from `norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end`. This value is used by later computations in this function.
702	`end`	Ends the current function, conditional block, loop, or protected block.
703		Blank line used to separate logical blocks and improve readability.
704	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
705	`% Local helper from original package file: vehicle14_core_numeric.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
706	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
707	`function [M,F,aux] = vehicle14_core_numeric(t, x, mf_fun, par, args, road, meta)`	Defines the local MATLAB function `vehicle14_core_numeric`. Main numerical evaluator: computes inputs, calls the generated dynamics function, and returns M and F.
708	`%VEHICLE14_CORE_NUMERIC Evaluate M(x)*xdot = F(x) using selected input/component model.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
709	`q = x(1:14);`	Assigns `q` from `x(1:14)`. This value is used by later computations in this function.
710	`u = x(15:28);`	Assigns `u` from `x(15:28)`. This value is used by later computations in this function.
711	`source = lower(getfield_default_runner(args, 'model_source', 'outer'));`	Assigns `source` from `lower(getfield_default_runner(args, 'model_source', 'outer'))`. This value is used by later computations in this function.
712		Blank line used to separate logical blocks and improve readability.
713	`if strcmp(source, 'outer')`	Starts a conditional branch. The following block executes only when this logical test is true.
714	`% New user-editable outer-model path. First ask only for steering so`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
715	`% the generated sensor geometry uses steering-consistent front K/C maps.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
716	`[IN0, ctrl] = vehicle14_outer_models_user('control', t, x, [], par, args, road, meta);`	Calls the user outer-model file in steering-only mode so steering-dependent sensor geometry is evaluated consistently.
717	`[~,~,Sdm] = mf_fun(q, u, par.P, IN0);`	Executes this MATLAB statement in `vehicle14_core_numeric`. It performs the operation shown by the sanitized code line.
718	`S = full(Sdm);`	Assigns `S` from `full(Sdm)`. This value is used by later computations in this function.
719		Blank line used to separate logical blocks and improve readability.
720	`% Now compute the complete IN vector from the user-editable outer model.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
721	`[IN, outer] = vehicle14_outer_models_user('full', t, x, S, par, args, road, meta);`	Calls the user outer-model file in full mode to compute tyre/contact wrenches, wheel torques, aero inputs, and the complete IN vector.
722	`[Mdm,Fdm,~] = mf_fun(q, u, par.P, IN);`	Executes this MATLAB statement in `vehicle14_core_numeric`. It performs the operation shown by the sanitized code line.
723	`M = full(Mdm);`	Assigns `M` from `full(Mdm)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
724	`F = full(Fdm);`	Assigns `F` from `full(Fdm)`. This value is used by later computations in this function.
725		Blank line used to separate logical blocks and improve readability.
726	`if nargout > 2`	Starts a conditional branch. The following block executes only when this logical test is true.
727	`aux.S = S;`	Assigns `aux.S` from `S`. This value is used by later computations in this function.
728	`aux.IN = IN;`	Assigns `aux.IN` from `IN`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
729	`aux.control = ctrl;`	Assigns `aux.control` from `ctrl`. This value is used by later computations in this function.
730	`aux.outer = outer;`	Assigns `aux.outer` from `outer`. This value is used by later computations in this function.
731	`aux.contact = outer.contact;`	Assigns `aux.contact` from `outer.contact`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
732	`aux.throttle = outer.throttle;`	Assigns `aux.throttle` from `outer.throttle`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
733	`aux.brake = outer.brake;`	Assigns `aux.brake` from `outer.brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
734	`aux.delta = outer.delta;`	Assigns `aux.delta` from `outer.delta`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
735	`aux.delta_dot = outer.delta_dot;`	Assigns `aux.delta_dot` from `outer.delta_dot`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
736	`aux.T_drive_total = outer.T_drive_total;`	Assigns `aux.T_drive_total` from `outer.T_drive_total`. This value is used by later computations in this function.
737	`aux.T_brake_total = outer.T_brake_total;`	Assigns `aux.T_brake_total` from `outer.T_brake_total`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
738	`aux.model_source = source;`	Assigns `aux.model_source` from `source`. This value is used by later computations in this function.
739	`end`	Ends the current function, conditional block, loop, or protected block.
740		Blank line used to separate logical blocks and improve readability.
741	`elseif strcmp(source, 'original')`	Starts an alternative conditional branch, tested only if previous branches were false.
742	`% Original built-in demo component path from the previous compact runner.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
743	`[throttle, brake, delta, delta_dot] = vehicle14_command_inputs(t, par, args);`	Executes this MATLAB statement in `vehicle14_core_numeric`. It performs the operation shown by the sanitized code line.
744	`IN0 = zeros(meta.nin,1);`	Assigns `IN0` from `zeros(meta.nin,1)`. This value is used by later computations in this function.
745	`IN0(meta.input_index.delta) = delta;`	Assigns `IN0(meta.input_index.delta)` from `delta`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
746	`IN0(meta.input_index.delta_dot) = delta_dot;`	Assigns `IN0(meta.input_index.delta_dot)` from `delta_dot`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
747	`[~,~,Sdm] = mf_fun(q, u, par.P, IN0);`	Executes this MATLAB statement in `vehicle14_core_numeric`. It performs the operation shown by the sanitized code line.
748	`S = full(Sdm);`	Assigns `S` from `full(Sdm)`. This value is used by later computations in this function.
749	`[IN, cdiag] = vehicle14_component_inputs_numeric(t, x, S, par, args, road, meta, throttle, brake, delta, delta_dot);`	Executes this MATLAB statement in `vehicle14_core_numeric`. It performs the operation shown by the sanitized code line.
750	`[Mdm,Fdm,~] = mf_fun(q, u, par.P, IN);`	Executes this MATLAB statement in `vehicle14_core_numeric`. It performs the operation shown by the sanitized code line.
751	`M = full(Mdm);`	Assigns `M` from `full(Mdm)`. This value is used by later computations in this function.
752	`F = full(Fdm);`	Assigns `F` from `full(Fdm)`. This value is used by later computations in this function.
753		Blank line used to separate logical blocks and improve readability.
754	`if nargout > 2`	Starts a conditional branch. The following block executes only when this logical test is true.
755	`aux.S = S;`	Assigns `aux.S` from `S`. This value is used by later computations in this function.
756	`aux.IN = IN;`	Assigns `aux.IN` from `IN`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
757	`aux.contact = cdiag;`	Assigns `aux.contact` from `cdiag`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
758	`aux.throttle = throttle;`	Assigns `aux.throttle` from `throttle`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
759	`aux.brake = brake;`	Assigns `aux.brake` from `brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
760	`aux.delta = delta;`	Assigns `aux.delta` from `delta`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
761	`aux.delta_dot = delta_dot;`	Assigns `aux.delta_dot` from `delta_dot`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
762	`aux.T_drive_total = throttle * par.T_drive_max;`	Assigns `aux.T_drive_total` from `throttle * par.T_drive_max`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
763	`aux.T_brake_total = brake * par.T_brake_max;`	Assigns `aux.T_brake_total` from `brake * par.T_brake_max`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
764	`aux.model_source = source;`	Assigns `aux.model_source` from `source`. This value is used by later computations in this function.
765	`end`	Ends the current function, conditional block, loop, or protected block.
766	`else`	Starts the fallback branch for the current conditional block.
767	`error('Unknown args.model_source: %s. Use outer or original.', source);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
768	`end`	Ends the current function, conditional block, loop, or protected block.
769	`end`	Ends the current function, conditional block, loop, or protected block.
770		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
771	`function val = getfield_default_runner(s, name, default_val)`	Defines the local MATLAB function `getfield_default_runner`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
772	`if isstruct(s) && isfield(s, name)`	Starts a conditional branch. The following block executes only when this logical test is true.
773	`val = s.(name);`	Assigns `val` from `s.(name)`. This value is used by later computations in this function.
774	`else`	Starts the fallback branch for the current conditional block.
775	`val = default_val;`	Assigns `val` from `default_val`. This value is used by later computations in this function.
776	`end`	Ends the current function, conditional block, loop, or protected block.
777	`end`	Ends the current function, conditional block, loop, or protected block.
778	`function [IN, diag] = vehicle14_component_inputs_numeric(t_abs, x, S, par, args, road, meta, throttle, brake, delta, delta_dot)`	Defines the local MATLAB function `vehicle14_component_inputs_numeric`. Built-in original component model for contact, tyre, torque, and aero inputs.
779	`%VEHICLE14_COMPONENT_INPUTS_NUMERIC Numeric 3D-ribbon tyre/contact layer outside generated-core.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
780	`qv = x(1:14);`	Assigns `qv` from `x(1:14)`. This value is used by later computations in this function.
781	`uv = x(15:28);`	Assigns `uv` from `x(15:28)`. This value is used by later computations in this function.
782	`idx = meta.sensor_index;`	Assigns `idx` from `meta.sensor_index`. This value is used by later computations in this function.
783	`in = meta.input_index;`	Assigns `in` from `meta.input_index`. This value is used by later computations in this function.
784	`IN = zeros(meta.nin,1);`	Assigns `IN` from `zeros(meta.nin,1)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
785	`IN(in.delta) = delta;`	Writes steering angle or steering rate into the external input vector before generated geometry/dynamics evaluation.
786	`IN(in.delta_dot) = delta_dot;`	Writes steering angle or steering rate into the external input vector before generated geometry/dynamics evaluation.
787		Blank line used to separate logical blocks and improve readability.
788	`T_drive = throttle * par.T_drive_max;`	Assigns `T_drive` from `throttle * par.T_drive_max`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
789	`T_brake = brake * par.T_brake_max;`	Assigns `T_brake` from `brake * par.T_brake_max`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
790	`kb = par.brake_bias_front;`	Assigns `kb` from `par.brake_bias_front`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
791	`omRL = uv(13); omRR = uv(14);`	Assigns `omRL` from `uv(13); omRR = uv(14)`. This value is used by later computations in this function.
792	`diff_bias = par.diff_gain * par.diff_width * tanh((omRL - omRR)/par.diff_width);`	Assigns `diff_bias` from `par.diff_gain * par.diff_width * tanh((omRL - omRR)/par.diff_width)`. This value is used by later computations in this function.
793	`wheel_torque.FL = -0.5 * kb * T_brake;`	Assigns `wheel_torque.FL` from `-0.5 * kb * T_brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
794	`wheel_torque.FR = -0.5 * kb * T_brake;`	Assigns `wheel_torque.FR` from `-0.5 * kb * T_brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
795	`wheel_torque.RL = +0.5 * T_drive - diff_bias - 0.5*(1-kb)*T_brake;`	Assigns `wheel_torque.RL` from `+0.5 * T_drive - diff_bias - 0.5*(1-kb)*T_brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
796	`wheel_torque.RR = +0.5 * T_drive + diff_bias - 0.5*(1-kb)*T_brake;`	Assigns `wheel_torque.RR` from `+0.5 * T_drive + diff_bias - 0.5*(1-kb)*T_brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
797		Blank line used to separate logical blocks and improve readability.
798	`names = {'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This value is used by later computations in this function.
799	`diag.names = names;`	Assigns `diag.names` from `names`. This value is used by later computations in this function.
800	`for i = 1:4`	Starts a loop; the following block is repeated for the specified index range.
801	`nm = names{i};`	Assigns `nm` from `names{i}`. This value is used by later computations in this function.
802	`p_wc_N = [S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];`	Assigns `p_wc_N` from `[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))]`. This value is used by later computations in this function.
803	`v_wc_N = [S(idx.([nm '_Wc_vx'])); S(idx.([nm '_Wc_vy'])); S(idx.([nm '_Wc_vz']))];`	Assigns `v_wc_N` from `[S(idx.([nm '_Wc_vx'])); S(idx.([nm '_Wc_vy'])); S(idx.([nm '_Wc_vz']))]`. This value is used by later computations in this function.
804	`head_N = [S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];`	Assigns `head_N` from `[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))]`. This value is used by later computations in this function.
805	`spin_N = [S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];`	Assigns `spin_N` from `[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))]`. This value is used by later computations in this function.
806	`carrier_omega_N = [S(idx.([nm '_carrier_wx'])); S(idx.([nm '_carrier_wy'])); S(idx.([nm '_carrier_wz']))];`	Assigns `carrier_omega_N` from `[S(idx.([nm '_carrier_wx'])); S(idx.([nm '_carrier_wy'])); S(idx.([nm '_carrier_wz']))]`. This value is used by later computations in this function.
807	`omega = uv(10+i);`	Assigns `omega` from `uv(10+i)`. This value is used by later computations in this function.
808	`ck = contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega);`	Assigns `ck` from `contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
809	`[F_B, M_B, d] = ribbon_road_tyre_contact_to_body_wrench(qv, par, road, p_wc_N, v_wc_N, head_N, spin_N, omega, carrier_omega_N, ck);`	Executes this MATLAB statement in `vehicle14_component_inputs_numeric`. It performs the operation shown by the sanitized code line.
810	`IN(in.(['FBx' nm])) = F_B(1); IN(in.(['FBy' nm])) = F_B(2); IN(in.(['FBz' nm])) = F_B(3);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.
811	`IN(in.(['MBx' nm])) = M_B(1); IN(in.(['MBy' nm])) = M_B(2); IN(in.(['MBz' nm])) = M_B(3);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.
812	`IN(in.(['T' nm])) = wheel_torque.(nm);`	Writes one value into the external input vector used by the generated dynamics function.

Line	Sanitized code line	Explanation
813	`diag.Fx(i,1) = d.Fx; diag.Fy(i,1) = d.Fy; diag.Fz(i,1) = d.Fz;`	Assigns `diag.Fx(i,1)` from `d.Fx; diag.Fy(i,1) = d.Fy; diag.Fz(i,1) = d.Fz`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.

Line	Sanitized code line	Explanation
814	`diag.kappa(i,1) = d.kappa; diag.alpha(i,1) = d.alpha; diag.compression(i,1) = d.compression;`	Assigns `diag.kappa(i,1)` from `d.kappa; diag.alpha(i,1) = d.alpha; diag.compression(i,1) = d.compression`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
815	`diag.road_z_at_wheel(i,1) = d.road_patch_z; diag.contact_x(i,1)=d.contact_x; diag.contact_y(i,1)=d.contact_y; diag.contact_z(i,1)=d.contact_z;`	Assigns `diag.road_z_at_wheel(i,1)` from `d.road_patch_z; diag.contact_x(i,1)=d.contact_x; diag.contact_y(i,1)=d.contact_y; diag.contact_z(i,1)=d.contact_z`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
816	`end`	Ends the current function, conditional block, loop, or protected block.
817		Blank line used to separate logical blocks and improve readability.
818	`% Aero: zero during drop/settle; smooth placeholder aero during maneuver.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
819	`if t_abs < args.settle_time || args.disable_aero`	Starts a conditional branch. The following block executes only when this logical test is true.
820	`IN(in.FdragF) = 0; IN(in.FdragR) = 0; IN(in.FdownF) = 0; IN(in.FdownR) = 0;`	Writes aero drag or downforce input into the external input vector.
821	`else`	Starts the fallback branch for the current conditional block.
822	`vx = uv(1); qdyn = 0.5*par.rho*vx*vx;`	Assigns `vx` from `uv(1); qdyn = 0.5*par.rho*vx*vx`. This value is used by later computations in this function.
823	`[p_af_N,~] = body_point_state_num(qv, uv, [par.aero_front_x;0;par.aero_z]);`	Executes this MATLAB statement in `vehicle14_component_inputs_numeric`. It performs the operation shown by the sanitized code line.
824	`[p_ar_N,~] = body_point_state_num(qv, uv, [par.aero_rear_x;0;par.aero_z]);`	Executes this MATLAB statement in `vehicle14_component_inputs_numeric`. It performs the operation shown by the sanitized code line.
825	`[hF,~,~,~] = road_height_normal_at_point(road, p_af_N);`	Executes this MATLAB statement in `vehicle14_component_inputs_numeric`. It performs the operation shown by the sanitized code line.
826	`[hR,~,~,~] = road_height_normal_at_point(road, p_ar_N);`	Executes this MATLAB statement in `vehicle14_component_inputs_numeric`. It performs the operation shown by the sanitized code line.
827	`multF_raw = 1 + par.aero_h_sens_front*(par.aero_h_ref - hF);`	Assigns `multF_raw` from `1 + par.aero_h_sens_front*(par.aero_h_ref - hF)`. This value is used by later computations in this function.
828	`multR_raw = 1 + par.aero_h_sens_rear*(par.aero_h_ref - hR);`	Assigns `multR_raw` from `1 + par.aero_h_sens_rear*(par.aero_h_ref - hR)`. This value is used by later computations in this function.
829	`multF = smooth_max(0.20, multF_raw, 1e-3);`	Assigns `multF` from `smooth_max(0.20, multF_raw, 1e-3)`. This value is used by later computations in this function.
830	`multR = smooth_max(0.20, multR_raw, 1e-3);`	Assigns `multR` from `smooth_max(0.20, multR_raw, 1e-3)`. This value is used by later computations in this function.
831	`D = 0.5*par.rho*par.CdA*vx*smooth_abs(vx,0.25);`	Assigns `D` from `0.5*par.rho*par.CdA*vx*smooth_abs(vx,0.25)`. This value is used by later computations in this function.
832	`IN(in.FdragF) = -0.5*D; IN(in.FdragR) = -0.5*D;`	Writes aero drag or downforce input into the external input vector.
833	`IN(in.FdownF) = -qdyn*par.ClA_front*multF;`	Writes aero drag or downforce input into the external input vector.
834	`IN(in.FdownR) = -qdyn*par.ClA_rear*multR;`	Writes aero drag or downforce input into the external input vector.
835	`end`	Ends the current function, conditional block, loop, or protected block.
836	`diag.throttle = throttle; diag.brake = brake; diag.delta = delta; diag.delta_dot = delta_dot;`	Assigns `diag.throttle` from `throttle; diag.brake = brake; diag.delta = delta; diag.delta_dot = delta_dot`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
837	`end`	Ends the current function, conditional block, loop, or protected block.
838		Blank line used to separate logical blocks and improve readability.
839	`function ck = contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega)`	Defines the local MATLAB function `contact_kinematics_from_pose`. Computes contact-point velocity and compression rate from wheel pose and motion.
840	`head_N = normalize_num(head_N, [1;0;0]); spin_N = normalize_num(spin_N, [0;1;0]);`	Assigns `head_N` from `normalize_num(head_N, [1;0;0]); spin_N = normalize_num(spin_N, [0;1;0])`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
841	`geom0 = oriented_tyre_contact_geometry(par, road, p_wc_N, head_N, spin_N);`	Assigns `geom0` from `oriented_tyre_contact_geometry(par, road, p_wc_N, head_N, spin_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
842	`head_dot_N = cross(carrier_omega_N, head_N);`	Assigns `head_dot_N` from `cross(carrier_omega_N, head_N)`. This value is used by later computations in this function.
843	`spin_dot_N = cross(carrier_omega_N, spin_N);`	Assigns `spin_dot_N` from `cross(carrier_omega_N, spin_N)`. This value is used by later computations in this function.
844	`h = 1e-5;`	Assigns `h` from `1e-5`. This value is used by later computations in this function.
845	`geomp = oriented_tyre_contact_geometry(par, road, p_wc_N + h*v_wc_N, head_N + h*head_dot_N, spin_N + h*spin_dot_N);`	Assigns `geomp` from `oriented_tyre_contact_geometry(par, road, p_wc_N + h*v_wc_N, head_N + h*head_dot_N, spin_N + h*spin_dot_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
846	`geomm = oriented_tyre_contact_geometry(par, road, p_wc_N - h*v_wc_N, head_N - h*head_dot_N, spin_N - h*spin_dot_N);`	Assigns `geomm` from `oriented_tyre_contact_geometry(par, road, p_wc_N - h*v_wc_N, head_N - h*head_dot_N, spin_N - h*spin_dot_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
847	`ck.geom = geom0;`	Assigns `ck.geom` from `geom0`. This value is used by later computations in this function.
848	`ck.contact_point_velocity_geom_N = (geomp.p_contact_N - geomm.p_contact_N)/(2*h);`	Assigns `ck.contact_point_velocity_geom_N` from `(geomp.p_contact_N - geomm.p_contact_N)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
849	`ck.contact_unloaded_velocity_geom_N = (geomp.p_contact_unloaded_N - geomm.p_contact_unloaded_N)/(2*h);`	Assigns `ck.contact_unloaded_velocity_geom_N` from `(geomp.p_contact_unloaded_N - geomm.p_contact_unloaded_N)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
850	`ck.radial_down_rate_N = (geomp.radial_down_N - geomm.radial_down_N)/(2*h);`	Assigns `ck.radial_down_rate_N` from `(geomp.radial_down_N - geomm.radial_down_N)/(2*h)`. This value is used by later computations in this function.
851	`ck.road_normal_rate_N = (geomp.n_road_N - geomm.n_road_N)/(2*h);`	Assigns `ck.road_normal_rate_N` from `(geomp.n_road_N - geomm.n_road_N)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
852	`ck.t_long_rate_N = (geomp.t_long_N - geomm.t_long_N)/(2*h);`	Assigns `ck.t_long_rate_N` from `(geomp.t_long_N - geomm.t_long_N)/(2*h)`. This value is used by later computations in this function.
853	`ck.t_lat_rate_N = (geomp.t_lat_N - geomm.t_lat_N)/(2*h);`	Assigns `ck.t_lat_rate_N` from `(geomp.t_lat_N - geomm.t_lat_N)/(2*h)`. This value is used by later computations in this function.
854	`ck.compression_rate_normal = (geomp.compression_normal - geomm.compression_normal)/(2*h);`	Assigns `ck.compression_rate_normal` from `(geomp.compression_normal - geomm.compression_normal)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
855	`ck.compression_rate_radial = (geomp.compression_radial - geomm.compression_radial)/(2*h);`	Assigns `ck.compression_rate_radial` from `(geomp.compression_radial - geomm.compression_radial)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
856	`ck.contact_gap_rate_normal = (geomp.contact_gap_normal - geomm.contact_gap_normal)/(2*h);`	Assigns `ck.contact_gap_rate_normal` from `(geomp.contact_gap_normal - geomm.contact_gap_normal)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
857	`ck.contact_gap_rate_radial = (geomp.contact_gap_radial - geomm.contact_gap_radial)/(2*h);`	Assigns `ck.contact_gap_rate_radial` from `(geomp.contact_gap_radial - geomm.contact_gap_radial)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
858	`ck.road_plane_residual_rate = (geomp.road_plane_residual - geomm.road_plane_residual)/(2*h);`	Assigns `ck.road_plane_residual_rate` from `(geomp.road_plane_residual - geomm.road_plane_residual)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
859	`r_wc_to_contact_N = geom0.r_wc_to_contact_N;`	Assigns `r_wc_to_contact_N` from `geom0.r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
860	`spin_axis_N = geom0.spin_axis_N;`	Assigns `spin_axis_N` from `geom0.spin_axis_N`. This value is used by later computations in this function.
861	`ck.contact_patch_velocity_carrier_N = v_wc_N + cross(carrier_omega_N, r_wc_to_contact_N);`	Assigns `ck.contact_patch_velocity_carrier_N` from `v_wc_N + cross(carrier_omega_N, r_wc_to_contact_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
862	`ck.contact_patch_velocity_spin_N = cross(omega*spin_axis_N, r_wc_to_contact_N);`	Assigns `ck.contact_patch_velocity_spin_N` from `cross(omega*spin_axis_N, r_wc_to_contact_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
863	`ck.contact_patch_velocity_material_N = ck.contact_patch_velocity_carrier_N + ck.contact_patch_velocity_spin_N;`	Assigns `ck.contact_patch_velocity_material_N` from `ck.contact_patch_velocity_carrier_N + ck.contact_patch_velocity_spin_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
864	`ck.carrier_omega_N = carrier_omega_N;`	Assigns `ck.carrier_omega_N` from `carrier_omega_N`. This value is used by later computations in this function.
865	`ck.road_normal_velocity_residual = dot(ck.contact_point_velocity_geom_N, geom0.n_road_N);`	Assigns `ck.road_normal_velocity_residual` from `dot(ck.contact_point_velocity_geom_N, geom0.n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
866	`end`	Ends the current function, conditional block, loop, or protected block.
867		Blank line used to separate logical blocks and improve readability.
868	`function geom = oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`	Defines the local MATLAB function `oriented_tyre_contact_geometry`. Finds the tyre contact point and compression using wheel and road geometry.

Line	Sanitized code line	Explanation
869	`[s,n,p_road_N,e_long_N,e_lat_N,n_road_N] = road_project_point(road, p_wc_N);`	Executes this MATLAB statement in `oriented_tyre_contact_geometry`. It performs the operation shown by the sanitized code line.
870	`wheel_heading_N = normalize_num(wheel_heading_N, e_long_N);`	Assigns `wheel_heading_N` from `normalize_num(wheel_heading_N, e_long_N)`. This value is used by later computations in this function.
871	`spin_axis_N = normalize_num(wheel_spin_axis_N, e_lat_N);`	Assigns `spin_axis_N` from `normalize_num(wheel_spin_axis_N, e_lat_N)`. This value is used by later computations in this function.
872	`t_long_raw = cross(spin_axis_N, n_road_N);`	Assigns `t_long_raw` from `cross(spin_axis_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
873	`if norm(t_long_raw) < 1e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
874	`t_long_raw = wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N;`	Assigns `t_long_raw` from `wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
875	`end`	Ends the current function, conditional block, loop, or protected block.
876	`t_long_N = normalize_num(t_long_raw, e_long_N);`	Assigns `t_long_N` from `normalize_num(t_long_raw, e_long_N)`. This value is used by later computations in this function.
877	`if dot(t_long_N, wheel_heading_N) < 0, t_long_N = -t_long_N; end`	Starts a conditional branch. The following block executes only when this logical test is true.
878	`t_lat_N = normalize_num(cross(n_road_N, t_long_N), e_lat_N);`	Assigns `t_lat_N` from `normalize_num(cross(n_road_N, t_long_N), e_lat_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
879	`normal_in_radial_plane = n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N;`	Assigns `normal_in_radial_plane` from `n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
880	`gamma = norm(normal_in_radial_plane);`	Assigns `gamma` from `norm(normal_in_radial_plane)`. This value is used by later computations in this function.
881	`if gamma < 1e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
882	`gamma = 1.0; radial_down_N = -n_road_N; degenerate = true;`	Assigns `gamma` from `1.0; radial_down_N = -n_road_N; degenerate = true`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
883	`else`	Starts the fallback branch for the current conditional block.
884	`radial_down_N = -normal_in_radial_plane/gamma; degenerate = false;`	Assigns `radial_down_N` from `-normal_in_radial_plane/gamma; degenerate = false`. This value is used by later computations in this function.
885	`end`	Ends the current function, conditional block, loop, or protected block.
886	`contact_to_wc_unit_N = -radial_down_N;`	Assigns `contact_to_wc_unit_N` from `-radial_down_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
887	`r_wc_to_unloaded_N = par.rw*radial_down_N;`	Assigns `r_wc_to_unloaded_N` from `par.rw*radial_down_N`. This value is used by later computations in this function.
888	`p_contact_unloaded_N = p_wc_N + r_wc_to_unloaded_N;`	Assigns `p_contact_unloaded_N` from `p_wc_N + r_wc_to_unloaded_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
889	`signed_center_distance = dot(p_wc_N - p_road_N, n_road_N);`	Assigns `signed_center_distance` from `dot(p_wc_N - p_road_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
890	`radial_depth_normal = par.rw*gamma;`	Assigns `radial_depth_normal` from `par.rw*gamma`. This value is used by later computations in this function.
891	`compression_normal = radial_depth_normal - signed_center_distance;`	Assigns `compression_normal` from `radial_depth_normal - signed_center_distance`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
892	`distance_center_to_road_along_radial = signed_center_distance/gamma;`	Assigns `distance_center_to_road_along_radial` from `signed_center_distance/gamma`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
893	`compression_radial = par.rw - distance_center_to_road_along_radial;`	Assigns `compression_radial` from `par.rw - distance_center_to_road_along_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
894	`r_wc_to_contact_N = distance_center_to_road_along_radial*radial_down_N;`	Assigns `r_wc_to_contact_N` from `distance_center_to_road_along_radial*radial_down_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
895	`p_contact_N = p_wc_N + r_wc_to_contact_N;`	Assigns `p_contact_N` from `p_wc_N + r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
896	`road_plane_residual = dot(p_contact_N - p_road_N, n_road_N);`	Assigns `road_plane_residual` from `dot(p_contact_N - p_road_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
897	`geom.s=s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N;`	Assigns `geom.s` from `s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
898	`geom.t_long_N=t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N; geom.contact_to_wc_unit_N=contact_to_wc_unit_N;`	Assigns `geom.t_long_N` from `t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N; geom.contact_to_wc_unit_N=contact_to_wc_unit_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
899	`geom.r_wc_to_contact_N=r_wc_to_contact_N; geom.p_contact_N=p_contact_N; geom.p_contact_road_N=p_contact_N; geom.r_wc_to_unloaded_N=r_wc_to_unloaded_N; geom.p_contact_unloaded_N=p_contact_unloaded_N;`	Assigns `geom.r_wc_to_contact_N` from `r_wc_to_contact_N; geom.p_contact_N=p_contact_N; geom.p_contact_road_N=p_contact_N; geom.r_wc_to_unloaded_N=r_wc_to_unloaded_N; geom.p_contact_unlo...`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
900	`geom.signed_center_distance=signed_center_distance; geom.radial_projection_norm=gamma; geom.radial_depth=radial_depth_normal; geom.radial_depth_normal=radial_depth_normal;`	Assigns `geom.signed_center_distance` from `signed_center_distance; geom.radial_projection_norm=gamma; geom.radial_depth=radial_depth_normal; geom.radial_depth_normal=radial_depth_normal`. This value is used by later computations in this function.
901	`geom.distance_center_to_road_along_radial=distance_center_to_road_along_radial; geom.compression=compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=compression_radial;`	Assigns `geom.distance_center_to_road_along_radial` from `distance_center_to_road_along_radial; geom.compression=compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=comp...`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
902	`geom.contact_gap_normal=-compression_normal; geom.contact_gap_radial=-compression_radial; geom.road_plane_residual=road_plane_residual; geom.degenerate_contact=degenerate;`	Assigns `geom.contact_gap_normal` from `-compression_normal; geom.contact_gap_radial=-compression_radial; geom.road_plane_residual=road_plane_residual; geom.degenerate_contact=degenerate`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
903	`end`	Ends the current function, conditional block, loop, or protected block.
904		Blank line used to separate logical blocks and improve readability.
905	`function [F_B, M_B, diag] = ribbon_road_tyre_contact_to_body_wrench(qv, par, road, p_wc_N, v_wc_N, wheel_heading_N, wheel_spin_axis_N, omega, carrier_omega_N, ck)`	Defines the local MATLAB function `ribbon_road_tyre_contact_to_body_wrench`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
906	`R_NB = body_rotation_from_q(qv); R_BN = R_NB.';`	Assigns `R_NB` from `body_rotation_from_q(qv); R_BN = R_NB.'`. This value is used by later computations in this function.
907	`geom = oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N);`	Assigns `geom` from `oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
908	`n_road_N = geom.n_road_N; t_long_N = geom.t_long_N; t_lat_N = geom.t_lat_N; spin_axis_N = geom.spin_axis_N; r_wc_to_contact_N = geom.r_wc_to_contact_N;`	Assigns `n_road_N` from `geom.n_road_N; t_long_N = geom.t_long_N; t_lat_N = geom.t_lat_N; spin_axis_N = geom.spin_axis_N; r_wc_to_contact_N = geom.r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
909	`compression = geom.compression;`	Assigns `compression` from `geom.compression`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
910	`compression_rate = ck.compression_rate_normal;`	Assigns `compression_rate` from `ck.compression_rate_normal`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
911	`compression_rate_radial = ck.compression_rate_radial;`	Assigns `compression_rate_radial` from `ck.compression_rate_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
912	`v_contact_geom_N = ck.contact_point_velocity_geom_N;`	Assigns `v_contact_geom_N` from `ck.contact_point_velocity_geom_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
913	`v_contact_carrier_N = ck.contact_patch_velocity_carrier_N;`	Assigns `v_contact_carrier_N` from `ck.contact_patch_velocity_carrier_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
914	`v_spin_contact_N = ck.contact_patch_velocity_spin_N;`	Assigns `v_spin_contact_N` from `ck.contact_patch_velocity_spin_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
915	`v_contact_material_N = ck.contact_patch_velocity_material_N;`	Assigns `v_contact_material_N` from `ck.contact_patch_velocity_material_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
916	`Fz = smoothplus(par.kt*compression + par.ct*compression_rate, par.normal_smooth_eps);`	Assigns `Fz` from `smoothplus(par.kt*compression + par.ct*compression_rate, par.normal_smooth_eps)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
917	`Vx = dot(v_contact_carrier_N, t_long_N); Vy = dot(v_contact_carrier_N, t_lat_N);`	Assigns `Vx` from `dot(v_contact_carrier_N, t_long_N); Vy = dot(v_contact_carrier_N, t_lat_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
918	`Vx_material = dot(v_contact_material_N, t_long_N); Vy_material = dot(v_contact_material_N, t_lat_N);`	Assigns `Vx_material` from `dot(v_contact_material_N, t_long_N); Vy_material = dot(v_contact_material_N, t_lat_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
919	`rolling_speed = -dot(v_spin_contact_N, t_long_N);`	Assigns `rolling_speed` from `-dot(v_spin_contact_N, t_long_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
920	`Vden = sqrt(Vx*Vx + par.v_eps*par.v_eps);`	Assigns `Vden` from `sqrt(Vx*Vx + par.v_eps*par.v_eps)`. This value is used by later computations in this function.
921	`kappa = (rolling_speed - Vx)/Vden;`	Assigns `kappa` from `(rolling_speed - Vx)/Vden`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
922	`alpha = atan2(Vy, Vden);`	Assigns `alpha` from `atan2(Vy, Vden)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
923	`Fx0 = par.Ck*kappa; Fy0 = -par.Ca*alpha;`	Assigns `Fx0` from `par.Ck*kappa; Fy0 = -par.Ca*alpha`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
924	`limit = par.mu*Fz + 1e-9;`	Assigns `limit` from `par.mu*Fz + 1e-9`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.

Line	Sanitized code line	Explanation
925	`norm_tan = sqrt(Fx0*Fx0 + Fy0*Fy0 + 1e-12);`	Assigns `norm_tan` from `sqrt(Fx0*Fx0 + Fy0*Fy0 + 1e-12)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
926	`scale = (1 + (norm_tan/limit)^4)^(-0.25);`	Assigns `scale` from `(1 + (norm_tan/limit)^4)^(-0.25)`. This value is used by later computations in this function.
927	`Fx = Fx0*scale; Fy = Fy0*scale;`	Assigns `Fx` from `Fx0*scale; Fy = Fy0*scale`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
928	`F_contact_N = Fx*t_long_N + Fy*t_lat_N + Fz*n_road_N;`	Assigns `F_contact_N` from `Fx*t_long_N + Fy*t_lat_N + Fz*n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
929	`F_B = R_BN*F_contact_N;`	Assigns `F_B` from `R_BN*F_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
930	`r_contact_B = R_BN*r_wc_to_contact_N;`	Assigns `r_contact_B` from `R_BN*r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
931	`M_B = cross(r_contact_B, F_B);`	Assigns `M_B` from `cross(r_contact_B, F_B)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
932	`diag.s=geom.s; diag.n=geom.n; diag.compression=compression; diag.compression_rate=compression_rate; diag.compression_rate_radial=compression_rate_radial;`	Assigns `diag.s` from `geom.s; diag.n=geom.n; diag.compression=compression; diag.compression_rate=compression_rate; diag.compression_rate_radial=compression_rate_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
933	`diag.Fz=Fz; diag.Fx=Fx; diag.Fy=Fy; diag.kappa=kappa; diag.alpha=alpha; diag.Vx_contact_carrier=Vx; diag.Vy_contact_carrier=Vy; diag.Vx_contact_material=Vx_material; diag.Vy_contact_material=Vy_material; diag.rolling_speed=rolling_speed;`	Assigns `diag.Fz` from `Fz; diag.Fx=Fx; diag.Fy=Fy; diag.kappa=kappa; diag.alpha=alpha; diag.Vx_contact_carrier=Vx; diag.Vy_contact_carrier=Vy; diag.Vx_contact_material=Vx...`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
934	`diag.contact_x=geom.p_contact_N(1); diag.contact_y=geom.p_contact_N(2); diag.contact_z=geom.p_contact_N(3); diag.road_patch_x=geom.p_road_N(1); diag.road_patch_y=geom.p_road_N(2); diag.road_patch_z=geom.p_road_N(3);`	Assigns `diag.contact_x` from `geom.p_contact_N(1); diag.contact_y=geom.p_contact_N(2); diag.contact_z=geom.p_contact_N(3); diag.road_patch_x=geom.p_road_N(1); diag.road_patch_y=...`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
935	`diag.road_normal_velocity_residual=ck.road_normal_velocity_residual; diag.road_plane_residual_rate=ck.road_plane_residual_rate; diag.contact_vx=v_contact_geom_N(1); diag.contact_vy=v_contact_geom_N(2); diag.contact_vz=v_contact_geom_N(3);`	Assigns `diag.road_normal_velocity_residual` from `ck.road_normal_velocity_residual; diag.road_plane_residual_rate=ck.road_plane_residual_rate; diag.contact_vx=v_contact_geom_N(1); diag.contact_vy=v...`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
936	`end`	Ends the current function, conditional block, loop, or protected block.
937		Blank line used to separate logical blocks and improve readability.
938	`function [h,e_z,s,n] = road_height_normal_at_point(road,p_N)`	Defines the local MATLAB function `road_height_normal_at_point`. Evaluates the selected road surface and normal near a point.
939	`[s,n,p_road,~,~,e_z] = road_project_point(road,p_N);`	Executes this MATLAB statement in `road_height_normal_at_point`. It performs the operation shown by the sanitized code line.
940	`h = dot(p_N-p_road,e_z);`	Assigns `h` from `dot(p_N-p_road,e_z)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
941	`end`	Ends the current function, conditional block, loop, or protected block.
942		Blank line used to separate logical blocks and improve readability.
943	`function [pN,vN] = body_point_state_num(qv, uv, r_B)`	Defines the local MATLAB function `body_point_state_num`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
944	`R = body_rotation_from_q(qv); pP = qv(1:3); vP = R*uv(1:3); omega = R*uv(4:6); rN = R*r_B;`	Assigns `R` from `body_rotation_from_q(qv); pP = qv(1:3); vP = R*uv(1:3); omega = R*uv(4:6); rN = R*r_B`. This value is used by later computations in this function.
945	`pN = pP + rN; vN = vP + cross(omega,rN);`	Assigns `pN` from `pP + rN; vN = vP + cross(omega,rN)`. This value is used by later computations in this function.
946	`end`	Ends the current function, conditional block, loop, or protected block.
947		Blank line used to separate logical blocks and improve readability.
948	`function R = body_rotation_from_q(qv)`	Defines the local MATLAB function `body_rotation_from_q`. Builds the yaw-pitch-roll body rotation matrix from q.
949	`R = rot_z(qv(4))*rot_y(qv(5))*rot_x(qv(6));`	Assigns `R` from `rot_z(qv(4))*rot_y(qv(5))*rot_x(qv(6))`. This value is used by later computations in this function.
950	`end`	Ends the current function, conditional block, loop, or protected block.
951	`function R = rot_x(a)`	Defines the local MATLAB function `rot_x`. Returns a 3-by-3 rotation matrix about the named axis.
952	`ca = cos(a);`	Assigns `ca` from `cos(a)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
953	`sa = sin(a);`	Assigns `sa` from `sin(a)`. This value is used by later computations in this function.
954	`R = [1 0 0; 0 ca -sa; 0 sa ca];`	Assigns `R` from `[1 0 0; 0 ca -sa; 0 sa ca]`. This value is used by later computations in this function.
955	`end`	Ends the current function, conditional block, loop, or protected block.
956	`function R = rot_y(a)`	Defines the local MATLAB function `rot_y`. Returns a 3-by-3 rotation matrix about the named axis.
957	`ca = cos(a);`	Assigns `ca` from `cos(a)`. This value is used by later computations in this function.
958	`sa = sin(a);`	Assigns `sa` from `sin(a)`. This value is used by later computations in this function.
959	`R = [ca 0 sa; 0 1 0; -sa 0 ca];`	Assigns `R` from `[ca 0 sa; 0 1 0; -sa 0 ca]`. This value is used by later computations in this function.
960	`end`	Ends the current function, conditional block, loop, or protected block.
961	`function R = rot_z(a)`	Defines the local MATLAB function `rot_z`. Returns a 3-by-3 rotation matrix about the named axis.
962	`ca = cos(a);`	Assigns `ca` from `cos(a)`. This value is used by later computations in this function.
963	`sa = sin(a);`	Assigns `sa` from `sin(a)`. This value is used by later computations in this function.
964	`R = [ca -sa 0; sa ca 0; 0 0 1];`	Assigns `R` from `[ca -sa 0; sa ca 0; 0 0 1]`. This value is used by later computations in this function.
965	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
966	`function y = smoothplus(x,epsv)`	Defines the local MATLAB function `smoothplus`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
967	`y = 0.5*(x + sqrt(x*x + epsv*epsv));`	Assigns `y` from `0.5*(x + sqrt(x*x + epsv*epsv))`. This value is used by later computations in this function.
968	`end`	Ends the current function, conditional block, loop, or protected block.
969	`function y = smooth_max(a,b,epsv)`	Defines the local MATLAB function `smooth_max`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
970	`d = b - a;`	Assigns `d` from `b - a`. This value is used by later computations in this function.
971	`y = a + smoothplus(d,epsv);`	Assigns `y` from `a + smoothplus(d,epsv)`. This value is used by later computations in this function.
972	`end`	Ends the current function, conditional block, loop, or protected block.
973	`function y = smooth_abs(x,epsv)`	Defines the local MATLAB function `smooth_abs`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
974	`y = sqrt(x*x + epsv*epsv);`	Assigns `y` from `sqrt(x*x + epsv*epsv)`. This value is used by later computations in this function.
975	`end`	Ends the current function, conditional block, loop, or protected block.
976	`function y = normalize_num(v,fallback)`	Defines the local MATLAB function `normalize_num`. Normalizes vectors while protecting against tiny norms.
977	`n = norm(v);`	Assigns `n` from `norm(v)`. This value is used by later computations in this function.
978	`if n < 1e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
979	`y = fallback/norm(fallback);`	Assigns `y` from `fallback/norm(fallback)`. This value is used by later computations in this function.
980	`else`	Starts the fallback branch for the current conditional block.

Line	Sanitized code line	Explanation
981	`y = v/n;`	Assigns `y` from `v/n`. This value is used by later computations in this function.
982	`end`	Ends the current function, conditional block, loop, or protected block.
983	`end`	Ends the current function, conditional block, loop, or protected block.
984		Blank line used to separate logical blocks and improve readability.
985	`function [s,n,p_road,e_long,e_lat,e_z] = road_project_point(road,p)`	Defines the local MATLAB function `road_project_point`. Projects a point to road coordinates and basis vectors.
986	`s = p(1);`	Assigns `s` from `p(1)`. This value is used by later computations in this function.
987	`for k=1:max(0,road.projection_iterations)`	Starts a loop; the following block is repeated for the specified index range.
988	`[f,~,~,~,~,~] = projection_residual_given_s(road,p,s);`	Executes this MATLAB statement in `road_project_point`. It performs the operation shown by the sanitized code line.
989	`if abs(f) < road.projection_tol, break; end`	Starts a conditional branch. The following block executes only when this logical test is true.
990	`ds=max(1e-4,1e-6*(1+abs(s)));`	Assigns `ds` from `max(1e-4,1e-6*(1+abs(s)))`. This value is used by later computations in this function.
991	`fp=projection_residual_given_s(road,p,s+ds); fm=projection_residual_given_s(road,p,s-ds);`	Assigns `fp` from `projection_residual_given_s(road,p,s+ds); fm=projection_residual_given_s(road,p,s-ds)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
992	`dfds=(fp-fm)/(2*ds);`	Assigns `dfds` from `(fp-fm)/(2*ds)`. This value is used by later computations in this function.
993	`if ~isfinite(dfds) || abs(dfds)<1e-12, break; end`	Starts a conditional branch. The following block executes only when this logical test is true.
994	`step=min(max(f/dfds,-5),5); s=s-step; if abs(step)<road.projection_tol, break; end`	Assigns `step` from `min(max(f/dfds,-5),5); s=s-step; if abs(step)<road.projection_tol, break; end`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
995	`end`	Ends the current function, conditional block, loop, or protected block.
996	`[~,n,p_road,e_long,e_lat,e_z,~] = projection_residual_given_s(road,p,s);`	Executes this MATLAB statement in `road_project_point`. It performs the operation shown by the sanitized code line.
997	`end`	Ends the current function, conditional block, loop, or protected block.
998		Blank line used to separate logical blocks and improve readability.
999	`function [f,n,p_road,e_long,e_lat,e_z,p_s] = projection_residual_given_s(road,p,s)`	Defines the local MATLAB function `projection_residual_given_s`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
1000	`[C,~,~,e_y,~,~] = centerline_quantities(road,s);`	Executes this MATLAB statement in `projection_residual_given_s`. It performs the operation shown by the sanitized code line.
1001	`n = dot(p-C,e_y);`	Assigns `n` from `dot(p-C,e_y)`. This value is used by later computations in this function.
1002	`[p_road,e_long,e_lat,e_z,p_s,~] = road_surface_at(road,s,n);`	Executes this MATLAB statement in `projection_residual_given_s`. It performs the operation shown by the sanitized code line.
1003	`r = p-p_road; f=dot(r,p_s);`	Assigns `r` from `p-p_road; f=dot(r,p_s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
1004	`end`	Ends the current function, conditional block, loop, or protected block.
1005		Blank line used to separate logical blocks and improve readability.
1006	`function [p_road,e_long,e_lat,e_z,p_s,p_n] = road_surface_at(road,s,n)`	Defines the local MATLAB function `road_surface_at`. Evaluates either flat or ribbon road height and normal.
1007	`[C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s);`	Executes this MATLAB statement in `road_surface_at`. It performs the operation shown by the sanitized code line.
1008	`p_road = C + n*ey; p_s = Cs + n*dey; p_n = ey;`	Assigns `p_road` from `C + n*ey; p_s = Cs + n*dey; p_n = ey`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
1009	`e_long=normalize_num(p_s,ex); e_lat=normalize_num(p_n,ey); e_z=normalize_num(cross(e_long,e_lat),ezc);`	Assigns `e_long` from `normalize_num(p_s,ex); e_lat=normalize_num(p_n,ey); e_z=normalize_num(cross(e_long,e_lat),ezc)`. This value is used by later computations in this function.
1010	`if e_z(3)<0, e_z=-e_z; e_lat=-e_lat; end`	Starts a conditional branch. The following block executes only when this logical test is true.
1011	`end`	Ends the current function, conditional block, loop, or protected block.
1012		Blank line used to separate logical blocks and improve readability.
1013	`function [C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s)`	Defines the local MATLAB function `centerline_quantities`. Computes the straight road centreline frame quantities.

Line	Sanitized code line	Explanation
1014	`if isfield(road,'type') && strcmpi(road.type,'flat')`	Starts a conditional branch. The following block executes only when this logical test is true.
1015	`C = [s;0;0];`	Assigns `C` from `[s;0;0]`. This value is used by later computations in this function.
1016	`Cs = [1;0;0];`	Assigns `Cs` from `[1;0;0]`. This value is used by later computations in this function.
1017	`ex = [1;0;0];`	Assigns `ex` from `[1;0;0]`. This value is used by later computations in this function.
1018	`ey = [0;1;0];`	Assigns `ey` from `[0;1;0]`. This value is used by later computations in this function.
1019	`dey = [0;0;0];`	Assigns `dey` from `[0;0;0]`. This value is used by later computations in this function.
1020	`ezc = [0;0;1];`	Assigns `ezc` from `[0;0;1]`. This value is used by later computations in this function.
1021	`return;`	Returns immediately from the current function.
1022	`end`	Ends the current function, conditional block, loop, or protected block.
1023	`ke=2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength;`	Assigns `ke` from `2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
1024	`zc=road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s);`	Assigns `zc` from `road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
1025	`bank=road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s);`	Assigns `bank` from `road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
1026	`C=[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L];`	Assigns `C` from `[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L]`. This value is used by later computations in this function.
1027	`dez=[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num(cross(ex,ey),[0;0;1]);`	Assigns `dez` from `[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num(cross(ex,ey),[0;0;1])`. This value is used by later computations in this function.
1028	`end`	Ends the current function, conditional block, loop, or protected block.
1029		Blank line used to separate logical blocks and improve readability.
1030	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1031	`% Local helper from original package file: vehicle14_rhs_full_numeric.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1032	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1033	`function dx = vehicle14_rhs_full_numeric(t, x, mf_fun, par, args, road, meta)`	Defines the local MATLAB function `vehicle14_rhs_full_numeric`. Returns xdot = M\F for the fallback explicit solver path.
1034	`%VEHICLE14_RHS_FULL_NUMERIC Explicit RHS xdot = M\F for MATLAB explicit/stiff ODE solvers.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1035	`[M,F] = vehicle14_core_numeric(t, x, mf_fun, par, args, road, meta);`	Executes this MATLAB statement in `vehicle14_rhs_full_numeric`. It performs the operation shown by the sanitized code line.
1036	`dx = M \ F;`	Assigns `dx` from `M \ F`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
1037	`end`	Ends the current function, conditional block, loop, or protected block.
1038		Blank line used to separate logical blocks and improve readability.
1039	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1040	`% Local helper from original package file: vehicle14_residual_full_numeric.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1041	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1042	`function res = vehicle14_residual_full_numeric(t, x, xdot, mf_fun, par, args, road, meta)`	Defines the local MATLAB function `vehicle14_residual_full_numeric`. Returns M*xdot - F for the preferred implicit solver path.
1043	`%VEHICLE14_RESIDUAL_FULL_NUMERIC Implicit residual M(x)*xdot - F(x) for ode15i.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1044	`[M,F] = vehicle14_core_numeric(t, x, mf_fun, par, args, road, meta);`	Executes this MATLAB statement in `vehicle14_residual_full_numeric`. It performs the operation shown by the sanitized code line.
1045	`res = M*xdot(:) - F;`	Assigns `res` from `M*xdot(:) - F`. This value is used by later computations in this function.
1046	`end`	Ends the current function, conditional block, loop, or protected block.
1047		Blank line used to separate logical blocks and improve readability.
1048	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1049	`% Local helper from original package file: vehicle14_save_plots_matlab.m`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1050	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1051	`function vehicle14_save_plots_matlab(t, x, mf_fun, par, args, road, meta, plot_dir)`	Defines the local MATLAB function `vehicle14_save_plots_matlab`. Creates diagnostic figures from a saved or current solution.
1052	`%VEHICLE14_SAVE_PLOTS_MATLAB Generate diagnostic PNG plots from the MATLAB/CasADi solution.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1053	`if nargin < 8 || isempty(plot_dir), plot_dir = args.plot_dir; end`	Starts a conditional branch. The following block executes only when this logical test is true.
1054	`if ~exist(plot_dir, 'dir'), mkdir(plot_dir); end`	Starts a conditional branch. The following block executes only when this logical test is true.
1055	`N = numel(t);`	Assigns `N` from `numel(t)`. This value is used by later computations in this function.
1056	`throttle=zeros(N,1); brake=zeros(N,1); delta=zeros(N,1); Fz=zeros(4,N); Fx=zeros(4,N); Fy=zeros(4,N); comp=zeros(4,N); roadz=zeros(4,N);`	Assigns `throttle` from `zeros(N,1); brake=zeros(N,1); delta=zeros(N,1); Fz=zeros(4,N); Fx=zeros(4,N); Fy=zeros(4,N); comp=zeros(4,N); roadz=zeros(4,N)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
1057	`for k=1:N`	Starts a loop; the following block is repeated for the specified index range.
1058	`[~,~,aux] = vehicle14_core_numeric(t(k), x(k,:).', mf_fun, par, args, road, meta);`	Executes this MATLAB statement in `vehicle14_save_plots_matlab`. It performs the operation shown by the sanitized code line.
1059	`throttle(k)=aux.throttle; brake(k)=aux.brake; delta(k)=aux.delta;`	Assigns `throttle(k)` from `aux.throttle; brake(k)=aux.brake; delta(k)=aux.delta`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.

Line	Sanitized code line	Explanation
1060	`Fz(:,k)=aux.contact.Fz; Fx(:,k)=aux.contact.Fx; Fy(:,k)=aux.contact.Fy; comp(:,k)=aux.contact.compression; roadz(:,k)=aux.contact.road_z_at_wheel;`	Executes this MATLAB statement in `vehicle14_save_plots_matlab`. It performs the operation shown by the sanitized code line.
1061	`end`	Ends the current function, conditional block, loop, or protected block.
1062	`names={'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This value is used by later computations in this function.
1063	`fig=figure('Visible','off'); plot3(x(:,1),x(:,2),x(:,3),'LineWidth',1.2); grid on; xlabel('X [m]'); ylabel('Y [m]'); zlabel('Z [m]'); title('Chassis CoM trajectory'); saveas(fig, fullfile(plot_dir,'trajectory_3d.png')); close(fig);`	Assigns `fig` from `figure('Visible','off'); plot3(x(:,1),x(:,2),x(:,3),'LineWidth',1.2); grid on; xlabel('X [m]'); ylabel('Y [m]'); zlabel('Z [m]'); title('Chassis Co...`. This value is used by later computations in this function.
1064	`fig=figure('Visible','off'); plot(t, rad2deg(x(:,6)),'DisplayName','roll [deg]'); hold on; plot(t, rad2deg(x(:,5)),'DisplayName','pitch [deg]'); plot(t, x(:,15),'DisplayName','vx [m/s]'); plot(t, x(:,20),'DisplayName','yaw rate r [rad/s]...`	Assigns `fig` from `figure('Visible','off'); plot(t, rad2deg(x(:,6)),'DisplayName','roll [deg]'); hold on; plot(t, rad2deg(x(:,5)),'DisplayName','pitch [deg]'); plot(t...`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
1065	`fig=figure('Visible','off'); plot(t, throttle,'DisplayName','throttle'); hold on; plot(t, brake,'DisplayName','brake'); plot(t, rad2deg(delta),'DisplayName','steering [deg]'); grid on; xlabel('time [s]'); title(sprintf('Driver commands (...`	Assigns `fig` from `figure('Visible','off'); plot(t, throttle,'DisplayName','throttle'); hold on; plot(t, brake,'DisplayName','brake'); plot(t, rad2deg(delta),'Display...`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
1066	`fig=figure('Visible','off'); hold on; for i=1:4, plot(t,Fz(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('Fz [N]'); title('Wheel normal loads'); legend show; saveas(fig, fullfile(plot_dir,'wheel_normal_loads.png'...`	Assigns `fig` from `figure('Visible','off'); hold on; for i=1:4, plot(t,Fz(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('Fz [N]'); title('Whe...`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
1067	`fig=figure('Visible','off'); hold on; for i=1:4, plot(t,comp(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('compression [m]'); title('Tyre compression'); legend show; saveas(fig, fullfile(plot_dir,'tyre_compressi...`	Assigns `fig` from `figure('Visible','off'); hold on; for i=1:4, plot(t,comp(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('compression [m]');...`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
1068	`fig=figure('Visible','off'); plot(x(:,1),x(:,3),'DisplayName','CoM Z'); hold on; plot(x(:,1),mean(roadz,1),'DisplayName','mean road Z at wheels'); grid on; xlabel('X [m]'); ylabel('Z [m]'); title('3D ribbon road excitation'); legend show...`	Assigns `fig` from `figure('Visible','off'); plot(x(:,1),x(:,3),'DisplayName','CoM Z'); hold on; plot(x(:,1),mean(roadz,1),'DisplayName','mean road Z at wheels'); grid...`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
1069	`fig=figure('Visible','off'); plot(t, x(:,7:10)); grid on; xlabel('time [s]'); ylabel('z_i [m]'); title('Suspension/wheel-centre heave coordinates'); legend(names{:}); saveas(fig, fullfile(plot_dir,'suspension_travels.png')); close(fig);`	Assigns `fig` from `figure('Visible','off'); plot(t, x(:,7:10)); grid on; xlabel('time [s]'); ylabel('z_i [m]'); title('Suspension/wheel-centre heave coordinates'); le...`. This value is used by later computations in this function.
1070	`fprintf('Saved MATLAB diagnostic plots to %s\n', plot_dir);`	Prints status information to the MATLAB command window so the user can see progress.
1071	`end`	Ends the current function, conditional block, loop, or protected block.
1072		Blank line used to separate logical blocks and improve readability.
1073	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1074	`% Local animation renderer, compact package version`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1075	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1076	`function render_vehicle14_matlab_animation(mat_file, out_mp4, out_gif)`	Defines the local MATLAB function `render_vehicle14_matlab_animation`. Renders the solved trajectory to MP4 and GIF.
1077	`%RENDER_VEHICLE14_MATLAB_ANIMATION Simple MATLAB MP4/GIF renderer for the saved solution.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1078	`if nargin < 1 || isempty(mat_file), mat_file = 'vehicle14_matlab_casadi_solution.mat'; end`	Starts a conditional branch. The following block executes only when this logical test is true.
1079	`if nargin < 2 || isempty(out_mp4), out_mp4 = 'vehicle14_matlab_casadi_animation.mp4'; end`	Starts a conditional branch. The following block executes only when this logical test is true.
1080	`if nargin < 3 || isempty(out_gif), out_gif = 'vehicle14_matlab_casadi_animation.gif'; end`	Starts a conditional branch. The following block executes only when this logical test is true.
1081	`load(mat_file, 't', 'x', 'par', 'args', 'road', 'meta');`	Loads a saved MAT-file so plots or animation can be regenerated without rerunning the solver.
1082	`[mf_fun, meta2] = vehicle14_build_casadi_function(); %#ok<ASGLU>`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1083	`fps = 20; playback_speed = 1.0;`	Assigns `fps` from `20; playback_speed = 1.0`. This value is used by later computations in this function.
1084	`nFrames = min(450, max(2, ceil((t(end)-t(1))*fps/playback_speed)+1));`	Assigns `nFrames` from `min(450, max(2, ceil((t(end)-t(1))*fps/playback_speed)+1))`. This value is used by later computations in this function.
1085	`tFrames = linspace(t(1), t(end), nFrames);`	Assigns `tFrames` from `linspace(t(1), t(end), nFrames)`. This value is used by later computations in this function.
1086	`xFrames = interp1(t, x, tFrames, 'linear');`	Assigns `xFrames` from `interp1(t, x, tFrames, 'linear')`. This value is used by later computations in this function.
1087	`fig = figure('Color','w','Position',[100 100 950 720]);`	Assigns `fig` from `figure('Color','w','Position',[100 100 950 720])`. This value is used by later computations in this function.
1088	`try`	Starts a protected block. If an error occurs, execution jumps to the matching catch block.
1089	`v = VideoWriter(out_mp4, 'MPEG-4');`	Assigns `v` from `VideoWriter(out_mp4, 'MPEG-4')`. This value is used by later computations in this function.
1090	`catch`	Handles an error from the preceding protected block; used for solver or rendering fallback.
1091	`[p0,n0,~] = fileparts(out_mp4);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1092	`out_mp4 = fullfile(p0, [n0 '.avi']);`	Assigns `out_mp4` from `fullfile(p0, [n0 '.avi'])`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
1093	`v = VideoWriter(out_mp4, 'Motion JPEG AVI');`	Assigns `v` from `VideoWriter(out_mp4, 'Motion JPEG AVI')`. This value is used by later computations in this function.
1094	`end`	Ends the current function, conditional block, loop, or protected block.
1095	`v.FrameRate = fps; open(v);`	Assigns `v.FrameRate` from `fps; open(v)`. This value is used by later computations in this function.
1096	`for k = 1:nFrames`	Starts a loop; the following block is repeated for the specified index range.
1097	`clf(fig); ax = axes('Parent',fig); hold(ax,'on'); grid(ax,'on'); axis(ax,'equal'); view(ax, 3);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1098	`q = xFrames(k,1:14).'; u = xFrames(k,15:28).';`	Assigns `q` from `xFrames(k,1:14).'; u = xFrames(k,15:28).'`. This value is used by later computations in this function.
1099	`xk = [q; u];`	Assigns `xk` from `[q; u]`. This value is used by later computations in this function.
1100	`source = lower(getfield_default_runner(args, 'model_source', 'outer'));`	Assigns `source` from `lower(getfield_default_runner(args, 'model_source', 'outer'))`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
1101	`if strcmp(source, 'outer')`	Starts a conditional branch. The following block executes only when this logical test is true.
1102	`[IN0,~] = vehicle14_outer_models_user('control', tFrames(k), xk, [], par, args, road, meta);`	Calls the user outer-model file in steering-only mode so steering-dependent sensor geometry is evaluated consistently.
1103	`else`	Starts the fallback branch for the current conditional block.
1104	`[~,~,delta,delta_dot] = vehicle14_command_inputs(tFrames(k), par, args);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1105	`IN0=zeros(meta.nin,1); IN0(meta.input_index.delta)=delta; IN0(meta.input_index.delta_dot)=delta_dot;`	Assigns `IN0` from `zeros(meta.nin,1); IN0(meta.input_index.delta)=delta; IN0(meta.input_index.delta_dot)=delta_dot`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
1106	`end`	Ends the current function, conditional block, loop, or protected block.
1107	`[~,~,Sdm] = mf_fun(q,u,par.P,IN0); S=full(Sdm); idx=meta.sensor_index;`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1108	`draw_local_road(ax, road, q(1), 10.0, road.width/2, 0.8, 1.2);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1109	`draw_chassis(ax,q,par);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1110	`names={'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This value is used by later computations in this function.
1111	`for i=1:4`	Starts a loop; the following block is repeated for the specified index range.
1112	`nm=names{i};`	Assigns `nm` from `names{i}`. This value is used by later computations in this function.
1113	`wc=[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];`	Assigns `wc` from `[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))]`. This value is used by later computations in this function.
1114	`head=[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];`	Assigns `head` from `[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))]`. This value is used by later computations in this function.
1115	`spin=[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];`	Assigns `spin` from `[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))]`. This value is used by later computations in this function.
1116	`draw_wheel(ax,wc,head,spin,par.rw);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1117	`end`	Ends the current function, conditional block, loop, or protected block.
1118	`xlabel(ax,'X [m]'); ylabel(ax,'Y [m]'); zlabel(ax,'Z [m]');`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1119	`title(ax,sprintf('vehicle14 MATLAB/CasADi | t = %.2f s | roll %.2f deg | pitch %.2f deg', tFrames(k), rad2deg(q(6)), rad2deg(q(5))));`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1120	`xlim(ax,[q(1)-4 q(1)+8]); ylim(ax,[q(2)-4 q(2)+4]); zlim(ax,[q(3)-1.0 q(3)+1.8]);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.

Line	Sanitized code line	Explanation
1121	`frame = getframe(fig); writeVideo(v,frame);`	Assigns `frame` from `getframe(fig); writeVideo(v,frame)`. This value is used by later computations in this function.
1122	`[A,map] = rgb2ind(frame2im(frame),256);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1123	`if k == 1`	Starts a conditional branch. The following block executes only when this logical test is true.
1124	`imwrite(A,map,out_gif,'gif','LoopCount',Inf,'DelayTime',1/fps);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1125	`else`	Starts the fallback branch for the current conditional block.
1126	`imwrite(A,map,out_gif,'gif','WriteMode','append','DelayTime',1/fps);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1127	`end`	Ends the current function, conditional block, loop, or protected block.
1128	`end`	Ends the current function, conditional block, loop, or protected block.
1129	`close(v); close(fig);`	Executes this MATLAB statement in `render_vehicle14_matlab_animation`. It performs the operation shown by the sanitized code line.
1130	`fprintf('Saved animation: %s and %s\n', out_mp4, out_gif);`	Prints status information to the MATLAB command window so the user can see progress.
1131	`end`	Ends the current function, conditional block, loop, or protected block.
1132		Blank line used to separate logical blocks and improve readability.
1133	`function draw_chassis(ax,q,par)`	Defines the local MATLAB function `draw_chassis`. Draws part of the vehicle or road for the animation.
1134	`R=body_rotation_from_q(q); P=q(1:3); Lf=par.lf; Lr=par.lr; W=max(par.tf,par.tr)/2; H=0.18;`	Assigns `R` from `body_rotation_from_q(q); P=q(1:3); Lf=par.lf; Lr=par.lr; W=max(par.tf,par.tr)/2; H=0.18`. This value is used by later computations in this function.
1135	`ptsB=[ Lf W H; Lf -W H; -Lr -W H; -Lr W H; Lf W -H; Lf -W -H; -Lr -W -H; -Lr W -H].';`	Assigns `ptsB` from `[ Lf W H; Lf -W H; -Lr -W H; -Lr W H; Lf W -H; Lf -W -H; -Lr -W -H; -Lr W -H].'`. This value is used by later computations in this function.
1136	`pts=(P + R*ptsB).';`	Assigns `pts` from `(P + R*ptsB).'`. This value is used by later computations in this function.
1137	`edges=[1 2;2 3;3 4;4 1;5 6;6 7;7 8;8 5;1 5;2 6;3 7;4 8];`	Assigns `edges` from `[1 2;2 3;3 4;4 1;5 6;6 7;7 8;8 5;1 5;2 6;3 7;4 8]`. This value is used by later computations in this function.
1138	`for e=1:size(edges,1), plot3(ax,pts(edges(e,:),1),pts(edges(e,:),2),pts(edges(e,:),3),'k-','LineWidth',1.6); end`	Starts a loop; the following block is repeated for the specified index range.
1139	`end`	Ends the current function, conditional block, loop, or protected block.
1140		Blank line used to separate logical blocks and improve readability.
1141	`function draw_wheel(ax,wc,head,spin,rw)`	Defines the local MATLAB function `draw_wheel`. Draws part of the vehicle or road for the animation.
1142	`spin=normalize_num(spin,[0;1;0]); e1=head-dot(head,spin)*spin; e1=normalize_num(e1,[1;0;0]); e3=normalize_num(cross(spin,e1),[0;0;1]);`	Assigns `spin` from `normalize_num(spin,[0;1;0]); e1=head-dot(head,spin)*spin; e1=normalize_num(e1,[1;0;0]); e3=normalize_num(cross(spin,e1),[0;0;1])`. This value is used by later computations in this function.
1143	`th=linspace(0,2*pi,40); C=wc + rw*(e1*cos(th)+e3*sin(th)); plot3(ax,C(1,:),C(2,:),C(3,:),'LineWidth',1.4);`	Assigns `th` from `linspace(0,2*pi,40); C=wc + rw*(e1*cos(th)+e3*sin(th)); plot3(ax,C(1,:),C(2,:),C(3,:),'LineWidth',1.4)`. This value is used by later computations in this function.
1144	`plot3(ax,[wc(1)-0.12*spin(1), wc(1)+0.12*spin(1)], [wc(2)-0.12*spin(2), wc(2)+0.12*spin(2)], [wc(3)-0.12*spin(3), wc(3)+0.12*spin(3)], 'LineWidth',2.0);`	Executes this MATLAB statement in `draw_wheel`. It performs the operation shown by the sanitized code line.
1145	`end`	Ends the current function, conditional block, loop, or protected block.
1146		Blank line used to separate logical blocks and improve readability.
1147	`function draw_local_road(ax,road,x0,xhalf,nhalf,ds,dn)`	Defines the local MATLAB function `draw_local_road`. Draws part of the vehicle or road for the animation.

Line	Sanitized code line	Explanation
1148	`sVals=(x0-xhalf):ds:(x0+xhalf); nVals=(-nhalf):dn:nhalf; [SS,NN]=meshgrid(sVals,nVals); X=zeros(size(SS));Y=X;Z=X;`	Assigns `sVals` from `(x0-xhalf):ds:(x0+xhalf); nVals=(-nhalf):dn:nhalf; [SS,NN]=meshgrid(sVals,nVals); X=zeros(size(SS));Y=X;Z=X`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
1149	`for i=1:numel(SS), [p,~,~,~]=road_surface_at_local(road,SS(i),NN(i)); X(i)=p(1);Y(i)=p(2);Z(i)=p(3); end`	Starts a loop; the following block is repeated for the specified index range.
1150	`surf(ax,X,Y,Z,'FaceAlpha',0.25,'EdgeAlpha',0.25,'FaceColor',[0.8 0.8 0.8],'EdgeColor',[0.4 0.4 0.4]);`	Executes this MATLAB statement in `draw_local_road`. It performs the operation shown by the sanitized code line.
1151	`end`	Ends the current function, conditional block, loop, or protected block.
1152	`% Uses local road_surface_at_local/centerline_quantities_local helpers defined in this runner.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
1153		Blank line used to separate logical blocks and improve readability.

File B - Generated dynamics matrix function

Evaluates the generated mass matrix M, forcing vector F, and sensor vector S from q, u, parameters P, and external input vector IN. This file is generated and should not be edited by hand.

Lines checked against uploaded code: 790. All code snippets below are sanitized aliases when required; line numbers still refer to the current uploaded file.

Line	Block/function	Purpose
1	vehicle14_generated_dynamics_matrices_casadi	Helper function used by this file. Its line-by-line rows below describe the exact operations.

Line-by-line explanation

Line	Sanitized code line	Explanation
1	`function [M,F,S] = vehicle14_generated_dynamics_matrices_casadi(q,u,P,IN)`	Defines the local MATLAB function `vehicle14_generated_dynamics_matrices_casadi`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
2	`%VEHICLE14_generated-core_MATRICES_CASADI Auto-generated from the uploaded symbolic generator 14-DOF model.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
3	`% Inputs: q(14), u(14), P(78), IN(34). Outputs are CasADi SX expressions.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
4	`% Do not edit this file by hand; regenerate it from export_vehicle14_matlab_casadi.py if the Python model changes.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
5	`import casadi.*`	Imports a MATLAB package namespace so that functions/classes from that package can be called without full qualification.
6	`M = SX.zeros(28,28);`	Allocates the generated mass matrix M with the required dimensions.
7	`F = SX.zeros(28,1);`	Allocates the generated forcing vector F with the required dimensions.
8	`S = SX.zeros(99,1);`	Allocates the generated sensor vector S with the required dimensions.
9	`zz0 = 4*P(2);`	Creates temporary common subexpression `zz0`. It stores `4*P(2)` so the generated expressions are shorter and faster to evaluate.
10	`zz1 = P(1) + zz0;`	Creates temporary common subexpression `zz1`. It stores `P(1) + zz0` so the generated expressions are shorter and faster to evaluate.
11	`zz2 = q(9).^2;`	Creates temporary common subexpression `zz2`. It stores `q(9).^2` so the generated expressions are shorter and faster to evaluate.
12	`zz3 = -P(12) + P(13);`	Creates temporary common subexpression `zz3`. It stores `-P(12) + P(13)` so the generated expressions are shorter and faster to evaluate.
13	`zz4 = P(37).*q(9) + P(39).*zz2 + zz3;`	Creates temporary common subexpression `zz4`. It stores `P(37).*q(9) + P(39).*zz2 + zz3` so the generated expressions are shorter and faster to evaluate.
14	`zz5 = P(2).*zz4;`	Creates temporary common subexpression `zz5`. It stores `P(2).*zz4` so the generated expressions are shorter and faster to evaluate.
15	`zz6 = q(10).^2;`	Creates temporary common subexpression `zz6`. It stores `q(10).^2` so the generated expressions are shorter and faster to evaluate.
16	`zz7 = P(37).*q(10) + P(39).*zz6 + zz3;`	Creates temporary common subexpression `zz7`. It stores `P(37).*q(10) + P(39).*zz6 + zz3` so the generated expressions are shorter and faster to evaluate.
17	`zz8 = P(2).*zz7;`	Creates temporary common subexpression `zz8`. It stores `P(2).*zz7` so the generated expressions are shorter and faster to evaluate.
18	`zz9 = q(7).^2;`	Creates temporary common subexpression `zz9`. It stores `q(7).^2` so the generated expressions are shorter and faster to evaluate.
19	`zz10 = IN(1).*P(27);`	Creates temporary common subexpression `zz10`. It stores `IN(1).*P(27)` so the generated expressions are shorter and faster to evaluate.
20	`zz11 = P(47).*zz10;`	Creates temporary common subexpression `zz11`. It stores `P(47).*zz10` so the generated expressions are shorter and faster to evaluate.
21	`zz12 = P(27).*P(46);`	Creates temporary common subexpression `zz12`. It stores `P(27).*P(46)` so the generated expressions are shorter and faster to evaluate.
22	`zz13 = P(27).^2;`	Creates temporary common subexpression `zz13`. It stores `P(27).^2` so the generated expressions are shorter and faster to evaluate.
23	`zz14 = IN(1).^2.*zz13;`	Creates temporary common subexpression `zz14`. It stores `IN(1).^2.*zz13` so the generated expressions are shorter and faster to evaluate.
24	`zz15 = IN(1).*zz12 + P(48).*zz14 + zz3;`	Creates temporary common subexpression `zz15`. It stores `IN(1).*zz12 + P(48).*zz14 + zz3` so the generated expressions are shorter and faster to evaluate.
25	`zz16 = P(36).*q(7) + P(38).*zz9 + q(7).*zz11 + zz15;`	Creates temporary common subexpression `zz16`. It stores `P(36).*q(7) + P(38).*zz9 + q(7).*zz11 + zz15` so the generated expressions are shorter and faster to evaluate.
26	`zz17 = P(2).*zz16;`	Creates temporary common subexpression `zz17`. It stores `P(2).*zz16` so the generated expressions are shorter and faster to evaluate.
27	`zz18 = q(8).^2;`	Creates temporary common subexpression `zz18`. It stores `q(8).^2` so the generated expressions are shorter and faster to evaluate.
28	`zz19 = P(36).*q(8) + P(38).*zz18 + q(8).*zz11 + zz15;`	Creates temporary common subexpression `zz19`. It stores `P(36).*q(8) + P(38).*zz18 + q(8).*zz11 + zz15` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
29	`zz20 = P(2).*zz19;`	Creates temporary common subexpression `zz20`. It stores `P(2).*zz19` so the generated expressions are shorter and faster to evaluate.
30	`zz21 = zz17 + zz20 + zz5 + zz8;`	Creates temporary common subexpression `zz21`. It stores `zz17 + zz20 + zz5 + zz8` so the generated expressions are shorter and faster to evaluate.
31	`zz22 = P(11)/2;`	Creates temporary common subexpression `zz22`. It stores `P(11)/2` so the generated expressions are shorter and faster to evaluate.
32	`zz23 = P(34).*q(10) + P(35).*zz6 - zz22;`	Creates temporary common subexpression `zz23`. It stores `P(34).*q(10) + P(35).*zz6 - zz22` so the generated expressions are shorter and faster to evaluate.
33	`zz24 = P(2).*zz23;`	Creates temporary common subexpression `zz24`. It stores `P(2).*zz23` so the generated expressions are shorter and faster to evaluate.
34	`zz25 = P(34).*q(9) + P(35).*zz2 + zz22;`	Creates temporary common subexpression `zz25`. It stores `P(34).*q(9) + P(35).*zz2 + zz22` so the generated expressions are shorter and faster to evaluate.
35	`zz26 = P(2).*zz25;`	Creates temporary common subexpression `zz26`. It stores `P(2).*zz25` so the generated expressions are shorter and faster to evaluate.
36	`zz27 = P(10)/2;`	Creates temporary common subexpression `zz27`. It stores `P(10)/2` so the generated expressions are shorter and faster to evaluate.
37	`zz28 = P(44).*zz10;`	Creates temporary common subexpression `zz28`. It stores `P(44).*zz10` so the generated expressions are shorter and faster to evaluate.
38	`zz29 = P(27).*P(43);`	Creates temporary common subexpression `zz29`. It stores `P(27).*P(43)` so the generated expressions are shorter and faster to evaluate.
39	`zz30 = IN(1).*zz29 + P(45).*zz14;`	Creates temporary common subexpression `zz30`. It stores `IN(1).*zz29 + P(45).*zz14` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
40	`zz31 = P(32).*q(7) + P(33).*zz9 + q(7).*zz28 + zz27 + zz30;`	Creates temporary common subexpression `zz31`. It stores `P(32).*q(7) + P(33).*zz9 + q(7).*zz28 + zz27 + zz30` so the generated expressions are shorter and faster to evaluate.
41	`zz32 = P(2).*zz31;`	Creates temporary common subexpression `zz32`. It stores `P(2).*zz31` so the generated expressions are shorter and faster to evaluate.
42	`zz33 = P(32).*q(8) + P(33).*zz18 + q(8).*zz28 - zz27 + zz30;`	Creates temporary common subexpression `zz33`. It stores `P(32).*q(8) + P(33).*zz18 + q(8).*zz28 - zz27 + zz30` so the generated expressions are shorter and faster to evaluate.
43	`zz34 = P(2).*zz33;`	Creates temporary common subexpression `zz34`. It stores `P(2).*zz33` so the generated expressions are shorter and faster to evaluate.
44	`zz35 = zz24 + zz26 + zz32 + zz34;`	Creates temporary common subexpression `zz35`. It stores `zz24 + zz26 + zz32 + zz34` so the generated expressions are shorter and faster to evaluate.
45	`zz36 = 2*P(29);`	Creates temporary common subexpression `zz36`. It stores `2*P(29)` so the generated expressions are shorter and faster to evaluate.
46	`zz37 = q(7).*zz36;`	Creates temporary common subexpression `zz37`. It stores `q(7).*zz36` so the generated expressions are shorter and faster to evaluate.
47	`zz38 = P(41).*zz10;`	Creates temporary common subexpression `zz38`. It stores `P(41).*zz10` so the generated expressions are shorter and faster to evaluate.
48	`zz39 = P(28) + zz38;`	Creates temporary common subexpression `zz39`. It stores `P(28) + zz38` so the generated expressions are shorter and faster to evaluate.
49	`zz40 = zz37 + zz39;`	Creates temporary common subexpression `zz40`. It stores `zz37 + zz39` so the generated expressions are shorter and faster to evaluate.
50	`zz41 = P(2).*zz40;`	Creates temporary common subexpression `zz41`. It stores `P(2).*zz40` so the generated expressions are shorter and faster to evaluate.
51	`zz42 = q(8).*zz36;`	Creates temporary common subexpression `zz42`. It stores `q(8).*zz36` so the generated expressions are shorter and faster to evaluate.
52	`zz43 = zz39 + zz42;`	Creates temporary common subexpression `zz43`. It stores `zz39 + zz42` so the generated expressions are shorter and faster to evaluate.
53	`zz44 = P(2).*zz43;`	Creates temporary common subexpression `zz44`. It stores `P(2).*zz43` so the generated expressions are shorter and faster to evaluate.
54	`zz45 = 2*P(31);`	Creates temporary common subexpression `zz45`. It stores `2*P(31)` so the generated expressions are shorter and faster to evaluate.
55	`zz46 = q(9).*zz45;`	Creates temporary common subexpression `zz46`. It stores `q(9).*zz45` so the generated expressions are shorter and faster to evaluate.
56	`zz47 = P(30) + zz46;`	Creates temporary common subexpression `zz47`. It stores `P(30) + zz46` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
57	`zz48 = P(2).*zz47;`	Creates temporary common subexpression `zz48`. It stores `P(2).*zz47` so the generated expressions are shorter and faster to evaluate.
58	`zz49 = q(10).*zz45;`	Creates temporary common subexpression `zz49`. It stores `q(10).*zz45` so the generated expressions are shorter and faster to evaluate.
59	`zz50 = P(30) + zz49;`	Creates temporary common subexpression `zz50`. It stores `P(30) + zz49` so the generated expressions are shorter and faster to evaluate.
60	`zz51 = P(2).*zz50;`	Creates temporary common subexpression `zz51`. It stores `P(2).*zz50` so the generated expressions are shorter and faster to evaluate.
61	`zz52 = -P(9);`	Creates temporary common subexpression `zz52`. It stores `-P(9)` so the generated expressions are shorter and faster to evaluate.
62	`zz53 = P(30).*q(9) + P(31).*zz2 + zz52;`	Creates temporary common subexpression `zz53`. It stores `P(30).*q(9) + P(31).*zz2 + zz52` so the generated expressions are shorter and faster to evaluate.
63	`zz54 = P(2).*zz53;`	Creates temporary common subexpression `zz54`. It stores `P(2).*zz53` so the generated expressions are shorter and faster to evaluate.
64	`zz55 = P(30).*q(10) + P(31).*zz6 + zz52;`	Creates temporary common subexpression `zz55`. It stores `P(30).*q(10) + P(31).*zz6 + zz52` so the generated expressions are shorter and faster to evaluate.
65	`zz56 = P(2).*zz55;`	Creates temporary common subexpression `zz56`. It stores `P(2).*zz55` so the generated expressions are shorter and faster to evaluate.
66	`zz57 = P(27).*P(40);`	Creates temporary common subexpression `zz57`. It stores `P(27).*P(40)` so the generated expressions are shorter and faster to evaluate.
67	`zz58 = IN(1).*zz57 + P(8) + P(42).*zz14;`	Creates temporary common subexpression `zz58`. It stores `IN(1).*zz57 + P(8) + P(42).*zz14` so the generated expressions are shorter and faster to evaluate.
68	`zz59 = P(28).*q(7) + P(29).*zz9 + q(7).*zz38 + zz58;`	Creates temporary common subexpression `zz59`. It stores `P(28).*q(7) + P(29).*zz9 + q(7).*zz38 + zz58` so the generated expressions are shorter and faster to evaluate.
69	`zz60 = P(2).*zz59;`	Creates temporary common subexpression `zz60`. It stores `P(2).*zz59` so the generated expressions are shorter and faster to evaluate.
70	`zz61 = P(28).*q(8) + P(29).*zz18 + q(8).*zz38 + zz58;`	Creates temporary common subexpression `zz61`. It stores `P(28).*q(8) + P(29).*zz18 + q(8).*zz38 + zz58` so the generated expressions are shorter and faster to evaluate.
71	`zz62 = P(2).*zz61;`	Creates temporary common subexpression `zz62`. It stores `P(2).*zz61` so the generated expressions are shorter and faster to evaluate.
72	`zz63 = zz54 + zz56 + zz60 + zz62;`	Creates temporary common subexpression `zz63`. It stores `zz54 + zz56 + zz60 + zz62` so the generated expressions are shorter and faster to evaluate.
73	`zz64 = 2*P(33);`	Creates temporary common subexpression `zz64`. It stores `2*P(33)` so the generated expressions are shorter and faster to evaluate.
74	`zz65 = q(7).*zz64;`	Creates temporary common subexpression `zz65`. It stores `q(7).*zz64` so the generated expressions are shorter and faster to evaluate.
75	`zz66 = P(32) + zz28;`	Creates temporary common subexpression `zz66`. It stores `P(32) + zz28` so the generated expressions are shorter and faster to evaluate.
76	`zz67 = zz65 + zz66;`	Creates temporary common subexpression `zz67`. It stores `zz65 + zz66` so the generated expressions are shorter and faster to evaluate.
77	`zz68 = P(2).*zz67;`	Creates temporary common subexpression `zz68`. It stores `P(2).*zz67` so the generated expressions are shorter and faster to evaluate.
78	`zz69 = q(8).*zz64;`	Creates temporary common subexpression `zz69`. It stores `q(8).*zz64` so the generated expressions are shorter and faster to evaluate.
79	`zz70 = zz66 + zz69;`	Creates temporary common subexpression `zz70`. It stores `zz66 + zz69` so the generated expressions are shorter and faster to evaluate.
80	`zz71 = P(2).*zz70;`	Creates temporary common subexpression `zz71`. It stores `P(2).*zz70` so the generated expressions are shorter and faster to evaluate.
81	`zz72 = 2*P(35);`	Creates temporary common subexpression `zz72`. It stores `2*P(35)` so the generated expressions are shorter and faster to evaluate.
82	`zz73 = q(9).*zz72;`	Creates temporary common subexpression `zz73`. It stores `q(9).*zz72` so the generated expressions are shorter and faster to evaluate.
83	`zz74 = P(34) + zz73;`	Creates temporary common subexpression `zz74`. It stores `P(34) + zz73` so the generated expressions are shorter and faster to evaluate.
84	`zz75 = P(2).*zz74;`	Creates temporary common subexpression `zz75`. It stores `P(2).*zz74` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
85	`zz76 = q(10).*zz72;`	Creates temporary common subexpression `zz76`. It stores `q(10).*zz72` so the generated expressions are shorter and faster to evaluate.
86	`zz77 = P(34) + zz76;`	Creates temporary common subexpression `zz77`. It stores `P(34) + zz76` so the generated expressions are shorter and faster to evaluate.
87	`zz78 = P(2).*zz77;`	Creates temporary common subexpression `zz78`. It stores `P(2).*zz77` so the generated expressions are shorter and faster to evaluate.
88	`zz79 = 2*P(38);`	Creates temporary common subexpression `zz79`. It stores `2*P(38)` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
89	`zz80 = q(7).*zz79;`	Creates temporary common subexpression `zz80`. It stores `q(7).*zz79` so the generated expressions are shorter and faster to evaluate.
90	`zz81 = P(36) + zz11;`	Creates temporary common subexpression `zz81`. It stores `P(36) + zz11` so the generated expressions are shorter and faster to evaluate.
91	`zz82 = zz80 + zz81;`	Creates temporary common subexpression `zz82`. It stores `zz80 + zz81` so the generated expressions are shorter and faster to evaluate.
92	`zz83 = P(2).*zz82;`	Creates temporary common subexpression `zz83`. It stores `P(2).*zz82` so the generated expressions are shorter and faster to evaluate.
93	`zz84 = q(8).*zz79;`	Creates temporary common subexpression `zz84`. It stores `q(8).*zz79` so the generated expressions are shorter and faster to evaluate.
94	`zz85 = zz81 + zz84;`	Creates temporary common subexpression `zz85`. It stores `zz81 + zz84` so the generated expressions are shorter and faster to evaluate.
95	`zz86 = P(2).*zz85;`	Creates temporary common subexpression `zz86`. It stores `P(2).*zz85` so the generated expressions are shorter and faster to evaluate.
96	`zz87 = 2*P(39);`	Creates temporary common subexpression `zz87`. It stores `2*P(39)` so the generated expressions are shorter and faster to evaluate.
97	`zz88 = q(9).*zz87;`	Creates temporary common subexpression `zz88`. It stores `q(9).*zz87` so the generated expressions are shorter and faster to evaluate.
98	`zz89 = P(37) + zz88;`	Creates temporary common subexpression `zz89`. It stores `P(37) + zz88` so the generated expressions are shorter and faster to evaluate.
99	`zz90 = P(2).*zz89;`	Creates temporary common subexpression `zz90`. It stores `P(2).*zz89` so the generated expressions are shorter and faster to evaluate.
100	`zz91 = q(10).*zz87;`	Creates temporary common subexpression `zz91`. It stores `q(10).*zz87` so the generated expressions are shorter and faster to evaluate.
101	`zz92 = P(37) + zz91;`	Creates temporary common subexpression `zz92`. It stores `P(37) + zz91` so the generated expressions are shorter and faster to evaluate.
102	`zz93 = P(2).*zz92;`	Creates temporary common subexpression `zz93`. It stores `P(2).*zz92` so the generated expressions are shorter and faster to evaluate.
103	`zz94 = -zz4;`	Creates temporary common subexpression `zz94`. It stores `-zz4` so the generated expressions are shorter and faster to evaluate.
104	`zz95 = -zz7;`	Creates temporary common subexpression `zz95`. It stores `-zz7` so the generated expressions are shorter and faster to evaluate.
105	`zz96 = -zz16;`	Creates temporary common subexpression `zz96`. It stores `-zz16` so the generated expressions are shorter and faster to evaluate.
106	`zz97 = -zz19;`	Creates temporary common subexpression `zz97`. It stores `-zz19` so the generated expressions are shorter and faster to evaluate.
107	`zz98 = P(52) + P(53).*q(9) + P(54).*zz2;`	Creates temporary common subexpression `zz98`. It stores `P(52) + P(53).*q(9) + P(54).*zz2` so the generated expressions are shorter and faster to evaluate.
108	`zz99 = sin(zz98);`	Creates temporary common subexpression `zz99`. It stores `sin(zz98)` so the generated expressions are shorter and faster to evaluate.
109	`zz100 = P(70) + P(71).*q(9) + P(72).*zz2;`	Creates temporary common subexpression `zz100`. It stores `P(70) + P(71).*q(9) + P(72).*zz2` so the generated expressions are shorter and faster to evaluate.
110	`zz101 = cos(zz100);`	Creates temporary common subexpression `zz101`. It stores `cos(zz100)` so the generated expressions are shorter and faster to evaluate.
111	`zz102 = P(7).*zz101.^2;`	Creates temporary common subexpression `zz102`. It stores `P(7).*zz101.^2` so the generated expressions are shorter and faster to evaluate.
112	`zz103 = P(52) + P(53).*q(10) + P(54).*zz6;`	Creates temporary common subexpression `zz103`. It stores `P(52) + P(53).*q(10) + P(54).*zz6` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
113	`zz104 = sin(zz103);`	Creates temporary common subexpression `zz104`. It stores `sin(zz103)` so the generated expressions are shorter and faster to evaluate.
114	`zz105 = P(70) + P(71).*q(10) + P(72).*zz6;`	Creates temporary common subexpression `zz105`. It stores `P(70) + P(71).*q(10) + P(72).*zz6` so the generated expressions are shorter and faster to evaluate.
115	`zz106 = cos(zz105);`	Creates temporary common subexpression `zz106`. It stores `cos(zz105)` so the generated expressions are shorter and faster to evaluate.
116	`zz107 = P(7).*zz106.^2;`	Creates temporary common subexpression `zz107`. It stores `P(7).*zz106.^2` so the generated expressions are shorter and faster to evaluate.
117	`zz108 = P(56).*zz10;`	Creates temporary common subexpression `zz108`. It stores `P(56).*zz10` so the generated expressions are shorter and faster to evaluate.
118	`zz109 = P(27).*P(55);`	Creates temporary common subexpression `zz109`. It stores `P(27).*P(55)` so the generated expressions are shorter and faster to evaluate.
119	`zz110 = IN(1).*zz109 + P(49) + P(57).*zz14;`	Creates temporary common subexpression `zz110`. It stores `IN(1).*zz109 + P(49) + P(57).*zz14` so the generated expressions are shorter and faster to evaluate.
120	`zz111 = P(50).*q(7) + P(51).*zz9 + q(7).*zz108 + zz110;`	Creates temporary common subexpression `zz111`. It stores `P(50).*q(7) + P(51).*zz9 + q(7).*zz108 + zz110` so the generated expressions are shorter and faster to evaluate.
121	`zz112 = sin(zz111);`	Creates temporary common subexpression `zz112`. It stores `sin(zz111)` so the generated expressions are shorter and faster to evaluate.
122	`zz113 = P(74).*zz10;`	Creates temporary common subexpression `zz113`. It stores `P(74).*zz10` so the generated expressions are shorter and faster to evaluate.
123	`zz114 = P(27).*P(73);`	Creates temporary common subexpression `zz114`. It stores `P(27).*P(73)` so the generated expressions are shorter and faster to evaluate.
124	`zz115 = IN(1).*zz114 + P(67) + P(75).*zz14;`	Creates temporary common subexpression `zz115`. It stores `IN(1).*zz114 + P(67) + P(75).*zz14` so the generated expressions are shorter and faster to evaluate.
125	`zz116 = P(68).*q(7) + P(69).*zz9 + q(7).*zz113 + zz115;`	Creates temporary common subexpression `zz116`. It stores `P(68).*q(7) + P(69).*zz9 + q(7).*zz113 + zz115` so the generated expressions are shorter and faster to evaluate.
126	`zz117 = cos(zz116);`	Creates temporary common subexpression `zz117`. It stores `cos(zz116)` so the generated expressions are shorter and faster to evaluate.
127	`zz118 = P(7).*zz117.^2;`	Creates temporary common subexpression `zz118`. It stores `P(7).*zz117.^2` so the generated expressions are shorter and faster to evaluate.
128	`zz119 = P(50).*q(8) + P(51).*zz18 + q(8).*zz108 + zz110;`	Creates temporary common subexpression `zz119`. It stores `P(50).*q(8) + P(51).*zz18 + q(8).*zz108 + zz110` so the generated expressions are shorter and faster to evaluate.
129	`zz120 = sin(zz119);`	Creates temporary common subexpression `zz120`. It stores `sin(zz119)` so the generated expressions are shorter and faster to evaluate.
130	`zz121 = P(68).*q(8) + P(69).*zz18 + q(8).*zz113 + zz115;`	Creates temporary common subexpression `zz121`. It stores `P(68).*q(8) + P(69).*zz18 + q(8).*zz113 + zz115` so the generated expressions are shorter and faster to evaluate.
131	`zz122 = cos(zz121);`	Creates temporary common subexpression `zz122`. It stores `cos(zz121)` so the generated expressions are shorter and faster to evaluate.
132	`zz123 = P(7).*zz122.^2;`	Creates temporary common subexpression `zz123`. It stores `P(7).*zz122.^2` so the generated expressions are shorter and faster to evaluate.
133	`zz124 = cos(zz98);`	Creates temporary common subexpression `zz124`. It stores `cos(zz98)` so the generated expressions are shorter and faster to evaluate.
134	`zz125 = zz102.*zz124.*zz99;`	Creates temporary common subexpression `zz125`. It stores `zz102.*zz124.*zz99` so the generated expressions are shorter and faster to evaluate.
135	`zz126 = cos(zz103);`	Creates temporary common subexpression `zz126`. It stores `cos(zz103)` so the generated expressions are shorter and faster to evaluate.
136	`zz127 = zz104.*zz107.*zz126;`	Creates temporary common subexpression `zz127`. It stores `zz104.*zz107.*zz126` so the generated expressions are shorter and faster to evaluate.
137	`zz128 = cos(zz111);`	Creates temporary common subexpression `zz128`. It stores `cos(zz111)` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
138	`zz129 = zz112.*zz118.*zz128;`	Creates temporary common subexpression `zz129`. It stores `zz112.*zz118.*zz128` so the generated expressions are shorter and faster to evaluate.
139	`zz130 = cos(zz119);`	Creates temporary common subexpression `zz130`. It stores `cos(zz119)` so the generated expressions are shorter and faster to evaluate.
140	`zz131 = zz120.*zz123.*zz130;`	Creates temporary common subexpression `zz131`. It stores `zz120.*zz123.*zz130` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
141	`zz132 = sin(zz100);`	Creates temporary common subexpression `zz132`. It stores `sin(zz100)` so the generated expressions are shorter and faster to evaluate.
142	`zz133 = P(7).*zz132;`	Creates temporary common subexpression `zz133`. It stores `P(7).*zz132` so the generated expressions are shorter and faster to evaluate.
143	`zz134 = zz101.*zz99;`	Creates temporary common subexpression `zz134`. It stores `zz101.*zz99` so the generated expressions are shorter and faster to evaluate.
144	`zz135 = zz133.*zz134;`	Creates temporary common subexpression `zz135`. It stores `zz133.*zz134` so the generated expressions are shorter and faster to evaluate.
145	`zz136 = sin(zz105);`	Creates temporary common subexpression `zz136`. It stores `sin(zz105)` so the generated expressions are shorter and faster to evaluate.
146	`zz137 = P(7).*zz136;`	Creates temporary common subexpression `zz137`. It stores `P(7).*zz136` so the generated expressions are shorter and faster to evaluate.
147	`zz138 = zz104.*zz106;`	Creates temporary common subexpression `zz138`. It stores `zz104.*zz106` so the generated expressions are shorter and faster to evaluate.
148	`zz139 = zz137.*zz138;`	Creates temporary common subexpression `zz139`. It stores `zz137.*zz138` so the generated expressions are shorter and faster to evaluate.
149	`zz140 = sin(zz116);`	Creates temporary common subexpression `zz140`. It stores `sin(zz116)` so the generated expressions are shorter and faster to evaluate.
150	`zz141 = P(7).*zz140;`	Creates temporary common subexpression `zz141`. It stores `P(7).*zz140` so the generated expressions are shorter and faster to evaluate.
151	`zz142 = zz112.*zz117;`	Creates temporary common subexpression `zz142`. It stores `zz112.*zz117` so the generated expressions are shorter and faster to evaluate.
152	`zz143 = zz141.*zz142;`	Creates temporary common subexpression `zz143`. It stores `zz141.*zz142` so the generated expressions are shorter and faster to evaluate.
153	`zz144 = sin(zz121);`	Creates temporary common subexpression `zz144`. It stores `sin(zz121)` so the generated expressions are shorter and faster to evaluate.
154	`zz145 = P(7).*zz144;`	Creates temporary common subexpression `zz145`. It stores `P(7).*zz144` so the generated expressions are shorter and faster to evaluate.
155	`zz146 = zz120.*zz122;`	Creates temporary common subexpression `zz146`. It stores `zz120.*zz122` so the generated expressions are shorter and faster to evaluate.
156	`zz147 = zz145.*zz146;`	Creates temporary common subexpression `zz147`. It stores `zz145.*zz146` so the generated expressions are shorter and faster to evaluate.
157	`zz148 = 2*q(7);`	Creates temporary common subexpression `zz148`. It stores `2*q(7)` so the generated expressions are shorter and faster to evaluate.
158	`zz149 = P(60).*zz148;`	Creates temporary common subexpression `zz149`. It stores `P(60).*zz148` so the generated expressions are shorter and faster to evaluate.
159	`zz150 = P(65).*zz10;`	Creates temporary common subexpression `zz150`. It stores `P(65).*zz10` so the generated expressions are shorter and faster to evaluate.
160	`zz151 = P(59) + zz150;`	Creates temporary common subexpression `zz151`. It stores `P(59) + zz150` so the generated expressions are shorter and faster to evaluate.
161	`zz152 = zz149 + zz151;`	Creates temporary common subexpression `zz152`. It stores `zz149 + zz151` so the generated expressions are shorter and faster to evaluate.
162	`zz153 = P(7).*zz152;`	Creates temporary common subexpression `zz153`. It stores `P(7).*zz152` so the generated expressions are shorter and faster to evaluate.
163	`zz154 = P(51).*zz148;`	Creates temporary common subexpression `zz154`. It stores `P(51).*zz148` so the generated expressions are shorter and faster to evaluate.
164	`zz155 = P(50) + zz108;`	Creates temporary common subexpression `zz155`. It stores `P(50) + zz108` so the generated expressions are shorter and faster to evaluate.
165	`zz156 = zz154 + zz155;`	Creates temporary common subexpression `zz156`. It stores `zz154 + zz155` so the generated expressions are shorter and faster to evaluate.
166	`zz157 = zz141.*zz156;`	Creates temporary common subexpression `zz157`. It stores `zz141.*zz156` so the generated expressions are shorter and faster to evaluate.
167	`zz158 = 2*q(8);`	Creates temporary common subexpression `zz158`. It stores `2*q(8)` so the generated expressions are shorter and faster to evaluate.
168	`zz159 = P(60).*zz158;`	Creates temporary common subexpression `zz159`. It stores `P(60).*zz158` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
169	`zz160 = zz151 + zz159;`	Creates temporary common subexpression `zz160`. It stores `zz151 + zz159` so the generated expressions are shorter and faster to evaluate.
170	`zz161 = P(7).*zz160;`	Creates temporary common subexpression `zz161`. It stores `P(7).*zz160` so the generated expressions are shorter and faster to evaluate.
171	`zz162 = P(51).*zz158;`	Creates temporary common subexpression `zz162`. It stores `P(51).*zz158` so the generated expressions are shorter and faster to evaluate.
172	`zz163 = zz155 + zz162;`	Creates temporary common subexpression `zz163`. It stores `zz155 + zz162` so the generated expressions are shorter and faster to evaluate.
173	`zz164 = zz145.*zz163;`	Creates temporary common subexpression `zz164`. It stores `zz145.*zz163` so the generated expressions are shorter and faster to evaluate.
174	`zz165 = 2*q(9);`	Creates temporary common subexpression `zz165`. It stores `2*q(9)` so the generated expressions are shorter and faster to evaluate.
175	`zz166 = P(63).*zz165;`	Creates temporary common subexpression `zz166`. It stores `P(63).*zz165` so the generated expressions are shorter and faster to evaluate.
176	`zz167 = P(62) + zz166;`	Creates temporary common subexpression `zz167`. It stores `P(62) + zz166` so the generated expressions are shorter and faster to evaluate.
177	`zz168 = P(7).*zz167;`	Creates temporary common subexpression `zz168`. It stores `P(7).*zz167` so the generated expressions are shorter and faster to evaluate.
178	`zz169 = P(54).*zz165;`	Creates temporary common subexpression `zz169`. It stores `P(54).*zz165` so the generated expressions are shorter and faster to evaluate.
179	`zz170 = P(53) + zz169;`	Creates temporary common subexpression `zz170`. It stores `P(53) + zz169` so the generated expressions are shorter and faster to evaluate.
180	`zz171 = zz133.*zz170;`	Creates temporary common subexpression `zz171`. It stores `zz133.*zz170` so the generated expressions are shorter and faster to evaluate.
181	`zz172 = 2*q(10);`	Creates temporary common subexpression `zz172`. It stores `2*q(10)` so the generated expressions are shorter and faster to evaluate.
182	`zz173 = P(63).*zz172;`	Creates temporary common subexpression `zz173`. It stores `P(63).*zz172` so the generated expressions are shorter and faster to evaluate.
183	`zz174 = P(62) + zz173;`	Creates temporary common subexpression `zz174`. It stores `P(62) + zz173` so the generated expressions are shorter and faster to evaluate.
184	`zz175 = P(7).*zz174;`	Creates temporary common subexpression `zz175`. It stores `P(7).*zz174` so the generated expressions are shorter and faster to evaluate.
185	`zz176 = P(54).*zz172;`	Creates temporary common subexpression `zz176`. It stores `P(54).*zz172` so the generated expressions are shorter and faster to evaluate.
186	`zz177 = P(53) + zz176;`	Creates temporary common subexpression `zz177`. It stores `P(53) + zz176` so the generated expressions are shorter and faster to evaluate.
187	`zz178 = zz137.*zz177;`	Creates temporary common subexpression `zz178`. It stores `zz137.*zz177` so the generated expressions are shorter and faster to evaluate.
188	`zz179 = P(7).*zz117;`	Creates temporary common subexpression `zz179`. It stores `P(7).*zz117` so the generated expressions are shorter and faster to evaluate.
189	`zz180 = -zz112.*zz179;`	Creates temporary common subexpression `zz180`. It stores `-zz112.*zz179` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
190	`zz181 = P(7).*zz122;`	Creates temporary common subexpression `zz181`. It stores `P(7).*zz122` so the generated expressions are shorter and faster to evaluate.
191	`zz182 = -zz120.*zz181;`	Creates temporary common subexpression `zz182`. It stores `-zz120.*zz181` so the generated expressions are shorter and faster to evaluate.
192	`zz183 = P(7).*zz101;`	Creates temporary common subexpression `zz183`. It stores `P(7).*zz101` so the generated expressions are shorter and faster to evaluate.
193	`zz184 = -zz183.*zz99;`	Creates temporary common subexpression `zz184`. It stores `-zz183.*zz99` so the generated expressions are shorter and faster to evaluate.
194	`zz185 = P(7).*zz106;`	Creates temporary common subexpression `zz185`. It stores `P(7).*zz106` so the generated expressions are shorter and faster to evaluate.
195	`zz186 = -zz104.*zz185;`	Creates temporary common subexpression `zz186`. It stores `-zz104.*zz185` so the generated expressions are shorter and faster to evaluate.
196	`zz187 = -zz53;`	Creates temporary common subexpression `zz187`. It stores `-zz53` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
197	`zz188 = -zz55;`	Creates temporary common subexpression `zz188`. It stores `-zz55` so the generated expressions are shorter and faster to evaluate.
198	`zz189 = -zz59;`	Creates temporary common subexpression `zz189`. It stores `-zz59` so the generated expressions are shorter and faster to evaluate.
199	`zz190 = -zz61;`	Creates temporary common subexpression `zz190`. It stores `-zz61` so the generated expressions are shorter and faster to evaluate.
200	`zz191 = zz117.*zz128;`	Creates temporary common subexpression `zz191`. It stores `zz117.*zz128` so the generated expressions are shorter and faster to evaluate.
201	`zz192 = zz16.*zz40;`	Creates temporary common subexpression `zz192`. It stores `zz16.*zz40` so the generated expressions are shorter and faster to evaluate.
202	`zz193 = zz122.*zz130;`	Creates temporary common subexpression `zz193`. It stores `zz122.*zz130` so the generated expressions are shorter and faster to evaluate.
203	`zz194 = zz19.*zz43;`	Creates temporary common subexpression `zz194`. It stores `zz19.*zz43` so the generated expressions are shorter and faster to evaluate.
204	`zz195 = zz101.*zz124;`	Creates temporary common subexpression `zz195`. It stores `zz101.*zz124` so the generated expressions are shorter and faster to evaluate.
205	`zz196 = zz4.*zz47;`	Creates temporary common subexpression `zz196`. It stores `zz4.*zz47` so the generated expressions are shorter and faster to evaluate.
206	`zz197 = zz106.*zz126;`	Creates temporary common subexpression `zz197`. It stores `zz106.*zz126` so the generated expressions are shorter and faster to evaluate.
207	`zz198 = zz50.*zz7;`	Creates temporary common subexpression `zz198`. It stores `zz50.*zz7` so the generated expressions are shorter and faster to evaluate.
208	`zz199 = zz128.*zz179;`	Creates temporary common subexpression `zz199`. It stores `zz128.*zz179` so the generated expressions are shorter and faster to evaluate.
209	`zz200 = zz130.*zz181;`	Creates temporary common subexpression `zz200`. It stores `zz130.*zz181` so the generated expressions are shorter and faster to evaluate.
210	`zz201 = zz124.*zz183;`	Creates temporary common subexpression `zz201`. It stores `zz124.*zz183` so the generated expressions are shorter and faster to evaluate.
211	`zz202 = zz126.*zz185;`	Creates temporary common subexpression `zz202`. It stores `zz126.*zz185` so the generated expressions are shorter and faster to evaluate.
212	`zz203 = -zz25;`	Creates temporary common subexpression `zz203`. It stores `-zz25` so the generated expressions are shorter and faster to evaluate.
213	`zz204 = -zz23;`	Creates temporary common subexpression `zz204`. It stores `-zz23` so the generated expressions are shorter and faster to evaluate.
214	`zz205 = -zz31;`	Creates temporary common subexpression `zz205`. It stores `-zz31` so the generated expressions are shorter and faster to evaluate.
215	`zz206 = -zz33;`	Creates temporary common subexpression `zz206`. It stores `-zz33` so the generated expressions are shorter and faster to evaluate.
216	`zz207 = P(7).*zz132.^2;`	Creates temporary common subexpression `zz207`. It stores `P(7).*zz132.^2` so the generated expressions are shorter and faster to evaluate.
217	`zz208 = P(7).*zz136.^2;`	Creates temporary common subexpression `zz208`. It stores `P(7).*zz136.^2` so the generated expressions are shorter and faster to evaluate.
218	`zz209 = P(7).*zz140.^2;`	Creates temporary common subexpression `zz209`. It stores `P(7).*zz140.^2` so the generated expressions are shorter and faster to evaluate.
219	`zz210 = P(7).*zz144.^2;`	Creates temporary common subexpression `zz210`. It stores `P(7).*zz144.^2` so the generated expressions are shorter and faster to evaluate.
220	`zz211 = zz140.*zz156;`	Creates temporary common subexpression `zz211`. It stores `zz140.*zz156` so the generated expressions are shorter and faster to evaluate.
221	`zz212 = zz152 + zz211;`	Creates temporary common subexpression `zz212`. It stores `zz152 + zz211` so the generated expressions are shorter and faster to evaluate.
222	`zz213 = P(7).*zz212;`	Creates temporary common subexpression `zz213`. It stores `P(7).*zz212` so the generated expressions are shorter and faster to evaluate.
223	`zz214 = zz141.*zz212;`	Creates temporary common subexpression `zz214`. It stores `zz141.*zz212` so the generated expressions are shorter and faster to evaluate.
224	`zz215 = zz144.*zz163;`	Creates temporary common subexpression `zz215`. It stores `zz144.*zz163` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
225	`zz216 = zz160 + zz215;`	Creates temporary common subexpression `zz216`. It stores `zz160 + zz215` so the generated expressions are shorter and faster to evaluate.
226	`zz217 = P(7).*zz216;`	Creates temporary common subexpression `zz217`. It stores `P(7).*zz216` so the generated expressions are shorter and faster to evaluate.
227	`zz218 = zz145.*zz216;`	Creates temporary common subexpression `zz218`. It stores `zz145.*zz216` so the generated expressions are shorter and faster to evaluate.
228	`zz219 = zz132.*zz170;`	Creates temporary common subexpression `zz219`. It stores `zz132.*zz170` so the generated expressions are shorter and faster to evaluate.
229	`zz220 = zz167 + zz219;`	Creates temporary common subexpression `zz220`. It stores `zz167 + zz219` so the generated expressions are shorter and faster to evaluate.
230	`zz221 = P(7).*zz220;`	Creates temporary common subexpression `zz221`. It stores `P(7).*zz220` so the generated expressions are shorter and faster to evaluate.
231	`zz222 = zz133.*zz220;`	Creates temporary common subexpression `zz222`. It stores `zz133.*zz220` so the generated expressions are shorter and faster to evaluate.
232	`zz223 = zz136.*zz177;`	Creates temporary common subexpression `zz223`. It stores `zz136.*zz177` so the generated expressions are shorter and faster to evaluate.
233	`zz224 = zz174 + zz223;`	Creates temporary common subexpression `zz224`. It stores `zz174 + zz223` so the generated expressions are shorter and faster to evaluate.
234	`zz225 = P(7).*zz224;`	Creates temporary common subexpression `zz225`. It stores `P(7).*zz224` so the generated expressions are shorter and faster to evaluate.
235	`zz226 = zz137.*zz224;`	Creates temporary common subexpression `zz226`. It stores `zz137.*zz224` so the generated expressions are shorter and faster to evaluate.
236	`zz227 = cos(q(4));`	Creates temporary common subexpression `zz227`. It stores `cos(q(4))` so the generated expressions are shorter and faster to evaluate.
237	`zz228 = cos(q(5));`	Creates temporary common subexpression `zz228`. It stores `cos(q(5))` so the generated expressions are shorter and faster to evaluate.
238	`zz229 = sin(q(4));`	Creates temporary common subexpression `zz229`. It stores `sin(q(4))` so the generated expressions are shorter and faster to evaluate.
239	`zz230 = cos(q(6));`	Creates temporary common subexpression `zz230`. It stores `cos(q(6))` so the generated expressions are shorter and faster to evaluate.
240	`zz231 = zz229.*zz230;`	Creates temporary common subexpression `zz231`. It stores `zz229.*zz230` so the generated expressions are shorter and faster to evaluate.
241	`zz232 = sin(q(5));`	Creates temporary common subexpression `zz232`. It stores `sin(q(5))` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
242	`zz233 = sin(q(6));`	Creates temporary common subexpression `zz233`. It stores `sin(q(6))` so the generated expressions are shorter and faster to evaluate.
243	`zz234 = -zz227.*zz232.*zz233 + zz231;`	Creates temporary common subexpression `zz234`. It stores `-zz227.*zz232.*zz233 + zz231` so the generated expressions are shorter and faster to evaluate.
244	`zz235 = zz229.*zz233;`	Creates temporary common subexpression `zz235`. It stores `zz229.*zz233` so the generated expressions are shorter and faster to evaluate.
245	`zz236 = zz227.*zz230;`	Creates temporary common subexpression `zz236`. It stores `zz227.*zz230` so the generated expressions are shorter and faster to evaluate.
246	`zz237 = zz232.*zz236 + zz235;`	Creates temporary common subexpression `zz237`. It stores `zz232.*zz236 + zz235` so the generated expressions are shorter and faster to evaluate.
247	`zz238 = -zz227.*zz233 + zz231.*zz232;`	Creates temporary common subexpression `zz238`. It stores `-zz227.*zz233 + zz231.*zz232` so the generated expressions are shorter and faster to evaluate.
248	`zz239 = zz232.*zz235 + zz236;`	Creates temporary common subexpression `zz239`. It stores `zz232.*zz235 + zz236` so the generated expressions are shorter and faster to evaluate.
249	`zz240 = zz228.*zz233;`	Creates temporary common subexpression `zz240`. It stores `zz228.*zz233` so the generated expressions are shorter and faster to evaluate.
250	`zz241 = zz228.*zz230;`	Creates temporary common subexpression `zz241`. It stores `zz228.*zz230` so the generated expressions are shorter and faster to evaluate.
251	`zz242 = 1./zz228;`	Creates temporary common subexpression `zz242`. It stores `1./zz228` so the generated expressions are shorter and faster to evaluate.
252	`zz243 = u(5).*zz233;`	Creates temporary common subexpression `zz243`. It stores `u(5).*zz233` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
253	`zz244 = u(6).*zz230;`	Creates temporary common subexpression `zz244`. It stores `u(6).*zz230` so the generated expressions are shorter and faster to evaluate.
254	`zz245 = tan(q(5));`	Creates temporary common subexpression `zz245`. It stores `tan(q(5))` so the generated expressions are shorter and faster to evaluate.
255	`zz246 = IN(32).*zz232;`	Creates temporary common subexpression `zz246`. It stores `IN(32).*zz232` so the generated expressions are shorter and faster to evaluate.
256	`zz247 = IN(34).*zz232;`	Creates temporary common subexpression `zz247`. It stores `IN(34).*zz232` so the generated expressions are shorter and faster to evaluate.
257	`zz248 = P(14).*zz232;`	Creates temporary common subexpression `zz248`. It stores `P(14).*zz232` so the generated expressions are shorter and faster to evaluate.
258	`zz249 = u(9).^2;`	Creates temporary common subexpression `zz249`. It stores `u(9).^2` so the generated expressions are shorter and faster to evaluate.
259	`zz250 = P(37).*u(9) + u(9).*zz88;`	Creates temporary common subexpression `zz250`. It stores `P(37).*u(9) + u(9).*zz88` so the generated expressions are shorter and faster to evaluate.
260	`zz251 = P(34).*u(9) + u(9).*zz73;`	Creates temporary common subexpression `zz251`. It stores `P(34).*u(9) + u(9).*zz73` so the generated expressions are shorter and faster to evaluate.
261	`zz252 = u(3) + u(4).*zz25 - u(5).*zz53 + u(9).*zz89;`	Creates temporary common subexpression `zz252`. It stores `u(3) + u(4).*zz25 - u(5).*zz53 + u(9).*zz89` so the generated expressions are shorter and faster to evaluate.
262	`zz253 = u(2) - u(4).*zz4 + u(6).*zz53 + u(9).*zz74;`	Creates temporary common subexpression `zz253`. It stores `u(2) - u(4).*zz4 + u(6).*zz53 + u(9).*zz74` so the generated expressions are shorter and faster to evaluate.
263	`zz254 = u(5).*zz250 + u(5).*zz252 - u(6).*zz251 - u(6).*zz253 + zz249.*zz45;`	Creates temporary common subexpression `zz254`. It stores `u(5).*zz250 + u(5).*zz252 - u(6).*zz251 - u(6).*zz253 + zz249.*zz45` so the generated expressions are shorter and faster to evaluate.
264	`zz255 = u(10).^2;`	Creates temporary common subexpression `zz255`. It stores `u(10).^2` so the generated expressions are shorter and faster to evaluate.
265	`zz256 = P(37).*u(10) + u(10).*zz91;`	Creates temporary common subexpression `zz256`. It stores `P(37).*u(10) + u(10).*zz91` so the generated expressions are shorter and faster to evaluate.
266	`zz257 = P(34).*u(10) + u(10).*zz76;`	Creates temporary common subexpression `zz257`. It stores `P(34).*u(10) + u(10).*zz76` so the generated expressions are shorter and faster to evaluate.
267	`zz258 = u(3) + u(4).*zz23 - u(5).*zz55 + u(10).*zz92;`	Creates temporary common subexpression `zz258`. It stores `u(3) + u(4).*zz23 - u(5).*zz55 + u(10).*zz92` so the generated expressions are shorter and faster to evaluate.
268	`zz259 = u(2) - u(4).*zz7 + u(6).*zz55 + u(10).*zz77;`	Creates temporary common subexpression `zz259`. It stores `u(2) - u(4).*zz7 + u(6).*zz55 + u(10).*zz77` so the generated expressions are shorter and faster to evaluate.
269	`zz260 = u(5).*zz256 + u(5).*zz258 - u(6).*zz257 - u(6).*zz259 + zz255.*zz45;`	Creates temporary common subexpression `zz260`. It stores `u(5).*zz256 + u(5).*zz258 - u(6).*zz257 - u(6).*zz259 + zz255.*zz45` so the generated expressions are shorter and faster to evaluate.
270	`zz261 = IN(2).*P(27);`	Creates temporary common subexpression `zz261`. It stores `IN(2).*P(27)` so the generated expressions are shorter and faster to evaluate.
271	`zz262 = P(41).*zz261;`	Creates temporary common subexpression `zz262`. It stores `P(41).*zz261` so the generated expressions are shorter and faster to evaluate.
272	`zz263 = u(7).^2;`	Creates temporary common subexpression `zz263`. It stores `u(7).^2` so the generated expressions are shorter and faster to evaluate.
273	`zz264 = P(36).*u(7) + u(7).*zz11 + u(7).*zz80;`	Creates temporary common subexpression `zz264`. It stores `P(36).*u(7) + u(7).*zz11 + u(7).*zz80` so the generated expressions are shorter and faster to evaluate.
274	`zz265 = P(32).*u(7) + u(7).*zz28 + u(7).*zz65;`	Creates temporary common subexpression `zz265`. It stores `P(32).*u(7) + u(7).*zz28 + u(7).*zz65` so the generated expressions are shorter and faster to evaluate.
275	`zz266 = P(27).*q(7);`	Creates temporary common subexpression `zz266`. It stores `P(27).*q(7)` so the generated expressions are shorter and faster to evaluate.
276	`zz267 = 2*IN(1).*zz13;`	Creates temporary common subexpression `zz267`. It stores `2*IN(1).*zz13` so the generated expressions are shorter and faster to evaluate.
277	`zz268 = P(48).*zz267 + zz12;`	Creates temporary common subexpression `zz268`. It stores `P(48).*zz267 + zz12` so the generated expressions are shorter and faster to evaluate.
278	`zz269 = IN(2).*(P(47).*zz266 + zz268) + u(3) + u(4).*zz31 - u(5).*zz59 + u(7).*zz82;`	Creates temporary common subexpression `zz269`. It stores `IN(2).*(P(47).*zz266 + zz268) + u(3) + u(4).*zz31 - u(5).*zz59 + u(7).*zz82` so the generated expressions are shorter and faster to evaluate.
279	`zz270 = P(45).*zz267 + zz29;`	Creates temporary common subexpression `zz270`. It stores `P(45).*zz267 + zz29` so the generated expressions are shorter and faster to evaluate.
280	`zz271 = IN(2).*(P(44).*zz266 + zz270) + u(2) - u(4).*zz16 + u(6).*zz59 + u(7).*zz67;`	Creates temporary common subexpression `zz271`. It stores `IN(2).*(P(44).*zz266 + zz270) + u(2) - u(4).*zz16 + u(6).*zz59 + u(7).*zz67` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
281	`zz272 = u(5).*zz264 + u(5).*zz269 - u(6).*zz265 - u(6).*zz271 + u(7).*zz262 + zz263.*zz36;`	Creates temporary common subexpression `zz272`. It stores `u(5).*zz264 + u(5).*zz269 - u(6).*zz265 - u(6).*zz271 + u(7).*zz262 + zz263.*zz36` so the generated expressions are shorter and faster to evaluate.
282	`zz273 = u(8).^2;`	Creates temporary common subexpression `zz273`. It stores `u(8).^2` so the generated expressions are shorter and faster to evaluate.
283	`zz274 = P(36).*u(8) + u(8).*zz11 + u(8).*zz84;`	Creates temporary common subexpression `zz274`. It stores `P(36).*u(8) + u(8).*zz11 + u(8).*zz84` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
284	`zz275 = P(32).*u(8) + u(8).*zz28 + u(8).*zz69;`	Creates temporary common subexpression `zz275`. It stores `P(32).*u(8) + u(8).*zz28 + u(8).*zz69` so the generated expressions are shorter and faster to evaluate.
285	`zz276 = P(27).*q(8);`	Creates temporary common subexpression `zz276`. It stores `P(27).*q(8)` so the generated expressions are shorter and faster to evaluate.
286	`zz277 = IN(2).*(P(47).*zz276 + zz268) + u(3) + u(4).*zz33 - u(5).*zz61 + u(8).*zz85;`	Creates temporary common subexpression `zz277`. It stores `IN(2).*(P(47).*zz276 + zz268) + u(3) + u(4).*zz33 - u(5).*zz61 + u(8).*zz85` so the generated expressions are shorter and faster to evaluate.
287	`zz278 = IN(2).*(P(44).*zz276 + zz270) + u(2) - u(4).*zz19 + u(6).*zz61 + u(8).*zz70;`	Creates temporary common subexpression `zz278`. It stores `IN(2).*(P(44).*zz276 + zz270) + u(2) - u(4).*zz19 + u(6).*zz61 + u(8).*zz70` so the generated expressions are shorter and faster to evaluate.
288	`zz279 = u(5).*zz274 + u(5).*zz277 - u(6).*zz275 - u(6).*zz278 + u(8).*zz262 + zz273.*zz36;`	Creates temporary common subexpression `zz279`. It stores `u(5).*zz274 + u(5).*zz277 - u(6).*zz275 - u(6).*zz278 + u(8).*zz262 + zz273.*zz36` so the generated expressions are shorter and faster to evaluate.
289	`zz280 = P(1).*P(14);`	Creates temporary common subexpression `zz280`. It stores `P(1).*P(14)` so the generated expressions are shorter and faster to evaluate.
290	`zz281 = P(14).*zz0;`	Creates temporary common subexpression `zz281`. It stores `P(14).*zz0` so the generated expressions are shorter and faster to evaluate.
291	`zz282 = P(30).*u(9) + u(9).*zz46;`	Creates temporary common subexpression `zz282`. It stores `P(30).*u(9) + u(9).*zz46` so the generated expressions are shorter and faster to evaluate.
292	`zz283 = u(1) + u(5).*zz4 - u(6).*zz25 + u(9).*zz47;`	Creates temporary common subexpression `zz283`. It stores `u(1) + u(5).*zz4 - u(6).*zz25 + u(9).*zz47` so the generated expressions are shorter and faster to evaluate.
293	`zz284 = -u(4).*zz250 - u(4).*zz252 + u(6).*zz282 + u(6).*zz283 + zz249.*zz72;`	Creates temporary common subexpression `zz284`. It stores `-u(4).*zz250 - u(4).*zz252 + u(6).*zz282 + u(6).*zz283 + zz249.*zz72` so the generated expressions are shorter and faster to evaluate.
294	`zz285 = P(30).*u(10) + u(10).*zz49;`	Creates temporary common subexpression `zz285`. It stores `P(30).*u(10) + u(10).*zz49` so the generated expressions are shorter and faster to evaluate.
295	`zz286 = u(1) + u(5).*zz7 - u(6).*zz23 + u(10).*zz50;`	Creates temporary common subexpression `zz286`. It stores `u(1) + u(5).*zz7 - u(6).*zz23 + u(10).*zz50` so the generated expressions are shorter and faster to evaluate.
296	`zz287 = -u(4).*zz256 - u(4).*zz258 + u(6).*zz285 + u(6).*zz286 + zz255.*zz72;`	Creates temporary common subexpression `zz287`. It stores `-u(4).*zz256 - u(4).*zz258 + u(6).*zz285 + u(6).*zz286 + zz255.*zz72` so the generated expressions are shorter and faster to evaluate.
297	`zz288 = P(44).*zz261;`	Creates temporary common subexpression `zz288`. It stores `P(44).*zz261` so the generated expressions are shorter and faster to evaluate.
298	`zz289 = P(28).*u(7) + u(7).*zz37 + u(7).*zz38;`	Creates temporary common subexpression `zz289`. It stores `P(28).*u(7) + u(7).*zz37 + u(7).*zz38` so the generated expressions are shorter and faster to evaluate.
299	`zz290 = P(42).*zz267 + zz57;`	Creates temporary common subexpression `zz290`. It stores `P(42).*zz267 + zz57` so the generated expressions are shorter and faster to evaluate.
300	`zz291 = IN(2).*(P(41).*zz266 + zz290) + u(1) + u(5).*zz16 - u(6).*zz31 + u(7).*zz40;`	Creates temporary common subexpression `zz291`. It stores `IN(2).*(P(41).*zz266 + zz290) + u(1) + u(5).*zz16 - u(6).*zz31 + u(7).*zz40` so the generated expressions are shorter and faster to evaluate.
301	`zz292 = -u(4).*zz264 - u(4).*zz269 + u(6).*zz289 + u(6).*zz291 + u(7).*zz288 + zz263.*zz64;`	Creates temporary common subexpression `zz292`. It stores `-u(4).*zz264 - u(4).*zz269 + u(6).*zz289 + u(6).*zz291 + u(7).*zz288 + zz263.*zz64` so the generated expressions are shorter and faster to evaluate.
302	`zz293 = P(28).*u(8) + u(8).*zz38 + u(8).*zz42;`	Creates temporary common subexpression `zz293`. It stores `P(28).*u(8) + u(8).*zz38 + u(8).*zz42` so the generated expressions are shorter and faster to evaluate.
303	`zz294 = IN(2).*(P(41).*zz276 + zz290) + u(1) + u(5).*zz19 - u(6).*zz33 + u(8).*zz43;`	Creates temporary common subexpression `zz294`. It stores `IN(2).*(P(41).*zz276 + zz290) + u(1) + u(5).*zz19 - u(6).*zz33 + u(8).*zz43` so the generated expressions are shorter and faster to evaluate.
304	`zz295 = -u(4).*zz274 - u(4).*zz277 + u(6).*zz293 + u(6).*zz294 + u(8).*zz288 + zz273.*zz64;`	Creates temporary common subexpression `zz295`. It stores `-u(4).*zz274 - u(4).*zz277 + u(6).*zz293 + u(6).*zz294 + u(8).*zz288 + zz273.*zz64` so the generated expressions are shorter and faster to evaluate.
305	`zz296 = u(4).*zz251 + u(4).*zz253 - u(5).*zz282 - u(5).*zz283 + zz249.*zz87;`	Creates temporary common subexpression `zz296`. It stores `u(4).*zz251 + u(4).*zz253 - u(5).*zz282 - u(5).*zz283 + zz249.*zz87` so the generated expressions are shorter and faster to evaluate.
306	`zz297 = u(4).*zz257 + u(4).*zz259 - u(5).*zz285 - u(5).*zz286 + zz255.*zz87;`	Creates temporary common subexpression `zz297`. It stores `u(4).*zz257 + u(4).*zz259 - u(5).*zz285 - u(5).*zz286 + zz255.*zz87` so the generated expressions are shorter and faster to evaluate.
307	`zz298 = P(47).*zz261;`	Creates temporary common subexpression `zz298`. It stores `P(47).*zz261` so the generated expressions are shorter and faster to evaluate.
308	`zz299 = u(4).*zz265 + u(4).*zz271 - u(5).*zz289 - u(5).*zz291 + u(7).*zz298 + zz263.*zz79;`	Creates temporary common subexpression `zz299`. It stores `u(4).*zz265 + u(4).*zz271 - u(5).*zz289 - u(5).*zz291 + u(7).*zz298 + zz263.*zz79` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
309	`zz300 = u(4).*zz275 + u(4).*zz278 - u(5).*zz293 - u(5).*zz294 + u(8).*zz298 + zz273.*zz79;`	Creates temporary common subexpression `zz300`. It stores `u(4).*zz275 + u(4).*zz278 - u(5).*zz293 - u(5).*zz294 + u(8).*zz298 + zz273.*zz79` so the generated expressions are shorter and faster to evaluate.
310	`zz301 = u(5).*u(6);`	Creates temporary common subexpression `zz301`. It stores `u(5).*u(6)` so the generated expressions are shorter and faster to evaluate.
311	`zz302 = P(5).*u(6) + P(6).*u(4);`	Creates temporary common subexpression `zz302`. It stores `P(5).*u(6) + P(6).*u(4)` so the generated expressions are shorter and faster to evaluate.
312	`zz303 = P(14).*zz241;`	Creates temporary common subexpression `zz303`. It stores `P(14).*zz241` so the generated expressions are shorter and faster to evaluate.
313	`zz304 = P(14).*zz240;`	Creates temporary common subexpression `zz304`. It stores `P(14).*zz240` so the generated expressions are shorter and faster to evaluate.
314	`zz305 = q(7) - q(8);`	Creates temporary common subexpression `zz305`. It stores `q(7) - q(8)` so the generated expressions are shorter and faster to evaluate.
315	`zz306 = u(7) - u(8);`	Creates temporary common subexpression `zz306`. It stores `u(7) - u(8)` so the generated expressions are shorter and faster to evaluate.
316	`zz307 = P(15).*q(7) + P(17).*u(7) + P(19).*zz305 + P(21).*zz306 + P(23);`	Creates temporary common subexpression `zz307`. It stores `P(15).*q(7) + P(17).*u(7) + P(19).*zz305 + P(21).*zz306 + P(23)` so the generated expressions are shorter and faster to evaluate.
317	`zz308 = P(15).*q(8) + P(17).*u(8) - P(19).*zz305 - P(21).*zz306 + P(24);`	Creates temporary common subexpression `zz308`. It stores `P(15).*q(8) + P(17).*u(8) - P(19).*zz305 - P(21).*zz306 + P(24)` so the generated expressions are shorter and faster to evaluate.
318	`zz309 = q(9) - q(10);`	Creates temporary common subexpression `zz309`. It stores `q(9) - q(10)` so the generated expressions are shorter and faster to evaluate.
319	`zz310 = u(9) - u(10);`	Creates temporary common subexpression `zz310`. It stores `u(9) - u(10)` so the generated expressions are shorter and faster to evaluate.
320	`zz311 = P(16).*q(9) + P(18).*u(9) + P(20).*zz309 + P(22).*zz310 + P(25);`	Creates temporary common subexpression `zz311`. It stores `P(16).*q(9) + P(18).*u(9) + P(20).*zz309 + P(22).*zz310 + P(25)` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
321	`zz312 = P(16).*q(10) + P(18).*u(10) - P(20).*zz309 - P(22).*zz310 + P(26);`	Creates temporary common subexpression `zz312`. It stores `P(16).*q(10) + P(18).*u(10) - P(20).*zz309 - P(22).*zz310 + P(26)` so the generated expressions are shorter and faster to evaluate.
322	`zz313 = -zz312;`	Creates temporary common subexpression `zz313`. It stores `-zz312` so the generated expressions are shorter and faster to evaluate.
323	`zz314 = -zz311;`	Creates temporary common subexpression `zz314`. It stores `-zz311` so the generated expressions are shorter and faster to evaluate.
324	`zz315 = -zz307;`	Creates temporary common subexpression `zz315`. It stores `-zz307` so the generated expressions are shorter and faster to evaluate.
325	`zz316 = -zz308;`	Creates temporary common subexpression `zz316`. It stores `-zz308` so the generated expressions are shorter and faster to evaluate.
326	`zz317 = 2*P(63);`	Creates temporary common subexpression `zz317`. It stores `2*P(63)` so the generated expressions are shorter and faster to evaluate.
327	`zz318 = 2*P(54);`	Creates temporary common subexpression `zz318`. It stores `2*P(54)` so the generated expressions are shorter and faster to evaluate.
328	`zz319 = P(72).*zz165;`	Creates temporary common subexpression `zz319`. It stores `P(72).*zz165` so the generated expressions are shorter and faster to evaluate.
329	`zz320 = P(71).*u(9) + u(9).*zz319;`	Creates temporary common subexpression `zz320`. It stores `P(71).*u(9) + u(9).*zz319` so the generated expressions are shorter and faster to evaluate.
330	`zz321 = P(53).*u(9) + u(6) + u(9).*zz169;`	Creates temporary common subexpression `zz321`. It stores `P(53).*u(9) + u(6) + u(9).*zz169` so the generated expressions are shorter and faster to evaluate.
331	`zz322 = u(4).*u(6);`	Creates temporary common subexpression `zz322`. It stores `u(4).*u(6)` so the generated expressions are shorter and faster to evaluate.
332	`zz323 = u(4).*zz99;`	Creates temporary common subexpression `zz323`. It stores `u(4).*zz99` so the generated expressions are shorter and faster to evaluate.
333	`zz324 = zz101.*zz320.*zz321 + zz132.*zz249.*zz318 - zz132.*zz320.*(u(5).*zz124 - zz323) - zz134.*(u(5).*zz321 - zz301) + zz195.*(-u(4).*zz321 + zz322) + zz249.*zz317;`	Creates temporary common subexpression `zz324`. It stores `zz101.*zz320.*zz321 + zz132.*zz249.*zz318 - zz132.*zz320.*(u(5).*zz124 - zz323) - zz134.*(u(5).*zz321 - zz3...` so the generated expressions are shorter and faster to evaluate.
334	`zz325 = P(7).*zz324;`	Creates temporary common subexpression `zz325`. It stores `P(7).*zz324` so the generated expressions are shorter and faster to evaluate.
335	`zz326 = P(72).*zz172;`	Creates temporary common subexpression `zz326`. It stores `P(72).*zz172` so the generated expressions are shorter and faster to evaluate.
336	`zz327 = P(71).*u(10) + u(10).*zz326;`	Creates temporary common subexpression `zz327`. It stores `P(71).*u(10) + u(10).*zz326` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
337	`zz328 = P(53).*u(10) + u(6) + u(10).*zz176;`	Creates temporary common subexpression `zz328`. It stores `P(53).*u(10) + u(6) + u(10).*zz176` so the generated expressions are shorter and faster to evaluate.
338	`zz329 = u(4).*zz104;`	Creates temporary common subexpression `zz329`. It stores `u(4).*zz104` so the generated expressions are shorter and faster to evaluate.
339	`zz330 = zz106.*zz327.*zz328 + zz136.*zz255.*zz318 - zz136.*zz327.*(u(5).*zz126 - zz329) - zz138.*(u(5).*zz328 - zz301) + zz197.*(-u(4).*zz328 + zz322) + zz255.*zz317;`	Creates temporary common subexpression `zz330`. It stores `zz106.*zz327.*zz328 + zz136.*zz255.*zz318 - zz136.*zz327.*(u(5).*zz126 - zz329) - zz138.*(u(5).*zz328 - zz3...` so the generated expressions are shorter and faster to evaluate.
340	`zz331 = P(7).*zz330;`	Creates temporary common subexpression `zz331`. It stores `P(7).*zz330` so the generated expressions are shorter and faster to evaluate.
341	`zz332 = u(7).*zz261;`	Creates temporary common subexpression `zz332`. It stores `u(7).*zz261` so the generated expressions are shorter and faster to evaluate.
342	`zz333 = 2*zz263;`	Creates temporary common subexpression `zz333`. It stores `2*zz263` so the generated expressions are shorter and faster to evaluate.
343	`zz334 = P(50).*u(7) + u(6) + u(7).*zz108 + u(7).*zz154;`	Creates temporary common subexpression `zz334`. It stores `P(50).*u(7) + u(6) + u(7).*zz108 + u(7).*zz154` so the generated expressions are shorter and faster to evaluate.
344	`zz335 = P(69).*zz148;`	Creates temporary common subexpression `zz335`. It stores `P(69).*zz148` so the generated expressions are shorter and faster to evaluate.
345	`zz336 = P(75).*zz267 + zz114;`	Creates temporary common subexpression `zz336`. It stores `P(75).*zz267 + zz114` so the generated expressions are shorter and faster to evaluate.
346	`zz337 = IN(2).*(P(74).*zz266 + zz336) + P(68).*u(7) + u(7).*zz113 + u(7).*zz335;`	Creates temporary common subexpression `zz337`. It stores `IN(2).*(P(74).*zz266 + zz336) + P(68).*u(7) + u(7).*zz113 + u(7).*zz335` so the generated expressions are shorter and faster to evaluate.
347	`zz338 = P(57).*zz267 + zz109;`	Creates temporary common subexpression `zz338`. It stores `P(57).*zz267 + zz109` so the generated expressions are shorter and faster to evaluate.
348	`zz339 = IN(2).*(P(56).*zz266 + zz338) + zz334;`	Creates temporary common subexpression `zz339`. It stores `IN(2).*(P(56).*zz266 + zz338) + zz334` so the generated expressions are shorter and faster to evaluate.
349	`zz340 = u(4).*zz112;`	Creates temporary common subexpression `zz340`. It stores `u(4).*zz112` so the generated expressions are shorter and faster to evaluate.
350	`zz341 = P(60).*zz333 + P(65).*zz332 + zz117.*zz334.*zz337 - zz140.*zz337.*(u(5).*zz128 - zz340) + zz140.*(P(51).*zz333 + P(56).*zz332) - zz142.*(u(5).*zz339 - zz301) + zz191.*(-u(4).*zz339 + zz322);`	Creates temporary common subexpression `zz341`. It stores `P(60).*zz333 + P(65).*zz332 + zz117.*zz334.*zz337 - zz140.*zz337.*(u(5).*zz128 - zz340) + zz140.*(P(51).*zz...` so the generated expressions are shorter and faster to evaluate.
351	`zz342 = P(7).*zz341;`	Creates temporary common subexpression `zz342`. It stores `P(7).*zz341` so the generated expressions are shorter and faster to evaluate.
352	`zz343 = u(8).*zz261;`	Creates temporary common subexpression `zz343`. It stores `u(8).*zz261` so the generated expressions are shorter and faster to evaluate.
353	`zz344 = 2*zz273;`	Creates temporary common subexpression `zz344`. It stores `2*zz273` so the generated expressions are shorter and faster to evaluate.
354	`zz345 = P(50).*u(8) + u(6) + u(8).*zz108 + u(8).*zz162;`	Creates temporary common subexpression `zz345`. It stores `P(50).*u(8) + u(6) + u(8).*zz108 + u(8).*zz162` so the generated expressions are shorter and faster to evaluate.
355	`zz346 = P(69).*zz158;`	Creates temporary common subexpression `zz346`. It stores `P(69).*zz158` so the generated expressions are shorter and faster to evaluate.
356	`zz347 = IN(2).*(P(74).*zz276 + zz336) + P(68).*u(8) + u(8).*zz113 + u(8).*zz346;`	Creates temporary common subexpression `zz347`. It stores `IN(2).*(P(74).*zz276 + zz336) + P(68).*u(8) + u(8).*zz113 + u(8).*zz346` so the generated expressions are shorter and faster to evaluate.
357	`zz348 = IN(2).*(P(56).*zz276 + zz338) + zz345;`	Creates temporary common subexpression `zz348`. It stores `IN(2).*(P(56).*zz276 + zz338) + zz345` so the generated expressions are shorter and faster to evaluate.
358	`zz349 = u(4).*zz120;`	Creates temporary common subexpression `zz349`. It stores `u(4).*zz120` so the generated expressions are shorter and faster to evaluate.
359	`zz350 = P(60).*zz344 + P(65).*zz343 + zz122.*zz345.*zz347 - zz144.*zz347.*(u(5).*zz130 - zz349) + zz144.*(P(51).*zz344 + P(56).*zz343) - zz146.*(u(5).*zz348 - zz301) + zz193.*(-u(4).*zz348 + zz322);`	Creates temporary common subexpression `zz350`. It stores `P(60).*zz344 + P(65).*zz343 + zz122.*zz345.*zz347 - zz144.*zz347.*(u(5).*zz130 - zz349) + zz144.*(P(51).*zz...` so the generated expressions are shorter and faster to evaluate.
360	`zz351 = P(7).*zz350;`	Creates temporary common subexpression `zz351`. It stores `P(7).*zz350` so the generated expressions are shorter and faster to evaluate.
361	`zz352 = cos(q(13));`	Creates temporary common subexpression `zz352`. It stores `cos(q(13))` so the generated expressions are shorter and faster to evaluate.
362	`zz353 = P(61) + P(62).*q(9) + P(63).*zz2;`	Creates temporary common subexpression `zz353`. It stores `P(61) + P(62).*q(9) + P(63).*zz2` so the generated expressions are shorter and faster to evaluate.
363	`zz354 = cos(zz353);`	Creates temporary common subexpression `zz354`. It stores `cos(zz353)` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
364	`zz355 = sin(zz353);`	Creates temporary common subexpression `zz355`. It stores `sin(zz353)` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
365	`zz356 = zz355.*zz99;`	Creates temporary common subexpression `zz356`. It stores `zz355.*zz99` so the generated expressions are shorter and faster to evaluate.
366	`zz357 = zz124.*zz354 - zz132.*zz356;`	Creates temporary common subexpression `zz357`. It stores `zz124.*zz354 - zz132.*zz356` so the generated expressions are shorter and faster to evaluate.
367	`zz358 = sin(q(13));`	Creates temporary common subexpression `zz358`. It stores `sin(q(13))` so the generated expressions are shorter and faster to evaluate.
368	`zz359 = zz124.*zz355;`	Creates temporary common subexpression `zz359`. It stores `zz124.*zz355` so the generated expressions are shorter and faster to evaluate.
369	`zz360 = zz354.*zz99;`	Creates temporary common subexpression `zz360`. It stores `zz354.*zz99` so the generated expressions are shorter and faster to evaluate.
370	`zz361 = zz132.*zz360 + zz359;`	Creates temporary common subexpression `zz361`. It stores `zz132.*zz360 + zz359` so the generated expressions are shorter and faster to evaluate.
371	`zz362 = zz352.*zz357 - zz358.*zz361;`	Creates temporary common subexpression `zz362`. It stores `zz352.*zz357 - zz358.*zz361` so the generated expressions are shorter and faster to evaluate.
372	`zz363 = zz354.*zz358;`	Creates temporary common subexpression `zz363`. It stores `zz354.*zz358` so the generated expressions are shorter and faster to evaluate.
373	`zz364 = zz352.*zz355;`	Creates temporary common subexpression `zz364`. It stores `zz352.*zz355` so the generated expressions are shorter and faster to evaluate.
374	`zz365 = zz363 + zz364;`	Creates temporary common subexpression `zz365`. It stores `zz363 + zz364` so the generated expressions are shorter and faster to evaluate.
375	`zz366 = zz355.*zz358;`	Creates temporary common subexpression `zz366`. It stores `zz355.*zz358` so the generated expressions are shorter and faster to evaluate.
376	`zz367 = zz101.*zz352.*zz354 - zz101.*zz366;`	Creates temporary common subexpression `zz367`. It stores `zz101.*zz352.*zz354 - zz101.*zz366` so the generated expressions are shorter and faster to evaluate.
377	`zz368 = zz352.*zz361 + zz357.*zz358;`	Creates temporary common subexpression `zz368`. It stores `zz352.*zz361 + zz357.*zz358` so the generated expressions are shorter and faster to evaluate.
378	`zz369 = zz132.*zz359 + zz360;`	Creates temporary common subexpression `zz369`. It stores `zz132.*zz359 + zz360` so the generated expressions are shorter and faster to evaluate.
379	`zz370 = -zz124.*zz132.*zz354 + zz356;`	Creates temporary common subexpression `zz370`. It stores `-zz124.*zz132.*zz354 + zz356` so the generated expressions are shorter and faster to evaluate.
380	`zz371 = zz352.*zz370 + zz358.*zz369;`	Creates temporary common subexpression `zz371`. It stores `zz352.*zz370 + zz358.*zz369` so the generated expressions are shorter and faster to evaluate.
381	`zz372 = u(4).*zz368 + u(5).*zz371 + zz320.*zz365 + zz321.*zz367;`	Creates temporary common subexpression `zz372`. It stores `u(4).*zz368 + u(5).*zz371 + zz320.*zz365 + zz321.*zz367` so the generated expressions are shorter and faster to evaluate.
382	`zz373 = P(62).*u(9) + u(9).*zz166;`	Creates temporary common subexpression `zz373`. It stores `P(62).*u(9) + u(9).*zz166` so the generated expressions are shorter and faster to evaluate.
383	`zz374 = u(5).*zz101.*zz124 + u(13) - zz101.*zz323 + zz132.*zz321 + zz373;`	Creates temporary common subexpression `zz374`. It stores `u(5).*zz101.*zz124 + u(13) - zz101.*zz323 + zz132.*zz321 + zz373` so the generated expressions are shorter and faster to evaluate.
384	`zz375 = P(7).*zz374;`	Creates temporary common subexpression `zz375`. It stores `P(7).*zz374` so the generated expressions are shorter and faster to evaluate.
385	`zz376 = zz372.*zz375;`	Creates temporary common subexpression `zz376`. It stores `zz372.*zz375` so the generated expressions are shorter and faster to evaluate.
386	`zz377 = cos(q(14));`	Creates temporary common subexpression `zz377`. It stores `cos(q(14))` so the generated expressions are shorter and faster to evaluate.
387	`zz378 = P(61) + P(62).*q(10) + P(63).*zz6;`	Creates temporary common subexpression `zz378`. It stores `P(61) + P(62).*q(10) + P(63).*zz6` so the generated expressions are shorter and faster to evaluate.
388	`zz379 = cos(zz378);`	Creates temporary common subexpression `zz379`. It stores `cos(zz378)` so the generated expressions are shorter and faster to evaluate.
389	`zz380 = sin(zz378);`	Creates temporary common subexpression `zz380`. It stores `sin(zz378)` so the generated expressions are shorter and faster to evaluate.
390	`zz381 = zz104.*zz380;`	Creates temporary common subexpression `zz381`. It stores `zz104.*zz380` so the generated expressions are shorter and faster to evaluate.
391	`zz382 = zz126.*zz379 - zz136.*zz381;`	Creates temporary common subexpression `zz382`. It stores `zz126.*zz379 - zz136.*zz381` so the generated expressions are shorter and faster to evaluate.
392	`zz383 = sin(q(14));`	Creates temporary common subexpression `zz383`. It stores `sin(q(14))` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
393	`zz384 = zz126.*zz380;`	Creates temporary common subexpression `zz384`. It stores `zz126.*zz380` so the generated expressions are shorter and faster to evaluate.
394	`zz385 = zz104.*zz379;`	Creates temporary common subexpression `zz385`. It stores `zz104.*zz379` so the generated expressions are shorter and faster to evaluate.
395	`zz386 = zz136.*zz385 + zz384;`	Creates temporary common subexpression `zz386`. It stores `zz136.*zz385 + zz384` so the generated expressions are shorter and faster to evaluate.
396	`zz387 = zz377.*zz382 - zz383.*zz386;`	Creates temporary common subexpression `zz387`. It stores `zz377.*zz382 - zz383.*zz386` so the generated expressions are shorter and faster to evaluate.
397	`zz388 = zz379.*zz383;`	Creates temporary common subexpression `zz388`. It stores `zz379.*zz383` so the generated expressions are shorter and faster to evaluate.
398	`zz389 = zz377.*zz380;`	Creates temporary common subexpression `zz389`. It stores `zz377.*zz380` so the generated expressions are shorter and faster to evaluate.
399	`zz390 = zz388 + zz389;`	Creates temporary common subexpression `zz390`. It stores `zz388 + zz389` so the generated expressions are shorter and faster to evaluate.
400	`zz391 = zz380.*zz383;`	Creates temporary common subexpression `zz391`. It stores `zz380.*zz383` so the generated expressions are shorter and faster to evaluate.
401	`zz392 = zz106.*zz377.*zz379 - zz106.*zz391;`	Creates temporary common subexpression `zz392`. It stores `zz106.*zz377.*zz379 - zz106.*zz391` so the generated expressions are shorter and faster to evaluate.
402	`zz393 = zz377.*zz386 + zz382.*zz383;`	Creates temporary common subexpression `zz393`. It stores `zz377.*zz386 + zz382.*zz383` so the generated expressions are shorter and faster to evaluate.
403	`zz394 = zz136.*zz384 + zz385;`	Creates temporary common subexpression `zz394`. It stores `zz136.*zz384 + zz385` so the generated expressions are shorter and faster to evaluate.
404	`zz395 = -zz126.*zz136.*zz379 + zz381;`	Creates temporary common subexpression `zz395`. It stores `-zz126.*zz136.*zz379 + zz381` so the generated expressions are shorter and faster to evaluate.
405	`zz396 = zz377.*zz395 + zz383.*zz394;`	Creates temporary common subexpression `zz396`. It stores `zz377.*zz395 + zz383.*zz394` so the generated expressions are shorter and faster to evaluate.
406	`zz397 = u(4).*zz393 + u(5).*zz396 + zz327.*zz390 + zz328.*zz392;`	Creates temporary common subexpression `zz397`. It stores `u(4).*zz393 + u(5).*zz396 + zz327.*zz390 + zz328.*zz392` so the generated expressions are shorter and faster to evaluate.
407	`zz398 = P(62).*u(10) + u(10).*zz173;`	Creates temporary common subexpression `zz398`. It stores `P(62).*u(10) + u(10).*zz173` so the generated expressions are shorter and faster to evaluate.
408	`zz399 = u(5).*zz106.*zz126 + u(14) - zz106.*zz329 + zz136.*zz328 + zz398;`	Creates temporary common subexpression `zz399`. It stores `u(5).*zz106.*zz126 + u(14) - zz106.*zz329 + zz136.*zz328 + zz398` so the generated expressions are shorter and faster to evaluate.
409	`zz400 = P(7).*zz399;`	Creates temporary common subexpression `zz400`. It stores `P(7).*zz399` so the generated expressions are shorter and faster to evaluate.
410	`zz401 = zz397.*zz400;`	Creates temporary common subexpression `zz401`. It stores `zz397.*zz400` so the generated expressions are shorter and faster to evaluate.
411	`zz402 = zz352.*zz354 - zz366;`	Creates temporary common subexpression `zz402`. It stores `zz352.*zz354 - zz366` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
412	`zz403 = -zz101.*zz363 - zz101.*zz364;`	Creates temporary common subexpression `zz403`. It stores `-zz101.*zz363 - zz101.*zz364` so the generated expressions are shorter and faster to evaluate.
413	`zz404 = zz352.*zz369 - zz358.*zz370;`	Creates temporary common subexpression `zz404`. It stores `zz352.*zz369 - zz358.*zz370` so the generated expressions are shorter and faster to evaluate.
414	`zz405 = zz375.*(u(4).*zz362 + u(5).*zz404 + zz320.*zz402 + zz321.*zz403);`	Creates temporary common subexpression `zz405`. It stores `zz375.*(u(4).*zz362 + u(5).*zz404 + zz320.*zz402 + zz321.*zz403)` so the generated expressions are shorter and faster to evaluate.
415	`zz406 = zz377.*zz379 - zz391;`	Creates temporary common subexpression `zz406`. It stores `zz377.*zz379 - zz391` so the generated expressions are shorter and faster to evaluate.
416	`zz407 = -zz106.*zz388 - zz106.*zz389;`	Creates temporary common subexpression `zz407`. It stores `-zz106.*zz388 - zz106.*zz389` so the generated expressions are shorter and faster to evaluate.
417	`zz408 = zz377.*zz394 - zz383.*zz395;`	Creates temporary common subexpression `zz408`. It stores `zz377.*zz394 - zz383.*zz395` so the generated expressions are shorter and faster to evaluate.
418	`zz409 = zz400.*(u(4).*zz387 + u(5).*zz408 + zz327.*zz406 + zz328.*zz407);`	Creates temporary common subexpression `zz409`. It stores `zz400.*(u(4).*zz387 + u(5).*zz408 + zz327.*zz406 + zz328.*zz407)` so the generated expressions are shorter and faster to evaluate.
419	`zz410 = cos(q(11));`	Creates temporary common subexpression `zz410`. It stores `cos(q(11))` so the generated expressions are shorter and faster to evaluate.
420	`zz411 = P(27).*P(64);`	Creates temporary common subexpression `zz411`. It stores `P(27).*P(64)` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
421	`zz412 = IN(1).*zz411 + P(58) + P(66).*zz14;`	Creates temporary common subexpression `zz412`. It stores `IN(1).*zz411 + P(58) + P(66).*zz14` so the generated expressions are shorter and faster to evaluate.
422	`zz413 = P(59).*q(7) + P(60).*zz9 + q(7).*zz150 + zz412;`	Creates temporary common subexpression `zz413`. It stores `P(59).*q(7) + P(60).*zz9 + q(7).*zz150 + zz412` so the generated expressions are shorter and faster to evaluate.
423	`zz414 = cos(zz413);`	Creates temporary common subexpression `zz414`. It stores `cos(zz413)` so the generated expressions are shorter and faster to evaluate.
424	`zz415 = sin(zz413);`	Creates temporary common subexpression `zz415`. It stores `sin(zz413)` so the generated expressions are shorter and faster to evaluate.
425	`zz416 = zz112.*zz415;`	Creates temporary common subexpression `zz416`. It stores `zz112.*zz415` so the generated expressions are shorter and faster to evaluate.
426	`zz417 = zz128.*zz414 - zz140.*zz416;`	Creates temporary common subexpression `zz417`. It stores `zz128.*zz414 - zz140.*zz416` so the generated expressions are shorter and faster to evaluate.
427	`zz418 = sin(q(11));`	Creates temporary common subexpression `zz418`. It stores `sin(q(11))` so the generated expressions are shorter and faster to evaluate.
428	`zz419 = zz128.*zz415;`	Creates temporary common subexpression `zz419`. It stores `zz128.*zz415` so the generated expressions are shorter and faster to evaluate.
429	`zz420 = zz112.*zz414;`	Creates temporary common subexpression `zz420`. It stores `zz112.*zz414` so the generated expressions are shorter and faster to evaluate.
430	`zz421 = zz140.*zz420 + zz419;`	Creates temporary common subexpression `zz421`. It stores `zz140.*zz420 + zz419` so the generated expressions are shorter and faster to evaluate.
431	`zz422 = zz410.*zz417 - zz418.*zz421;`	Creates temporary common subexpression `zz422`. It stores `zz410.*zz417 - zz418.*zz421` so the generated expressions are shorter and faster to evaluate.
432	`zz423 = zz414.*zz418;`	Creates temporary common subexpression `zz423`. It stores `zz414.*zz418` so the generated expressions are shorter and faster to evaluate.
433	`zz424 = zz410.*zz415;`	Creates temporary common subexpression `zz424`. It stores `zz410.*zz415` so the generated expressions are shorter and faster to evaluate.
434	`zz425 = zz423 + zz424;`	Creates temporary common subexpression `zz425`. It stores `zz423 + zz424` so the generated expressions are shorter and faster to evaluate.
435	`zz426 = zz415.*zz418;`	Creates temporary common subexpression `zz426`. It stores `zz415.*zz418` so the generated expressions are shorter and faster to evaluate.
436	`zz427 = zz117.*zz410.*zz414 - zz117.*zz426;`	Creates temporary common subexpression `zz427`. It stores `zz117.*zz410.*zz414 - zz117.*zz426` so the generated expressions are shorter and faster to evaluate.
437	`zz428 = zz410.*zz421 + zz417.*zz418;`	Creates temporary common subexpression `zz428`. It stores `zz410.*zz421 + zz417.*zz418` so the generated expressions are shorter and faster to evaluate.
438	`zz429 = zz140.*zz419 + zz420;`	Creates temporary common subexpression `zz429`. It stores `zz140.*zz419 + zz420` so the generated expressions are shorter and faster to evaluate.
439	`zz430 = -zz128.*zz140.*zz414 + zz416;`	Creates temporary common subexpression `zz430`. It stores `-zz128.*zz140.*zz414 + zz416` so the generated expressions are shorter and faster to evaluate.
440	`zz431 = zz410.*zz430 + zz418.*zz429;`	Creates temporary common subexpression `zz431`. It stores `zz410.*zz430 + zz418.*zz429` so the generated expressions are shorter and faster to evaluate.
441	`zz432 = u(4).*zz428 + u(5).*zz431 + zz337.*zz425 + zz339.*zz427;`	Creates temporary common subexpression `zz432`. It stores `u(4).*zz428 + u(5).*zz431 + zz337.*zz425 + zz339.*zz427` so the generated expressions are shorter and faster to evaluate.
442	`zz433 = P(66).*zz267 + zz411;`	Creates temporary common subexpression `zz433`. It stores `P(66).*zz267 + zz411` so the generated expressions are shorter and faster to evaluate.
443	`zz434 = IN(2).*(P(65).*zz266 + zz433) + P(59).*u(7) + u(7).*zz149 + u(7).*zz150;`	Creates temporary common subexpression `zz434`. It stores `IN(2).*(P(65).*zz266 + zz433) + P(59).*u(7) + u(7).*zz149 + u(7).*zz150` so the generated expressions are shorter and faster to evaluate.
444	`zz435 = u(5).*zz117.*zz128 + u(11) - zz117.*zz340 + zz140.*zz339 + zz434;`	Creates temporary common subexpression `zz435`. It stores `u(5).*zz117.*zz128 + u(11) - zz117.*zz340 + zz140.*zz339 + zz434` so the generated expressions are shorter and faster to evaluate.
445	`zz436 = P(7).*zz435;`	Creates temporary common subexpression `zz436`. It stores `P(7).*zz435` so the generated expressions are shorter and faster to evaluate.
446	`zz437 = zz432.*zz436;`	Creates temporary common subexpression `zz437`. It stores `zz432.*zz436` so the generated expressions are shorter and faster to evaluate.
447	`zz438 = cos(q(12));`	Creates temporary common subexpression `zz438`. It stores `cos(q(12))` so the generated expressions are shorter and faster to evaluate.
448	`zz439 = P(59).*q(8) + P(60).*zz18 + q(8).*zz150 + zz412;`	Creates temporary common subexpression `zz439`. It stores `P(59).*q(8) + P(60).*zz18 + q(8).*zz150 + zz412` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
449	`zz440 = cos(zz439);`	Creates temporary common subexpression `zz440`. It stores `cos(zz439)` so the generated expressions are shorter and faster to evaluate.
450	`zz441 = sin(zz439);`	Creates temporary common subexpression `zz441`. It stores `sin(zz439)` so the generated expressions are shorter and faster to evaluate.
451	`zz442 = zz120.*zz441;`	Creates temporary common subexpression `zz442`. It stores `zz120.*zz441` so the generated expressions are shorter and faster to evaluate.
452	`zz443 = zz130.*zz440 - zz144.*zz442;`	Creates temporary common subexpression `zz443`. It stores `zz130.*zz440 - zz144.*zz442` so the generated expressions are shorter and faster to evaluate.
453	`zz444 = sin(q(12));`	Creates temporary common subexpression `zz444`. It stores `sin(q(12))` so the generated expressions are shorter and faster to evaluate.
454	`zz445 = zz130.*zz441;`	Creates temporary common subexpression `zz445`. It stores `zz130.*zz441` so the generated expressions are shorter and faster to evaluate.
455	`zz446 = zz120.*zz440;`	Creates temporary common subexpression `zz446`. It stores `zz120.*zz440` so the generated expressions are shorter and faster to evaluate.
456	`zz447 = zz144.*zz446 + zz445;`	Creates temporary common subexpression `zz447`. It stores `zz144.*zz446 + zz445` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
457	`zz448 = zz438.*zz443 - zz444.*zz447;`	Creates temporary common subexpression `zz448`. It stores `zz438.*zz443 - zz444.*zz447` so the generated expressions are shorter and faster to evaluate.
458	`zz449 = zz440.*zz444;`	Creates temporary common subexpression `zz449`. It stores `zz440.*zz444` so the generated expressions are shorter and faster to evaluate.
459	`zz450 = zz438.*zz441;`	Creates temporary common subexpression `zz450`. It stores `zz438.*zz441` so the generated expressions are shorter and faster to evaluate.
460	`zz451 = zz449 + zz450;`	Creates temporary common subexpression `zz451`. It stores `zz449 + zz450` so the generated expressions are shorter and faster to evaluate.
461	`zz452 = zz441.*zz444;`	Creates temporary common subexpression `zz452`. It stores `zz441.*zz444` so the generated expressions are shorter and faster to evaluate.
462	`zz453 = zz122.*zz438.*zz440 - zz122.*zz452;`	Creates temporary common subexpression `zz453`. It stores `zz122.*zz438.*zz440 - zz122.*zz452` so the generated expressions are shorter and faster to evaluate.
463	`zz454 = zz438.*zz447 + zz443.*zz444;`	Creates temporary common subexpression `zz454`. It stores `zz438.*zz447 + zz443.*zz444` so the generated expressions are shorter and faster to evaluate.
464	`zz455 = zz144.*zz445 + zz446;`	Creates temporary common subexpression `zz455`. It stores `zz144.*zz445 + zz446` so the generated expressions are shorter and faster to evaluate.
465	`zz456 = -zz130.*zz144.*zz440 + zz442;`	Creates temporary common subexpression `zz456`. It stores `-zz130.*zz144.*zz440 + zz442` so the generated expressions are shorter and faster to evaluate.
466	`zz457 = zz438.*zz456 + zz444.*zz455;`	Creates temporary common subexpression `zz457`. It stores `zz438.*zz456 + zz444.*zz455` so the generated expressions are shorter and faster to evaluate.
467	`zz458 = u(4).*zz454 + u(5).*zz457 + zz347.*zz451 + zz348.*zz453;`	Creates temporary common subexpression `zz458`. It stores `u(4).*zz454 + u(5).*zz457 + zz347.*zz451 + zz348.*zz453` so the generated expressions are shorter and faster to evaluate.
468	`zz459 = IN(2).*(P(65).*zz276 + zz433) + P(59).*u(8) + u(8).*zz150 + u(8).*zz159;`	Creates temporary common subexpression `zz459`. It stores `IN(2).*(P(65).*zz276 + zz433) + P(59).*u(8) + u(8).*zz150 + u(8).*zz159` so the generated expressions are shorter and faster to evaluate.
469	`zz460 = u(5).*zz122.*zz130 + u(12) - zz122.*zz349 + zz144.*zz348 + zz459;`	Creates temporary common subexpression `zz460`. It stores `u(5).*zz122.*zz130 + u(12) - zz122.*zz349 + zz144.*zz348 + zz459` so the generated expressions are shorter and faster to evaluate.
470	`zz461 = P(7).*zz460;`	Creates temporary common subexpression `zz461`. It stores `P(7).*zz460` so the generated expressions are shorter and faster to evaluate.
471	`zz462 = zz458.*zz461;`	Creates temporary common subexpression `zz462`. It stores `zz458.*zz461` so the generated expressions are shorter and faster to evaluate.
472	`zz463 = zz410.*zz414 - zz426;`	Creates temporary common subexpression `zz463`. It stores `zz410.*zz414 - zz426` so the generated expressions are shorter and faster to evaluate.
473	`zz464 = -zz117.*zz423 - zz117.*zz424;`	Creates temporary common subexpression `zz464`. It stores `-zz117.*zz423 - zz117.*zz424` so the generated expressions are shorter and faster to evaluate.
474	`zz465 = zz410.*zz429 - zz418.*zz430;`	Creates temporary common subexpression `zz465`. It stores `zz410.*zz429 - zz418.*zz430` so the generated expressions are shorter and faster to evaluate.
475	`zz466 = zz436.*(u(4).*zz422 + u(5).*zz465 + zz337.*zz463 + zz339.*zz464);`	Creates temporary common subexpression `zz466`. It stores `zz436.*(u(4).*zz422 + u(5).*zz465 + zz337.*zz463 + zz339.*zz464)` so the generated expressions are shorter and faster to evaluate.
476	`zz467 = zz438.*zz440 - zz452;`	Creates temporary common subexpression `zz467`. It stores `zz438.*zz440 - zz452` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
477	`zz468 = -zz122.*zz449 - zz122.*zz450;`	Creates temporary common subexpression `zz468`. It stores `-zz122.*zz449 - zz122.*zz450` so the generated expressions are shorter and faster to evaluate.
478	`zz469 = zz438.*zz455 - zz444.*zz456;`	Creates temporary common subexpression `zz469`. It stores `zz438.*zz455 - zz444.*zz456` so the generated expressions are shorter and faster to evaluate.
479	`zz470 = zz461.*(u(4).*zz448 + u(5).*zz469 + zz347.*zz467 + zz348.*zz468);`	Creates temporary common subexpression `zz470`. It stores `zz461.*(u(4).*zz448 + u(5).*zz469 + zz347.*zz467 + zz348.*zz468)` so the generated expressions are shorter and faster to evaluate.
480	`zz471 = P(3).*u(4) + P(6).*u(6);`	Creates temporary common subexpression `zz471`. It stores `P(3).*u(4) + P(6).*u(6)` so the generated expressions are shorter and faster to evaluate.
481	`zz472 = zz367.*zz405;`	Creates temporary common subexpression `zz472`. It stores `zz367.*zz405` so the generated expressions are shorter and faster to evaluate.
482	`zz473 = zz392.*zz409;`	Creates temporary common subexpression `zz473`. It stores `zz392.*zz409` so the generated expressions are shorter and faster to evaluate.
483	`zz474 = zz427.*zz466;`	Creates temporary common subexpression `zz474`. It stores `zz427.*zz466` so the generated expressions are shorter and faster to evaluate.
484	`zz475 = zz453.*zz470;`	Creates temporary common subexpression `zz475`. It stores `zz453.*zz470` so the generated expressions are shorter and faster to evaluate.
485	`zz476 = P(68) + zz113;`	Creates temporary common subexpression `zz476`. It stores `P(68) + zz113` so the generated expressions are shorter and faster to evaluate.
486	`zz477 = zz335 + zz476;`	Creates temporary common subexpression `zz477`. It stores `zz335 + zz476` so the generated expressions are shorter and faster to evaluate.
487	`zz478 = -IN(6).*zz142 + IN(7).*zz191 + IN(8).*zz140;`	Creates temporary common subexpression `zz478`. It stores `-IN(6).*zz142 + IN(7).*zz191 + IN(8).*zz140` so the generated expressions are shorter and faster to evaluate.
488	`zz479 = zz346 + zz476;`	Creates temporary common subexpression `zz479`. It stores `zz346 + zz476` so the generated expressions are shorter and faster to evaluate.
489	`zz480 = -IN(12).*zz146 + IN(13).*zz193 + IN(14).*zz144;`	Creates temporary common subexpression `zz480`. It stores `-IN(12).*zz146 + IN(13).*zz193 + IN(14).*zz144` so the generated expressions are shorter and faster to evaluate.
490	`zz481 = P(71) + zz319;`	Creates temporary common subexpression `zz481`. It stores `P(71) + zz319` so the generated expressions are shorter and faster to evaluate.
491	`zz482 = -IN(18).*zz134 + IN(19).*zz195 + IN(20).*zz132;`	Creates temporary common subexpression `zz482`. It stores `-IN(18).*zz134 + IN(19).*zz195 + IN(20).*zz132` so the generated expressions are shorter and faster to evaluate.
492	`zz483 = P(71) + zz326;`	Creates temporary common subexpression `zz483`. It stores `P(71) + zz326` so the generated expressions are shorter and faster to evaluate.
493	`zz484 = -IN(24).*zz138 + IN(25).*zz197 + IN(26).*zz136;`	Creates temporary common subexpression `zz484`. It stores `-IN(24).*zz138 + IN(25).*zz197 + IN(26).*zz136` so the generated expressions are shorter and faster to evaluate.
494	`zz485 = zz228.*zz59;`	Creates temporary common subexpression `zz485`. It stores `zz228.*zz59` so the generated expressions are shorter and faster to evaluate.
495	`zz486 = -zz234;`	Creates temporary common subexpression `zz486`. It stores `-zz234` so the generated expressions are shorter and faster to evaluate.
496	`zz487 = zz228.*zz291;`	Creates temporary common subexpression `zz487`. It stores `zz228.*zz291` so the generated expressions are shorter and faster to evaluate.
497	`zz488 = zz227.*zz228;`	Creates temporary common subexpression `zz488`. It stores `zz227.*zz228` so the generated expressions are shorter and faster to evaluate.
498	`zz489 = zz112.*zz486 + zz128.*zz488;`	Creates temporary common subexpression `zz489`. It stores `zz112.*zz486 + zz128.*zz488` so the generated expressions are shorter and faster to evaluate.
499	`zz490 = -zz112.*zz488 + zz128.*zz486;`	Creates temporary common subexpression `zz490`. It stores `-zz112.*zz488 + zz128.*zz486` so the generated expressions are shorter and faster to evaluate.
500	`zz491 = zz228.*zz229;`	Creates temporary common subexpression `zz491`. It stores `zz228.*zz229` so the generated expressions are shorter and faster to evaluate.
501	`zz492 = zz112.*zz239 + zz128.*zz491;`	Creates temporary common subexpression `zz492`. It stores `zz112.*zz239 + zz128.*zz491` so the generated expressions are shorter and faster to evaluate.
502	`zz493 = -zz112.*zz491 + zz128.*zz239;`	Creates temporary common subexpression `zz493`. It stores `-zz112.*zz491 + zz128.*zz239` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
503	`zz494 = zz112.*zz228.*zz233 - zz128.*zz232;`	Creates temporary common subexpression `zz494`. It stores `zz112.*zz228.*zz233 - zz128.*zz232` so the generated expressions are shorter and faster to evaluate.
504	`zz495 = zz112.*zz232 + zz128.*zz240;`	Creates temporary common subexpression `zz495`. It stores `zz112.*zz232 + zz128.*zz240` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
505	`zz496 = zz117.*zz490 + zz140.*zz237;`	Creates temporary common subexpression `zz496`. It stores `zz117.*zz490 + zz140.*zz237` so the generated expressions are shorter and faster to evaluate.
506	`zz497 = zz117.*zz493 + zz140.*zz238;`	Creates temporary common subexpression `zz497`. It stores `zz117.*zz493 + zz140.*zz238` so the generated expressions are shorter and faster to evaluate.
507	`zz498 = zz117.*zz495 + zz140.*zz241;`	Creates temporary common subexpression `zz498`. It stores `zz117.*zz495 + zz140.*zz241` so the generated expressions are shorter and faster to evaluate.
508	`zz499 = u(4).*zz488 + u(5).*zz486;`	Creates temporary common subexpression `zz499`. It stores `u(4).*zz488 + u(5).*zz486` so the generated expressions are shorter and faster to evaluate.
509	`zz500 = u(4).*zz491 + u(5).*zz239;`	Creates temporary common subexpression `zz500`. It stores `u(4).*zz491 + u(5).*zz239` so the generated expressions are shorter and faster to evaluate.
510	`zz501 = -u(4).*zz232 + u(5).*zz240;`	Creates temporary common subexpression `zz501`. It stores `-u(4).*zz232 + u(5).*zz240` so the generated expressions are shorter and faster to evaluate.
511	`zz502 = zz120.*zz486 + zz130.*zz488;`	Creates temporary common subexpression `zz502`. It stores `zz120.*zz486 + zz130.*zz488` so the generated expressions are shorter and faster to evaluate.
512	`zz503 = -zz120.*zz488 + zz130.*zz486;`	Creates temporary common subexpression `zz503`. It stores `-zz120.*zz488 + zz130.*zz486` so the generated expressions are shorter and faster to evaluate.
513	`zz504 = zz120.*zz239 + zz130.*zz491;`	Creates temporary common subexpression `zz504`. It stores `zz120.*zz239 + zz130.*zz491` so the generated expressions are shorter and faster to evaluate.
514	`zz505 = -zz120.*zz491 + zz130.*zz239;`	Creates temporary common subexpression `zz505`. It stores `-zz120.*zz491 + zz130.*zz239` so the generated expressions are shorter and faster to evaluate.
515	`zz506 = zz120.*zz228.*zz233 - zz130.*zz232;`	Creates temporary common subexpression `zz506`. It stores `zz120.*zz228.*zz233 - zz130.*zz232` so the generated expressions are shorter and faster to evaluate.
516	`zz507 = zz120.*zz232 + zz130.*zz240;`	Creates temporary common subexpression `zz507`. It stores `zz120.*zz232 + zz130.*zz240` so the generated expressions are shorter and faster to evaluate.
517	`zz508 = zz122.*zz503 + zz144.*zz237;`	Creates temporary common subexpression `zz508`. It stores `zz122.*zz503 + zz144.*zz237` so the generated expressions are shorter and faster to evaluate.
518	`zz509 = zz122.*zz505 + zz144.*zz238;`	Creates temporary common subexpression `zz509`. It stores `zz122.*zz505 + zz144.*zz238` so the generated expressions are shorter and faster to evaluate.
519	`zz510 = zz122.*zz507 + zz144.*zz241;`	Creates temporary common subexpression `zz510`. It stores `zz122.*zz507 + zz144.*zz241` so the generated expressions are shorter and faster to evaluate.
520	`zz511 = zz124.*zz488 + zz486.*zz99;`	Creates temporary common subexpression `zz511`. It stores `zz124.*zz488 + zz486.*zz99` so the generated expressions are shorter and faster to evaluate.
521	`zz512 = zz124.*zz486 - zz488.*zz99;`	Creates temporary common subexpression `zz512`. It stores `zz124.*zz486 - zz488.*zz99` so the generated expressions are shorter and faster to evaluate.
522	`zz513 = zz124.*zz491 + zz239.*zz99;`	Creates temporary common subexpression `zz513`. It stores `zz124.*zz491 + zz239.*zz99` so the generated expressions are shorter and faster to evaluate.
523	`zz514 = zz124.*zz239 - zz491.*zz99;`	Creates temporary common subexpression `zz514`. It stores `zz124.*zz239 - zz491.*zz99` so the generated expressions are shorter and faster to evaluate.
524	`zz515 = -zz124.*zz232 + zz228.*zz233.*zz99;`	Creates temporary common subexpression `zz515`. It stores `-zz124.*zz232 + zz228.*zz233.*zz99` so the generated expressions are shorter and faster to evaluate.
525	`zz516 = zz124.*zz240 + zz232.*zz99;`	Creates temporary common subexpression `zz516`. It stores `zz124.*zz240 + zz232.*zz99` so the generated expressions are shorter and faster to evaluate.
526	`zz517 = zz101.*zz512 + zz132.*zz237;`	Creates temporary common subexpression `zz517`. It stores `zz101.*zz512 + zz132.*zz237` so the generated expressions are shorter and faster to evaluate.
527	`zz518 = zz101.*zz514 + zz132.*zz238;`	Creates temporary common subexpression `zz518`. It stores `zz101.*zz514 + zz132.*zz238` so the generated expressions are shorter and faster to evaluate.
528	`zz519 = zz101.*zz516 + zz132.*zz241;`	Creates temporary common subexpression `zz519`. It stores `zz101.*zz516 + zz132.*zz241` so the generated expressions are shorter and faster to evaluate.
529	`zz520 = zz104.*zz486 + zz126.*zz488;`	Creates temporary common subexpression `zz520`. It stores `zz104.*zz486 + zz126.*zz488` so the generated expressions are shorter and faster to evaluate.
530	`zz521 = -zz104.*zz488 + zz126.*zz486;`	Creates temporary common subexpression `zz521`. It stores `-zz104.*zz488 + zz126.*zz486` so the generated expressions are shorter and faster to evaluate.
531	`zz522 = zz104.*zz239 + zz126.*zz491;`	Creates temporary common subexpression `zz522`. It stores `zz104.*zz239 + zz126.*zz491` so the generated expressions are shorter and faster to evaluate.
532	`zz523 = -zz104.*zz491 + zz126.*zz239;`	Creates temporary common subexpression `zz523`. It stores `-zz104.*zz491 + zz126.*zz239` so the generated expressions are shorter and faster to evaluate.

Line	Sanitized code line	Explanation
533	`zz524 = zz104.*zz228.*zz233 - zz126.*zz232;`	Creates temporary common subexpression `zz524`. It stores `zz104.*zz228.*zz233 - zz126.*zz232` so the generated expressions are shorter and faster to evaluate.
534	`zz525 = zz104.*zz232 + zz126.*zz240;`	Creates temporary common subexpression `zz525`. It stores `zz104.*zz232 + zz126.*zz240` so the generated expressions are shorter and faster to evaluate.
535	`zz526 = zz106.*zz521 + zz136.*zz237;`	Creates temporary common subexpression `zz526`. It stores `zz106.*zz521 + zz136.*zz237` so the generated expressions are shorter and faster to evaluate.
536	`zz527 = zz106.*zz523 + zz136.*zz238;`	Creates temporary common subexpression `zz527`. It stores `zz106.*zz523 + zz136.*zz238` so the generated expressions are shorter and faster to evaluate.
537	`zz528 = zz106.*zz525 + zz136.*zz241;`	Creates temporary common subexpression `zz528`. It stores `zz106.*zz525 + zz136.*zz241` so the generated expressions are shorter and faster to evaluate.
538	`zz529 = P(78).*zz241 + q(3);`	Creates temporary common subexpression `zz529`. It stores `P(78).*zz241 + q(3)` so the generated expressions are shorter and faster to evaluate.
539	`M(1,1) = 1;`	Assigns `M(1,1)` from `1`. This value is used by later computations in this function.
540	`M(2,2) = 1;`	Assigns `M(2,2)` from `1`. This value is used by later computations in this function.
541	`M(3,3) = 1;`	Assigns `M(3,3)` from `1`. This value is used by later computations in this function.
542	`M(4,4) = 1;`	Assigns `M(4,4)` from `1`. This value is used by later computations in this function.
543	`M(5,5) = 1;`	Assigns `M(5,5)` from `1`. This value is used by later computations in this function.
544	`M(6,6) = 1;`	Assigns `M(6,6)` from `1`. This value is used by later computations in this function.
545	`M(7,7) = 1;`	Assigns `M(7,7)` from `1`. This value is used by later computations in this function.
546	`M(8,8) = 1;`	Assigns `M(8,8)` from `1`. This value is used by later computations in this function.
547	`M(9,9) = 1;`	Assigns `M(9,9)` from `1`. This value is used by later computations in this function.
548	`M(10,10) = 1;`	Assigns `M(10,10)` from `1`. This value is used by later computations in this function.
549	`M(11,11) = 1;`	Assigns `M(11,11)` from `1`. This value is used by later computations in this function.
550	`M(12,12) = 1;`	Assigns `M(12,12)` from `1`. This value is used by later computations in this function.
551	`M(13,13) = 1;`	Assigns `M(13,13)` from `1`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
552	`M(14,14) = 1;`	Assigns `M(14,14)` from `1`. This value is used by later computations in this function.
553	`M(15,15) = zz1;`	Assigns `M(15,15)` from `zz1`. This value is used by later computations in this function.
554	`M(15,19) = zz21;`	Assigns `M(15,19)` from `zz21`. This value is used by later computations in this function.
555	`M(15,20) = -zz35;`	Assigns `M(15,20)` from `-zz35`. This value is used by later computations in this function.
556	`M(15,21) = zz41;`	Assigns `M(15,21)` from `zz41`. This value is used by later computations in this function.
557	`M(15,22) = zz44;`	Assigns `M(15,22)` from `zz44`. This value is used by later computations in this function.
558	`M(15,23) = zz48;`	Assigns `M(15,23)` from `zz48`. This value is used by later computations in this function.
559	`M(15,24) = zz51;`	Assigns `M(15,24)` from `zz51`. This value is used by later computations in this function.
560	`M(16,16) = zz1;`	Assigns `M(16,16)` from `zz1`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
561	`M(16,18) = -zz21;`	Assigns `M(16,18)` from `-zz21`. This value is used by later computations in this function.
562	`M(16,20) = zz63;`	Assigns `M(16,20)` from `zz63`. This value is used by later computations in this function.
563	`M(16,21) = zz68;`	Assigns `M(16,21)` from `zz68`. This value is used by later computations in this function.
564	`M(16,22) = zz71;`	Assigns `M(16,22)` from `zz71`. This value is used by later computations in this function.
565	`M(16,23) = zz75;`	Assigns `M(16,23)` from `zz75`. This value is used by later computations in this function.
566	`M(16,24) = zz78;`	Assigns `M(16,24)` from `zz78`. This value is used by later computations in this function.
567	`M(17,17) = zz1;`	Assigns `M(17,17)` from `zz1`. This value is used by later computations in this function.
568	`M(17,18) = zz35;`	Assigns `M(17,18)` from `zz35`. This value is used by later computations in this function.
569	`M(17,19) = -zz63;`	Assigns `M(17,19)` from `-zz63`. This value is used by later computations in this function.
570	`M(17,21) = zz83;`	Assigns `M(17,21)` from `zz83`. This value is used by later computations in this function.
571	`M(17,22) = zz86;`	Assigns `M(17,22)` from `zz86`. This value is used by later computations in this function.
572	`M(17,23) = zz90;`	Assigns `M(17,23)` from `zz90`. This value is used by later computations in this function.
573	`M(17,24) = zz93;`	Assigns `M(17,24)` from `zz93`. This value is used by later computations in this function.
574	`M(18,16) = P(2).*zz94 + P(2).*zz95 + P(2).*zz96 + P(2).*zz97;`	Assigns `M(18,16)` from `P(2).*zz94 + P(2).*zz95 + P(2).*zz96 + P(2).*zz97`. This value is used by later computations in this function.
575	`M(18,17) = zz35;`	Assigns `M(18,17)` from `zz35`. This value is used by later computations in this function.
576	`M(18,18) = P(2).*(zz23.^2 - zz7.*zz95) + P(2).*(zz25.^2 - zz4.*zz94) + P(2).*(-zz16.*zz96 + zz31.^2) + P(2).*(-zz19.*zz97 + zz33.^2) + P(3) + zz102.*zz99.^2 + zz104.^2.*zz107 + zz112.^2.*zz118 + zz120.^2.*zz123;`	Assigns `M(18,18)` from `P(2).*(zz23.^2 - zz7.*zz95) + P(2).*(zz25.^2 - zz4.*zz94) + P(2).*(-zz16.*zz96 + zz31.^2) + P(2).*(-zz19.*zz97 + zz33.^2) + P(3) + zz102.*zz99.^2 +...`. This value is used by later computations in this function.
577	`M(18,19) = -zz125 - zz127 - zz129 - zz131 - zz24.*zz55 - zz26.*zz53 - zz32.*zz59 - zz34.*zz61;`	Assigns `M(18,19)` from `-zz125 - zz127 - zz129 - zz131 - zz24.*zz55 - zz26.*zz53 - zz32.*zz59 - zz34.*zz61`. This value is used by later computations in this function.
578	`M(18,20) = P(6) - zz135 - zz139 - zz143 - zz147 + zz54.*zz94 + zz56.*zz95 + zz60.*zz96 + zz62.*zz97;`	Assigns `M(18,20)` from `P(6) - zz135 - zz139 - zz143 - zz147 + zz54.*zz94 + zz56.*zz95 + zz60.*zz96 + zz62.*zz97`. This value is used by later computations in this function.
579	`M(18,21) = P(2).*(zz31.*zz82 + zz67.*zz96) - zz142.*zz153 - zz142.*zz157;`	Assigns `M(18,21)` from `P(2).*(zz31.*zz82 + zz67.*zz96) - zz142.*zz153 - zz142.*zz157`. This value is used by later computations in this function.
580	`M(18,22) = P(2).*(zz33.*zz85 + zz70.*zz97) - zz146.*zz161 - zz146.*zz164;`	Assigns `M(18,22)` from `P(2).*(zz33.*zz85 + zz70.*zz97) - zz146.*zz161 - zz146.*zz164`. This value is used by later computations in this function.
581	`M(18,23) = P(2).*(zz25.*zz89 + zz74.*zz94) - zz134.*zz168 - zz134.*zz171;`	Assigns `M(18,23)` from `P(2).*(zz25.*zz89 + zz74.*zz94) - zz134.*zz168 - zz134.*zz171`. This value is used by later computations in this function.
582	`M(18,24) = P(2).*(zz23.*zz92 + zz77.*zz95) - zz138.*zz175 - zz138.*zz178;`	Assigns `M(18,24)` from `P(2).*(zz23.*zz92 + zz77.*zz95) - zz138.*zz175 - zz138.*zz178`. This value is used by later computations in this function.
583	`M(18,25) = zz180;`	Assigns `M(18,25)` from `zz180`. This value is used by later computations in this function.
584	`M(18,26) = zz182;`	Assigns `M(18,26)` from `zz182`. This value is used by later computations in this function.
585	`M(18,27) = zz184;`	Assigns `M(18,27)` from `zz184`. This value is used by later computations in this function.
586	`M(18,28) = zz186;`	Assigns `M(18,28)` from `zz186`. This value is used by later computations in this function.
587	`M(19,15) = zz21;`	Assigns `M(19,15)` from `zz21`. This value is used by later computations in this function.
588	`M(19,17) = P(2).*zz187 + P(2).*zz188 + P(2).*zz189 + P(2).*zz190;`	Assigns `M(19,17)` from `P(2).*zz187 + P(2).*zz188 + P(2).*zz189 + P(2).*zz190`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
589	`M(19,18) = -zz125 - zz127 - zz129 - zz131 + zz187.*zz26 + zz188.*zz24 + zz189.*zz32 + zz190.*zz34;`	Assigns `M(19,18)` from `-zz125 - zz127 - zz129 - zz131 + zz187.*zz26 + zz188.*zz24 + zz189.*zz32 + zz190.*zz34`. This value is used by later computations in this function.
590	`M(19,19) = P(2).*(zz16.^2 - zz189.*zz59) + P(2).*(zz19.^2 - zz190.*zz61) + P(2).*(-zz187.*zz53 + zz4.^2) + P(2).*(-zz188.*zz55 + zz7.^2) + P(4) + zz102.*zz124.^2 + zz107.*zz126.^2 + zz118.*zz128.^2 + zz123.*zz130.^2;`	Assigns `M(19,19)` from `P(2).*(zz16.^2 - zz189.*zz59) + P(2).*(zz19.^2 - zz190.*zz61) + P(2).*(-zz187.*zz53 + zz4.^2) + P(2).*(-zz188.*zz55 + zz7.^2) + P(4) + zz102.*zz124...`. This value is used by later computations in this function.
591	`M(19,20) = P(7).*zz101.*zz124.*zz132 + P(7).*zz106.*zz126.*zz136 + P(7).*zz117.*zz128.*zz140 + P(7).*zz122.*zz130.*zz144 - zz17.*zz31 - zz20.*zz33 - zz23.*zz8 - zz25.*zz5;`	Assigns `M(19,20)` from `P(7).*zz101.*zz124.*zz132 + P(7).*zz106.*zz126.*zz136 + P(7).*zz117.*zz128.*zz140 + P(7).*zz122.*zz130.*zz144 - zz17.*zz31 - zz20.*zz33 - zz23.*zz8...`. This value is used by later computations in this function.
592	`M(19,21) = P(2).*(zz189.*zz82 + zz192) + zz153.*zz191 + zz157.*zz191;`	Assigns `M(19,21)` from `P(2).*(zz189.*zz82 + zz192) + zz153.*zz191 + zz157.*zz191`. This value is used by later computations in this function.
593	`M(19,22) = P(2).*(zz190.*zz85 + zz194) + zz161.*zz193 + zz164.*zz193;`	Assigns `M(19,22)` from `P(2).*(zz190.*zz85 + zz194) + zz161.*zz193 + zz164.*zz193`. This value is used by later computations in this function.
594	`M(19,23) = P(2).*(zz187.*zz89 + zz196) + zz168.*zz195 + zz171.*zz195;`	Assigns `M(19,23)` from `P(2).*(zz187.*zz89 + zz196) + zz168.*zz195 + zz171.*zz195`. This value is used by later computations in this function.
595	`M(19,24) = P(2).*(zz188.*zz92 + zz198) + zz175.*zz197 + zz178.*zz197;`	Assigns `M(19,24)` from `P(2).*(zz188.*zz92 + zz198) + zz175.*zz197 + zz178.*zz197`. This value is used by later computations in this function.
596	`M(19,25) = zz199;`	Assigns `M(19,25)` from `zz199`. This value is used by later computations in this function.
597	`M(19,26) = zz200;`	Assigns `M(19,26)` from `zz200`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
598	`M(19,27) = zz201;`	Assigns `M(19,27)` from `zz201`. This value is used by later computations in this function.
599	`M(19,28) = zz202;`	Assigns `M(19,28)` from `zz202`. This value is used by later computations in this function.
600	`M(20,15) = P(2).*zz203 + P(2).*zz204 + P(2).*zz205 + P(2).*zz206;`	Assigns `M(20,15)` from `P(2).*zz203 + P(2).*zz204 + P(2).*zz205 + P(2).*zz206`. This value is used by later computations in this function.
601	`M(20,16) = zz63;`	Assigns `M(20,16)` from `zz63`. This value is used by later computations in this function.
602	`M(20,18) = P(6) - zz135 - zz139 - zz143 - zz147 - zz17.*zz59 - zz20.*zz61 - zz5.*zz53 - zz55.*zz8;`	Assigns `M(20,18)` from `P(6) - zz135 - zz139 - zz143 - zz147 - zz17.*zz59 - zz20.*zz61 - zz5.*zz53 - zz55.*zz8`. This value is used by later computations in this function.
603	`M(20,19) = zz133.*zz195 + zz137.*zz197 + zz141.*zz191 + zz145.*zz193 + zz17.*zz205 + zz20.*zz206 + zz203.*zz5 + zz204.*zz8;`	Assigns `M(20,19)` from `zz133.*zz195 + zz137.*zz197 + zz141.*zz191 + zz145.*zz193 + zz17.*zz205 + zz20.*zz206 + zz203.*zz5 + zz204.*zz8`. This value is used by later computations in this function.
604	`M(20,20) = P(2).*(-zz203.*zz25 + zz53.^2) + P(2).*(-zz204.*zz23 + zz55.^2) + P(2).*(-zz205.*zz31 + zz59.^2) + P(2).*(-zz206.*zz33 + zz61.^2) + P(5) + zz207 + zz208 + zz209 + zz210;`	Assigns `M(20,20)` from `P(2).*(-zz203.*zz25 + zz53.^2) + P(2).*(-zz204.*zz23 + zz55.^2) + P(2).*(-zz205.*zz31 + zz59.^2) + P(2).*(-zz206.*zz33 + zz61.^2) + P(5) + zz207 + ...`. This value is used by later computations in this function.
605	`M(20,21) = P(2).*(zz205.*zz40 + zz59.*zz67) + zz141.*zz152 + zz156.*zz209;`	Assigns `M(20,21)` from `P(2).*(zz205.*zz40 + zz59.*zz67) + zz141.*zz152 + zz156.*zz209`. This value is used by later computations in this function.
606	`M(20,22) = P(2).*(zz206.*zz43 + zz61.*zz70) + zz145.*zz160 + zz163.*zz210;`	Assigns `M(20,22)` from `P(2).*(zz206.*zz43 + zz61.*zz70) + zz145.*zz160 + zz163.*zz210`. This value is used by later computations in this function.
607	`M(20,23) = P(2).*(zz203.*zz47 + zz53.*zz74) + zz133.*zz167 + zz170.*zz207;`	Assigns `M(20,23)` from `P(2).*(zz203.*zz47 + zz53.*zz74) + zz133.*zz167 + zz170.*zz207`. This value is used by later computations in this function.
608	`M(20,24) = P(2).*(zz204.*zz50 + zz55.*zz77) + zz137.*zz174 + zz177.*zz208;`	Assigns `M(20,24)` from `P(2).*(zz204.*zz50 + zz55.*zz77) + zz137.*zz174 + zz177.*zz208`. This value is used by later computations in this function.
609	`M(20,25) = zz141;`	Assigns `M(20,25)` from `zz141`. This value is used by later computations in this function.
610	`M(20,26) = zz145;`	Assigns `M(20,26)` from `zz145`. This value is used by later computations in this function.
611	`M(20,27) = zz133;`	Assigns `M(20,27)` from `zz133`. This value is used by later computations in this function.
612	`M(20,28) = zz137;`	Assigns `M(20,28)` from `zz137`. This value is used by later computations in this function.
613	`M(21,15) = zz41;`	Assigns `M(21,15)` from `zz41`. This value is used by later computations in this function.
614	`M(21,16) = zz68;`	Assigns `M(21,16)` from `zz68`. This value is used by later computations in this function.
615	`M(21,17) = zz83;`	Assigns `M(21,17)` from `zz83`. This value is used by later computations in this function.
616	`M(21,18) = P(2).*(-zz16.*zz67 + zz31.*zz82) - zz142.*zz213;`	Assigns `M(21,18)` from `P(2).*(-zz16.*zz67 + zz31.*zz82) - zz142.*zz213`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
617	`M(21,19) = P(2).*(zz192 - zz59.*zz82) + zz191.*zz213;`	Assigns `M(21,19)` from `P(2).*(zz192 - zz59.*zz82) + zz191.*zz213`. This value is used by later computations in this function.
618	`M(21,20) = P(2).*(-zz31.*zz40 + zz59.*zz67) + zz214;`	Assigns `M(21,20)` from `P(2).*(-zz31.*zz40 + zz59.*zz67) + zz214`. This value is used by later computations in this function.
619	`M(21,21) = P(2).*(zz40.^2 + zz67.^2 + zz82.^2) + zz152.*zz213 + zz156.*zz214;`	Assigns `M(21,21)` from `P(2).*(zz40.^2 + zz67.^2 + zz82.^2) + zz152.*zz213 + zz156.*zz214`. This value is used by later computations in this function.
620	`M(21,25) = zz213;`	Assigns `M(21,25)` from `zz213`. This value is used by later computations in this function.
621	`M(22,15) = zz44;`	Assigns `M(22,15)` from `zz44`. This value is used by later computations in this function.
622	`M(22,16) = zz71;`	Assigns `M(22,16)` from `zz71`. This value is used by later computations in this function.
623	`M(22,17) = zz86;`	Assigns `M(22,17)` from `zz86`. This value is used by later computations in this function.
624	`M(22,18) = P(2).*(-zz19.*zz70 + zz33.*zz85) - zz146.*zz217;`	Assigns `M(22,18)` from `P(2).*(-zz19.*zz70 + zz33.*zz85) - zz146.*zz217`. This value is used by later computations in this function.
625	`M(22,19) = P(2).*(zz194 - zz61.*zz85) + zz193.*zz217;`	Assigns `M(22,19)` from `P(2).*(zz194 - zz61.*zz85) + zz193.*zz217`. This value is used by later computations in this function.
626	`M(22,20) = P(2).*(-zz33.*zz43 + zz61.*zz70) + zz218;`	Assigns `M(22,20)` from `P(2).*(-zz33.*zz43 + zz61.*zz70) + zz218`. This value is used by later computations in this function.
627	`M(22,22) = P(2).*(zz43.^2 + zz70.^2 + zz85.^2) + zz160.*zz217 + zz163.*zz218;`	Assigns `M(22,22)` from `P(2).*(zz43.^2 + zz70.^2 + zz85.^2) + zz160.*zz217 + zz163.*zz218`. This value is used by later computations in this function.
628	`M(22,26) = zz217;`	Assigns `M(22,26)` from `zz217`. This value is used by later computations in this function.
629	`M(23,15) = zz48;`	Assigns `M(23,15)` from `zz48`. This value is used by later computations in this function.
630	`M(23,16) = zz75;`	Assigns `M(23,16)` from `zz75`. This value is used by later computations in this function.
631	`M(23,17) = zz90;`	Assigns `M(23,17)` from `zz90`. This value is used by later computations in this function.
632	`M(23,18) = P(2).*(zz25.*zz89 - zz4.*zz74) - zz134.*zz221;`	Assigns `M(23,18)` from `P(2).*(zz25.*zz89 - zz4.*zz74) - zz134.*zz221`. This value is used by later computations in this function.
633	`M(23,19) = P(2).*(zz196 - zz53.*zz89) + zz195.*zz221;`	Assigns `M(23,19)` from `P(2).*(zz196 - zz53.*zz89) + zz195.*zz221`. This value is used by later computations in this function.
634	`M(23,20) = P(2).*(-zz25.*zz47 + zz53.*zz74) + zz222;`	Assigns `M(23,20)` from `P(2).*(-zz25.*zz47 + zz53.*zz74) + zz222`. This value is used by later computations in this function.
635	`M(23,23) = P(2).*(zz47.^2 + zz74.^2 + zz89.^2) + zz167.*zz221 + zz170.*zz222;`	Assigns `M(23,23)` from `P(2).*(zz47.^2 + zz74.^2 + zz89.^2) + zz167.*zz221 + zz170.*zz222`. This value is used by later computations in this function.
636	`M(23,27) = zz221;`	Assigns `M(23,27)` from `zz221`. This value is used by later computations in this function.
637	`M(24,15) = zz51;`	Assigns `M(24,15)` from `zz51`. This value is used by later computations in this function.
638	`M(24,16) = zz78;`	Assigns `M(24,16)` from `zz78`. This value is used by later computations in this function.
639	`M(24,17) = zz93;`	Assigns `M(24,17)` from `zz93`. This value is used by later computations in this function.
640	`M(24,18) = P(2).*(zz23.*zz92 - zz7.*zz77) - zz138.*zz225;`	Assigns `M(24,18)` from `P(2).*(zz23.*zz92 - zz7.*zz77) - zz138.*zz225`. This value is used by later computations in this function.
641	`M(24,19) = P(2).*(zz198 - zz55.*zz92) + zz197.*zz225;`	Assigns `M(24,19)` from `P(2).*(zz198 - zz55.*zz92) + zz197.*zz225`. This value is used by later computations in this function.
642	`M(24,20) = P(2).*(-zz23.*zz50 + zz55.*zz77) + zz226;`	Assigns `M(24,20)` from `P(2).*(-zz23.*zz50 + zz55.*zz77) + zz226`. This value is used by later computations in this function.
643	`M(24,24) = P(2).*(zz50.^2 + zz77.^2 + zz92.^2) + zz174.*zz225 + zz177.*zz226;`	Assigns `M(24,24)` from `P(2).*(zz50.^2 + zz77.^2 + zz92.^2) + zz174.*zz225 + zz177.*zz226`. This value is used by later computations in this function.
644	`M(24,28) = zz225;`	Assigns `M(24,28)` from `zz225`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
645	`M(25,18) = zz180;`	Assigns `M(25,18)` from `zz180`. This value is used by later computations in this function.
646	`M(25,19) = zz199;`	Assigns `M(25,19)` from `zz199`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
647	`M(25,20) = zz141;`	Assigns `M(25,20)` from `zz141`. This value is used by later computations in this function.
648	`M(25,21) = zz153 + zz157;`	Assigns `M(25,21)` from `zz153 + zz157`. This value is used by later computations in this function.
649	`M(25,25) = P(7);`	Assigns `M(25,25)` from `P(7)`. This value is used by later computations in this function.
650	`M(26,18) = zz182;`	Assigns `M(26,18)` from `zz182`. This value is used by later computations in this function.
651	`M(26,19) = zz200;`	Assigns `M(26,19)` from `zz200`. This value is used by later computations in this function.
652	`M(26,20) = zz145;`	Assigns `M(26,20)` from `zz145`. This value is used by later computations in this function.
653	`M(26,22) = zz161 + zz164;`	Assigns `M(26,22)` from `zz161 + zz164`. This value is used by later computations in this function.
654	`M(26,26) = P(7);`	Assigns `M(26,26)` from `P(7)`. This value is used by later computations in this function.
655	`M(27,18) = zz184;`	Assigns `M(27,18)` from `zz184`. This value is used by later computations in this function.
656	`M(27,19) = zz201;`	Assigns `M(27,19)` from `zz201`. This value is used by later computations in this function.
657	`M(27,20) = zz133;`	Assigns `M(27,20)` from `zz133`. This value is used by later computations in this function.
658	`M(27,23) = zz168 + zz171;`	Assigns `M(27,23)` from `zz168 + zz171`. This value is used by later computations in this function.
659	`M(27,27) = P(7);`	Assigns `M(27,27)` from `P(7)`. This value is used by later computations in this function.
660	`M(28,18) = zz186;`	Assigns `M(28,18)` from `zz186`. This value is used by later computations in this function.
661	`M(28,19) = zz202;`	Assigns `M(28,19)` from `zz202`. This value is used by later computations in this function.
662	`M(28,20) = zz137;`	Assigns `M(28,20)` from `zz137`. This value is used by later computations in this function.
663	`M(28,24) = zz175 + zz178;`	Assigns `M(28,24)` from `zz175 + zz178`. This value is used by later computations in this function.
664	`M(28,28) = P(7);`	Assigns `M(28,28)` from `P(7)`. This value is used by later computations in this function.
665	`F(1) = u(1).*zz227.*zz228 - u(2).*zz234 + u(3).*zz237;`	Assigns `F(1)` from `u(1).*zz227.*zz228 - u(2).*zz234 + u(3).*zz237`. This value is used by later computations in this function.
666	`F(2) = u(1).*zz228.*zz229 + u(2).*zz239 + u(3).*zz238;`	Assigns `F(2)` from `u(1).*zz228.*zz229 + u(2).*zz239 + u(3).*zz238`. This value is used by later computations in this function.
667	`F(3) = -u(1).*zz232 + u(2).*zz240 + u(3).*zz241;`	Assigns `F(3)` from `-u(1).*zz232 + u(2).*zz240 + u(3).*zz241`. This value is used by later computations in this function.
668	`F(4) = zz242.*zz243 + zz242.*zz244;`	Assigns `F(4)` from `zz242.*zz243 + zz242.*zz244`. This value is used by later computations in this function.
669	`F(5) = u(5).*zz230 - u(6).*zz233;`	Assigns `F(5)` from `u(5).*zz230 - u(6).*zz233`. This value is used by later computations in this function.
670	`F(6) = u(4) + zz243.*zz245 + zz244.*zz245;`	Assigns `F(6)` from `u(4) + zz243.*zz245 + zz244.*zz245`. This value is used by later computations in this function.
671	`F(7) = u(7);`	Assigns `F(7)` from `u(7)`. This value is used by later computations in this function.
672	`F(8) = u(8);`	Assigns `F(8)` from `u(8)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
673	`F(9) = u(9);`	Assigns `F(9)` from `u(9)`. This value is used by later computations in this function.
674	`F(10) = u(10);`	Assigns `F(10)` from `u(10)`. This value is used by later computations in this function.
675	`F(11) = u(11);`	Assigns `F(11)` from `u(11)`. This value is used by later computations in this function.
676	`F(12) = u(12);`	Assigns `F(12)` from `u(12)`. This value is used by later computations in this function.
677	`F(13) = u(13);`	Assigns `F(13)` from `u(13)`. This value is used by later computations in this function.
678	`F(14) = u(14);`	Assigns `F(14)` from `u(14)`. This value is used by later computations in this function.
679	`F(15) = IN(3) + IN(9) + IN(15) + IN(21) + IN(31) + IN(33) + P(1).*zz248 - P(1).*(-u(2).*u(6) + u(3).*u(5)) - P(2).*zz254 - P(2).*zz260 - P(2).*zz272 - P(2).*zz279 + zz0.*zz248 - zz246 - zz247;`	Writes one value into the external input vector used by the generated dynamics function.
680	`F(16) = IN(4) + IN(10) + IN(16) + IN(22) + IN(32).*zz228.*zz233 + IN(34).*zz228.*zz233 - P(1).*(u(1).*u(6) - u(3).*u(4)) - P(2).*zz284 - P(2).*zz287 - P(2).*zz292 - P(2).*zz295 - zz240.*zz280 - zz240.*zz281;`	Writes one value into the external input vector used by the generated dynamics function.
681	`F(17) = IN(5) + IN(11) + IN(17) + IN(23) + IN(32).*zz228.*zz230 + IN(34).*zz228.*zz230 - P(1).*(-u(1).*u(5) + u(2).*u(4)) - P(2).*zz296 - P(2).*zz297 - P(2).*zz299 - P(2).*zz300 - zz241.*zz280 - zz241.*zz281;`	Writes one value into the external input vector used by the generated dynamics function.
682	`F(18) = -IN(4).*zz16 + IN(5).*zz31 + IN(6) - IN(10).*zz19 + IN(11).*zz33 + IN(12) - IN(16).*zz4 + IN(17).*zz25 + IN(18) - IN(22).*zz7 + IN(23).*zz23 + IN(24) - IN(32).*P(78).*zz240 - IN(34).*P(78).*zz240 + P(4).*zz301 - u(5).*zz302 + zz1...`	Writes one value into the external input vector used by the generated dynamics function.
683	`F(19) = IN(3).*zz16 - IN(5).*zz59 + IN(7) + IN(9).*zz19 - IN(11).*zz61 + IN(13) + IN(15).*zz4 - IN(17).*zz53 + IN(19) + IN(21).*zz7 - IN(23).*zz55 + IN(25) + IN(31).*P(78) - IN(32).*P(76).*zz241 + IN(33).*P(78) - IN(34).*P(77).*zz241 - P...`	Writes one value into the external input vector used by the generated dynamics function.
684	`F(20) = -IN(3).*zz31 + IN(4).*zz59 + IN(8) - IN(9).*zz33 + IN(10).*zz61 + IN(14) - IN(15).*zz25 + IN(16).*zz53 + IN(20) - IN(21).*zz23 + IN(22).*zz55 + IN(26) + IN(32).*P(76).*zz228.*zz233 + IN(34).*P(77).*zz228.*zz233 + P(2).*zz23.*zz26...`	Writes one value into the external input vector used by the generated dynamics function.
685	`F(21) = IN(3).*zz40 + IN(4).*zz67 + IN(5).*zz82 + IN(8).*zz156 + IN(27).*zz152 + IN(27).*zz211 + zz152.*zz478 - zz153.*zz341 + zz156.*zz437.*zz464 - zz156.*zz474 - zz157.*zz341 + zz248.*zz41 - zz272.*zz41 - zz292.*zz68 - zz299.*zz83 - zz...`	Writes one value into the external input vector used by the generated dynamics function.
686	`F(22) = IN(9).*zz43 + IN(10).*zz70 + IN(11).*zz85 + IN(14).*zz163 + IN(28).*zz160 + IN(28).*zz215 + zz160.*zz480 - zz161.*zz350 + zz163.*zz462.*zz468 - zz163.*zz475 - zz164.*zz350 + zz248.*zz44 - zz279.*zz44 - zz295.*zz71 - zz300.*zz86 -...`	Writes one value into the external input vector used by the generated dynamics function.
687	`F(23) = IN(15).*zz47 + IN(16).*zz74 + IN(17).*zz89 + IN(20).*zz170 + IN(29).*zz167 + IN(29).*zz219 + zz167.*zz482 - zz168.*zz324 + zz170.*zz376.*zz403 - zz170.*zz472 - zz171.*zz324 + zz248.*zz48 - zz254.*zz48 - zz284.*zz75 - zz296.*zz90 ...`	Writes one value into the external input vector used by the generated dynamics function.

Line	Sanitized code line	Explanation
688	`F(24) = IN(21).*zz50 + IN(22).*zz77 + IN(23).*zz92 + IN(26).*zz177 + IN(30).*zz174 + IN(30).*zz223 + zz174.*zz484 - zz175.*zz330 + zz177.*zz401.*zz407 - zz177.*zz473 - zz178.*zz330 + zz248.*zz51 - zz260.*zz51 - zz287.*zz78 - zz297.*zz93 ...;`	Writes one value into the external input vector used by the generated dynamics function.
689	`F(25) = IN(27) - zz342 + zz478;`	Writes one value into the external input vector used by the generated dynamics function.
690	`F(26) = IN(28) - zz351 + zz480;`	Writes one value into the external input vector used by the generated dynamics function.
691	`F(27) = IN(29) - zz325 + zz482;`	Writes one value into the external input vector used by the generated dynamics function.
692	`F(28) = IN(30) - zz331 + zz484;`	Writes one value into the external input vector used by the generated dynamics function.
693	`S(1) = q(1) + zz16.*zz237 + zz227.*zz485 + zz31.*zz486;`	Assigns `S(1)` from `q(1) + zz16.*zz237 + zz227.*zz485 + zz31.*zz486`. This value is used by later computations in this function.
694	`S(2) = q(2) + zz16.*zz238 + zz229.*zz485 + zz239.*zz31;`	Assigns `S(2)` from `q(2) + zz16.*zz238 + zz229.*zz485 + zz239.*zz31`. This value is used by later computations in this function.
695	`S(3) = q(3) + zz16.*zz241 - zz232.*zz59 + zz240.*zz31;`	Assigns `S(3)` from `q(3) + zz16.*zz241 - zz232.*zz59 + zz240.*zz31`. This value is used by later computations in this function.
696	`S(4) = zz227.*zz487 + zz237.*zz269 + zz271.*zz486;`	Assigns `S(4)` from `zz227.*zz487 + zz237.*zz269 + zz271.*zz486`. This value is used by later computations in this function.
697	`S(5) = zz229.*zz487 + zz238.*zz269 + zz239.*zz271;`	Assigns `S(5)` from `zz229.*zz487 + zz238.*zz269 + zz239.*zz271`. This value is used by later computations in this function.
698	`S(6) = -zz232.*zz291 + zz240.*zz271 + zz241.*zz269;`	Assigns `S(6)` from `-zz232.*zz291 + zz240.*zz271 + zz241.*zz269`. This value is used by later computations in this function.
699	`S(7) = -zz117.*zz269.*zz415 + zz271.*zz429 + zz291.*zz417;`	Assigns `S(7)` from `-zz117.*zz269.*zz415 + zz271.*zz429 + zz291.*zz417`. This value is used by later computations in this function.
700	`S(8) = zz140.*zz269 - zz142.*zz291 + zz191.*zz271;`	Assigns `S(8)` from `zz140.*zz269 - zz142.*zz291 + zz191.*zz271`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
701	`S(9) = zz59;`	Assigns `S(9)` from `zz59`. This value is used by later computations in this function.
702	`S(10) = zz31;`	Assigns `S(10)` from `zz31`. This value is used by later computations in this function.
703	`S(11) = zz16;`	Assigns `S(11)` from `zz16`. This value is used by later computations in this function.
704	`S(12) = zz111;`	Assigns `S(12)` from `zz111`. This value is used by later computations in this function.
705	`S(13) = zz116;`	Assigns `S(13)` from `zz116`. This value is used by later computations in this function.
706	`S(14) = zz413;`	Assigns `S(14)` from `zz413`. This value is used by later computations in this function.
707	`S(15) = zz10;`	Assigns `S(15)` from `zz10`. This value is used by later computations in this function.
708	`S(16) = zz414.*zz489 - zz415.*(zz117.*zz237 - zz140.*zz490);`	Assigns `S(16)` from `zz414.*zz489 - zz415.*(zz117.*zz237 - zz140.*zz490)`. This value is used by later computations in this function.
709	`S(17) = zz414.*zz492 - zz415.*(zz117.*zz238 - zz140.*zz493);`	Assigns `S(17)` from `zz414.*zz492 - zz415.*(zz117.*zz238 - zz140.*zz493)`. This value is used by later computations in this function.
710	`S(18) = zz414.*zz494 - zz415.*(zz117.*zz228.*zz230 - zz140.*zz495);`	Assigns `S(18)` from `zz414.*zz494 - zz415.*(zz117.*zz228.*zz230 - zz140.*zz495)`. This value is used by later computations in this function.
711	`S(19) = zz496;`	Assigns `S(19)` from `zz496`. This value is used by later computations in this function.
712	`S(20) = zz497;`	Assigns `S(20)` from `zz497`. This value is used by later computations in this function.
713	`S(21) = zz498;`	Assigns `S(21)` from `zz498`. This value is used by later computations in this function.
714	`S(22) = zz237.*zz339 + zz337.*zz489 + zz434.*zz496 + zz499;`	Assigns `S(22)` from `zz237.*zz339 + zz337.*zz489 + zz434.*zz496 + zz499`. This value is used by later computations in this function.
715	`S(23) = zz238.*zz339 + zz337.*zz492 + zz434.*zz497 + zz500;`	Assigns `S(23)` from `zz238.*zz339 + zz337.*zz492 + zz434.*zz497 + zz500`. This value is used by later computations in this function.
716	`S(24) = zz241.*zz339 + zz337.*zz494 + zz434.*zz498 + zz501;`	Assigns `S(24)` from `zz241.*zz339 + zz337.*zz494 + zz434.*zz498 + zz501`. This value is used by later computations in this function.
717	`S(25) = q(1) + zz19.*zz237 + zz33.*zz486 + zz488.*zz61;`	Assigns `S(25)` from `q(1) + zz19.*zz237 + zz33.*zz486 + zz488.*zz61`. This value is used by later computations in this function.
718	`S(26) = q(2) + zz19.*zz238 + zz239.*zz33 + zz491.*zz61;`	Assigns `S(26)` from `q(2) + zz19.*zz238 + zz239.*zz33 + zz491.*zz61`. This value is used by later computations in this function.
719	`S(27) = q(3) + zz19.*zz241 - zz232.*zz61 + zz240.*zz33;`	Assigns `S(27)` from `q(3) + zz19.*zz241 - zz232.*zz61 + zz240.*zz33`. This value is used by later computations in this function.
720	`S(28) = zz237.*zz277 + zz278.*zz486 + zz294.*zz488;`	Assigns `S(28)` from `zz237.*zz277 + zz278.*zz486 + zz294.*zz488`. This value is used by later computations in this function.
721	`S(29) = zz238.*zz277 + zz239.*zz278 + zz294.*zz491;`	Assigns `S(29)` from `zz238.*zz277 + zz239.*zz278 + zz294.*zz491`. This value is used by later computations in this function.
722	`S(30) = -zz232.*zz294 + zz240.*zz278 + zz241.*zz277;`	Assigns `S(30)` from `-zz232.*zz294 + zz240.*zz278 + zz241.*zz277`. This value is used by later computations in this function.
723	`S(31) = -zz122.*zz277.*zz441 + zz278.*zz455 + zz294.*zz443;`	Assigns `S(31)` from `-zz122.*zz277.*zz441 + zz278.*zz455 + zz294.*zz443`. This value is used by later computations in this function.
724	`S(32) = zz144.*zz277 - zz146.*zz294 + zz193.*zz278;`	Assigns `S(32)` from `zz144.*zz277 - zz146.*zz294 + zz193.*zz278`. This value is used by later computations in this function.
725	`S(33) = zz61;`	Assigns `S(33)` from `zz61`. This value is used by later computations in this function.
726	`S(34) = zz33;`	Assigns `S(34)` from `zz33`. This value is used by later computations in this function.
727	`S(35) = zz19;`	Assigns `S(35)` from `zz19`. This value is used by later computations in this function.
728	`S(36) = zz119;`	Assigns `S(36)` from `zz119`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
729	`S(37) = zz121;`	Assigns `S(37)` from `zz121`. This value is used by later computations in this function.
730	`S(38) = zz439;`	Assigns `S(38)` from `zz439`. This value is used by later computations in this function.
731	`S(39) = zz10;`	Assigns `S(39)` from `zz10`. This value is used by later computations in this function.
732	`S(40) = zz440.*zz502 - zz441.*(zz122.*zz237 - zz144.*zz503);`	Assigns `S(40)` from `zz440.*zz502 - zz441.*(zz122.*zz237 - zz144.*zz503)`. This value is used by later computations in this function.
733	`S(41) = zz440.*zz504 - zz441.*(zz122.*zz238 - zz144.*zz505);`	Assigns `S(41)` from `zz440.*zz504 - zz441.*(zz122.*zz238 - zz144.*zz505)`. This value is used by later computations in this function.
734	`S(42) = zz440.*zz506 - zz441.*(zz122.*zz228.*zz230 - zz144.*zz507);`	Assigns `S(42)` from `zz440.*zz506 - zz441.*(zz122.*zz228.*zz230 - zz144.*zz507)`. This value is used by later computations in this function.
735	`S(43) = zz508;`	Assigns `S(43)` from `zz508`. This value is used by later computations in this function.
736	`S(44) = zz509;`	Assigns `S(44)` from `zz509`. This value is used by later computations in this function.
737	`S(45) = zz510;`	Assigns `S(45)` from `zz510`. This value is used by later computations in this function.
738	`S(46) = zz237.*zz348 + zz347.*zz502 + zz459.*zz508 + zz499;`	Assigns `S(46)` from `zz237.*zz348 + zz347.*zz502 + zz459.*zz508 + zz499`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
739	`S(47) = zz238.*zz348 + zz347.*zz504 + zz459.*zz509 + zz500;`	Assigns `S(47)` from `zz238.*zz348 + zz347.*zz504 + zz459.*zz509 + zz500`. This value is used by later computations in this function.
740	`S(48) = zz241.*zz348 + zz347.*zz506 + zz459.*zz510 + zz501;`	Assigns `S(48)` from `zz241.*zz348 + zz347.*zz506 + zz459.*zz510 + zz501`. This value is used by later computations in this function.
741	`S(49) = q(1) + zz237.*zz4 + zz25.*zz486 + zz488.*zz53;`	Assigns `S(49)` from `q(1) + zz237.*zz4 + zz25.*zz486 + zz488.*zz53`. This value is used by later computations in this function.
742	`S(50) = q(2) + zz238.*zz4 + zz239.*zz25 + zz491.*zz53;`	Assigns `S(50)` from `q(2) + zz238.*zz4 + zz239.*zz25 + zz491.*zz53`. This value is used by later computations in this function.
743	`S(51) = q(3) - zz232.*zz53 + zz240.*zz25 + zz241.*zz4;`	Assigns `S(51)` from `q(3) - zz232.*zz53 + zz240.*zz25 + zz241.*zz4`. This value is used by later computations in this function.
744	`S(52) = zz237.*zz252 + zz253.*zz486 + zz283.*zz488;`	Assigns `S(52)` from `zz237.*zz252 + zz253.*zz486 + zz283.*zz488`. This value is used by later computations in this function.
745	`S(53) = zz238.*zz252 + zz239.*zz253 + zz283.*zz491;`	Assigns `S(53)` from `zz238.*zz252 + zz239.*zz253 + zz283.*zz491`. This value is used by later computations in this function.
746	`S(54) = -zz232.*zz283 + zz240.*zz253 + zz241.*zz252;`	Assigns `S(54)` from `-zz232.*zz283 + zz240.*zz253 + zz241.*zz252`. This value is used by later computations in this function.
747	`S(55) = -zz101.*zz252.*zz355 + zz253.*zz369 + zz283.*zz357;`	Assigns `S(55)` from `-zz101.*zz252.*zz355 + zz253.*zz369 + zz283.*zz357`. This value is used by later computations in this function.
748	`S(56) = zz132.*zz252 - zz134.*zz283 + zz195.*zz253;`	Assigns `S(56)` from `zz132.*zz252 - zz134.*zz283 + zz195.*zz253`. This value is used by later computations in this function.
749	`S(57) = zz53;`	Assigns `S(57)` from `zz53`. This value is used by later computations in this function.
750	`S(58) = zz25;`	Assigns `S(58)` from `zz25`. This value is used by later computations in this function.
751	`S(59) = zz4;`	Assigns `S(59)` from `zz4`. This value is used by later computations in this function.
752	`S(60) = zz98;`	Assigns `S(60)` from `zz98`. This value is used by later computations in this function.
753	`S(61) = zz100;`	Assigns `S(61)` from `zz100`. This value is used by later computations in this function.
754	`S(62) = zz353;`	Assigns `S(62)` from `zz353`. This value is used by later computations in this function.
755	`S(64) = zz354.*zz511 - zz355.*(zz101.*zz237 - zz132.*zz512);`	Assigns `S(64)` from `zz354.*zz511 - zz355.*(zz101.*zz237 - zz132.*zz512)`. This value is used by later computations in this function.
756	`S(65) = zz354.*zz513 - zz355.*(zz101.*zz238 - zz132.*zz514);`	Assigns `S(65)` from `zz354.*zz513 - zz355.*(zz101.*zz238 - zz132.*zz514)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
757	`S(66) = zz354.*zz515 - zz355.*(zz101.*zz228.*zz230 - zz132.*zz516);`	Assigns `S(66)` from `zz354.*zz515 - zz355.*(zz101.*zz228.*zz230 - zz132.*zz516)`. This value is used by later computations in this function.
758	`S(67) = zz517;`	Assigns `S(67)` from `zz517`. This value is used by later computations in this function.
759	`S(68) = zz518;`	Assigns `S(68)` from `zz518`. This value is used by later computations in this function.
760	`S(69) = zz519;`	Assigns `S(69)` from `zz519`. This value is used by later computations in this function.
761	`S(70) = zz237.*zz321 + zz320.*zz511 + zz373.*zz517 + zz499;`	Assigns `S(70)` from `zz237.*zz321 + zz320.*zz511 + zz373.*zz517 + zz499`. This value is used by later computations in this function.
762	`S(71) = zz238.*zz321 + zz320.*zz513 + zz373.*zz518 + zz500;`	Assigns `S(71)` from `zz238.*zz321 + zz320.*zz513 + zz373.*zz518 + zz500`. This value is used by later computations in this function.
763	`S(72) = zz241.*zz321 + zz320.*zz515 + zz373.*zz519 + zz501;`	Assigns `S(72)` from `zz241.*zz321 + zz320.*zz515 + zz373.*zz519 + zz501`. This value is used by later computations in this function.
764	`S(73) = q(1) + zz23.*zz486 + zz237.*zz7 + zz488.*zz55;`	Assigns `S(73)` from `q(1) + zz23.*zz486 + zz237.*zz7 + zz488.*zz55`. This value is used by later computations in this function.
765	`S(74) = q(2) + zz23.*zz239 + zz238.*zz7 + zz491.*zz55;`	Assigns `S(74)` from `q(2) + zz23.*zz239 + zz238.*zz7 + zz491.*zz55`. This value is used by later computations in this function.
766	`S(75) = q(3) + zz23.*zz240 - zz232.*zz55 + zz241.*zz7;`	Assigns `S(75)` from `q(3) + zz23.*zz240 - zz232.*zz55 + zz241.*zz7`. This value is used by later computations in this function.
767	`S(76) = zz237.*zz258 + zz259.*zz486 + zz286.*zz488;`	Assigns `S(76)` from `zz237.*zz258 + zz259.*zz486 + zz286.*zz488`. This value is used by later computations in this function.
768	`S(77) = zz238.*zz258 + zz239.*zz259 + zz286.*zz491;`	Assigns `S(77)` from `zz238.*zz258 + zz239.*zz259 + zz286.*zz491`. This value is used by later computations in this function.
769	`S(78) = -zz232.*zz286 + zz240.*zz259 + zz241.*zz258;`	Assigns `S(78)` from `-zz232.*zz286 + zz240.*zz259 + zz241.*zz258`. This value is used by later computations in this function.
770	`S(79) = -zz106.*zz258.*zz380 + zz259.*zz394 + zz286.*zz382;`	Assigns `S(79)` from `-zz106.*zz258.*zz380 + zz259.*zz394 + zz286.*zz382`. This value is used by later computations in this function.
771	`S(80) = zz136.*zz258 - zz138.*zz286 + zz197.*zz259;`	Assigns `S(80)` from `zz136.*zz258 - zz138.*zz286 + zz197.*zz259`. This value is used by later computations in this function.
772	`S(81) = zz55;`	Assigns `S(81)` from `zz55`. This value is used by later computations in this function.
773	`S(82) = zz23;`	Assigns `S(82)` from `zz23`. This value is used by later computations in this function.
774	`S(83) = zz7;`	Assigns `S(83)` from `zz7`. This value is used by later computations in this function.
775	`S(84) = zz103;`	Assigns `S(84)` from `zz103`. This value is used by later computations in this function.
776	`S(85) = zz105;`	Assigns `S(85)` from `zz105`. This value is used by later computations in this function.
777	`S(86) = zz378;`	Assigns `S(86)` from `zz378`. This value is used by later computations in this function.
778	`S(88) = zz379.*zz520 - zz380.*(zz106.*zz237 - zz136.*zz521);`	Assigns `S(88)` from `zz379.*zz520 - zz380.*(zz106.*zz237 - zz136.*zz521)`. This value is used by later computations in this function.
779	`S(89) = zz379.*zz522 - zz380.*(zz106.*zz238 - zz136.*zz523);`	Assigns `S(89)` from `zz379.*zz522 - zz380.*(zz106.*zz238 - zz136.*zz523)`. This value is used by later computations in this function.
780	`S(90) = zz379.*zz524 - zz380.*(zz106.*zz228.*zz230 - zz136.*zz525);`	Assigns `S(90)` from `zz379.*zz524 - zz380.*(zz106.*zz228.*zz230 - zz136.*zz525)`. This value is used by later computations in this function.
781	`S(91) = zz526;`	Assigns `S(91)` from `zz526`. This value is used by later computations in this function.
782	`S(92) = zz527;`	Assigns `S(92)` from `zz527`. This value is used by later computations in this function.
783	`S(93) = zz528;`	Assigns `S(93)` from `zz528`. This value is used by later computations in this function.
784	`S(94) = zz237.*zz328 + zz327.*zz520 + zz398.*zz526 + zz499;`	Assigns `S(94)` from `zz237.*zz328 + zz327.*zz520 + zz398.*zz526 + zz499`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
785	`S(95) = zz238.*zz328 + zz327.*zz522 + zz398.*zz527 + zz500;`	Assigns `S(95)` from `zz238.*zz328 + zz327.*zz522 + zz398.*zz527 + zz500`. This value is used by later computations in this function.
786	`S(96) = zz241.*zz328 + zz327.*zz524 + zz398.*zz528 + zz501;`	Assigns `S(96)` from `zz241.*zz328 + zz327.*zz524 + zz398.*zz528 + zz501`. This value is used by later computations in this function.
787	`S(97) = -P(76).*zz232 + zz529;`	Assigns `S(97)` from `-P(76).*zz232 + zz529`. This value is used by later computations in this function.
788	`S(98) = -P(77).*zz232 + zz529;`	Assigns `S(98)` from `-P(77).*zz232 + zz529`. This value is used by later computations in this function.
789	`S(99) = u(1);`	Assigns `S(99)` from `u(1)`. This value is used by later computations in this function.
790	`end`	Ends the current function, conditional block, loop, or protected block.

File C - User-editable outer model

Defines the driver signals, total drive/brake commands, powertrain and brake torque split, tyre/contact model, road-contact calculations, aero model, and assembly of IN(34). This is the main file to edit for vehicle-model experiments.

Lines checked against uploaded code: 794. All code snippets below are sanitized aliases when required; line numbers still refer to the current uploaded file.

Line	Block/function	Purpose
1	vehicle14_outer_models_user	User-editable outer model that assembles the complete IN vector.
122	user_outer_model_options	Collects all user-editable outer-model constants and model switches.
195	user_control_signals	Defines throttle, braking, steering, and total torque commands.
269	steering_only	Helper function used by this file. Its line-by-line rows below describe the exact operations.
297	user_powertrain_brake_model	Converts total drive/brake commands into four wheel torques.
358	user_tyre_contact_wrench	Computes tyre/contact force, moment, normal load, slip, and diagnostics for one wheel.
426	tyre_force_tan_ellipse	Combined-slip tyre law inspired by the older MATLAB model.
455	tyre_force_linear_saturated	Fallback linear tyre law with friction saturation.
467	muxmax_muymax_func	Computes load-sensitive longitudinal and lateral friction limits.
485	smooth_ramp_func	Smoothly prevents negative friction-limit values.
489	user_aero_model	Computes drag and downforce inputs from body speed and ride-height proxies.
533	contact_kinematics_from_pose	Computes contact-point velocity and compression rate from wheel pose and motion.
563	oriented_tyre_contact_geometry	Finds the tyre contact point and compression using wheel and road geometry.
627	road_height_normal_at_point	Evaluates the selected road surface and normal near a point.
632	body_point_state_num	Helper function used by this file. Its line-by-line rows below describe the exact operations.
642	body_rotation_from_q	Builds the yaw-pitch-roll body rotation matrix from q.
646	rot_x	Returns a 3-by-3 rotation matrix about the named axis.
652	rot_y	Returns a 3-by-3 rotation matrix about the named axis.
658	rot_z	Returns a 3-by-3 rotation matrix about the named axis.
664	road_project_point	Projects a point to road coordinates and basis vectors.
687	projection_residual_given_s	Helper function used by this file. Its line-by-line rows below describe the exact operations.
695	road_surface_at	Evaluates either flat or ribbon road height and normal.
709	centerline_quantities	Computes the straight road centreline frame quantities.
740	smooth_step	Provides a smooth 0-to-1 transition to avoid discontinuous commands.
751	smoothplus	Helper function used by this file. Its line-by-line rows below describe the exact operations.
755	smooth_max	Helper function used by this file. Its line-by-line rows below describe the exact operations.
760	smooth_abs	Helper function used by this file. Its line-by-line rows below describe the exact operations.
764	normalize_num	Normalizes vectors while protecting against tiny norms.
773	getfield_default	Reads a struct field with a safe default fallback.
781	empty_contact_diag	Initializes contact diagnostic arrays.

Line-by-line explanation

Line	Sanitized code line	Explanation
1	`function [IN, aux] = vehicle14_outer_models_user(mode, t, x, S, par, args, road, meta)`	Defines the local MATLAB function `vehicle14_outer_models_user`. User-editable outer model that assembles the complete IN vector.
2	`%VEHICLE14_OUTER_MODELS_USER User-editable outer models for the 14-DOF generated dynamics/CasADi vehicle.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
3	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
4	`% This is the main file you should edit when you want to change:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
5	`% 1) control signals: total driving torque, total braking torque, steering;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
6	`% 2) powertrain model and torque split;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
7	`% 3) braking model and brake torque split;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
8	`% 4) tyre/contact model;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
9	`% 5) aero model.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
10	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
11	`% The generated file vehicle14_generated_dynamics_matrices_casadi.m should remain untouched.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
12	`% This file converts your outer-skeleton models into the 34-input vector IN`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
13	`% expected by the generated generated dynamics/CasADi equations of motion.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
14	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
15	`% Calling convention used by run_vehicle14_matlab_casadi_demo.m:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
16	`% [IN0, ctrl] = vehicle14_outer_models_user('control', t, x, [], par, args, road, meta)`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
17	`% [IN, aux] = vehicle14_outer_models_user('full', t, x, S, par, args, road, meta)`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
18	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
19	`% The 'control' mode returns only delta and delta_dot in IN. This is needed`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
20	`% before the sensor evaluation because the front wheel-plate geometry depends`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
21	`% on steering. The 'full' mode returns all tyre, powertrain/brake, and aero`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
22	`% inputs after the sensor vector S has been computed.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
23		Blank line used to separate logical blocks and improve readability.
24	`if nargin < 1 || isempty(mode)`	Starts a conditional branch. The following block executes only when this logical test is true.
25	`mode = 'full';`	Assigns `mode` from `'full'`. This value is used by later computations in this function.
26	`end`	Ends the current function, conditional block, loop, or protected block.
27	`mode = lower(char(mode));`	Assigns `mode` from `lower(char(mode))`. This value is used by later computations in this function.
28		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
29	`cfg = user_outer_model_options(par, args);`	Assigns `cfg` from `user_outer_model_options(par, args)`. This value is used by later computations in this function.
30	`ctrl = user_control_signals(t, x, par, args, cfg);`	Assigns `ctrl` from `user_control_signals(t, x, par, args, cfg)`. This value is used by later computations in this function.
31		Blank line used to separate logical blocks and improve readability.
32	`IN = zeros(meta.nin,1);`	Assigns `IN` from `zeros(meta.nin,1)`. This value is used by later computations in this function.
33	`IN(meta.input_index.delta) = ctrl.delta;`	Writes steering angle or steering rate into the external input vector before generated geometry/dynamics evaluation.
34	`IN(meta.input_index.delta_dot) = ctrl.delta_dot;`	Writes steering angle or steering rate into the external input vector before generated geometry/dynamics evaluation.
35		Blank line used to separate logical blocks and improve readability.
36	`aux = ctrl;`	Assigns `aux` from `ctrl`. This value is used by later computations in this function.
37	`aux.cfg = cfg;`	Assigns `aux.cfg` from `cfg`. This value is used by later computations in this function.
38	`aux.contact = empty_contact_diag();`	Assigns `aux.contact` from `empty_contact_diag()`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
39		Blank line used to separate logical blocks and improve readability.
40	`if strcmp(mode, 'control')`	Starts a conditional branch. The following block executes only when this logical test is true.
41	`return;`	Returns immediately from the current function.
42	`elseif ~strcmp(mode, 'full')`	Starts an alternative conditional branch, tested only if previous branches were false.
43	`error('Unknown vehicle14_outer_models_user mode: %s. Use control or full.', mode);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
44	`end`	Ends the current function, conditional block, loop, or protected block.
45		Blank line used to separate logical blocks and improve readability.
46	`if isempty(S)`	Starts a conditional branch. The following block executes only when this logical test is true.
47	`error('Full outer-model mode requires the sensor vector S from the generated generated dynamics/CasADi model.');`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
48	`end`	Ends the current function, conditional block, loop, or protected block.
49		Blank line used to separate logical blocks and improve readability.
50	`qv = x(1:14);`	Assigns `qv` from `x(1:14)`. This value is used by later computations in this function.
51	`uv = x(15:28);`	Assigns `uv` from `x(15:28)`. This value is used by later computations in this function.
52	`in = meta.input_index;`	Assigns `in` from `meta.input_index`. This value is used by later computations in this function.
53		Blank line used to separate logical blocks and improve readability.
54	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
55	`% 1. Powertrain and braking model: total torques -> four wheel torques.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
56	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
57	`[wheel_torque, torque_diag] = user_powertrain_brake_model(ctrl, x, S, par, args, cfg, meta);`	Executes this MATLAB statement in `vehicle14_outer_models_user`. It performs the operation shown by the sanitized code line.
58	`IN(in.TFL) = wheel_torque.FL;`	Writes a wheel torque into the external input vector for the corresponding wheel-spin equation.
59	`IN(in.TFR) = wheel_torque.FR;`	Writes a wheel torque into the external input vector for the corresponding wheel-spin equation.
60	`IN(in.TRL) = wheel_torque.RL;`	Writes a wheel torque into the external input vector for the corresponding wheel-spin equation.
61	`IN(in.TRR) = wheel_torque.RR;`	Writes a wheel torque into the external input vector for the corresponding wheel-spin equation.
62		Blank line used to separate logical blocks and improve readability.
63	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
64	`% 2. Tyre/contact model: road contact -> body-frame wheel-centre wrenches.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
65	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
66	`names = {'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This value is used by later computations in this function.
67	`idx = meta.sensor_index;`	Assigns `idx` from `meta.sensor_index`. This value is used by later computations in this function.
68	`contact = empty_contact_diag();`	Assigns `contact` from `empty_contact_diag()`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
69	`contact.names = names;`	Assigns `contact.names` from `names`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
70		Blank line used to separate logical blocks and improve readability.
71	`for ii = 1:4`	Starts a loop; the following block is repeated for the specified index range.
72	`nm = names{ii};`	Assigns `nm` from `names{ii}`. This value is used by later computations in this function.
73	`p_wc_N = [S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];`	Assigns `p_wc_N` from `[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))]`. This value is used by later computations in this function.
74	`v_wc_N = [S(idx.([nm '_Wc_vx'])); S(idx.([nm '_Wc_vy'])); S(idx.([nm '_Wc_vz']))];`	Assigns `v_wc_N` from `[S(idx.([nm '_Wc_vx'])); S(idx.([nm '_Wc_vy'])); S(idx.([nm '_Wc_vz']))]`. This value is used by later computations in this function.
75	`head_N = [S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];`	Assigns `head_N` from `[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))]`. This value is used by later computations in this function.
76	`spin_N = [S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];`	Assigns `spin_N` from `[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))]`. This value is used by later computations in this function.
77	`carrier_omega_N = [S(idx.([nm '_carrier_wx'])); S(idx.([nm '_carrier_wy'])); S(idx.([nm '_carrier_wz']))];`	Assigns `carrier_omega_N` from `[S(idx.([nm '_carrier_wx'])); S(idx.([nm '_carrier_wy'])); S(idx.([nm '_carrier_wz']))]`. This value is used by later computations in this function.
78	`omega = uv(10+ii);`	Assigns `omega` from `uv(10+ii)`. This value is used by later computations in this function.
79		Blank line used to separate logical blocks and improve readability.
80	`ck = contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega);`	Assigns `ck` from `contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
81	`[F_B, M_B, d] = user_tyre_contact_wrench(qv, par, road, p_wc_N, v_wc_N, head_N, spin_N, omega, carrier_omega_N, ck, cfg);`	Executes this MATLAB statement in `vehicle14_outer_models_user`. It performs the operation shown by the sanitized code line.
82		Blank line used to separate logical blocks and improve readability.
83	`IN(in.(['FBx' nm])) = F_B(1);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.
84	`IN(in.(['FBy' nm])) = F_B(2);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.

Line	Sanitized code line	Explanation
85	`IN(in.(['FBz' nm])) = F_B(3);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.
86	`IN(in.(['MBx' nm])) = M_B(1);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.
87	`IN(in.(['MBy' nm])) = M_B(2);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.
88	`IN(in.(['MBz' nm])) = M_B(3);`	Writes a body-frame tyre/contact force or moment component into the external input vector for one wheel.
89		Blank line used to separate logical blocks and improve readability.
90	`contact.Fx(ii,1) = d.Fx;`	Assigns `contact.Fx(ii,1)` from `d.Fx`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
91	`contact.Fy(ii,1) = d.Fy;`	Assigns `contact.Fy(ii,1)` from `d.Fy`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
92	`contact.Fz(ii,1) = d.Fz;`	Assigns `contact.Fz(ii,1)` from `d.Fz`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
93	`contact.kappa(ii,1) = d.kappa;`	Assigns `contact.kappa(ii,1)` from `d.kappa`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
94	`contact.alpha(ii,1) = d.alpha;`	Assigns `contact.alpha(ii,1)` from `d.alpha`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
95	`contact.compression(ii,1) = d.compression;`	Assigns `contact.compression(ii,1)` from `d.compression`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
96	`contact.road_z_at_wheel(ii,1) = d.road_patch_z;`	Assigns `contact.road_z_at_wheel(ii,1)` from `d.road_patch_z`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
97	`contact.contact_x(ii,1) = d.contact_x;`	Assigns `contact.contact_x(ii,1)` from `d.contact_x`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
98	`contact.contact_y(ii,1) = d.contact_y;`	Assigns `contact.contact_y(ii,1)` from `d.contact_y`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
99	`contact.contact_z(ii,1) = d.contact_z;`	Assigns `contact.contact_z(ii,1)` from `d.contact_z`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
100	`contact.mu_ratio(ii,1) = d.mu_ratio;`	Assigns `contact.mu_ratio(ii,1)` from `d.mu_ratio`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
101	`end`	Ends the current function, conditional block, loop, or protected block.
102		Blank line used to separate logical blocks and improve readability.
103	`% ------------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
104	`% 3. Aero model.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
105	`% ----------------------------------------------------------------------`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
106	`[FdragF, FdownF, FdragR, FdownR, aero_diag] = user_aero_model(t, x, S, par, args, road, meta, cfg);`	Executes this MATLAB statement in `vehicle14_outer_models_user`. It performs the operation shown by the sanitized code line.
107	`IN(in.FdragF) = FdragF;`	Writes aero drag or downforce input into the external input vector.
108	`IN(in.FdownF) = FdownF;`	Writes aero drag or downforce input into the external input vector.
109	`IN(in.FdragR) = FdragR;`	Writes aero drag or downforce input into the external input vector.
110	`IN(in.FdownR) = FdownR;`	Writes aero drag or downforce input into the external input vector.
111		Blank line used to separate logical blocks and improve readability.
112	`aux.contact = contact;`	Assigns `aux.contact` from `contact`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
113	`aux.torque = torque_diag;`	Assigns `aux.torque` from `torque_diag`. This value is used by later computations in this function.
114	`aux.aero = aero_diag;`	Assigns `aux.aero` from `aero_diag`. This value is used by later computations in this function.
115	`aux.wheel_torque = wheel_torque;`	Assigns `aux.wheel_torque` from `wheel_torque`. This value is used by later computations in this function.
116	`end`	Ends the current function, conditional block, loop, or protected block.
117		Blank line used to separate logical blocks and improve readability.
118	`% ========================================================================`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
119	`% USER-EDITABLE SECTION`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
120	`% ========================================================================`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
121		Blank line used to separate logical blocks and improve readability.
122	`function cfg = user_outer_model_options(par, args)`	Defines the local MATLAB function `user_outer_model_options`. Collects all user-editable outer-model constants and model switches.
123	`%USER_OUTER_MODEL_OPTIONS All editable model choices and constants in one place.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
124	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
125	`% This section is inspired by the user's older MATLAB model:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
126	`% - define total Tt, total Tb, and delta as user signals;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
127	`% - split brake torque with front bias kb and optional lateral redistribution;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
128	`% - use a combined-slip tan-ellipse tyre model when tyre parameters exist;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
129	`% - use smooth fallbacks so the demo still runs with the default vehicle14 parameters.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
130		Blank line used to separate logical blocks and improve readability.
131	`cfg.control_profile = getfield_default(args, 'maneuver_profile', 'aggressive-3d');`	Sets user outer-model option `cfg.control_profile` to `getfield_default(args, 'maneuver_profile', 'aggressive-3d')`. This option controls controls, tyre, brake, differential, or aero behavior.
132	`cfg.drop_settle_time = getfield_default(args, 'settle_time', 2.0);`	Sets user outer-model option `cfg.drop_settle_time` to `getfield_default(args, 'settle_time', 2.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
133	`cfg.maneuver_time = getfield_default(args, 'maneuver_time', 16.0);`	Sets user outer-model option `cfg.maneuver_time` to `getfield_default(args, 'maneuver_time', 16.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
134		Blank line used to separate logical blocks and improve readability.
135	`cfg.T_drive_max = getfield_default(par, 'T_drive_max', 1900.0); % total positive rear driving torque [Nm]`	Sets user outer-model option `cfg.T_drive_max` to `getfield_default(par, 'T_drive_max', 1900.0); % total positive rear driving torque [Nm]`. This option controls controls, tyre, brake, differential, or aero behavior.
136	`cfg.T_brake_max = getfield_default(par, 'T_brake_max', 4500.0); % total positive braking magnitude [Nm]`	Sets user outer-model option `cfg.T_brake_max` to `getfield_default(par, 'T_brake_max', 4500.0); % total positive braking magnitude [Nm]`. This option controls controls, tyre, brake, differential, or aero behavior.
137	`cfg.steer_max = getfield_default(par, 'steer_max', deg2rad(7.0));`	Sets user outer-model option `cfg.steer_max` to `getfield_default(par, 'steer_max', deg2rad(7.0))`. This option controls controls, tyre, brake, differential, or aero behavior.
138		Blank line used to separate logical blocks and improve readability.
139	`cfg.throttle_level = getfield_default(args, 'throttle', 0.65);`	Sets user outer-model option `cfg.throttle_level` to `getfield_default(args, 'throttle', 0.65)`. This option controls controls, tyre, brake, differential, or aero behavior.
140	`cfg.throttle_start = getfield_default(args, 'throttle_start', 0.20);`	Sets user outer-model option `cfg.throttle_start` to `getfield_default(args, 'throttle_start', 0.20)`. This option controls controls, tyre, brake, differential, or aero behavior.

Line	Sanitized code line	Explanation
141	`cfg.throttle_ramp = getfield_default(args, 'throttle_ramp', 0.45);`	Sets user outer-model option `cfg.throttle_ramp` to `getfield_default(args, 'throttle_ramp', 0.45)`. This option controls controls, tyre, brake, differential, or aero behavior.
142	`cfg.steer_deg = getfield_default(args, 'steer_deg', 7.0);`	Sets user outer-model option `cfg.steer_deg` to `getfield_default(args, 'steer_deg', 7.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
143	`cfg.steer_start = getfield_default(args, 'steer_start', 1.0);`	Sets user outer-model option `cfg.steer_start` to `getfield_default(args, 'steer_start', 1.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
144	`cfg.steer_ramp = getfield_default(args, 'steer_ramp', 0.5);`	Sets user outer-model option `cfg.steer_ramp` to `getfield_default(args, 'steer_ramp', 0.5)`. This option controls controls, tyre, brake, differential, or aero behavior.
145	`cfg.steer_freq = getfield_default(args, 'steer_freq', 0.55);`	Sets user outer-model option `cfg.steer_freq` to `getfield_default(args, 'steer_freq', 0.55)`. This option controls controls, tyre, brake, differential, or aero behavior.
146	`cfg.brake_start = getfield_default(args, 'brake_start', 11.5);`	Sets user outer-model option `cfg.brake_start` to `getfield_default(args, 'brake_start', 11.5)`. This option controls controls, tyre, brake, differential, or aero behavior.
147	`cfg.brake_level = getfield_default(args, 'brake_level', 0.28);`	Sets user outer-model option `cfg.brake_level` to `getfield_default(args, 'brake_level', 0.28)`. This option controls controls, tyre, brake, differential, or aero behavior.

Line	Sanitized code line	Explanation
148	`cfg.brake_ramp = getfield_default(args, 'brake_ramp', 0.35);`	Sets user outer-model option `cfg.brake_ramp` to `getfield_default(args, 'brake_ramp', 0.35)`. This option controls controls, tyre, brake, differential, or aero behavior.
149	`cfg.brake_hold = getfield_default(args, 'brake_hold', 1.15);`	Sets user outer-model option `cfg.brake_hold` to `getfield_default(args, 'brake_hold', 1.15)`. This option controls controls, tyre, brake, differential, or aero behavior.
150		Blank line used to separate logical blocks and improve readability.
151	`cfg.brake_bias_front = getfield_default(par, 'brake_bias_front', 0.65);`	Sets user outer-model option `cfg.brake_bias_front` to `getfield_default(par, 'brake_bias_front', 0.65)`. This option controls controls, tyre, brake, differential, or aero behavior.
152	`cfg.enable_lateral_brake_redistribution = true;`	Sets user outer-model option `cfg.enable_lateral_brake_redistribution` to `true`. This option controls controls, tyre, brake, differential, or aero behavior.
153	`cfg.brake_lateral_tuning_f = 0.50; % inspired by tuning_const_f in the old file`	Sets user outer-model option `cfg.brake_lateral_tuning_f` to `0.50; % inspired by tuning_const_f in the old file`. This option controls controls, tyre, brake, differential, or aero behavior.
154	`cfg.brake_lateral_tuning_r = 0.50; % inspired by tuning_const_r in the old file`	Sets user outer-model option `cfg.brake_lateral_tuning_r` to `0.50; % inspired by tuning_const_r in the old file`. This option controls controls, tyre, brake, differential, or aero behavior.
155	`cfg.ay_bar_max_for_split = 2.0*getfield_default(par, 'g', 9.81);`	Sets user outer-model option `cfg.ay_bar_max_for_split` to `2.0*getfield_default(par, 'g', 9.81)`. This option controls controls, tyre, brake, differential, or aero behavior.
156	`cfg.max_left_right_brake_bias = 0.35; % clamp zf/zr so each side remains positive`	Sets user outer-model option `cfg.max_left_right_brake_bias` to `0.35; % clamp zf/zr so each side remains positive`. This option controls controls, tyre, brake, differential, or aero behavior.
157		Blank line used to separate logical blocks and improve readability.
158	`cfg.enable_rear_diff = true;`	Sets user outer-model option `cfg.enable_rear_diff` to `true`. This option controls controls, tyre, brake, differential, or aero behavior.
159	`cfg.diff_gain = getfield_default(par, 'diff_gain', 35.0);`	Sets user outer-model option `cfg.diff_gain` to `getfield_default(par, 'diff_gain', 35.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
160	`cfg.diff_width = getfield_default(par, 'diff_width', 15.0);`	Sets user outer-model option `cfg.diff_width` to `getfield_default(par, 'diff_width', 15.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
161		Blank line used to separate logical blocks and improve readability.
162	`cfg.tyre_model = 'tan-ellipse'; % 'tan-ellipse' or 'linear-saturated'`	Sets user outer-model option `cfg.tyre_model` to `'tan-ellipse'; % 'tan-ellipse' or 'linear-saturated'`. This option controls controls, tyre, brake, differential, or aero behavior.
163	`cfg.my_effective_mu_scaling = getfield_default(par, 'my_effective_mu_scaling', 1.0);`	Sets user outer-model option `cfg.my_effective_mu_scaling` to `getfield_default(par, 'my_effective_mu_scaling', 1.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
164	`cfg.v_eps = getfield_default(par, 'v_eps', 1.0);`	Sets user outer-model option `cfg.v_eps` to `getfield_default(par, 'v_eps', 1.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
165	`cfg.kt = getfield_default(par, 'kt', 220000.0);`	Sets user outer-model option `cfg.kt` to `getfield_default(par, 'kt', 220000.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
166	`cfg.ct = getfield_default(par, 'ct', 1400.0);`	Sets user outer-model option `cfg.ct` to `getfield_default(par, 'ct', 1400.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
167	`cfg.normal_smooth_eps = getfield_default(par, 'normal_smooth_eps', 30.0);`	Sets user outer-model option `cfg.normal_smooth_eps` to `getfield_default(par, 'normal_smooth_eps', 30.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
168	`cfg.mu = getfield_default(par, 'mu', 1.15);`	Sets user outer-model option `cfg.mu` to `getfield_default(par, 'mu', 1.15)`. This option controls controls, tyre, brake, differential, or aero behavior.

Line	Sanitized code line	Explanation
169	`cfg.Ck = getfield_default(par, 'Ck', 90000.0);`	Sets user outer-model option `cfg.Ck` to `getfield_default(par, 'Ck', 90000.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
170	`cfg.Ca = getfield_default(par, 'Ca', 85000.0);`	Sets user outer-model option `cfg.Ca` to `getfield_default(par, 'Ca', 85000.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
171		Blank line used to separate logical blocks and improve readability.
172	`% Tan-ellipse parameters. If your my_params() has the old fields, these`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
173	`% values can be replaced by par/veh fields. The fallback values are moderate`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
174	`% and solver-friendly for a runnable demonstration.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
175	`cfg.Bx = getfield_default(par, 'Bx', 10.0);`	Sets user outer-model option `cfg.Bx` to `getfield_default(par, 'Bx', 10.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
176	`cfg.Cx = getfield_default(par, 'Cx', 1.65);`	Sets user outer-model option `cfg.Cx` to `getfield_default(par, 'Cx', 1.65)`. This option controls controls, tyre, brake, differential, or aero behavior.
177	`cfg.By = getfield_default(par, 'By', 7.5);`	Sets user outer-model option `cfg.By` to `getfield_default(par, 'By', 7.5)`. This option controls controls, tyre, brake, differential, or aero behavior.
178	`cfg.Cy = getfield_default(par, 'Cy', 1.30);`	Sets user outer-model option `cfg.Cy` to `getfield_default(par, 'Cy', 1.30)`. This option controls controls, tyre, brake, differential, or aero behavior.
179	`cfg.Qx = getfield_default(par, 'Qx', 1.60);`	Sets user outer-model option `cfg.Qx` to `getfield_default(par, 'Qx', 1.60)`. This option controls controls, tyre, brake, differential, or aero behavior.
180	`cfg.Qy = getfield_default(par, 'Qy', 1.50);`	Sets user outer-model option `cfg.Qy` to `getfield_default(par, 'Qy', 1.50)`. This option controls controls, tyre, brake, differential, or aero behavior.
181	`cfg.eps1 = getfield_default(par, 'eps1', 1.0e-8);`	Sets user outer-model option `cfg.eps1` to `getfield_default(par, 'eps1', 1.0e-8)`. This option controls controls, tyre, brake, differential, or aero behavior.
182	`cfg.eps2 = getfield_default(par, 'eps2', 1.0e-6);`	Sets user outer-model option `cfg.eps2` to `getfield_default(par, 'eps2', 1.0e-6)`. This option controls controls, tyre, brake, differential, or aero behavior.
183	`cfg.Fz1 = getfield_default(par, 'Fz1', 1000.0);`	Sets user outer-model option `cfg.Fz1` to `getfield_default(par, 'Fz1', 1000.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
184	`cfg.Fz2 = getfield_default(par, 'Fz2', 6000.0);`	Sets user outer-model option `cfg.Fz2` to `getfield_default(par, 'Fz2', 6000.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
185	`cfg.d1x = getfield_default(par, 'd1x', cfg.mu);`	Sets user outer-model option `cfg.d1x` to `getfield_default(par, 'd1x', cfg.mu)`. This option controls controls, tyre, brake, differential, or aero behavior.
186	`cfg.d2x = getfield_default(par, 'd2x', 0.0);`	Sets user outer-model option `cfg.d2x` to `getfield_default(par, 'd2x', 0.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
187	`cfg.d1y = getfield_default(par, 'd1y', cfg.mu);`	Sets user outer-model option `cfg.d1y` to `getfield_default(par, 'd1y', cfg.mu)`. This option controls controls, tyre, brake, differential, or aero behavior.
188	`cfg.d2y = getfield_default(par, 'd2y', 0.0);`	Sets user outer-model option `cfg.d2y` to `getfield_default(par, 'd2y', 0.0)`. This option controls controls, tyre, brake, differential, or aero behavior.
189	`cfg.rou_smooth = getfield_default(par, 'rou_smooth', 1.0e-3);`	Sets user outer-model option `cfg.rou_smooth` to `getfield_default(par, 'rou_smooth', 1.0e-3)`. This option controls controls, tyre, brake, differential, or aero behavior.
190		Blank line used to separate logical blocks and improve readability.
191	`cfg.disable_aero = getfield_default(args, 'disable_aero', false);`	Sets user outer-model option `cfg.disable_aero` to `getfield_default(args, 'disable_aero', false)`. This option controls controls, tyre, brake, differential, or aero behavior.

Line	Sanitized code line	Explanation
192	`cfg.aero_h_min_multiplier = 0.20;`	Sets user outer-model option `cfg.aero_h_min_multiplier` to `0.20`. This option controls controls, tyre, brake, differential, or aero behavior.
193	`end`	Ends the current function, conditional block, loop, or protected block.
194		Blank line used to separate logical blocks and improve readability.
195	`function ctrl = user_control_signals(t_abs, x, par, args, cfg)`	Defines the local MATLAB function `user_control_signals`. Defines throttle, braking, steering, and total torque commands.
196	`%USER_CONTROL_SIGNALS Define total drive torque, total braking torque, and steering.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.

Line	Sanitized code line	Explanation
197	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
198	`% This is the direct analogue of the old MATLAB variables:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
199	`% Tt = total driving torque, positive [Nm]`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
200	`% Tb = total braking torque magnitude, positive here [Nm]`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
201	`% delta = steering angle [rad]`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
202	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
203	`% Edit this function when you want to impose another maneuver, controller,`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
204	`% MPC reference, or measured actuator trace.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
205		Blank line used to separate logical blocks and improve readability.
206	`if t_abs < cfg.drop_settle_time`	Starts a conditional branch. The following block executes only when this logical test is true.
207	`throttle = 0.0;`	Assigns `throttle` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
208	`brake = 0.0;`	Assigns `brake` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
209	`delta = 0.0;`	Assigns `delta` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
210	`else`	Starts the fallback branch for the current conditional block.
211	`tau = t_abs - cfg.drop_settle_time;`	Assigns `tau` from `t_abs - cfg.drop_settle_time`. This value is used by later computations in this function.
212	`profile = lower(cfg.control_profile);`	Assigns `profile` from `lower(cfg.control_profile)`. This value is used by later computations in this function.
213		Blank line used to separate logical blocks and improve readability.
214	`if strcmp(profile, 'straight-launch')`	Starts a conditional branch. The following block executes only when this logical test is true.
215	`throttle = cfg.throttle_level * smooth_step(tau, cfg.throttle_start, cfg.throttle_start + cfg.throttle_ramp);`	Assigns `throttle` from `cfg.throttle_level * smooth_step(tau, cfg.throttle_start, cfg.throttle_start + cfg.throttle_ramp)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
216	`brake = 0.0;`	Assigns `brake` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
217	`delta = 0.0;`	Assigns `delta` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
218	`elseif strcmp(profile, 'creep-steer')`	Starts an alternative conditional branch, tested only if previous branches were false.
219	`throttle = cfg.throttle_level * smooth_step(tau, cfg.throttle_start, cfg.throttle_start + cfg.throttle_ramp);`	Assigns `throttle` from `cfg.throttle_level * smooth_step(tau, cfg.throttle_start, cfg.throttle_start + cfg.throttle_ramp)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
220	`brake = 0.0;`	Assigns `brake` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
221	`if cfg.brake_start >= 0.0 && tau >= cfg.brake_start`	Starts a conditional branch. The following block executes only when this logical test is true.
222	`bw = smooth_step(tau, cfg.brake_start, cfg.brake_start + cfg.brake_ramp);`	Assigns `bw` from `smooth_step(tau, cfg.brake_start, cfg.brake_start + cfg.brake_ramp)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
223	`brake = cfg.brake_level*bw;`	Assigns `brake` from `cfg.brake_level*bw`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
224	`throttle = throttle*(1.0 - bw);`	Assigns `throttle` from `throttle*(1.0 - bw)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.

Line	Sanitized code line	Explanation
225	`end`	Ends the current function, conditional block, loop, or protected block.
226	`steer_amp = deg2rad(cfg.steer_deg);`	Assigns `steer_amp` from `deg2rad(cfg.steer_deg)`. This value is used by later computations in this function.
227	`env = smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp);`	Assigns `env` from `smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp)`. This value is used by later computations in this function.
228	`delta = steer_amp*env*sin(2*pi*cfg.steer_freq*max(tau - cfg.steer_start, 0.0));`	Assigns `delta` from `steer_amp*env*sin(2*pi*cfg.steer_freq*max(tau - cfg.steer_start, 0.0))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
229	`elseif strcmp(profile, 'source-demo')`	Starts an alternative conditional branch, tested only if previous branches were false.
230	`throttle = 0.55*smooth_step(tau, 0.20, 0.80);`	Assigns `throttle` from `0.55*smooth_step(tau, 0.20, 0.80)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
231	`brake = 0.0;`	Assigns `brake` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
232	`delta = cfg.steer_max*smooth_step(tau, 0.80, 1.20)*sin(2*pi*0.35*max(tau - 0.80, 0.0));`	Assigns `delta` from `cfg.steer_max*smooth_step(tau, 0.80, 1.20)*sin(2*pi*0.35*max(tau - 0.80, 0.0))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
233	`elseif strcmp(profile, 'aggressive-3d')`	Starts an alternative conditional branch, tested only if previous branches were false.
234	`launch = smooth_step(tau, 0.10, 0.55);`	Assigns `launch` from `smooth_step(tau, 0.10, 0.55)`. This value is used by later computations in this function.
235	`throttle = cfg.throttle_level * launch;`	Assigns `throttle` from `cfg.throttle_level * launch`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
236	`brake_window = 0.0;`	Assigns `brake_window` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
237	`if cfg.brake_start >= 0.0`	Starts a conditional branch. The following block executes only when this logical test is true.

Line	Sanitized code line	Explanation
238	`brake_window = smooth_step(tau, cfg.brake_start, cfg.brake_start + cfg.brake_ramp) * ...`	Assigns `brake_window` from `smooth_step(tau, cfg.brake_start, cfg.brake_start + cfg.brake_ramp) * ...`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
239	`(1.0 - smooth_step(tau, cfg.brake_start + cfg.brake_hold, cfg.brake_start + cfg.brake_hold + cfg.brake_ramp));`	Executes this MATLAB statement in `user_control_signals`. It performs the operation shown by the sanitized code line.
240	`end`	Ends the current function, conditional block, loop, or protected block.
241	`brake = cfg.brake_level * brake_window;`	Assigns `brake` from `cfg.brake_level * brake_window`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
242	`throttle = throttle * (1.0 - 0.85*brake_window);`	Assigns `throttle` from `throttle * (1.0 - 0.85*brake_window)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
243	`steer_amp = deg2rad(cfg.steer_deg);`	Assigns `steer_amp` from `deg2rad(cfg.steer_deg)`. This value is used by later computations in this function.
244	`env = smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp) * ...`	Assigns `env` from `smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp) * ...`. This value is used by later computations in this function.
245	`(1.0 - smooth_step(tau, cfg.maneuver_time - 1.0, cfg.maneuver_time - 0.2));`	Executes this MATLAB statement in `user_control_signals`. It performs the operation shown by the sanitized code line.
246	`tt = max(tau - cfg.steer_start, 0.0);`	Assigns `tt` from `max(tau - cfg.steer_start, 0.0)`. This value is used by later computations in this function.
247	`raw = 0.78*sin(2*pi*cfg.steer_freq*tt) + 0.28*sin(2*pi*1.75*cfg.steer_freq*tt + 0.45);`	Assigns `raw` from `0.78*sin(2*pi*cfg.steer_freq*tt) + 0.28*sin(2*pi*1.75*cfg.steer_freq*tt + 0.45)`. This value is used by later computations in this function.
248	`delta = steer_amp * env * min(max(raw, -1.0), 1.0);`	Assigns `delta` from `steer_amp * env * min(max(raw, -1.0), 1.0)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
249	`else`	Starts the fallback branch for the current conditional block.
250	`error('Unknown control_profile/maneuver_profile: %s', profile);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
251	`end`	Ends the current function, conditional block, loop, or protected block.
252	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
253		Blank line used to separate logical blocks and improve readability.
254	`% Finite-difference steering rate. This keeps the user signal definition`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
255	`% simple and avoids adding delta as an independent state in this demo.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
256	`h = 1.0e-5;`	Assigns `h` from `1.0e-5`. This value is used by later computations in this function.
257	`delta_p = steering_only(t_abs + h, cfg);`	Assigns `delta_p` from `steering_only(t_abs + h, cfg)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
258	`delta_m = steering_only(t_abs - h, cfg);`	Assigns `delta_m` from `steering_only(t_abs - h, cfg)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
259	`delta_dot = (delta_p - delta_m)/(2*h);`	Assigns `delta_dot` from `(delta_p - delta_m)/(2*h)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
260		Blank line used to separate logical blocks and improve readability.
261	`ctrl.throttle = throttle;`	Assigns `ctrl.throttle` from `throttle`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
262	`ctrl.brake = brake;`	Assigns `ctrl.brake` from `brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
263	`ctrl.T_drive_total = cfg.T_drive_max * throttle;`	Assigns `ctrl.T_drive_total` from `cfg.T_drive_max * throttle`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
264	`ctrl.T_brake_total = cfg.T_brake_max * brake;`	Assigns `ctrl.T_brake_total` from `cfg.T_brake_max * brake`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
265	`ctrl.delta = delta;`	Assigns `ctrl.delta` from `delta`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
266	`ctrl.delta_dot = delta_dot;`	Assigns `ctrl.delta_dot` from `delta_dot`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
267	`end`	Ends the current function, conditional block, loop, or protected block.
268		Blank line used to separate logical blocks and improve readability.
269	`function delta = steering_only(t_abs, cfg)`	Defines the local MATLAB function `steering_only`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
270	`%STEERING_ONLY Helper used only for delta_dot finite difference.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
271	`if t_abs < cfg.drop_settle_time`	Starts a conditional branch. The following block executes only when this logical test is true.
272	`delta = 0.0;`	Assigns `delta` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
273	`return;`	Returns immediately from the current function.
274	`end`	Ends the current function, conditional block, loop, or protected block.
275	`tau = t_abs - cfg.drop_settle_time;`	Assigns `tau` from `t_abs - cfg.drop_settle_time`. This value is used by later computations in this function.
276	`profile = lower(cfg.control_profile);`	Assigns `profile` from `lower(cfg.control_profile)`. This value is used by later computations in this function.
277	`if strcmp(profile, 'straight-launch')`	Starts a conditional branch. The following block executes only when this logical test is true.
278	`delta = 0.0;`	Assigns `delta` from `0.0`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
279	`elseif strcmp(profile, 'creep-steer')`	Starts an alternative conditional branch, tested only if previous branches were false.
280	`steer_amp = deg2rad(cfg.steer_deg);`	Assigns `steer_amp` from `deg2rad(cfg.steer_deg)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
281	`env = smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp);`	Assigns `env` from `smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
282	`delta = steer_amp*env*sin(2*pi*cfg.steer_freq*max(tau - cfg.steer_start, 0.0));`	Assigns `delta` from `steer_amp*env*sin(2*pi*cfg.steer_freq*max(tau - cfg.steer_start, 0.0))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
283	`elseif strcmp(profile, 'source-demo')`	Starts an alternative conditional branch, tested only if previous branches were false.
284	`delta = cfg.steer_max*smooth_step(tau, 0.80, 1.20)*sin(2*pi*0.35*max(tau - 0.80, 0.0));`	Assigns `delta` from `cfg.steer_max*smooth_step(tau, 0.80, 1.20)*sin(2*pi*0.35*max(tau - 0.80, 0.0))`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
285	`elseif strcmp(profile, 'aggressive-3d')`	Starts an alternative conditional branch, tested only if previous branches were false.
286	`steer_amp = deg2rad(cfg.steer_deg);`	Assigns `steer_amp` from `deg2rad(cfg.steer_deg)`. This value is used by later computations in this function.
287	`env = smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp) * ...`	Assigns `env` from `smooth_step(tau, cfg.steer_start, cfg.steer_start + cfg.steer_ramp) * ...`. This value is used by later computations in this function.
288	`(1.0 - smooth_step(tau, cfg.maneuver_time - 1.0, cfg.maneuver_time - 0.2));`	Executes this MATLAB statement in `steering_only`. It performs the operation shown by the sanitized code line.
289	`tt = max(tau - cfg.steer_start, 0.0);`	Assigns `tt` from `max(tau - cfg.steer_start, 0.0)`. This value is used by later computations in this function.
290	`raw = 0.78*sin(2*pi*cfg.steer_freq*tt) + 0.28*sin(2*pi*1.75*cfg.steer_freq*tt + 0.45);`	Assigns `raw` from `0.78*sin(2*pi*cfg.steer_freq*tt) + 0.28*sin(2*pi*1.75*cfg.steer_freq*tt + 0.45)`. This value is used by later computations in this function.
291	`delta = steer_amp * env * min(max(raw, -1.0), 1.0);`	Assigns `delta` from `steer_amp * env * min(max(raw, -1.0), 1.0)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
292	`else`	Starts the fallback branch for the current conditional block.
293	`error('Unknown control_profile/maneuver_profile: %s', profile);`	Raises a MATLAB error when an invalid option or inconsistent input is detected.
294	`end`	Ends the current function, conditional block, loop, or protected block.
295	`end`	Ends the current function, conditional block, loop, or protected block.
296		Blank line used to separate logical blocks and improve readability.
297	`function [wheel_torque, diag] = user_powertrain_brake_model(ctrl, x, S, par, args, cfg, meta)`	Defines the local MATLAB function `user_powertrain_brake_model`. Converts total drive/brake commands into four wheel torques.
298	`%USER_POWERTRAIN_BRAKE_MODEL Split total Tt/Tb into wheel torques.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
299	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
300	`% Sign convention for wheel torques sent to the generated dynamics model:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
301	`% positive torque drives wheel spin forward;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
302	`% negative torque brakes the wheel.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
303	`%`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
304	`% This follows the spirit of the old MATLAB code:`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
305	`% kb = front brake bias;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
306	`% Tbf = kb*Tb; Tbr = (1-kb)*Tb;`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
307	`% optional left/right brake redistribution based on lateral acceleration.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
308		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
309	`uv = x(15:28);`	Assigns `uv` from `x(15:28)`. This value is used by later computations in this function.
310	`Tt = ctrl.T_drive_total; % positive drive torque magnitude`	Assigns `Tt` from `ctrl.T_drive_total; % positive drive torque magnitude`. This value is used by later computations in this function.
311	`Tb = ctrl.T_brake_total; % positive brake torque magnitude`	Assigns `Tb` from `ctrl.T_brake_total; % positive brake torque magnitude`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
312	`kb = cfg.brake_bias_front;`	Assigns `kb` from `cfg.brake_bias_front`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
313		Blank line used to separate logical blocks and improve readability.
314	`% Rear-drive open-differential-like split with smooth speed-sensitive bias.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
315	`omRL = uv(13);`	Assigns `omRL` from `uv(13)`. This value is used by later computations in this function.
316	`omRR = uv(14);`	Assigns `omRR` from `uv(14)`. This value is used by later computations in this function.
317	`if cfg.enable_rear_diff`	Starts a conditional branch. The following block executes only when this logical test is true.
318	`diff_bias = cfg.diff_gain * cfg.diff_width * tanh((omRL - omRR)/cfg.diff_width);`	Assigns `diff_bias` from `cfg.diff_gain * cfg.diff_width * tanh((omRL - omRR)/cfg.diff_width)`. This value is used by later computations in this function.
319	`else`	Starts the fallback branch for the current conditional block.
320	`diff_bias = 0.0;`	Assigns `diff_bias` from `0.0`. This value is used by later computations in this function.
321	`end`	Ends the current function, conditional block, loop, or protected block.
322	`Tdrive_RL = 0.5*Tt - diff_bias;`	Assigns `Tdrive_RL` from `0.5*Tt - diff_bias`. This value is used by later computations in this function.
323	`Tdrive_RR = 0.5*Tt + diff_bias;`	Assigns `Tdrive_RR` from `0.5*Tt + diff_bias`. This value is used by later computations in this function.
324		Blank line used to separate logical blocks and improve readability.
325	`% Approximate lateral acceleration proxy for brake redistribution. This is`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
326	`% deliberately algebraic in the current state and therefore compatible with`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
327	`% ode15i. You can replace it with your own controller signal if desired.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
328	`vx = uv(1);`	Assigns `vx` from `uv(1)`. This value is used by later computations in this function.
329	`r = uv(6);`	Assigns `r` from `uv(6)`. This value is used by later computations in this function.
330	`ay_proxy = vx*r;`	Assigns `ay_proxy` from `vx*r`. This value is used by later computations in this function.
331	`if cfg.enable_lateral_brake_redistribution`	Starts a conditional branch. The following block executes only when this logical test is true.

Line	Sanitized code line	Explanation
332	`zf = cfg.brake_lateral_tuning_f * 0.5/cfg.ay_bar_max_for_split * ay_proxy;`	Assigns `zf` from `cfg.brake_lateral_tuning_f * 0.5/cfg.ay_bar_max_for_split * ay_proxy`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
333	`zr = cfg.brake_lateral_tuning_r * 0.5/cfg.ay_bar_max_for_split * ay_proxy;`	Assigns `zr` from `cfg.brake_lateral_tuning_r * 0.5/cfg.ay_bar_max_for_split * ay_proxy`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
334	`zf = min(max(zf, -cfg.max_left_right_brake_bias), cfg.max_left_right_brake_bias);`	Assigns `zf` from `min(max(zf, -cfg.max_left_right_brake_bias), cfg.max_left_right_brake_bias)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
335	`zr = min(max(zr, -cfg.max_left_right_brake_bias), cfg.max_left_right_brake_bias);`	Assigns `zr` from `min(max(zr, -cfg.max_left_right_brake_bias), cfg.max_left_right_brake_bias)`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
336	`else`	Starts the fallback branch for the current conditional block.

Line	Sanitized code line	Explanation
337	`zf = 0.0;`	Assigns `zf` from `0.0`. This value is used by later computations in this function.
338	`zr = 0.0;`	Assigns `zr` from `0.0`. This value is used by later computations in this function.
339	`end`	Ends the current function, conditional block, loop, or protected block.
340		Blank line used to separate logical blocks and improve readability.
341	`Tbf = kb*Tb;`	Assigns `Tbf` from `kb*Tb`. This value is used by later computations in this function.
342	`Tbr = (1.0-kb)*Tb;`	Assigns `Tbr` from `(1.0-kb)*Tb`. This value is used by later computations in this function.
343		Blank line used to separate logical blocks and improve readability.
344	`wheel_torque.FL = -(0.5 - zf)*Tbf;`	Assigns `wheel_torque.FL` from `-(0.5 - zf)*Tbf`. This value is used by later computations in this function.
345	`wheel_torque.FR = -(0.5 + zf)*Tbf;`	Assigns `wheel_torque.FR` from `-(0.5 + zf)*Tbf`. This value is used by later computations in this function.
346	`wheel_torque.RL = Tdrive_RL - (0.5 - zr)*Tbr;`	Assigns `wheel_torque.RL` from `Tdrive_RL - (0.5 - zr)*Tbr`. This value is used by later computations in this function.
347	`wheel_torque.RR = Tdrive_RR - (0.5 + zr)*Tbr;`	Assigns `wheel_torque.RR` from `Tdrive_RR - (0.5 + zr)*Tbr`. This value is used by later computations in this function.
348		Blank line used to separate logical blocks and improve readability.
349	`diag.T_drive_total = Tt;`	Assigns `diag.T_drive_total` from `Tt`. This value is used by later computations in this function.
350	`diag.T_brake_total = Tb;`	Assigns `diag.T_brake_total` from `Tb`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
351	`diag.brake_bias_front = kb;`	Assigns `diag.brake_bias_front` from `kb`. This value is used by later computations in this function. This line is part of the driver-command or steering logic.
352	`diag.zf = zf;`	Assigns `diag.zf` from `zf`. This value is used by later computations in this function.
353	`diag.zr = zr;`	Assigns `diag.zr` from `zr`. This value is used by later computations in this function.
354	`diag.ay_proxy = ay_proxy;`	Assigns `diag.ay_proxy` from `ay_proxy`. This value is used by later computations in this function.
355	`diag.diff_bias = diff_bias;`	Assigns `diag.diff_bias` from `diff_bias`. This value is used by later computations in this function.
356	`end`	Ends the current function, conditional block, loop, or protected block.
357		Blank line used to separate logical blocks and improve readability.
358	`function [F_B, M_B, diag] = user_tyre_contact_wrench(qv, par, road, p_wc_N, v_wc_N, wheel_heading_N, wheel_spin_axis_N, omega, carrier_omega_N, ck, cfg)`	Defines the local MATLAB function `user_tyre_contact_wrench`. Computes tyre/contact force, moment, normal load, slip, and diagnostics for one wheel.
359	`%USER_TYRE_CONTACT_WRENCH 3D road/contact/tyre model external to generated-core.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
360	`R_NB = body_rotation_from_q(qv);`	Assigns `R_NB` from `body_rotation_from_q(qv)`. This value is used by later computations in this function.
361	`R_BN = R_NB.';`	Assigns `R_BN` from `R_NB.'`. This value is used by later computations in this function.
362	`geom = oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N);`	Assigns `geom` from `oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
363		Blank line used to separate logical blocks and improve readability.
364	`n_road_N = geom.n_road_N;`	Assigns `n_road_N` from `geom.n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
365	`t_long_N = geom.t_long_N;`	Assigns `t_long_N` from `geom.t_long_N`. This value is used by later computations in this function.
366	`t_lat_N = geom.t_lat_N;`	Assigns `t_lat_N` from `geom.t_lat_N`. This value is used by later computations in this function.
367	`r_wc_to_contact_N = geom.r_wc_to_contact_N;`	Assigns `r_wc_to_contact_N` from `geom.r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
368	`compression = geom.compression;`	Assigns `compression` from `geom.compression`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
369	`compression_rate = ck.compression_rate_normal;`	Assigns `compression_rate` from `ck.compression_rate_normal`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
370	`compression_rate_radial = ck.compression_rate_radial;`	Assigns `compression_rate_radial` from `ck.compression_rate_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
371	`v_contact_geom_N = ck.contact_point_velocity_geom_N;`	Assigns `v_contact_geom_N` from `ck.contact_point_velocity_geom_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
372	`v_contact_carrier_N = ck.contact_patch_velocity_carrier_N;`	Assigns `v_contact_carrier_N` from `ck.contact_patch_velocity_carrier_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
373	`v_spin_contact_N = ck.contact_patch_velocity_spin_N;`	Assigns `v_spin_contact_N` from `ck.contact_patch_velocity_spin_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
374	`v_contact_material_N = ck.contact_patch_velocity_material_N;`	Assigns `v_contact_material_N` from `ck.contact_patch_velocity_material_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
375		Blank line used to separate logical blocks and improve readability.
376	`Fz = smoothplus(cfg.kt*compression + cfg.ct*compression_rate, cfg.normal_smooth_eps);`	Assigns `Fz` from `smoothplus(cfg.kt*compression + cfg.ct*compression_rate, cfg.normal_smooth_eps)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.
377	`Vx = dot(v_contact_carrier_N, t_long_N);`	Assigns `Vx` from `dot(v_contact_carrier_N, t_long_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
378	`Vy = dot(v_contact_carrier_N, t_lat_N);`	Assigns `Vy` from `dot(v_contact_carrier_N, t_lat_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
379	`Vx_material = dot(v_contact_material_N, t_long_N);`	Assigns `Vx_material` from `dot(v_contact_material_N, t_long_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
380	`Vy_material = dot(v_contact_material_N, t_lat_N);`	Assigns `Vy_material` from `dot(v_contact_material_N, t_lat_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
381	`rolling_speed = -dot(v_spin_contact_N, t_long_N);`	Assigns `rolling_speed` from `-dot(v_spin_contact_N, t_long_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
382	`Vden = sqrt(Vx*Vx + cfg.v_eps*cfg.v_eps);`	Assigns `Vden` from `sqrt(Vx*Vx + cfg.v_eps*cfg.v_eps)`. This value is used by later computations in this function.
383	`kappa = (rolling_speed - Vx)/Vden;`	Assigns `kappa` from `(rolling_speed - Vx)/Vden`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
384	`alpha = atan2(Vy, Vden);`	Assigns `alpha` from `atan2(Vy, Vden)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
385		Blank line used to separate logical blocks and improve readability.
386	`if strcmpi(cfg.tyre_model, 'tan-ellipse')`	Starts a conditional branch. The following block executes only when this logical test is true.
387	`[Fx, Fy, mu_ratio] = tyre_force_tan_ellipse(cfg, kappa, alpha, Fz);`	Executes this MATLAB statement in `user_tyre_contact_wrench`. It performs the operation shown by the sanitized code line.
388	`else`	Starts the fallback branch for the current conditional block.
389	`[Fx, Fy, mu_ratio] = tyre_force_linear_saturated(cfg, kappa, alpha, Fz);`	Executes this MATLAB statement in `user_tyre_contact_wrench`. It performs the operation shown by the sanitized code line.
390	`end`	Ends the current function, conditional block, loop, or protected block.
391		Blank line used to separate logical blocks and improve readability.
392	`F_contact_N = Fx*t_long_N + Fy*t_lat_N + Fz*n_road_N;`	Assigns `F_contact_N` from `Fx*t_long_N + Fy*t_lat_N + Fz*n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation. This line is part of the tyre force/slip calculation.

Line	Sanitized code line	Explanation
393	`F_B = R_BN*F_contact_N;`	Assigns `F_B` from `R_BN*F_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
394	`r_contact_B = R_BN*r_wc_to_contact_N;`	Assigns `r_contact_B` from `R_BN*r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
395	`M_B = cross(r_contact_B, F_B);`	Assigns `M_B` from `cross(r_contact_B, F_B)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
396		Blank line used to separate logical blocks and improve readability.
397	`diag.s = geom.s;`	Assigns `diag.s` from `geom.s`. This value is used by later computations in this function.
398	`diag.n = geom.n;`	Assigns `diag.n` from `geom.n`. This value is used by later computations in this function.
399	`diag.compression = compression;`	Assigns `diag.compression` from `compression`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
400	`diag.compression_rate = compression_rate;`	Assigns `diag.compression_rate` from `compression_rate`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
401	`diag.compression_rate_radial = compression_rate_radial;`	Assigns `diag.compression_rate_radial` from `compression_rate_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
402	`diag.Fz = Fz;`	Assigns `diag.Fz` from `Fz`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
403	`diag.Fx = Fx;`	Assigns `diag.Fx` from `Fx`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
404	`diag.Fy = Fy;`	Assigns `diag.Fy` from `Fy`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
405	`diag.mu_ratio = mu_ratio;`	Assigns `diag.mu_ratio` from `mu_ratio`. This value is used by later computations in this function.
406	`diag.kappa = kappa;`	Assigns `diag.kappa` from `kappa`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
407	`diag.alpha = alpha;`	Assigns `diag.alpha` from `alpha`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
408	`diag.Vx_contact_carrier = Vx;`	Assigns `diag.Vx_contact_carrier` from `Vx`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
409	`diag.Vy_contact_carrier = Vy;`	Assigns `diag.Vy_contact_carrier` from `Vy`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
410	`diag.Vx_contact_material = Vx_material;`	Assigns `diag.Vx_contact_material` from `Vx_material`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
411	`diag.Vy_contact_material = Vy_material;`	Assigns `diag.Vy_contact_material` from `Vy_material`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
412	`diag.rolling_speed = rolling_speed;`	Assigns `diag.rolling_speed` from `rolling_speed`. This value is used by later computations in this function.
413	`diag.contact_x = geom.p_contact_N(1);`	Assigns `diag.contact_x` from `geom.p_contact_N(1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
414	`diag.contact_y = geom.p_contact_N(2);`	Assigns `diag.contact_y` from `geom.p_contact_N(2)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
415	`diag.contact_z = geom.p_contact_N(3);`	Assigns `diag.contact_z` from `geom.p_contact_N(3)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
416	`diag.road_patch_x = geom.p_road_N(1);`	Assigns `diag.road_patch_x` from `geom.p_road_N(1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
417	`diag.road_patch_y = geom.p_road_N(2);`	Assigns `diag.road_patch_y` from `geom.p_road_N(2)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
418	`diag.road_patch_z = geom.p_road_N(3);`	Assigns `diag.road_patch_z` from `geom.p_road_N(3)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
419	`diag.road_normal_velocity_residual = ck.road_normal_velocity_residual;`	Assigns `diag.road_normal_velocity_residual` from `ck.road_normal_velocity_residual`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
420	`diag.road_plane_residual_rate = ck.road_plane_residual_rate;`	Assigns `diag.road_plane_residual_rate` from `ck.road_plane_residual_rate`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
421	`diag.contact_vx = v_contact_geom_N(1);`	Assigns `diag.contact_vx` from `v_contact_geom_N(1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
422	`diag.contact_vy = v_contact_geom_N(2);`	Assigns `diag.contact_vy` from `v_contact_geom_N(2)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
423	`diag.contact_vz = v_contact_geom_N(3);`	Assigns `diag.contact_vz` from `v_contact_geom_N(3)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
424	`end`	Ends the current function, conditional block, loop, or protected block.
425		Blank line used to separate logical blocks and improve readability.
426	`function [Fx, Fy, mu_ratio] = tyre_force_tan_ellipse(cfg, kappa, alpha, Fz)`	Defines the local MATLAB function `tyre_force_tan_ellipse`. Combined-slip tyre law inspired by the older MATLAB model.
427	`%TYRE_FORCE_TAN_ELLIPSE Combined-slip tyre inspired by the uploaded old MATLAB model.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
428	`Qx = cfg.Qx;`	Assigns `Qx` from `cfg.Qx`. This value is used by later computations in this function.
429	`Qy = cfg.Qy;`	Assigns `Qy` from `cfg.Qy`. This value is used by later computations in this function.
430	`eps1 = cfg.eps1;`	Assigns `eps1` from `cfg.eps1`. This value is used by later computations in this function.
431	`eps2 = cfg.eps2;`	Assigns `eps2` from `cfg.eps2`. This value is used by later computations in this function.
432		Blank line used to separate logical blocks and improve readability.
433	`k_max = tan(pi/(2*cfg.Cx))/cfg.Bx;`	Assigns `k_max` from `tan(pi/(2*cfg.Cx))/cfg.Bx`. This value is used by later computations in this function.
434	`alpha_peak = tan(pi/(2*cfg.Cy))/cfg.By;`	Assigns `alpha_peak` from `tan(pi/(2*cfg.Cy))/cfg.By`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
435	`tan_alpha_max = tan(alpha_peak);`	Assigns `tan_alpha_max` from `tan(alpha_peak)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
436		Blank line used to separate logical blocks and improve readability.
437	`[mu_x_max, mu_y_max] = muxmax_muymax_func(cfg, Fz);`	Executes this MATLAB statement in `tyre_force_tan_ellipse`. It performs the operation shown by the sanitized code line.
438		Blank line used to separate logical blocks and improve readability.
439	`k_n = kappa / k_max;`	Assigns `k_n` from `kappa / k_max`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
440	`a_n = (-tan(alpha)) / tan_alpha_max;`	Assigns `a_n` from `(-tan(alpha)) / tan_alpha_max`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
441	`rou = sqrt(k_n.^2 + a_n.^2 + eps1) + eps2;`	Assigns `rou` from `sqrt(k_n.^2 + a_n.^2 + eps1) + eps2`. This value is used by later computations in this function.
442		Blank line used to separate logical blocks and improve readability.
443	`Sx = pi/(2*atan2(Qx, 1));`	Assigns `Sx` from `pi/(2*atan2(Qx, 1))`. This value is used by later computations in this function.
444	`Sy = pi/(2*atan2(Qy, 1));`	Assigns `Sy` from `pi/(2*atan2(Qy, 1))`. This value is used by later computations in this function.
445	`mu_scale = cfg.my_effective_mu_scaling;`	Assigns `mu_scale` from `cfg.my_effective_mu_scaling`. This value is used by later computations in this function.
446		Blank line used to separate logical blocks and improve readability.
447	`Fx = (mu_scale * mu_x_max) .* Fz .* sin(Qx * atan2(Sx * rou, 1)) .* (k_n ./ rou);`	Assigns `Fx` from `(mu_scale * mu_x_max) .* Fz .* sin(Qx * atan2(Sx * rou, 1)) .* (k_n ./ rou)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
448	`Fy = (mu_scale * mu_y_max) .* Fz .* sin(Qy * atan2(Sy * rou, 1)) .* (a_n ./ rou);`	Assigns `Fy` from `(mu_scale * mu_y_max) .* Fz .* sin(Qy * atan2(Sy * rou, 1)) .* (a_n ./ rou)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.

Line	Sanitized code line	Explanation
449		Blank line used to separate logical blocks and improve readability.
450	`denx = max(abs((mu_scale * mu_x_max).*Fz), 1.0e-9);`	Assigns `denx` from `max(abs((mu_scale * mu_x_max).*Fz), 1.0e-9)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
451	`deny = max(abs((mu_scale * mu_y_max).*Fz), 1.0e-9);`	Assigns `deny` from `max(abs((mu_scale * mu_y_max).*Fz), 1.0e-9)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.

Line	Sanitized code line	Explanation
452	`mu_ratio = hypot(Fx./denx, Fy./deny);`	Assigns `mu_ratio` from `hypot(Fx./denx, Fy./deny)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
453	`end`	Ends the current function, conditional block, loop, or protected block.
454		Blank line used to separate logical blocks and improve readability.
455	`function [Fx, Fy, mu_ratio] = tyre_force_linear_saturated(cfg, kappa, alpha, Fz)`	Defines the local MATLAB function `tyre_force_linear_saturated`. Fallback linear tyre law with friction saturation.
456	`%TYRE_FORCE_LINEAR_SATURATED Fallback from the previous executable demo.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
457	`Fx0 = cfg.Ck*kappa;`	Assigns `Fx0` from `cfg.Ck*kappa`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
458	`Fy0 = -cfg.Ca*alpha;`	Assigns `Fy0` from `-cfg.Ca*alpha`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
459	`limit = cfg.mu*Fz + 1.0e-9;`	Assigns `limit` from `cfg.mu*Fz + 1.0e-9`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
460	`norm_tan = sqrt(Fx0*Fx0 + Fy0*Fy0 + 1.0e-12);`	Assigns `norm_tan` from `sqrt(Fx0*Fx0 + Fy0*Fy0 + 1.0e-12)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
461	`scale = (1.0 + (norm_tan/limit)^4)^(-0.25);`	Assigns `scale` from `(1.0 + (norm_tan/limit)^4)^(-0.25)`. This value is used by later computations in this function.
462	`Fx = Fx0*scale;`	Assigns `Fx` from `Fx0*scale`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
463	`Fy = Fy0*scale;`	Assigns `Fy` from `Fy0*scale`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
464	`mu_ratio = norm_tan/max(limit, 1.0e-9);`	Assigns `mu_ratio` from `norm_tan/max(limit, 1.0e-9)`. This value is used by later computations in this function.
465	`end`	Ends the current function, conditional block, loop, or protected block.
466		Blank line used to separate logical blocks and improve readability.
467	`function [mu_x_max, mu_y_max] = muxmax_muymax_func(cfg, Fz)`	Defines the local MATLAB function `muxmax_muymax_func`. Computes load-sensitive longitudinal and lateral friction limits.
468	`%MUXMAX_MUYMAX_FUNC Load-sensitive friction estimate from the old MATLAB model.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
469	`Fz1 = cfg.Fz1;`	Assigns `Fz1` from `cfg.Fz1`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
470	`Fz2 = cfg.Fz2;`	Assigns `Fz2` from `cfg.Fz2`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
471	`rou_smooth = cfg.rou_smooth;`	Assigns `rou_smooth` from `cfg.rou_smooth`. This value is used by later computations in this function.
472		Blank line used to separate logical blocks and improve readability.
473	`mu_x_max1 = (cfg.d1x*Fz1 + cfg.d2x)/Fz1;`	Assigns `mu_x_max1` from `(cfg.d1x*Fz1 + cfg.d2x)/Fz1`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
474	`mu_x_max2 = (cfg.d1x*Fz2 + cfg.d2x)/Fz2;`	Assigns `mu_x_max2` from `(cfg.d1x*Fz2 + cfg.d2x)/Fz2`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
475	`mu_y_max1 = (cfg.d1y*Fz1 + cfg.d2y)/Fz1;`	Assigns `mu_y_max1` from `(cfg.d1y*Fz1 + cfg.d2y)/Fz1`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
476	`mu_y_max2 = (cfg.d1y*Fz2 + cfg.d2y)/Fz2;`	Assigns `mu_y_max2` from `(cfg.d1y*Fz2 + cfg.d2y)/Fz2`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.

Line	Sanitized code line	Explanation
477		Blank line used to separate logical blocks and improve readability.
478	`mu_x_max_raw = (Fz - Fz1) * (mu_x_max2 - mu_x_max1) / (Fz2 - Fz1) + mu_x_max1;`	Assigns `mu_x_max_raw` from `(Fz - Fz1) * (mu_x_max2 - mu_x_max1) / (Fz2 - Fz1) + mu_x_max1`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
479	`mu_y_max_raw = (Fz - Fz1) * (mu_y_max2 - mu_y_max1) / (Fz2 - Fz1) + mu_y_max1;`	Assigns `mu_y_max_raw` from `(Fz - Fz1) * (mu_y_max2 - mu_y_max1) / (Fz2 - Fz1) + mu_y_max1`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
480		Blank line used to separate logical blocks and improve readability.
481	`mu_x_max = mu_x_max_raw + smooth_ramp_func(mu_x_max_raw, rou_smooth);`	Assigns `mu_x_max` from `mu_x_max_raw + smooth_ramp_func(mu_x_max_raw, rou_smooth)`. This value is used by later computations in this function.
482	`mu_y_max = mu_y_max_raw + smooth_ramp_func(mu_y_max_raw, rou_smooth);`	Assigns `mu_y_max` from `mu_y_max_raw + smooth_ramp_func(mu_y_max_raw, rou_smooth)`. This value is used by later computations in this function.
483	`end`	Ends the current function, conditional block, loop, or protected block.
484		Blank line used to separate logical blocks and improve readability.
485	`function smooth_ramp = smooth_ramp_func(x, rou_smooth)`	Defines the local MATLAB function `smooth_ramp_func`. Smoothly prevents negative friction-limit values.
486	`smooth_ramp = (-x) .* ( atan2(-x, rou_smooth)/pi + 1/2 ) + rou_smooth / pi;`	Assigns `smooth_ramp` from `(-x) .* ( atan2(-x, rou_smooth)/pi + 1/2 ) + rou_smooth / pi`. This value is used by later computations in this function.
487	`end`	Ends the current function, conditional block, loop, or protected block.
488		Blank line used to separate logical blocks and improve readability.
489	`function [FdragF, FdownF, FdragR, FdownR, diag] = user_aero_model(t_abs, x, S, par, args, road, meta, cfg)`	Defines the local MATLAB function `user_aero_model`. Computes drag and downforce inputs from body speed and ride-height proxies.
490	`%USER_AERO_MODEL Drag/downforce model. Edit here for aero maps.`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
491	`qv = x(1:14);`	Assigns `qv` from `x(1:14)`. This value is used by later computations in this function.
492	`uv = x(15:28);`	Assigns `uv` from `x(15:28)`. This value is used by later computations in this function.
493		Blank line used to separate logical blocks and improve readability.
494	`if t_abs < cfg.drop_settle_time || cfg.disable_aero`	Starts a conditional branch. The following block executes only when this logical test is true.
495	`FdragF = 0.0;`	Assigns `FdragF` from `0.0`. This value is used by later computations in this function.
496	`FdownF = 0.0;`	Assigns `FdownF` from `0.0`. This value is used by later computations in this function.
497	`FdragR = 0.0;`	Assigns `FdragR` from `0.0`. This value is used by later computations in this function.
498	`FdownR = 0.0;`	Assigns `FdownR` from `0.0`. This value is used by later computations in this function.
499	`diag.enabled = false;`	Assigns `diag.enabled` from `false`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
500	`return;`	Returns immediately from the current function.
501	`end`	Ends the current function, conditional block, loop, or protected block.
502		Blank line used to separate logical blocks and improve readability.
503	`vx = uv(1);`	Assigns `vx` from `uv(1)`. This value is used by later computations in this function.
504	`qdyn = 0.5*getfield_default(par, 'rho', 1.225)*vx*vx;`	Assigns `qdyn` from `0.5*getfield_default(par, 'rho', 1.225)*vx*vx`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
505	`[p_af_N,~] = body_point_state_num(qv, uv, [getfield_default(par, 'aero_front_x', 1.25);0;getfield_default(par, 'aero_z', 0.05)]);`	Executes this MATLAB statement in `user_aero_model`. It performs the operation shown by the sanitized code line.
506	`[p_ar_N,~] = body_point_state_num(qv, uv, [getfield_default(par, 'aero_rear_x', -1.20);0;getfield_default(par, 'aero_z', 0.05)]);`	Executes this MATLAB statement in `user_aero_model`. It performs the operation shown by the sanitized code line.
507	`[hF,~,~,~] = road_height_normal_at_point(road, p_af_N);`	Executes this MATLAB statement in `user_aero_model`. It performs the operation shown by the sanitized code line.
508	`[hR,~,~,~] = road_height_normal_at_point(road, p_ar_N);`	Executes this MATLAB statement in `user_aero_model`. It performs the operation shown by the sanitized code line.
509		Blank line used to separate logical blocks and improve readability.
510	`multF_raw = 1.0 + getfield_default(par, 'aero_h_sens_front', 1.0)*(getfield_default(par, 'aero_h_ref', 0.080) - hF);`	Assigns `multF_raw` from `1.0 + getfield_default(par, 'aero_h_sens_front', 1.0)*(getfield_default(par, 'aero_h_ref', 0.080) - hF)`. This value is used by later computations in this function.
511	`multR_raw = 1.0 + getfield_default(par, 'aero_h_sens_rear', 1.2)*(getfield_default(par, 'aero_h_ref', 0.080) - hR);`	Assigns `multR_raw` from `1.0 + getfield_default(par, 'aero_h_sens_rear', 1.2)*(getfield_default(par, 'aero_h_ref', 0.080) - hR)`. This value is used by later computations in this function.
512	`multF = smooth_max(cfg.aero_h_min_multiplier, multF_raw, 1.0e-3);`	Assigns `multF` from `smooth_max(cfg.aero_h_min_multiplier, multF_raw, 1.0e-3)`. This value is used by later computations in this function.
513	`multR = smooth_max(cfg.aero_h_min_multiplier, multR_raw, 1.0e-3);`	Assigns `multR` from `smooth_max(cfg.aero_h_min_multiplier, multR_raw, 1.0e-3)`. This value is used by later computations in this function.
514		Blank line used to separate logical blocks and improve readability.
515	`D = 0.5*getfield_default(par, 'rho', 1.225)*getfield_default(par, 'CdA', 0.65)*vx*smooth_abs(vx, 0.25);`	Assigns `D` from `0.5*getfield_default(par, 'rho', 1.225)*getfield_default(par, 'CdA', 0.65)*vx*smooth_abs(vx, 0.25)`. This value is used by later computations in this function.
516	`FdragF = -0.5*D;`	Assigns `FdragF` from `-0.5*D`. This value is used by later computations in this function.
517	`FdragR = -0.5*D;`	Assigns `FdragR` from `-0.5*D`. This value is used by later computations in this function.
518	`FdownF = -qdyn*getfield_default(par, 'ClA_front', 0.48)*multF;`	Assigns `FdownF` from `-qdyn*getfield_default(par, 'ClA_front', 0.48)*multF`. This value is used by later computations in this function.
519	`FdownR = -qdyn*getfield_default(par, 'ClA_rear', 0.72)*multR;`	Assigns `FdownR` from `-qdyn*getfield_default(par, 'ClA_rear', 0.72)*multR`. This value is used by later computations in this function.
520		Blank line used to separate logical blocks and improve readability.
521	`diag.enabled = true;`	Assigns `diag.enabled` from `true`. This value is used by later computations in this function.
522	`diag.hF = hF;`	Assigns `diag.hF` from `hF`. This value is used by later computations in this function.
523	`diag.hR = hR;`	Assigns `diag.hR` from `hR`. This value is used by later computations in this function.
524	`diag.multF = multF;`	Assigns `diag.multF` from `multF`. This value is used by later computations in this function.
525	`diag.multR = multR;`	Assigns `diag.multR` from `multR`. This value is used by later computations in this function.
526	`diag.drag_total = D;`	Assigns `diag.drag_total` from `D`. This value is used by later computations in this function.
527	`end`	Ends the current function, conditional block, loop, or protected block.
528		Blank line used to separate logical blocks and improve readability.
529	`% ========================================================================`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
530	`% GEOMETRY, CONTACT, ROAD, AND MATH HELPERS`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
531	`% ========================================================================`	Comment line. It documents usage, assumptions, or a code section; MATLAB does not execute it.
532		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
533	`function ck = contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega)`	Defines the local MATLAB function `contact_kinematics_from_pose`. Computes contact-point velocity and compression rate from wheel pose and motion.
534	`head_N = normalize_num(head_N, [1;0;0]);`	Assigns `head_N` from `normalize_num(head_N, [1;0;0])`. This value is used by later computations in this function.
535	`spin_N = normalize_num(spin_N, [0;1;0]);`	Assigns `spin_N` from `normalize_num(spin_N, [0;1;0])`. This value is used by later computations in this function.
536	`geom0 = oriented_tyre_contact_geometry(par, road, p_wc_N, head_N, spin_N);`	Assigns `geom0` from `oriented_tyre_contact_geometry(par, road, p_wc_N, head_N, spin_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
537	`head_dot_N = cross(carrier_omega_N, head_N);`	Assigns `head_dot_N` from `cross(carrier_omega_N, head_N)`. This value is used by later computations in this function.
538	`spin_dot_N = cross(carrier_omega_N, spin_N);`	Assigns `spin_dot_N` from `cross(carrier_omega_N, spin_N)`. This value is used by later computations in this function.
539	`h = 1.0e-5;`	Assigns `h` from `1.0e-5`. This value is used by later computations in this function.
540	`geomp = oriented_tyre_contact_geometry(par, road, p_wc_N + h*v_wc_N, head_N + h*head_dot_N, spin_N + h*spin_dot_N);`	Assigns `geomp` from `oriented_tyre_contact_geometry(par, road, p_wc_N + h*v_wc_N, head_N + h*head_dot_N, spin_N + h*spin_dot_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
541	`geomm = oriented_tyre_contact_geometry(par, road, p_wc_N - h*v_wc_N, head_N - h*head_dot_N, spin_N - h*spin_dot_N);`	Assigns `geomm` from `oriented_tyre_contact_geometry(par, road, p_wc_N - h*v_wc_N, head_N - h*head_dot_N, spin_N - h*spin_dot_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
542	`ck.geom = geom0;`	Assigns `ck.geom` from `geom0`. This value is used by later computations in this function.
543	`ck.contact_point_velocity_geom_N = (geomp.p_contact_N - geomm.p_contact_N)/(2*h);`	Assigns `ck.contact_point_velocity_geom_N` from `(geomp.p_contact_N - geomm.p_contact_N)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
544	`ck.contact_unloaded_velocity_geom_N = (geomp.p_contact_unloaded_N - geomm.p_contact_unloaded_N)/(2*h);`	Assigns `ck.contact_unloaded_velocity_geom_N` from `(geomp.p_contact_unloaded_N - geomm.p_contact_unloaded_N)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
545	`ck.radial_down_rate_N = (geomp.radial_down_N - geomm.radial_down_N)/(2*h);`	Assigns `ck.radial_down_rate_N` from `(geomp.radial_down_N - geomm.radial_down_N)/(2*h)`. This value is used by later computations in this function.
546	`ck.road_normal_rate_N = (geomp.n_road_N - geomm.n_road_N)/(2*h);`	Assigns `ck.road_normal_rate_N` from `(geomp.n_road_N - geomm.n_road_N)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
547	`ck.t_long_rate_N = (geomp.t_long_N - geomm.t_long_N)/(2*h);`	Assigns `ck.t_long_rate_N` from `(geomp.t_long_N - geomm.t_long_N)/(2*h)`. This value is used by later computations in this function.
548	`ck.t_lat_rate_N = (geomp.t_lat_N - geomm.t_lat_N)/(2*h);`	Assigns `ck.t_lat_rate_N` from `(geomp.t_lat_N - geomm.t_lat_N)/(2*h)`. This value is used by later computations in this function.
549	`ck.compression_rate_normal = (geomp.compression_normal - geomm.compression_normal)/(2*h);`	Assigns `ck.compression_rate_normal` from `(geomp.compression_normal - geomm.compression_normal)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
550	`ck.compression_rate_radial = (geomp.compression_radial - geomm.compression_radial)/(2*h);`	Assigns `ck.compression_rate_radial` from `(geomp.compression_radial - geomm.compression_radial)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
551	`ck.contact_gap_rate_normal = (geomp.contact_gap_normal - geomm.contact_gap_normal)/(2*h);`	Assigns `ck.contact_gap_rate_normal` from `(geomp.contact_gap_normal - geomm.contact_gap_normal)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
552	`ck.contact_gap_rate_radial = (geomp.contact_gap_radial - geomm.contact_gap_radial)/(2*h);`	Assigns `ck.contact_gap_rate_radial` from `(geomp.contact_gap_radial - geomm.contact_gap_radial)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
553	`ck.road_plane_residual_rate = (geomp.road_plane_residual - geomm.road_plane_residual)/(2*h);`	Assigns `ck.road_plane_residual_rate` from `(geomp.road_plane_residual - geomm.road_plane_residual)/(2*h)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
554	`r_wc_to_contact_N = geom0.r_wc_to_contact_N;`	Assigns `r_wc_to_contact_N` from `geom0.r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
555	`spin_axis_N = geom0.spin_axis_N;`	Assigns `spin_axis_N` from `geom0.spin_axis_N`. This value is used by later computations in this function.
556	`ck.contact_patch_velocity_carrier_N = v_wc_N + cross(carrier_omega_N, r_wc_to_contact_N);`	Assigns `ck.contact_patch_velocity_carrier_N` from `v_wc_N + cross(carrier_omega_N, r_wc_to_contact_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
557	`ck.contact_patch_velocity_spin_N = cross(omega*spin_axis_N, r_wc_to_contact_N);`	Assigns `ck.contact_patch_velocity_spin_N` from `cross(omega*spin_axis_N, r_wc_to_contact_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
558	`ck.contact_patch_velocity_material_N = ck.contact_patch_velocity_carrier_N + ck.contact_patch_velocity_spin_N;`	Assigns `ck.contact_patch_velocity_material_N` from `ck.contact_patch_velocity_carrier_N + ck.contact_patch_velocity_spin_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
559	`ck.carrier_omega_N = carrier_omega_N;`	Assigns `ck.carrier_omega_N` from `carrier_omega_N`. This value is used by later computations in this function.
560	`ck.road_normal_velocity_residual = dot(ck.contact_point_velocity_geom_N, geom0.n_road_N);`	Assigns `ck.road_normal_velocity_residual` from `dot(ck.contact_point_velocity_geom_N, geom0.n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
561	`end`	Ends the current function, conditional block, loop, or protected block.
562		Blank line used to separate logical blocks and improve readability.
563	`function geom = oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`	Defines the local MATLAB function `oriented_tyre_contact_geometry`. Finds the tyre contact point and compression using wheel and road geometry.
564	`[s,n,p_road_N,e_long_N,e_lat_N,n_road_N] = road_project_point(road, p_wc_N);`	Executes this MATLAB statement in `oriented_tyre_contact_geometry`. It performs the operation shown by the sanitized code line.
565	`wheel_heading_N = normalize_num(wheel_heading_N, e_long_N);`	Assigns `wheel_heading_N` from `normalize_num(wheel_heading_N, e_long_N)`. This value is used by later computations in this function.
566	`spin_axis_N = normalize_num(wheel_spin_axis_N, e_lat_N);`	Assigns `spin_axis_N` from `normalize_num(wheel_spin_axis_N, e_lat_N)`. This value is used by later computations in this function.
567	`t_long_raw = cross(spin_axis_N, n_road_N);`	Assigns `t_long_raw` from `cross(spin_axis_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
568	`if norm(t_long_raw) < 1.0e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
569	`t_long_raw = wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N;`	Assigns `t_long_raw` from `wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
570	`end`	Ends the current function, conditional block, loop, or protected block.
571	`t_long_N = normalize_num(t_long_raw, e_long_N);`	Assigns `t_long_N` from `normalize_num(t_long_raw, e_long_N)`. This value is used by later computations in this function.
572	`if dot(t_long_N, wheel_heading_N) < 0.0`	Starts a conditional branch. The following block executes only when this logical test is true.
573	`t_long_N = -t_long_N;`	Assigns `t_long_N` from `-t_long_N`. This value is used by later computations in this function.
574	`end`	Ends the current function, conditional block, loop, or protected block.
575	`t_lat_N = normalize_num(cross(n_road_N, t_long_N), e_lat_N);`	Assigns `t_lat_N` from `normalize_num(cross(n_road_N, t_long_N), e_lat_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
576	`normal_in_radial_plane = n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N;`	Assigns `normal_in_radial_plane` from `n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
577	`gamma = norm(normal_in_radial_plane);`	Assigns `gamma` from `norm(normal_in_radial_plane)`. This value is used by later computations in this function.
578	`if gamma < 1.0e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
579	`gamma = 1.0;`	Assigns `gamma` from `1.0`. This value is used by later computations in this function.
580	`radial_down_N = -n_road_N;`	Assigns `radial_down_N` from `-n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
581	`degenerate = true;`	Assigns `degenerate` from `true`. This value is used by later computations in this function.
582	`else`	Starts the fallback branch for the current conditional block.
583	`radial_down_N = -normal_in_radial_plane/gamma;`	Assigns `radial_down_N` from `-normal_in_radial_plane/gamma`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
584	`degenerate = false;`	Assigns `degenerate` from `false`. This value is used by later computations in this function.
585	`end`	Ends the current function, conditional block, loop, or protected block.
586	`contact_to_wc_unit_N = -radial_down_N;`	Assigns `contact_to_wc_unit_N` from `-radial_down_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
587	`r_wc_to_unloaded_N = par.rw*radial_down_N;`	Assigns `r_wc_to_unloaded_N` from `par.rw*radial_down_N`. This value is used by later computations in this function.
588	`p_contact_unloaded_N = p_wc_N + r_wc_to_unloaded_N;`	Assigns `p_contact_unloaded_N` from `p_wc_N + r_wc_to_unloaded_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
589	`signed_center_distance = dot(p_wc_N - p_road_N, n_road_N);`	Assigns `signed_center_distance` from `dot(p_wc_N - p_road_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
590	`radial_depth_normal = par.rw*gamma;`	Assigns `radial_depth_normal` from `par.rw*gamma`. This value is used by later computations in this function.
591	`compression_normal = radial_depth_normal - signed_center_distance;`	Assigns `compression_normal` from `radial_depth_normal - signed_center_distance`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
592	`distance_center_to_road_along_radial = signed_center_distance/gamma;`	Assigns `distance_center_to_road_along_radial` from `signed_center_distance/gamma`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
593	`compression_radial = par.rw - distance_center_to_road_along_radial;`	Assigns `compression_radial` from `par.rw - distance_center_to_road_along_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
594	`r_wc_to_contact_N = distance_center_to_road_along_radial*radial_down_N;`	Assigns `r_wc_to_contact_N` from `distance_center_to_road_along_radial*radial_down_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
595	`p_contact_N = p_wc_N + r_wc_to_contact_N;`	Assigns `p_contact_N` from `p_wc_N + r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
596	`road_plane_residual = dot(p_contact_N - p_road_N, n_road_N);`	Assigns `road_plane_residual` from `dot(p_contact_N - p_road_N, n_road_N)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
597	`geom.s = s;`	Assigns `geom.s` from `s`. This value is used by later computations in this function.
598	`geom.n = n;`	Assigns `geom.n` from `n`. This value is used by later computations in this function.
599	`geom.p_road_N = p_road_N;`	Assigns `geom.p_road_N` from `p_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
600	`geom.e_long_N = e_long_N;`	Assigns `geom.e_long_N` from `e_long_N`. This value is used by later computations in this function.
601	`geom.e_lat_N = e_lat_N;`	Assigns `geom.e_lat_N` from `e_lat_N`. This value is used by later computations in this function.
602	`geom.n_road_N = n_road_N;`	Assigns `geom.n_road_N` from `n_road_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
603	`geom.t_long_N = t_long_N;`	Assigns `geom.t_long_N` from `t_long_N`. This value is used by later computations in this function.
604	`geom.t_lat_N = t_lat_N;`	Assigns `geom.t_lat_N` from `t_lat_N`. This value is used by later computations in this function.
605	`geom.spin_axis_N = spin_axis_N;`	Assigns `geom.spin_axis_N` from `spin_axis_N`. This value is used by later computations in this function.
606	`geom.radial_down_N = radial_down_N;`	Assigns `geom.radial_down_N` from `radial_down_N`. This value is used by later computations in this function.
607	`geom.contact_to_wc_unit_N = contact_to_wc_unit_N;`	Assigns `geom.contact_to_wc_unit_N` from `contact_to_wc_unit_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
608	`geom.r_wc_to_contact_N = r_wc_to_contact_N;`	Assigns `geom.r_wc_to_contact_N` from `r_wc_to_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
609	`geom.p_contact_N = p_contact_N;`	Assigns `geom.p_contact_N` from `p_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
610	`geom.p_contact_road_N = p_contact_N;`	Assigns `geom.p_contact_road_N` from `p_contact_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
611	`geom.r_wc_to_unloaded_N = r_wc_to_unloaded_N;`	Assigns `geom.r_wc_to_unloaded_N` from `r_wc_to_unloaded_N`. This value is used by later computations in this function.
612	`geom.p_contact_unloaded_N = p_contact_unloaded_N;`	Assigns `geom.p_contact_unloaded_N` from `p_contact_unloaded_N`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
613	`geom.signed_center_distance = signed_center_distance;`	Assigns `geom.signed_center_distance` from `signed_center_distance`. This value is used by later computations in this function.
614	`geom.radial_projection_norm = gamma;`	Assigns `geom.radial_projection_norm` from `gamma`. This value is used by later computations in this function.
615	`geom.radial_depth = radial_depth_normal;`	Assigns `geom.radial_depth` from `radial_depth_normal`. This value is used by later computations in this function.
616	`geom.radial_depth_normal = radial_depth_normal;`	Assigns `geom.radial_depth_normal` from `radial_depth_normal`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
617	`geom.distance_center_to_road_along_radial = distance_center_to_road_along_radial;`	Assigns `geom.distance_center_to_road_along_radial` from `distance_center_to_road_along_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
618	`geom.compression = compression_normal;`	Assigns `geom.compression` from `compression_normal`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
619	`geom.compression_normal = compression_normal;`	Assigns `geom.compression_normal` from `compression_normal`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
620	`geom.compression_radial = compression_radial;`	Assigns `geom.compression_radial` from `compression_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
621	`geom.contact_gap_normal = -compression_normal;`	Assigns `geom.contact_gap_normal` from `-compression_normal`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
622	`geom.contact_gap_radial = -compression_radial;`	Assigns `geom.contact_gap_radial` from `-compression_radial`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
623	`geom.road_plane_residual = road_plane_residual;`	Assigns `geom.road_plane_residual` from `road_plane_residual`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
624	`geom.degenerate_contact = degenerate;`	Assigns `geom.degenerate_contact` from `degenerate`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
625	`end`	Ends the current function, conditional block, loop, or protected block.
626		Blank line used to separate logical blocks and improve readability.
627	`function [h,e_z,s,n] = road_height_normal_at_point(road,p_N)`	Defines the local MATLAB function `road_height_normal_at_point`. Evaluates the selected road surface and normal near a point.
628	`[s,n,p_road,~,~,e_z] = road_project_point(road,p_N);`	Executes this MATLAB statement in `road_height_normal_at_point`. It performs the operation shown by the sanitized code line.
629	`h = dot(p_N-p_road,e_z);`	Assigns `h` from `dot(p_N-p_road,e_z)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
630	`end`	Ends the current function, conditional block, loop, or protected block.
631		Blank line used to separate logical blocks and improve readability.
632	`function [pN,vN] = body_point_state_num(qv, uv, r_B)`	Defines the local MATLAB function `body_point_state_num`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
633	`R = body_rotation_from_q(qv);`	Assigns `R` from `body_rotation_from_q(qv)`. This value is used by later computations in this function.
634	`pP = qv(1:3);`	Assigns `pP` from `qv(1:3)`. This value is used by later computations in this function.
635	`vP = R*uv(1:3);`	Assigns `vP` from `R*uv(1:3)`. This value is used by later computations in this function.
636	`omega = R*uv(4:6);`	Assigns `omega` from `R*uv(4:6)`. This value is used by later computations in this function.
637	`rN = R*r_B;`	Assigns `rN` from `R*r_B`. This value is used by later computations in this function.
638	`pN = pP + rN;`	Assigns `pN` from `pP + rN`. This value is used by later computations in this function.
639	`vN = vP + cross(omega,rN);`	Assigns `vN` from `vP + cross(omega,rN)`. This value is used by later computations in this function.
640	`end`	Ends the current function, conditional block, loop, or protected block.
641		Blank line used to separate logical blocks and improve readability.
642	`function R = body_rotation_from_q(qv)`	Defines the local MATLAB function `body_rotation_from_q`. Builds the yaw-pitch-roll body rotation matrix from q.
643	`R = rot_z(qv(4))*rot_y(qv(5))*rot_x(qv(6));`	Assigns `R` from `rot_z(qv(4))*rot_y(qv(5))*rot_x(qv(6))`. This value is used by later computations in this function.
644	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
645		Blank line used to separate logical blocks and improve readability.
646	`function R = rot_x(a)`	Defines the local MATLAB function `rot_x`. Returns a 3-by-3 rotation matrix about the named axis.
647	`ca = cos(a);`	Assigns `ca` from `cos(a)`. This value is used by later computations in this function.
648	`sa = sin(a);`	Assigns `sa` from `sin(a)`. This value is used by later computations in this function.
649	`R = [1 0 0; 0 ca -sa; 0 sa ca];`	Assigns `R` from `[1 0 0; 0 ca -sa; 0 sa ca]`. This value is used by later computations in this function.
650	`end`	Ends the current function, conditional block, loop, or protected block.
651		Blank line used to separate logical blocks and improve readability.
652	`function R = rot_y(a)`	Defines the local MATLAB function `rot_y`. Returns a 3-by-3 rotation matrix about the named axis.
653	`ca = cos(a);`	Assigns `ca` from `cos(a)`. This value is used by later computations in this function.
654	`sa = sin(a);`	Assigns `sa` from `sin(a)`. This value is used by later computations in this function.
655	`R = [ca 0 sa; 0 1 0; -sa 0 ca];`	Assigns `R` from `[ca 0 sa; 0 1 0; -sa 0 ca]`. This value is used by later computations in this function.
656	`end`	Ends the current function, conditional block, loop, or protected block.
657		Blank line used to separate logical blocks and improve readability.
658	`function R = rot_z(a)`	Defines the local MATLAB function `rot_z`. Returns a 3-by-3 rotation matrix about the named axis.
659	`ca = cos(a);`	Assigns `ca` from `cos(a)`. This value is used by later computations in this function.
660	`sa = sin(a);`	Assigns `sa` from `sin(a)`. This value is used by later computations in this function.
661	`R = [ca -sa 0; sa ca 0; 0 0 1];`	Assigns `R` from `[ca -sa 0; sa ca 0; 0 0 1]`. This value is used by later computations in this function.
662	`end`	Ends the current function, conditional block, loop, or protected block.
663		Blank line used to separate logical blocks and improve readability.
664	`function [s,n,p_road,e_long,e_lat,e_z] = road_project_point(road,p)`	Defines the local MATLAB function `road_project_point`. Projects a point to road coordinates and basis vectors.
665	`s = p(1);`	Assigns `s` from `p(1)`. This value is used by later computations in this function.
666	`for kk = 1:max(0,road.projection_iterations)`	Starts a loop; the following block is repeated for the specified index range.
667	`[f,~,~,~,~,~] = projection_residual_given_s(road,p,s);`	Executes this MATLAB statement in `road_project_point`. It performs the operation shown by the sanitized code line.
668	`if abs(f) < road.projection_tol`	Starts a conditional branch. The following block executes only when this logical test is true.
669	`break;`	Executes this MATLAB statement in `road_project_point`. It performs the operation shown by the sanitized code line.
670	`end`	Ends the current function, conditional block, loop, or protected block.

Line	Sanitized code line	Explanation
671	`ds = max(1.0e-4, 1.0e-6*(1.0+abs(s)));`	Assigns `ds` from `max(1.0e-4, 1.0e-6*(1.0+abs(s)))`. This value is used by later computations in this function.
672	`fp = projection_residual_given_s(road,p,s+ds);`	Assigns `fp` from `projection_residual_given_s(road,p,s+ds)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
673	`fm = projection_residual_given_s(road,p,s-ds);`	Assigns `fm` from `projection_residual_given_s(road,p,s-ds)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
674	`dfds = (fp-fm)/(2*ds);`	Assigns `dfds` from `(fp-fm)/(2*ds)`. This value is used by later computations in this function.
675	`if ~isfinite(dfds) || abs(dfds) < 1.0e-12`	Starts a conditional branch. The following block executes only when this logical test is true.
676	`break;`	Executes this MATLAB statement in `road_project_point`. It performs the operation shown by the sanitized code line.
677	`end`	Ends the current function, conditional block, loop, or protected block.
678	`step = min(max(f/dfds, -5.0), 5.0);`	Assigns `step` from `min(max(f/dfds, -5.0), 5.0)`. This value is used by later computations in this function.
679	`s = s - step;`	Assigns `s` from `s - step`. This value is used by later computations in this function.
680	`if abs(step) < road.projection_tol`	Starts a conditional branch. The following block executes only when this logical test is true.
681	`break;`	Executes this MATLAB statement in `road_project_point`. It performs the operation shown by the sanitized code line.
682	`end`	Ends the current function, conditional block, loop, or protected block.
683	`end`	Ends the current function, conditional block, loop, or protected block.
684	`[~,n,p_road,e_long,e_lat,e_z,~] = projection_residual_given_s(road,p,s);`	Executes this MATLAB statement in `road_project_point`. It performs the operation shown by the sanitized code line.
685	`end`	Ends the current function, conditional block, loop, or protected block.
686		Blank line used to separate logical blocks and improve readability.
687	`function [f,n,p_road,e_long,e_lat,e_z,p_s] = projection_residual_given_s(road,p,s)`	Defines the local MATLAB function `projection_residual_given_s`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
688	`[C,~,~,e_y,~,~] = centerline_quantities(road,s);`	Executes this MATLAB statement in `projection_residual_given_s`. It performs the operation shown by the sanitized code line.
689	`n = dot(p-C,e_y);`	Assigns `n` from `dot(p-C,e_y)`. This value is used by later computations in this function.
690	`[p_road,e_long,e_lat,e_z,p_s,~] = road_surface_at(road,s,n);`	Executes this MATLAB statement in `projection_residual_given_s`. It performs the operation shown by the sanitized code line.
691	`r = p-p_road;`	Assigns `r` from `p-p_road`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
692	`f = dot(r,p_s);`	Assigns `f` from `dot(r,p_s)`. This value is used by later computations in this function.
693	`end`	Ends the current function, conditional block, loop, or protected block.
694		Blank line used to separate logical blocks and improve readability.
695	`function [p_road,e_long,e_lat,e_z,p_s,p_n] = road_surface_at(road,s,n)`	Defines the local MATLAB function `road_surface_at`. Evaluates either flat or ribbon road height and normal.
696	`[C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s);`	Executes this MATLAB statement in `road_surface_at`. It performs the operation shown by the sanitized code line.
697	`p_road = C + n*ey;`	Assigns `p_road` from `C + n*ey`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
698	`p_s = Cs + n*dey;`	Assigns `p_s` from `Cs + n*dey`. This value is used by later computations in this function.
699	`p_n = ey;`	Assigns `p_n` from `ey`. This value is used by later computations in this function.
700	`e_long = normalize_num(p_s,ex);`	Assigns `e_long` from `normalize_num(p_s,ex)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
701	`e_lat = normalize_num(p_n,ey);`	Assigns `e_lat` from `normalize_num(p_n,ey)`. This value is used by later computations in this function.
702	`e_z = normalize_num(cross(e_long,e_lat),ezc);`	Assigns `e_z` from `normalize_num(cross(e_long,e_lat),ezc)`. This value is used by later computations in this function.
703	`if e_z(3) < 0.0`	Starts a conditional branch. The following block executes only when this logical test is true.
704	`e_z = -e_z;`	Assigns `e_z` from `-e_z`. This value is used by later computations in this function.
705	`e_lat = -e_lat;`	Assigns `e_lat` from `-e_lat`. This value is used by later computations in this function.
706	`end`	Ends the current function, conditional block, loop, or protected block.
707	`end`	Ends the current function, conditional block, loop, or protected block.
708		Blank line used to separate logical blocks and improve readability.
709	`function [C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s)`	Defines the local MATLAB function `centerline_quantities`. Computes the straight road centreline frame quantities.
710	`if isfield(road,'type') && strcmpi(road.type,'flat')`	Starts a conditional branch. The following block executes only when this logical test is true.
711	`C = [s;0;0];`	Assigns `C` from `[s;0;0]`. This value is used by later computations in this function.
712	`Cs = [1;0;0];`	Assigns `Cs` from `[1;0;0]`. This value is used by later computations in this function.
713	`ex = [1;0;0];`	Assigns `ex` from `[1;0;0]`. This value is used by later computations in this function.
714	`ey = [0;1;0];`	Assigns `ey` from `[0;1;0]`. This value is used by later computations in this function.
715	`dey = [0;0;0];`	Assigns `dey` from `[0;0;0]`. This value is used by later computations in this function.
716	`ezc = [0;0;1];`	Assigns `ezc` from `[0;0;1]`. This value is used by later computations in this function.
717	`return;`	Returns immediately from the current function.
718	`end`	Ends the current function, conditional block, loop, or protected block.
719	`ke = 2*pi/road.elev_wavelength;`	Assigns `ke` from `2*pi/road.elev_wavelength`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.

Line	Sanitized code line	Explanation
720	`kb = 2*pi/road.bank_wavelength;`	Assigns `kb` from `2*pi/road.bank_wavelength`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
721	`zc = road.elev_amp*(1-cos(ke*s));`	Assigns `zc` from `road.elev_amp*(1-cos(ke*s))`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
722	`dz = road.elev_amp*ke*sin(ke*s);`	Assigns `dz` from `road.elev_amp*ke*sin(ke*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
723	`d2z = road.elev_amp*ke*ke*cos(ke*s);`	Assigns `d2z` from `road.elev_amp*ke*ke*cos(ke*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
724	`bank = road.bank_amp*sin(kb*s);`	Assigns `bank` from `road.bank_amp*sin(kb*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
725	`db = road.bank_amp*kb*cos(kb*s);`	Assigns `db` from `road.bank_amp*kb*cos(kb*s)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
726	`C = [s;0;zc];`	Assigns `C` from `[s;0;zc]`. This value is used by later computations in this function.
727	`Cs = [1;0;dz];`	Assigns `Cs` from `[1;0;dz]`. This value is used by later computations in this function.
728	`L = sqrt(1+dz*dz);`	Assigns `L` from `sqrt(1+dz*dz)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
729	`ex = [1/L;0;dz/L];`	Assigns `ex` from `[1/L;0;dz/L]`. This value is used by later computations in this function.
730	`ey0 = [0;1;0];`	Assigns `ey0` from `[0;1;0]`. This value is used by later computations in this function.
731	`ez0 = [-dz/L;0;1/L];`	Assigns `ez0` from `[-dz/L;0;1/L]`. This value is used by later computations in this function.
732	`dez = [-d2z/(L^3);0;-dz*d2z/(L^3)];`	Assigns `dez` from `[-d2z/(L^3);0;-dz*d2z/(L^3)]`. This value is used by later computations in this function.
733	`cb = cos(bank);`	Assigns `cb` from `cos(bank)`. This value is used by later computations in this function.
734	`sb = sin(bank);`	Assigns `sb` from `sin(bank)`. This value is used by later computations in this function.
735	`ey = cb*ey0 + sb*ez0;`	Assigns `ey` from `cb*ey0 + sb*ez0`. This value is used by later computations in this function.
736	`dey = -sb*db*ey0 + cb*db*ez0 + sb*dez;`	Assigns `dey` from `-sb*db*ey0 + cb*db*ez0 + sb*dez`. This value is used by later computations in this function.
737	`ezc = normalize_num(cross(ex,ey),[0;0;1]);`	Assigns `ezc` from `normalize_num(cross(ex,ey),[0;0;1])`. This value is used by later computations in this function.
738	`end`	Ends the current function, conditional block, loop, or protected block.
739		Blank line used to separate logical blocks and improve readability.
740	`function y = smooth_step(t, t0, t1)`	Defines the local MATLAB function `smooth_step`. Provides a smooth 0-to-1 transition to avoid discontinuous commands.
741	`if t <= t0`	Starts a conditional branch. The following block executes only when this logical test is true.
742	`y = 0.0;`	Assigns `y` from `0.0`. This value is used by later computations in this function.
743	`elseif t >= t1`	Starts an alternative conditional branch, tested only if previous branches were false.
744	`y = 1.0;`	Assigns `y` from `1.0`. This value is used by later computations in this function.
745	`else`	Starts the fallback branch for the current conditional block.
746	`a = (t-t0)/(t1-t0);`	Assigns `a` from `(t-t0)/(t1-t0)`. This value is used by later computations in this function.
747	`y = a*a*(3.0 - 2.0*a);`	Assigns `y` from `a*a*(3.0 - 2.0*a)`. This value is used by later computations in this function.
748	`end`	Ends the current function, conditional block, loop, or protected block.
749	`end`	Ends the current function, conditional block, loop, or protected block.
750		Blank line used to separate logical blocks and improve readability.
751	`function y = smoothplus(x,epsv)`	Defines the local MATLAB function `smoothplus`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
752	`y = 0.5*(x + sqrt(x*x + epsv*epsv));`	Assigns `y` from `0.5*(x + sqrt(x*x + epsv*epsv))`. This value is used by later computations in this function.
753	`end`	Ends the current function, conditional block, loop, or protected block.
754		Blank line used to separate logical blocks and improve readability.
755	`function y = smooth_max(a,b,epsv)`	Defines the local MATLAB function `smooth_max`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
756	`d = b - a;`	Assigns `d` from `b - a`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
757	`y = a + smoothplus(d,epsv);`	Assigns `y` from `a + smoothplus(d,epsv)`. This value is used by later computations in this function.
758	`end`	Ends the current function, conditional block, loop, or protected block.
759		Blank line used to separate logical blocks and improve readability.
760	`function y = smooth_abs(x,epsv)`	Defines the local MATLAB function `smooth_abs`. Helper function used by this file. Its line-by-line rows below describe the exact operations.
761	`y = sqrt(x*x + epsv*epsv);`	Assigns `y` from `sqrt(x*x + epsv*epsv)`. This value is used by later computations in this function.
762	`end`	Ends the current function, conditional block, loop, or protected block.
763		Blank line used to separate logical blocks and improve readability.
764	`function y = normalize_num(v,fallback)`	Defines the local MATLAB function `normalize_num`. Normalizes vectors while protecting against tiny norms.
765	`n = norm(v);`	Assigns `n` from `norm(v)`. This value is used by later computations in this function.
766	`if n < 1.0e-10`	Starts a conditional branch. The following block executes only when this logical test is true.
767	`y = fallback/norm(fallback);`	Assigns `y` from `fallback/norm(fallback)`. This value is used by later computations in this function.

Line	Sanitized code line	Explanation
768	`else`	Starts the fallback branch for the current conditional block.
769	`y = v/n;`	Assigns `y` from `v/n`. This value is used by later computations in this function.
770	`end`	Ends the current function, conditional block, loop, or protected block.
771	`end`	Ends the current function, conditional block, loop, or protected block.
772		Blank line used to separate logical blocks and improve readability.
773	`function val = getfield_default(s, name, default_val)`	Defines the local MATLAB function `getfield_default`. Reads a struct field with a safe default fallback.
774	`if isstruct(s) && isfield(s, name)`	Starts a conditional branch. The following block executes only when this logical test is true.
775	`val = s.(name);`	Assigns `val` from `s.(name)`. This value is used by later computations in this function.
776	`else`	Starts the fallback branch for the current conditional block.
777	`val = default_val;`	Assigns `val` from `default_val`. This value is used by later computations in this function.
778	`end`	Ends the current function, conditional block, loop, or protected block.
779	`end`	Ends the current function, conditional block, loop, or protected block.
780		Blank line used to separate logical blocks and improve readability.
781	`function d = empty_contact_diag()`	Defines the local MATLAB function `empty_contact_diag`. Initializes contact diagnostic arrays.
782	`d.names = {'FL','FR','RL','RR'};`	Assigns `d.names` from `{'FL','FR','RL','RR'}`. This value is used by later computations in this function.
783	`d.Fx = zeros(4,1);`	Assigns `d.Fx` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
784	`d.Fy = zeros(4,1);`	Assigns `d.Fy` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.

Line	Sanitized code line	Explanation
785	`d.Fz = zeros(4,1);`	Assigns `d.Fz` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
786	`d.kappa = zeros(4,1);`	Assigns `d.kappa` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
787	`d.alpha = zeros(4,1);`	Assigns `d.alpha` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the tyre force/slip calculation.
788	`d.compression = zeros(4,1);`	Assigns `d.compression` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
789	`d.road_z_at_wheel = zeros(4,1);`	Assigns `d.road_z_at_wheel` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
790	`d.contact_x = zeros(4,1);`	Assigns `d.contact_x` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
791	`d.contact_y = zeros(4,1);`	Assigns `d.contact_y` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
792	`d.contact_z = zeros(4,1);`	Assigns `d.contact_z` from `zeros(4,1)`. This value is used by later computations in this function. This line is part of the road-contact or tyre-compression calculation.
793	`d.mu_ratio = zeros(4,1);`	Assigns `d.mu_ratio` from `zeros(4,1)`. This value is used by later computations in this function.
794	`end`	Ends the current function, conditional block, loop, or protected block.

File D - Export/regeneration helper script

Reference Python script that loads the source mechanical model, extracts matrix and sensor expressions, converts them to MATLAB/CasADi syntax, writes helper files, and packages the result.

Lines checked against uploaded code: 928. All code snippets below are sanitized aliases when required; line numbers still refer to the current uploaded file.

Line	Block/function	Purpose
44	replace_source_symbols	Maps source symbolic names to vector-indexed generated-code variables.
81	mcode	Prints a symbolic expression in MATLAB/CasADi-compatible syntax.
110	matlab_cellstr	Formats Python string lists as MATLAB cell-string syntax.
116	valid_field	Converts names into valid MATLAB struct-field names.

Line-by-line explanation

Line	Sanitized code line	Explanation
1	`from __future__ import annotations`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
2	`from pathlib import Path`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
3	`import sys, types, re, textwrap, zipfile, shutil, json`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
4	`import symbolic-tool as sp`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
5	`import symbolic-tool.physics.mechanics as me`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
6	`from symbolic-tool.printing.octave import octave_code`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
7	`from dataclasses import fields`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
8		Blank line used to separate logical blocks and improve readability.
9	`SRC = Path('/mnt/data/vehicle14_generated-core_roadfree_wrench_v1_3d_ribbon_full_kc_v10_nicola_wp_checked(18).py')`	Builds a filesystem path used as an input file, output folder, or generated artifact location.
10	`OUT = Path('/mnt/data/vehicle14_matlab_casadi_package')`	Builds a filesystem path used as an input file, output folder, or generated artifact location.
11	`if OUT.exists():`	Starts a Python conditional branch; the indented block runs only if the condition is true.
12	`shutil.rmtree(OUT)`	Executes this Python statement in `top level` as part of the export/regeneration workflow.
13	`OUT.mkdir(parents=True)`	Assigns `OUT.mkdir(parents` from `True)`. This intermediate value supports the export/regeneration workflow.
14		Blank line used to separate logical blocks and improve readability.
15	`# Load a lightly patched copy of the uploaded module so the symbolic sensor expressions are retained.`	Comment line. It documents the exporter workflow; Python does not execute it.
16	`src = SRC.read_text()`	Assigns `src` from `SRC.read_text()`. This intermediate value supports the export/regeneration workflow.
17	`src = src.replace(' sensor_fun: Callable\n mass_matrix_full: sp.Matrix',`	Assigns `src` from `src.replace(' sensor_fun: Callable\n mass_matrix_full: sp.Matrix',`. This intermediate value supports the export/regeneration workflow.
18	`' sensor_fun: Callable\n sensor_exprs: Tuple[sp.Expr, ...]\n mass_matrix_full: sp.Matrix')`	Executes this Python statement in `top level` as part of the export/regeneration workflow.
19	`src = src.replace(' sensor_fun=sensor_fun,\n mass_matrix_full=KM.mass_matrix_full,',`	Assigns `src` from `src.replace(' sensor_fun=sensor_fun,\n mass_matrix_full=KM.mass_matrix_full,',`. This intermediate value supports the export/regeneration workflow.
20	`' sensor_fun=sensor_fun,\n sensor_exprs=tuple(sensor_exprs),\n mass_matrix_full=KM.mass_matrix_full,')`	Executes this Python statement in `top level` as part of the export/regeneration workflow.
21	`mod = types.ModuleType('veh14_export_mod')`	Assigns `mod` from `types.ModuleType('veh14_export_mod')`. This intermediate value supports the export/regeneration workflow.
22	`mod.__file__ = str(SRC)`	Assigns `mod.__file__` from `str(SRC)`. This intermediate value supports the export/regeneration workflow.
23	`sys.modules[mod.__name__] = mod`	Assigns `sys.modules[mod.__name__]` from `mod`. This intermediate value supports the export/regeneration workflow.
24	`exec(compile(src, str(SRC), 'exec'), mod.__dict__)`	Executes a temporary in-memory copy of the source model so expressions can be extracted without editing the uploaded file.
25	`model = mod.derive_vehicle14_generated-core()`	Calls the source model derivation routine to obtain matrices, forcing terms, names, and sensors for export.
26	`par = mod.VehicleParams()`	Assigns `par` from `mod.VehicleParams()`. This intermediate value supports the export/regeneration workflow.
27		Blank line used to separate logical blocks and improve readability.
28	`# ----------------------------------------------------------------------`	Comment line. It documents the exporter workflow; Python does not execute it.

Line	Sanitized code line	Explanation
29	`# Generate CasADi/SX matrix and sensor expressions.`	Comment line. It documents the exporter workflow; Python does not execute it.
30	`# ----------------------------------------------------------------------`	Comment line. It documents the exporter workflow; Python does not execute it.
31	`# Replacement is based on object names, not reconstructed assumptions.`	Comment line. It documents the exporter workflow; Python does not execute it.
32	`# Some parameter symbols in the source are declared positive=True, so plain`	Comment line. It documents the exporter workflow; Python does not execute it.
33	`# sp.symbols('mB') would not be equal to the original symbol.`	Comment line. It documents the exporter workflow; Python does not execute it.
34	`q_map = {name: sp.Symbol(f'q{i:02d}') for i, name in enumerate(model.q_names, 1)}`	Assigns `q_map` from `{name: sp.Symbol(f'q{i:02d}') for i, name in enumerate(model.q_names, 1)}`. This intermediate value supports the export/regeneration workflow.
35	`u_map = {name: sp.Symbol(f'u{i:02d}') for i, name in enumerate(model.u_names, 1)}`	Assigns `u_map` from `{name: sp.Symbol(f'u{i:02d}') for i, name in enumerate(model.u_names, 1)}`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
36	`p_map = {name: sp.Symbol(f'P{i:02d}') for i, name in enumerate(model.p_names, 1)}`	Assigns `p_map` from `{name: sp.Symbol(f'P{i:02d}') for i, name in enumerate(model.p_names, 1)}`. This intermediate value supports the export/regeneration workflow.
37	`in_map = {name: sp.Symbol(f'IN{i:02d}') for i, name in enumerate(model.input_names, 1)}`	Assigns `in_map` from `{name: sp.Symbol(f'IN{i:02d}') for i, name in enumerate(model.input_names, 1)}`. This intermediate value supports the export/regeneration workflow.
38	`rep = {}`	Assigns `rep` from `{}`. This intermediate value supports the export/regeneration workflow.
39	`for i, name in enumerate(model.q_names, 1): rep[f'q{i:02d}'] = f'q({i})'`	Starts a Python loop over the specified collection or range.
40	`for i, name in enumerate(model.u_names, 1): rep[f'u{i:02d}'] = f'u({i})'`	Starts a Python loop over the specified collection or range.
41	`for i, name in enumerate(model.p_names, 1): rep[f'P{i:02d}'] = f'P({i})'`	Starts a Python loop over the specified collection or range.
42	`for i, name in enumerate(model.input_names, 1): rep[f'IN{i:02d}'] = f'IN({i})'`	Starts a Python loop over the specified collection or range.
43		Blank line used to separate logical blocks and improve readability.
44	`def replace_source_symbols(expr):`	Defines Python helper function `replace_source_symbols`. Maps source symbolic names to vector-indexed generated-code variables.
45	`expr = sp.sympify(expr)`	Assigns `expr` from `sp.sympify(expr)`. This intermediate value supports the export/regeneration workflow.
46	`local = {}`	Assigns `local` from `{}`. This intermediate value supports the export/regeneration workflow.
47	`for sym in expr.atoms(sp.Symbol):`	Starts a Python loop over the specified collection or range.
48	`nm = str(sym)`	Assigns `nm` from `str(sym)`. This intermediate value supports the export/regeneration workflow.
49	`if nm in p_map:`	Starts a Python conditional branch; the indented block runs only if the condition is true.
50	`local[sym] = p_map[nm]`	Assigns `local[sym]` from `p_map[nm]`. This intermediate value supports the export/regeneration workflow.
51	`elif nm in in_map:`	Starts an alternative Python conditional branch.
52	`local[sym] = in_map[nm]`	Assigns `local[sym]` from `in_map[nm]`. This intermediate value supports the export/regeneration workflow.
53	`for fun in expr.atoms(sp.Function):`	Starts a Python loop over the specified collection or range.
54	`nm = str(fun).replace('(t)', '')`	Assigns `nm` from `str(fun).replace('(t)', '')`. This intermediate value supports the export/regeneration workflow.
55	`if nm in q_map:`	Starts a Python conditional branch; the indented block runs only if the condition is true.
56	`local[fun] = q_map[nm]`	Assigns `local[fun]` from `q_map[nm]`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
57	`elif nm in u_map:`	Starts an alternative Python conditional branch.
58	`local[fun] = u_map[nm]`	Assigns `local[fun]` from `u_map[nm]`. This intermediate value supports the export/regeneration workflow.
59	`return expr.xreplace(local)`	Returns the computed value from the current Python function.
60		Blank line used to separate logical blocks and improve readability.
61	`exprs = []`	Assigns `exprs` from `[]`. This intermediate value supports the export/regeneration workflow.
62	`targets = []`	Assigns `targets` from `[]`. This intermediate value supports the export/regeneration workflow.
63	`for i in range(model.mass_matrix_full.rows):`	Starts a Python loop over the specified collection or range.
64	`for j in range(model.mass_matrix_full.cols):`	Starts a Python loop over the specified collection or range.
65	`e = model.mass_matrix_full[i, j]`	Assigns `e` from `model.mass_matrix_full[i, j]`. This intermediate value supports the export/regeneration workflow.
66	`if e != 0:`	Starts a Python conditional branch; the indented block runs only if the condition is true.
67	`targets.append(('M', i+1, j+1))`	Executes this Python statement in `replace_source_symbols` as part of the export/regeneration workflow.
68	`exprs.append(replace_source_symbols(e))`	Executes this Python statement in `replace_source_symbols` as part of the export/regeneration workflow.
69	`for i, e in enumerate(model.forcing_full):`	Starts a Python loop over the specified collection or range.
70	`if e != 0:`	Starts a Python conditional branch; the indented block runs only if the condition is true.
71	`targets.append(('F', i+1, 1))`	Executes this Python statement in `replace_source_symbols` as part of the export/regeneration workflow.
72	`exprs.append(replace_source_symbols(e))`	Executes this Python statement in `replace_source_symbols` as part of the export/regeneration workflow.
73	`for i, e in enumerate(model.sensor_exprs):`	Starts a Python loop over the specified collection or range.
74	`if e != 0:`	Starts a Python conditional branch; the indented block runs only if the condition is true.
75	`targets.append(('S', i+1, 1))`	Executes this Python statement in `replace_source_symbols` as part of the export/regeneration workflow.
76	`exprs.append(replace_source_symbols(e))`	Executes this Python statement in `replace_source_symbols` as part of the export/regeneration workflow.
77		Blank line used to separate logical blocks and improve readability.
78	`repls, reduced = sp.cse(exprs, symbols=sp.numbered_symbols('zz'))`	Applies common-subexpression elimination so the generated MATLAB/CasADi formulas are shorter and faster.
79	`pattern = re.compile(r'\b(' + '|'.join(sorted(map(re.escape, rep.keys()), key=len, reverse=True)) + r')\b')`	Executes a temporary in-memory copy of the source model so expressions can be extracted without editing the uploaded file.
80		Blank line used to separate logical blocks and improve readability.
81	`def mcode(expr: sp.Expr) -> str:`	Defines Python helper function `mcode`. Prints a symbolic expression in MATLAB/CasADi-compatible syntax.
82	`c = octave_code(expr, assign_to=None)`	Converts one algebraic expression to MATLAB-compatible text.
83	`c = pattern.sub(lambda m: rep[m.group(1)], c)`	Assigns `c` from `pattern.sub(lambda m: rep[m.group(1)], c)`. This intermediate value supports the export/regeneration workflow.
84	`return c`	Returns the computed value from the current Python function.

Line	Sanitized code line	Explanation
85		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
86	`lines = []`	Assigns `lines` from `[]`. This intermediate value supports the export/regeneration workflow.
87	`lines.append("function [M,F,S] = vehicle14_generated_dynamics_matrices_casadi(q,u,P,IN)")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
88	`lines.append("%VEHICLE14_generated-core_MATRICES_CASADI Auto-generated from the uploaded symbolic generator 14-DOF model.")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
89	`lines.append("% Inputs: q(14), u(14), P(78), IN(34). Outputs are CasADi SX expressions.")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
90	`lines.append("% Do not edit this file by hand; regenerate it from export_vehicle14_matlab_casadi.py if the Python model changes.")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
91	`lines.append("import casadi.*")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
92	`lines.append("M = SX.zeros(28,28);")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
93	`lines.append("F = SX.zeros(28,1);")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
94	`lines.append(f"S = SX.zeros({len(model.sensor_names)},1);")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
95	`for sym, e in repls:`	Starts a Python loop over the specified collection or range.
96	`lines.append(f"{sym} = {mcode(e)};")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
97	`for (name, i, j), e in zip(targets, reduced):`	Starts a Python loop over the specified collection or range.
98	`if name == 'M':`	Starts a Python conditional branch; the indented block runs only if the condition is true.
99	`lines.append(f"M({i},{j}) = {mcode(e)};")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
100	`elif name == 'F':`	Starts an alternative Python conditional branch.
101	`lines.append(f"F({i}) = {mcode(e)};")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
102	`else:`	Starts the fallback branch of the current conditional block.
103	`lines.append(f"S({i}) = {mcode(e)};")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
104	`lines.append("end")`	Executes this Python statement in `mcode` as part of the export/regeneration workflow.
105	`(OUT/'vehicle14_generated_dynamics_matrices_casadi.m').write_text('\n'.join(lines) + '\n')`	Writes generated text content to an output file.
106		Blank line used to separate logical blocks and improve readability.
107	`# ----------------------------------------------------------------------`	Comment line. It documents the exporter workflow; Python does not execute it.
108	`# Meta and params.`	Comment line. It documents the exporter workflow; Python does not execute it.
109	`# ----------------------------------------------------------------------`	Comment line. It documents the exporter workflow; Python does not execute it.
110	`def matlab_cellstr(names):`	Defines Python helper function `matlab_cellstr`. Formats Python string lists as MATLAB cell-string syntax.
111	`chunks = []`	Assigns `chunks` from `[]`. This intermediate value supports the export/regeneration workflow.
112	`for n in names:`	Starts a Python loop over the specified collection or range.

Line	Sanitized code line	Explanation
113	`chunks.append("'" + str(n).replace("'", "''") + "'")`	Executes this Python statement in `matlab_cellstr` as part of the export/regeneration workflow.
114	`return '{' + ', '.join(chunks) + '}'`	Returns the computed value from the current Python function.
115		Blank line used to separate logical blocks and improve readability.
116	`def valid_field(name: str) -> str:`	Defines Python helper function `valid_field`. Converts names into valid MATLAB struct-field names.
117	`# The generated names are already valid, but keep this robust.`	Comment line. It documents the exporter workflow; Python does not execute it.
118	`out = re.sub(r'\W', '_', name)`	Assigns `out` from `re.sub(r'\W', '_', name)`. This intermediate value supports the export/regeneration workflow.
119	`if not re.match(r'[A-Za-z]', out):`	Starts a Python conditional branch; the indented block runs only if the condition is true.
120	`out = 'x_' + out`	Assigns `out` from `'x_' + out`. This intermediate value supports the export/regeneration workflow.
121	`return out`	Returns the computed value from the current Python function.
122		Blank line used to separate logical blocks and improve readability.
123	`meta_lines = [`	Assigns `meta_lines` from `[`. This intermediate value supports the export/regeneration workflow.
124	`"function meta = vehicle14_generated_dynamics_meta()",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
125	`"% Metadata matching vehicle14_generated_dynamics_matrices_casadi.m.",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
126	`f"meta.q_names = {matlab_cellstr(model.q_names)};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
127	`f"meta.u_names = {matlab_cellstr(model.u_names)};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
128	`f"meta.p_names = {matlab_cellstr(model.p_names)};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
129	`f"meta.input_names = {matlab_cellstr(model.input_names)};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
130	`f"meta.sensor_names = {matlab_cellstr(model.sensor_names)};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
131	`"meta.nq = 14; meta.nu = 14; meta.nx = 28;",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
132	`f"meta.np = {len(model.p_names)}; meta.nin = {len(model.input_names)}; meta.ns = {len(model.sensor_names)};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
133	`"meta.input_index = struct();",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
134	`]`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
135	`for i,n in enumerate(model.input_names,1):`	Starts a Python loop over the specified collection or range.
136	`meta_lines.append(f"meta.input_index.{valid_field(n)} = {i};")`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
137	`meta_lines.append("meta.sensor_index = struct();")`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
138	`for i,n in enumerate(model.sensor_names,1):`	Starts a Python loop over the specified collection or range.
139	`meta_lines.append(f"meta.sensor_index.{valid_field(n)} = {i};")`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
140	`meta_lines.append("end")`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
141	`(OUT/'vehicle14_generated_dynamics_meta.m').write_text('\n'.join(meta_lines)+'\n')`	Writes generated text content to an output file.
142		Blank line used to separate logical blocks and improve readability.
143	`# Params: all dataclass fields and the P vector in the exact order expected by the generated EOM.`	Comment line. It documents the exporter workflow; Python does not execute it.
144	`# Preserve NaN preload fields, but P uses resolved_preloads like the Python as_symbol_tuple method.`	Comment line. It documents the exporter workflow; Python does not execute it.
145	`param_lines = ["function par = vehicle14_params_default()", "% Default parameters copied from the uploaded Python VehicleParams dataclass."]`	Assigns `param_lines` from `["function par = vehicle14_params_default()", "% Default parameters copied from the uploaded Python VehicleParams dataclass."]`. This intermediate value supports the export/regeneration workflow.
146	`for f in fields(par):`	Starts a Python loop over the specified collection or range.
147	`val = getattr(par, f.name)`	Assigns `val` from `getattr(par, f.name)`. This intermediate value supports the export/regeneration workflow.
148	`if isinstance(val, float):`	Starts a Python conditional branch; the indented block runs only if the condition is true.
149	`if val != val:`	Starts a Python conditional branch; the indented block runs only if the condition is true.
150	`s='NaN'`	Assigns `s` from `'NaN'`. This intermediate value supports the export/regeneration workflow.
151	`else:`	Starts the fallback branch of the current conditional block.
152	`s=repr(float(val))`	Assigns `s` from `repr(float(val))`. This intermediate value supports the export/regeneration workflow.
153	`elif isinstance(val, int):`	Starts an alternative Python conditional branch.
154	`s=str(val)`	Assigns `s` from `str(val)`. This intermediate value supports the export/regeneration workflow.
155	`else:`	Starts the fallback branch of the current conditional block.
156	`s=repr(val)`	Assigns `s` from `repr(val)`. This intermediate value supports the export/regeneration workflow.
157	`param_lines.append(f"par.{f.name} = {s};")`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
158	`P_vals = list(par.as_symbol_tuple())`	Assigns `P_vals` from `list(par.as_symbol_tuple())`. This intermediate value supports the export/regeneration workflow.
159	`P_str = '; '.join('NaN' if (isinstance(v,float) and v!=v) else repr(float(v)) for v in P_vals)`	Assigns `P_str` from `'; '.join('NaN' if (isinstance(v,float) and v!=v) else repr(float(v)) for v in P_vals)`. This intermediate value supports the export/regeneration workflow.
160	`param_lines += [`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
161	`"par.corner_names = {'FL','FR','RL','RR'};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
162	`f"par.p_names = {matlab_cellstr(model.p_names)};",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
163	`f"par.P = [{P_str}];",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
164	`"end",`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
165	`]`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
166	`(OUT/'vehicle14_params_default.m').write_text('\n'.join(param_lines)+'\n')`	Writes generated text content to an output file.
167		Blank line used to separate logical blocks and improve readability.
168	`# Build function helper.`	Comment line. It documents the exporter workflow; Python does not execute it.

Line	Sanitized code line	Explanation
169	`(OUT/'vehicle14_build_casadi_function.m').write_text(r'''function [mf_fun, meta] = vehicle14_build_casadi_function()`	Writes generated text content to an output file.
170	`%VEHICLE14_BUILD_CASADI_FUNCTION Build the numeric CasADi Function used by MATLAB solvers.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
171	`import casadi.*`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
172	`meta = vehicle14_generated_dynamics_meta();`	Assigns `meta` from `vehicle14_generated_dynamics_meta()`. This intermediate value supports the export/regeneration workflow.
173	`q = SX.sym('q', meta.nq, 1);`	Assigns `q` from `SX.sym('q', meta.nq, 1)`. This intermediate value supports the export/regeneration workflow.
174	`u = SX.sym('u', meta.nu, 1);`	Assigns `u` from `SX.sym('u', meta.nu, 1)`. This intermediate value supports the export/regeneration workflow.
175	`P = SX.sym('P', meta.np, 1);`	Assigns `P` from `SX.sym('P', meta.np, 1)`. This intermediate value supports the export/regeneration workflow.
176	`IN = SX.sym('IN', meta.nin, 1);`	Assigns `IN` from `SX.sym('IN', meta.nin, 1)`. This intermediate value supports the export/regeneration workflow.
177	`[M,F,S] = vehicle14_generated_dynamics_matrices_casadi(q,u,P,IN);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
178	`mf_fun = Function('vehicle14_mf', {q,u,P,IN}, {M,F,S}, {'q','u','P','IN'}, {'M','F','S'});`	Assigns `mf_fun` from `Function('vehicle14_mf', {q,u,P,IN}, {M,F,S}, {'q','u','P','IN'}, {'M','F','S'})`. This intermediate value supports the export/regeneration workflow.
179	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
180	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
181		Blank line used to separate logical blocks and improve readability.
182	`# Demo options.`	Comment line. It documents the exporter workflow; Python does not execute it.
183	`(OUT/'vehicle14_demo_options.m').write_text(r'''function args = vehicle14_demo_options()`	Writes generated text content to an output file.
184	`%VEHICLE14_DEMO_OPTIONS Options matching the uploaded Python drop-then-maneuver demo.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
185	`args.drop_height = 0.25;`	Assigns `args.drop_height` from `0.25`. This intermediate value supports the export/regeneration workflow.
186	`args.settle_time = 2.0;`	Assigns `args.settle_time` from `2.0`. This intermediate value supports the export/regeneration workflow.
187	`args.maneuver_time = 16.0;`	Assigns `args.maneuver_time` from `16.0`. This intermediate value supports the export/regeneration workflow.
188	`args.t_final = args.settle_time + args.maneuver_time;`	Assigns `args.t_final` from `args.settle_time + args.maneuver_time`. This intermediate value supports the export/regeneration workflow.
189	`args.max_step = 0.006;`	Assigns `args.max_step` from `0.006`. This intermediate value supports the export/regeneration workflow.
190	`args.rtol = 1e-7;`	Assigns `args.rtol` from `1e-7`. This intermediate value supports the export/regeneration workflow.
191	`args.atol = 1e-9;`	Assigns `args.atol` from `1e-9`. This intermediate value supports the export/regeneration workflow.
192	`args.suspension_init = 'zero-force';`	Assigns `args.suspension_init` from `'zero-force'`. This intermediate value supports the export/regeneration workflow.
193	`args.initial_x = 0.0;`	Assigns `args.initial_x` from `0.0`. This intermediate value supports the export/regeneration workflow.
194	`args.initial_y = 0.0;`	Assigns `args.initial_y` from `0.0`. This intermediate value supports the export/regeneration workflow.
195	`args.initial_vx = 0.0;`	Assigns `args.initial_vx` from `0.0`. This intermediate value supports the export/regeneration workflow.
196	`args.initial_vz = 0.0;`	Assigns `args.initial_vz` from `0.0`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
197	`args.initial_wheel_omega = 0.0;`	Assigns `args.initial_wheel_omega` from `0.0`. This intermediate value supports the export/regeneration workflow.
198	`args.maneuver_profile = 'aggressive-3d';`	Assigns `args.maneuver_profile` from `'aggressive-3d'`. This intermediate value supports the export/regeneration workflow.
199	`args.throttle = 0.65;`	Assigns `args.throttle` from `0.65`. This intermediate value supports the export/regeneration workflow.
200	`args.throttle_start = 0.2;`	Assigns `args.throttle_start` from `0.2`. This intermediate value supports the export/regeneration workflow.
201	`args.throttle_ramp = 0.45;`	Assigns `args.throttle_ramp` from `0.45`. This intermediate value supports the export/regeneration workflow.
202	`args.steer_deg = 7.0;`	Assigns `args.steer_deg` from `7.0`. This intermediate value supports the export/regeneration workflow.
203	`args.steer_start = 1.0;`	Assigns `args.steer_start` from `1.0`. This intermediate value supports the export/regeneration workflow.
204	`args.steer_ramp = 0.5;`	Assigns `args.steer_ramp` from `0.5`. This intermediate value supports the export/regeneration workflow.
205	`args.steer_freq = 0.55;`	Assigns `args.steer_freq` from `0.55`. This intermediate value supports the export/regeneration workflow.
206	`args.brake_start = 11.5;`	Assigns `args.brake_start` from `11.5`. This intermediate value supports the export/regeneration workflow.
207	`args.brake_level = 0.28;`	Assigns `args.brake_level` from `0.28`. This intermediate value supports the export/regeneration workflow.
208	`args.brake_ramp = 0.35;`	Assigns `args.brake_ramp` from `0.35`. This intermediate value supports the export/regeneration workflow.
209	`args.brake_hold = 1.15;`	Assigns `args.brake_hold` from `1.15`. This intermediate value supports the export/regeneration workflow.
210	`args.disable_aero = false;`	Assigns `args.disable_aero` from `false`. This intermediate value supports the export/regeneration workflow.
211	`args.road_width = 12.0;`	Assigns `args.road_width` from `12.0`. This intermediate value supports the export/regeneration workflow.
212	`args.road_elev_amp = 1.0;`	Assigns `args.road_elev_amp` from `1.0`. This intermediate value supports the export/regeneration workflow.
213	`args.road_elev_wavelength = 80.0;`	Assigns `args.road_elev_wavelength` from `80.0`. This intermediate value supports the export/regeneration workflow.
214	`args.road_bank_deg = 2.0;`	Assigns `args.road_bank_deg` from `2.0`. This intermediate value supports the export/regeneration workflow.
215	`args.road_bank_wavelength = 60.0;`	Assigns `args.road_bank_wavelength` from `60.0`. This intermediate value supports the export/regeneration workflow.
216	`args.road_projection_iterations = 0;`	Assigns `args.road_projection_iterations` from `0`. This intermediate value supports the export/regeneration workflow.
217	`args.output_mat = 'vehicle14_matlab_casadi_solution.mat';`	Assigns `args.output_mat` from `'vehicle14_matlab_casadi_solution.mat'`. This intermediate value supports the export/regeneration workflow.
218	`args.plot_dir = 'vehicle14_matlab_casadi_plots';`	Assigns `args.plot_dir` from `'vehicle14_matlab_casadi_plots'`. This intermediate value supports the export/regeneration workflow.
219	`args.animation_mp4 = 'vehicle14_matlab_casadi_animation.mp4';`	Assigns `args.animation_mp4` from `'vehicle14_matlab_casadi_animation.mp4'`. This intermediate value supports the export/regeneration workflow.
220	`args.animation_gif = 'vehicle14_matlab_casadi_animation.gif';`	Assigns `args.animation_gif` from `'vehicle14_matlab_casadi_animation.gif'`. This intermediate value supports the export/regeneration workflow.
221	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
222	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
223		Blank line used to separate logical blocks and improve readability.
224	`# Road constructor.`	Comment line. It documents the exporter workflow; Python does not execute it.

Line	Sanitized code line	Explanation
225	`(OUT/'vehicle14_make_demo_road.m').write_text(r'''function road = vehicle14_make_demo_road(args)`	Writes generated text content to an output file.
226	`%VEHICLE14_MAKE_DEMO_ROAD Straight 3D ribbon road used by the executable demo.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
227	`road.width = args.road_width;`	Assigns `road.width` from `args.road_width`. This intermediate value supports the export/regeneration workflow.
228	`road.elev_amp = args.road_elev_amp;`	Assigns `road.elev_amp` from `args.road_elev_amp`. This intermediate value supports the export/regeneration workflow.
229	`road.elev_wavelength = args.road_elev_wavelength;`	Assigns `road.elev_wavelength` from `args.road_elev_wavelength`. This intermediate value supports the export/regeneration workflow.
230	`road.bank_amp = deg2rad(args.road_bank_deg);`	Assigns `road.bank_amp` from `deg2rad(args.road_bank_deg)`. This intermediate value supports the export/regeneration workflow.
231	`road.bank_wavelength = args.road_bank_wavelength;`	Assigns `road.bank_wavelength` from `args.road_bank_wavelength`. This intermediate value supports the export/regeneration workflow.
232	`road.projection_iterations = args.road_projection_iterations;`	Assigns `road.projection_iterations` from `args.road_projection_iterations`. This intermediate value supports the export/regeneration workflow.
233	`road.projection_tol = 1e-11;`	Assigns `road.projection_tol` from `1e-11`. This intermediate value supports the export/regeneration workflow.
234	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
235	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
236		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
237	`# Core numeric functions and contact layer.`	Comment line. It documents the exporter workflow; Python does not execute it.
238	`(OUT/'vehicle14_command_inputs.m').write_text(r'''function [throttle, brake, delta, delta_dot] = vehicle14_command_inputs(t_abs, par, args)`	Writes generated text content to an output file.
239	`%VEHICLE14_COMMAND_INPUTS Driver command profiles ported from the uploaded Python demo.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
240	`[throttle, brake, delta] = command_no_derivative(t_abs, par, args);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
241	`h = 1.0e-5;`	Assigns `h` from `1.0e-5`. This intermediate value supports the export/regeneration workflow.
242	`[~,~,delta_p] = command_no_derivative(t_abs + h, par, args);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
243	`[~,~,delta_m] = command_no_derivative(t_abs - h, par, args);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
244	`delta_dot = (delta_p - delta_m)/(2*h);`	Assigns `delta_dot` from `(delta_p - delta_m)/(2*h)`. This intermediate value supports the export/regeneration workflow.
245	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
246		Blank line used to separate logical blocks and improve readability.
247	`function [throttle, brake, delta] = command_no_derivative(t_abs, par, args)`	Assigns `function [throttle, brake, delta]` from `command_no_derivative(t_abs, par, args)`. This intermediate value supports the export/regeneration workflow.
248	`if t_abs < args.settle_time`	Starts a Python conditional branch; the indented block runs only if the condition is true.
249	`throttle = 0; brake = 0; delta = 0; return;`	Assigns `throttle` from `0; brake = 0; delta = 0; return`. This intermediate value supports the export/regeneration workflow.
250	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
251	`tau = t_abs - args.settle_time;`	Assigns `tau` from `t_abs - args.settle_time`. This intermediate value supports the export/regeneration workflow.
252	`profile = args.maneuver_profile;`	Assigns `profile` from `args.maneuver_profile`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
253	`if strcmp(profile, 'straight-launch')`	Starts a Python conditional branch; the indented block runs only if the condition is true.
254	`throttle = args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp);`	Assigns `throttle` from `args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp)`. This intermediate value supports the export/regeneration workflow.
255	`brake = 0; delta = 0; return;`	Assigns `brake` from `0; delta = 0; return`. This intermediate value supports the export/regeneration workflow.
256	`elseif strcmp(profile, 'creep-steer')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
257	`throttle = args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp);`	Assigns `throttle` from `args.throttle * smooth_step(tau, args.throttle_start, args.throttle_start + args.throttle_ramp)`. This intermediate value supports the export/regeneration workflow.
258	`brake = 0;`	Assigns `brake` from `0`. This intermediate value supports the export/regeneration workflow.
259	`if args.brake_start >= 0 && tau >= args.brake_start`	Starts a Python conditional branch; the indented block runs only if the condition is true.
260	`bw = smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp);`	Assigns `bw` from `smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp)`. This intermediate value supports the export/regeneration workflow.
261	`brake = args.brake_level*bw;`	Assigns `brake` from `args.brake_level*bw`. This intermediate value supports the export/regeneration workflow.
262	`throttle = throttle*(1-bw);`	Assigns `throttle` from `throttle*(1-bw)`. This intermediate value supports the export/regeneration workflow.
263	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
264	`steer_amp = deg2rad(args.steer_deg);`	Assigns `steer_amp` from `deg2rad(args.steer_deg)`. This intermediate value supports the export/regeneration workflow.
265	`env = smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp);`	Assigns `env` from `smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp)`. This intermediate value supports the export/regeneration workflow.
266	`delta = steer_amp*env*sin(2*pi*args.steer_freq*max(tau-args.steer_start,0));`	Assigns `delta` from `steer_amp*env*sin(2*pi*args.steer_freq*max(tau-args.steer_start,0))`. This intermediate value supports the export/regeneration workflow.
267	`return;`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
268	`elseif strcmp(profile, 'source-demo')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
269	`throttle = 0.55*smooth_step(tau, 0.2, 0.8);`	Assigns `throttle` from `0.55*smooth_step(tau, 0.2, 0.8)`. This intermediate value supports the export/regeneration workflow.
270	`brake = 0;`	Assigns `brake` from `0`. This intermediate value supports the export/regeneration workflow.
271	`delta = par.steer_max * smooth_step(tau, 0.8, 1.2) * sin(2*pi*0.35*max(tau-0.8,0));`	Assigns `delta` from `par.steer_max * smooth_step(tau, 0.8, 1.2) * sin(2*pi*0.35*max(tau-0.8,0))`. This intermediate value supports the export/regeneration workflow.
272	`return;`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
273	`elseif strcmp(profile, 'aggressive-3d')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
274	`launch = smooth_step(tau, 0.10, 0.55);`	Assigns `launch` from `smooth_step(tau, 0.10, 0.55)`. This intermediate value supports the export/regeneration workflow.
275	`throttle = args.throttle * launch;`	Assigns `throttle` from `args.throttle * launch`. This intermediate value supports the export/regeneration workflow.
276	`brake_window = 0;`	Assigns `brake_window` from `0`. This intermediate value supports the export/regeneration workflow.
277	`if args.brake_start >= 0`	Starts a Python conditional branch; the indented block runs only if the condition is true.
278	`brake_window = smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp) * ...`	Assigns `brake_window` from `smooth_step(tau, args.brake_start, args.brake_start + args.brake_ramp) * ...`. This intermediate value supports the export/regeneration workflow.
279	`(1 - smooth_step(tau, args.brake_start + args.brake_hold, args.brake_start + args.brake_hold + args.brake_ramp));`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
280	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
281	`brake = args.brake_level * brake_window;`	Assigns `brake` from `args.brake_level * brake_window`. This intermediate value supports the export/regeneration workflow.
282	`throttle = throttle * (1 - 0.85*brake_window);`	Assigns `throttle` from `throttle * (1 - 0.85*brake_window)`. This intermediate value supports the export/regeneration workflow.
283	`steer_amp = deg2rad(args.steer_deg);`	Assigns `steer_amp` from `deg2rad(args.steer_deg)`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
284	`env = smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp) * ...`	Assigns `env` from `smooth_step(tau, args.steer_start, args.steer_start + args.steer_ramp) * ...`. This intermediate value supports the export/regeneration workflow.
285	`(1 - smooth_step(tau, args.maneuver_time - 1.0, args.maneuver_time - 0.2));`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
286	`tt = max(tau - args.steer_start, 0);`	Assigns `tt` from `max(tau - args.steer_start, 0)`. This intermediate value supports the export/regeneration workflow.
287	`raw = 0.78*sin(2*pi*args.steer_freq*tt) + 0.28*sin(2*pi*1.75*args.steer_freq*tt + 0.45);`	Assigns `raw` from `0.78*sin(2*pi*args.steer_freq*tt) + 0.28*sin(2*pi*1.75*args.steer_freq*tt + 0.45)`. This intermediate value supports the export/regeneration workflow.
288	`delta = steer_amp * env * min(max(raw, -1), 1);`	Assigns `delta` from `steer_amp * env * min(max(raw, -1), 1)`. This intermediate value supports the export/regeneration workflow.
289	`return;`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
290	`else`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
291	`error('Unknown maneuver_profile: %s', profile);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
292	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
293	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
294		Blank line used to separate logical blocks and improve readability.
295	`function y = smooth_step(t, t0, t1)`	Assigns `function y` from `smooth_step(t, t0, t1)`. This intermediate value supports the export/regeneration workflow.
296	`if t <= t0`	Starts a Python conditional branch; the indented block runs only if the condition is true.
297	`y = 0;`	Assigns `y` from `0`. This intermediate value supports the export/regeneration workflow.
298	`elseif t >= t1`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
299	`y = 1;`	Assigns `y` from `1`. This intermediate value supports the export/regeneration workflow.
300	`else`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
301	`a = (t-t0)/(t1-t0);`	Assigns `a` from `(t-t0)/(t1-t0)`. This intermediate value supports the export/regeneration workflow.
302	`y = a*a*(3 - 2*a);`	Assigns `y` from `a*a*(3 - 2*a)`. This intermediate value supports the export/regeneration workflow.
303	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
304	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
305	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
306		Blank line used to separate logical blocks and improve readability.
307	`(OUT/'vehicle14_core_numeric.m').write_text(r'''function [M,F,aux] = vehicle14_core_numeric(t, x, mf_fun, par, args, road, meta)`	Writes generated text content to an output file.
308	`%VEHICLE14_CORE_NUMERIC Evaluate M(x)*xdot = F(x) with numeric contact/tyre inputs.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
309	`q = x(1:14);`	Assigns `q` from `x(1:14)`. This intermediate value supports the export/regeneration workflow.
310	`u = x(15:28);`	Assigns `u` from `x(15:28)`. This intermediate value supports the export/regeneration workflow.
311	`[throttle, brake, delta, delta_dot] = vehicle14_command_inputs(t, par, args);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
312	`IN0 = zeros(meta.nin,1);`	Assigns `IN0` from `zeros(meta.nin,1)`. This intermediate value supports the export/regeneration workflow.
313	`IN0(meta.input_index.delta) = delta;`	Assigns `IN0(meta.input_index.delta)` from `delta`. This intermediate value supports the export/regeneration workflow.
314	`IN0(meta.input_index.delta_dot) = delta_dot;`	Assigns `IN0(meta.input_index.delta_dot)` from `delta_dot`. This intermediate value supports the export/regeneration workflow.
315	`[~,~,Sdm] = mf_fun(q, u, par.P, IN0);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
316	`S = full(Sdm);`	Assigns `S` from `full(Sdm)`. This intermediate value supports the export/regeneration workflow.
317	`[IN, cdiag] = vehicle14_component_inputs_numeric(t, x, S, par, args, road, meta, throttle, brake, delta, delta_dot);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
318	`[Mdm,Fdm,~] = mf_fun(q, u, par.P, IN);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
319	`M = full(Mdm);`	Assigns `M` from `full(Mdm)`. This intermediate value supports the export/regeneration workflow.
320	`F = full(Fdm);`	Assigns `F` from `full(Fdm)`. This intermediate value supports the export/regeneration workflow.
321	`if nargout > 2`	Starts a Python conditional branch; the indented block runs only if the condition is true.
322	`aux.S = S;`	Assigns `aux.S` from `S`. This intermediate value supports the export/regeneration workflow.
323	`aux.IN = IN;`	Assigns `aux.IN` from `IN`. This intermediate value supports the export/regeneration workflow.
324	`aux.contact = cdiag;`	Assigns `aux.contact` from `cdiag`. This intermediate value supports the export/regeneration workflow.
325	`aux.throttle = throttle;`	Assigns `aux.throttle` from `throttle`. This intermediate value supports the export/regeneration workflow.
326	`aux.brake = brake;`	Assigns `aux.brake` from `brake`. This intermediate value supports the export/regeneration workflow.
327	`aux.delta = delta;`	Assigns `aux.delta` from `delta`. This intermediate value supports the export/regeneration workflow.
328	`aux.delta_dot = delta_dot;`	Assigns `aux.delta_dot` from `delta_dot`. This intermediate value supports the export/regeneration workflow.
329	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
330	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
331	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
332		Blank line used to separate logical blocks and improve readability.
333	`(OUT/'vehicle14_rhs_full_numeric.m').write_text(r'''function dx = vehicle14_rhs_full_numeric(t, x, mf_fun, par, args, road, meta)`	Writes generated text content to an output file.

Line	Sanitized code line	Explanation
334	%VEHICLE14_RHS_FULL_NUMERIC Explicit RHS xdot = M\F for MATLAB explicit/stiff ODE solvers.	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
335	[M,F] = vehicle14_core_numeric(t, x, mf_fun, par, args, road, meta);	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
336	dx = M \ F;	Assigns `dx` from `M \ F`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
337	end	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
338	''')	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
339		Blank line used to separate logical blocks and improve readability.
340	(OUT/'vehicle14_residual_full_numeric.m').write_text(r'''function res = vehicle14_residual_full_numeric(t, x, xdot, mf_fun, par, args, road, meta)	Writes generated text content to an output file.
341	%VEHICLE14_RESIDUAL_FULL_NUMERIC Implicit residual M(x)*xdot - F(x) for ode15i.	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
342	[M,F] = vehicle14_core_numeric(t, x, mf_fun, par, args, road, meta);	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
343	res = M*xdot(:) - F;	Assigns `res` from `M*xdot(:) - F`. This intermediate value supports the export/regeneration workflow.
344	end	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
345	''')	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
346		Blank line used to separate logical blocks and improve readability.
347	(OUT/'vehicle14_initial_state.m').write_text(r'''function x0 = vehicle14_initial_state(mf_fun, par, args, road, meta)	Writes generated text content to an output file.
348	%VEHICLE14_INITIAL_STATE Sensor-based drop initial condition with requested tyre gap.	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
349	q0 = zeros(14,1);	Assigns `q0` from `zeros(14,1)`. This intermediate value supports the export/regeneration workflow.
350	u0 = zeros(14,1);	Assigns `u0` from `zeros(14,1)`. This intermediate value supports the export/regeneration workflow.
351	q0(1) = args.initial_x;	Assigns `q0(1)` from `args.initial_x`. This intermediate value supports the export/regeneration workflow.
352	q0(2) = args.initial_y;	Assigns `q0(2)` from `args.initial_y`. This intermediate value supports the export/regeneration workflow.
353	q0(7:10) = initial_suspension_coordinates(par, args.suspension_init);	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
354	u0(1) = args.initial_vx;	Assigns `u0(1)` from `args.initial_vx`. This intermediate value supports the export/regeneration workflow.
355	u0(3) = args.initial_vz;	Assigns `u0(3)` from `args.initial_vz`. This intermediate value supports the export/regeneration workflow.
356	u0(11:14) = args.initial_wheel_omega;	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
357	q0(3) = par.hc + args.drop_height;	Assigns `q0(3)` from `par.hc + args.drop_height`. This intermediate value supports the export/regeneration workflow.
358	for k = 1:8	Starts a Python loop over the specified collection or range.
359	gap = min_contact_gap(q0, u0, mf_fun, par, road, meta);	Assigns `gap` from `min_contact_gap(q0, u0, mf_fun, par, road, meta)`. This intermediate value supports the export/regeneration workflow.
360	err = gap - args.drop_height;	Assigns `err` from `gap - args.drop_height`. This intermediate value supports the export/regeneration workflow.
361	if abs(err) < 1e-10, break; end	Starts a Python conditional branch; the indented block runs only if the condition is true.
362	dZ = 1e-5;	Assigns `dZ` from `1e-5`. This intermediate value supports the export/regeneration workflow.
363	qp = q0; qm = q0; qp(3) = qp(3) + dZ; qm(3) = qm(3) - dZ;	Assigns `qp` from `q0; qm = q0; qp(3) = qp(3) + dZ; qm(3) = qm(3) - dZ`. This intermediate value supports the export/regeneration workflow.
364	gp = min_contact_gap(qp, u0, mf_fun, par, road, meta);	Assigns `gp` from `min_contact_gap(qp, u0, mf_fun, par, road, meta)`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
365	gm = min_contact_gap(qm, u0, mf_fun, par, road, meta);	Assigns `gm` from `min_contact_gap(qm, u0, mf_fun, par, road, meta)`. This intermediate value supports the export/regeneration workflow.
366	dgap = (gp-gm)/(2*dZ);	Assigns `dgap` from `(gp-gm)/(2*dZ)`. This intermediate value supports the export/regeneration workflow.
367	if ~isfinite(dgap) || abs(dgap) < 1e-9	Starts a Python conditional branch; the indented block runs only if the condition is true.
368	q0(3) = q0(3) - err;	Assigns `q0(3)` from `q0(3) - err`. This intermediate value supports the export/regeneration workflow.
369	else	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
370	q0(3) = q0(3) - min(max(err/dgap, -0.5), 0.5);	Assigns `q0(3)` from `q0(3) - min(max(err/dgap, -0.5), 0.5)`. This intermediate value supports the export/regeneration workflow.
371	end	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
372	end	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
373	x0 = [q0; u0];	Assigns `x0` from `[q0; u0]`. This intermediate value supports the export/regeneration workflow.
374	end	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
375		Blank line used to separate logical blocks and improve readability.
376	function z = initial_suspension_coordinates(par, mode)	Assigns `function z` from `initial_suspension_coordinates(par, mode)`. This intermediate value supports the export/regeneration workflow.
377	Fpre = resolved_preloads(par);	Assigns `Fpre` from `resolved_preloads(par)`. This intermediate value supports the export/regeneration workflow.
378	if strcmp(mode, 'zero-force')	Starts a Python conditional branch; the indented block runs only if the condition is true.
379	z = [-Fpre(1)/par.ksf; -Fpre(2)/par.ksf; -Fpre(3)/par.ksr; -Fpre(4)/par.ksr];	Assigns `z` from `[-Fpre(1)/par.ksf; -Fpre(2)/par.ksf; -Fpre(3)/par.ksr; -Fpre(4)/par.ksr]`. This intermediate value supports the export/regeneration workflow.
380	elseif strcmp(mode, 'nominal') || strcmp(mode, 'static-source')	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
381	z = zeros(4,1);	Assigns `z` from `zeros(4,1)`. This intermediate value supports the export/regeneration workflow.
382	else	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
383	error('Unknown suspension_init: %s', mode);	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
384	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
385	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
386		Blank line used to separate logical blocks and improve readability.
387	`function Fpre = resolved_preloads(par)`	Assigns `function Fpre` from `resolved_preloads(par)`. This intermediate value supports the export/regeneration workflow.
388	`total_w = par.mB*par.g;`	Assigns `total_w` from `par.mB*par.g`. This intermediate value supports the export/regeneration workflow.
389	`front_axle = total_w*par.lr/(par.lf+par.lr);`	Assigns `front_axle` from `total_w*par.lr/(par.lf+par.lr)`. This intermediate value supports the export/regeneration workflow.
390	`rear_axle = total_w*par.lf/(par.lf+par.lr);`	Assigns `rear_axle` from `total_w*par.lf/(par.lf+par.lr)`. This intermediate value supports the export/regeneration workflow.
391	`defaults = [0.5*front_axle; 0.5*front_axle; 0.5*rear_axle; 0.5*rear_axle];`	Assigns `defaults` from `[0.5*front_axle; 0.5*front_axle; 0.5*rear_axle; 0.5*rear_axle]`. This intermediate value supports the export/regeneration workflow.
392	`user = [par.Fpre_FL; par.Fpre_FR; par.Fpre_RL; par.Fpre_RR];`	Assigns `user` from `[par.Fpre_FL; par.Fpre_FR; par.Fpre_RL; par.Fpre_RR]`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
393	`Fpre = defaults;`	Assigns `Fpre` from `defaults`. This intermediate value supports the export/regeneration workflow.
394	`for i=1:4`	Starts a Python loop over the specified collection or range.
395	`if ~isnan(user(i)), Fpre(i)=user(i); end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
396	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
397	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
398		Blank line used to separate logical blocks and improve readability.
399	`function gap = min_contact_gap(q0, u0, mf_fun, par, road, meta)`	Assigns `function gap` from `min_contact_gap(q0, u0, mf_fun, par, road, meta)`. This intermediate value supports the export/regeneration workflow.
400	`IN0 = zeros(meta.nin,1);`	Assigns `IN0` from `zeros(meta.nin,1)`. This intermediate value supports the export/regeneration workflow.
401	`[~,~,Sdm] = mf_fun(q0, u0, par.P, IN0);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
402	`S = full(Sdm);`	Assigns `S` from `full(Sdm)`. This intermediate value supports the export/regeneration workflow.
403	`idx = meta.sensor_index;`	Assigns `idx` from `meta.sensor_index`. This intermediate value supports the export/regeneration workflow.
404	`names = {'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This intermediate value supports the export/regeneration workflow.
405	`gaps = zeros(4,1);`	Assigns `gaps` from `zeros(4,1)`. This intermediate value supports the export/regeneration workflow.
406	`for i=1:4`	Starts a Python loop over the specified collection or range.
407	`nm = names{i};`	Assigns `nm` from `names{i}`. This intermediate value supports the export/regeneration workflow.
408	`p_wc = [S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];`	Assigns `p_wc` from `[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))]`. This intermediate value supports the export/regeneration workflow.
409	`head = [S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];`	Assigns `head` from `[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))]`. This intermediate value supports the export/regeneration workflow.
410	`spin = [S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];`	Assigns `spin` from `[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))]`. This intermediate value supports the export/regeneration workflow.
411	`geom = oriented_tyre_contact_geometry_local(par, road, p_wc, head, spin);`	Assigns `geom` from `oriented_tyre_contact_geometry_local(par, road, p_wc, head, spin)`. This intermediate value supports the export/regeneration workflow.
412	`gaps(i) = -geom.compression;`	Assigns `gaps(i)` from `-geom.compression`. This intermediate value supports the export/regeneration workflow.
413	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
414	`gap = min(gaps);`	Assigns `gap` from `min(gaps)`. This intermediate value supports the export/regeneration workflow.
415	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
416		Blank line used to separate logical blocks and improve readability.
417	`% Local minimal geometry copies needed before the main contact file is loaded.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
418	`function geom = oriented_tyre_contact_geometry_local(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`	Assigns `function geom` from `oriented_tyre_contact_geometry_local(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`. This intermediate value supports the export/regeneration workflow.
419	`[s,n,p_road_N,e_long_N,e_lat_N,n_road_N] = road_project_point_local(road, p_wc_N);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
420	`wheel_heading_N = normalize_num_local(wheel_heading_N, e_long_N);`	Assigns `wheel_heading_N` from `normalize_num_local(wheel_heading_N, e_long_N)`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
421	`spin_axis_N = normalize_num_local(wheel_spin_axis_N, e_lat_N);`	Assigns `spin_axis_N` from `normalize_num_local(wheel_spin_axis_N, e_lat_N)`. This intermediate value supports the export/regeneration workflow.
422	`t_long_raw = cross(spin_axis_N, n_road_N);`	Assigns `t_long_raw` from `cross(spin_axis_N, n_road_N)`. This intermediate value supports the export/regeneration workflow.
423	`if norm(t_long_raw) < 1e-10`	Starts a Python conditional branch; the indented block runs only if the condition is true.
424	`t_long_raw = wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N;`	Assigns `t_long_raw` from `wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N`. This intermediate value supports the export/regeneration workflow.
425	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
426	`t_long_N = normalize_num_local(t_long_raw, e_long_N);`	Assigns `t_long_N` from `normalize_num_local(t_long_raw, e_long_N)`. This intermediate value supports the export/regeneration workflow.
427	`if dot(t_long_N, wheel_heading_N) < 0, t_long_N = -t_long_N; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
428	`t_lat_N = normalize_num_local(cross(n_road_N,t_long_N), e_lat_N);`	Assigns `t_lat_N` from `normalize_num_local(cross(n_road_N,t_long_N), e_lat_N)`. This intermediate value supports the export/regeneration workflow.
429	`normal_in_radial_plane = n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N;`	Assigns `normal_in_radial_plane` from `n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N`. This intermediate value supports the export/regeneration workflow.
430	`gamma = norm(normal_in_radial_plane);`	Assigns `gamma` from `norm(normal_in_radial_plane)`. This intermediate value supports the export/regeneration workflow.
431	`if gamma < 1e-10`	Starts a Python conditional branch; the indented block runs only if the condition is true.
432	`gamma = 1.0; radial_down_N = -n_road_N;`	Assigns `gamma` from `1.0; radial_down_N = -n_road_N`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
433	`else`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
434	`radial_down_N = -normal_in_radial_plane/gamma;`	Assigns `radial_down_N` from `-normal_in_radial_plane/gamma`. This intermediate value supports the export/regeneration workflow.
435	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
436	`signed_center_distance = dot(p_wc_N - p_road_N, n_road_N);`	Assigns `signed_center_distance` from `dot(p_wc_N - p_road_N, n_road_N)`. This intermediate value supports the export/regeneration workflow.
437	`radial_depth_normal = par.rw*gamma;`	Assigns `radial_depth_normal` from `par.rw*gamma`. This intermediate value supports the export/regeneration workflow.
438	`compression_normal = radial_depth_normal - signed_center_distance;`	Assigns `compression_normal` from `radial_depth_normal - signed_center_distance`. This intermediate value supports the export/regeneration workflow.
439	`distance_center_to_road_along_radial = signed_center_distance/gamma;`	Assigns `distance_center_to_road_along_radial` from `signed_center_distance/gamma`. This intermediate value supports the export/regeneration workflow.
440	`compression_radial = par.rw - distance_center_to_road_along_radial;`	Assigns `compression_radial` from `par.rw - distance_center_to_road_along_radial`. This intermediate value supports the export/regeneration workflow.
441	`r_wc_to_contact_N = distance_center_to_road_along_radial*radial_down_N;`	Assigns `r_wc_to_contact_N` from `distance_center_to_road_along_radial*radial_down_N`. This intermediate value supports the export/regeneration workflow.
442	`p_contact_N = p_wc_N + r_wc_to_contact_N;`	Assigns `p_contact_N` from `p_wc_N + r_wc_to_contact_N`. This intermediate value supports the export/regeneration workflow.
443	`geom.s=s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N;`	Assigns `geom.s` from `s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N`. This intermediate value supports the export/regeneration workflow.
444	`geom.t_long_N=t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N;`	Assigns `geom.t_long_N` from `t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N`. This intermediate value supports the export/regeneration workflow.
445	`geom.r_wc_to_contact_N=r_wc_to_contact_N; geom.p_contact_N=p_contact_N;`	Assigns `geom.r_wc_to_contact_N` from `r_wc_to_contact_N; geom.p_contact_N=p_contact_N`. This intermediate value supports the export/regeneration workflow.
446	`geom.compression=compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=compression_radial;`	Assigns `geom.compression` from `compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=compression_radial`. This intermediate value supports the export/regeneration workflow.
447	`geom.contact_gap_normal=-compression_normal; geom.contact_gap_radial=-compression_radial;`	Assigns `geom.contact_gap_normal` from `-compression_normal; geom.contact_gap_radial=-compression_radial`. This intermediate value supports the export/regeneration workflow.
448	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
449		Blank line used to separate logical blocks and improve readability.
450	`function [s,n,p_road,e_long,e_lat,e_z] = road_project_point_local(road,p)`	Assigns `function [s,n,p_road,e_long,e_lat,e_z]` from `road_project_point_local(road,p)`. This intermediate value supports the export/regeneration workflow.
451	`s = p(1);`	Assigns `s` from `p(1)`. This intermediate value supports the export/regeneration workflow.
452	`[C,~,~,e_y,~,~] = centerline_quantities_local(road,s);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
453	`n = dot(p-C,e_y);`	Assigns `n` from `dot(p-C,e_y)`. This intermediate value supports the export/regeneration workflow.
454	`[p_road,e_long,e_lat,e_z] = road_surface_at_local(road,s,n);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
455	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
456		Blank line used to separate logical blocks and improve readability.
457	`function [p_road,e_long,e_lat,e_z] = road_surface_at_local(road,s,n)`	Assigns `function [p_road,e_long,e_lat,e_z]` from `road_surface_at_local(road,s,n)`. This intermediate value supports the export/regeneration workflow.
458	`[C,Cs,ex,ey,dey,ezc] = centerline_quantities_local(road,s);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
459	`p_road = C + n*ey; ps = Cs + n*dey; pn = ey;`	Assigns `p_road` from `C + n*ey; ps = Cs + n*dey; pn = ey`. This intermediate value supports the export/regeneration workflow.
460	`e_long = normalize_num_local(ps,ex); e_lat = normalize_num_local(pn,ey); e_z = normalize_num_local(cross(e_long,e_lat),ezc);`	Assigns `e_long` from `normalize_num_local(ps,ex); e_lat = normalize_num_local(pn,ey); e_z = normalize_num_local(cross(e_long,e_lat),ezc)`. This intermediate value supports the export/regeneration workflow.
461	`if e_z(3)<0, e_z=-e_z; e_lat=-e_lat; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
462	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
463		Blank line used to separate logical blocks and improve readability.
464	`function [C,Cs,ex,ey,dey,ezc] = centerline_quantities_local(road,s)`	Assigns `function [C,Cs,ex,ey,dey,ezc]` from `centerline_quantities_local(road,s)`. This intermediate value supports the export/regeneration workflow.
465	`ke=2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength;`	Assigns `ke` from `2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength`. This intermediate value supports the export/regeneration workflow.
466	`zc=road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s);`	Assigns `zc` from `road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s)`. This intermediate value supports the export/regeneration workflow.
467	`bank=road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s);`	Assigns `bank` from `road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s)`. This intermediate value supports the export/regeneration workflow.
468	`C=[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L];`	Assigns `C` from `[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L]`. This intermediate value supports the export/regeneration workflow.
469	`dez=[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num_local(cross(ex,ey),[0;0;1]);`	Assigns `dez` from `[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num_local(cross(ex,ey),[...`. This intermediate value supports the export/regeneration workflow.
470	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
471		Blank line used to separate logical blocks and improve readability.
472	`function y = normalize_num_local(v,fallback)`	Assigns `function y` from `normalize_num_local(v,fallback)`. This intermediate value supports the export/regeneration workflow.
473	`n=norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end`	Assigns `n` from `norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end`. This intermediate value supports the export/regeneration workflow.
474	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
475	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
476		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
477	`# The main component input/contact layer. Some road helper code duplicates initial_state locals so files are standalone.`	Comment line. It documents the exporter workflow; Python does not execute it.

Line	Sanitized code line	Explanation
478	`(OUT/'vehicle14_component_inputs_numeric.m').write_text(r'''function [IN, diag] = vehicle14_component_inputs_numeric(t_abs, x, S, par, args, road, meta, throttle, brake, delta, delta_dot)`	Writes generated text content to an output file.
479	`%VEHICLE14_COMPONENT_INPUTS_NUMERIC Numeric 3D-ribbon tyre/contact layer outside generated-core.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
480	`qv = x(1:14);`	Assigns `qv` from `x(1:14)`. This intermediate value supports the export/regeneration workflow.
481	`uv = x(15:28);`	Assigns `uv` from `x(15:28)`. This intermediate value supports the export/regeneration workflow.
482	`idx = meta.sensor_index;`	Assigns `idx` from `meta.sensor_index`. This intermediate value supports the export/regeneration workflow.
483	`in = meta.input_index;`	Assigns `in` from `meta.input_index`. This intermediate value supports the export/regeneration workflow.
484	`IN = zeros(meta.nin,1);`	Assigns `IN` from `zeros(meta.nin,1)`. This intermediate value supports the export/regeneration workflow.
485	`IN(in.delta) = delta;`	Assigns `IN(in.delta)` from `delta`. This intermediate value supports the export/regeneration workflow.
486	`IN(in.delta_dot) = delta_dot;`	Assigns `IN(in.delta_dot)` from `delta_dot`. This intermediate value supports the export/regeneration workflow.
487		Blank line used to separate logical blocks and improve readability.
488	`T_drive = throttle * par.T_drive_max;`	Assigns `T_drive` from `throttle * par.T_drive_max`. This intermediate value supports the export/regeneration workflow.
489	`T_brake = brake * par.T_brake_max;`	Assigns `T_brake` from `brake * par.T_brake_max`. This intermediate value supports the export/regeneration workflow.
490	`kb = par.brake_bias_front;`	Assigns `kb` from `par.brake_bias_front`. This intermediate value supports the export/regeneration workflow.
491	`omRL = uv(13); omRR = uv(14);`	Assigns `omRL` from `uv(13); omRR = uv(14)`. This intermediate value supports the export/regeneration workflow.
492	`diff_bias = par.diff_gain * par.diff_width * tanh((omRL - omRR)/par.diff_width);`	Assigns `diff_bias` from `par.diff_gain * par.diff_width * tanh((omRL - omRR)/par.diff_width)`. This intermediate value supports the export/regeneration workflow.
493	`wheel_torque.FL = -0.5 * kb * T_brake;`	Assigns `wheel_torque.FL` from `-0.5 * kb * T_brake`. This intermediate value supports the export/regeneration workflow.
494	`wheel_torque.FR = -0.5 * kb * T_brake;`	Assigns `wheel_torque.FR` from `-0.5 * kb * T_brake`. This intermediate value supports the export/regeneration workflow.
495	`wheel_torque.RL = +0.5 * T_drive - diff_bias - 0.5*(1-kb)*T_brake;`	Assigns `wheel_torque.RL` from `+0.5 * T_drive - diff_bias - 0.5*(1-kb)*T_brake`. This intermediate value supports the export/regeneration workflow.
496	`wheel_torque.RR = +0.5 * T_drive + diff_bias - 0.5*(1-kb)*T_brake;`	Assigns `wheel_torque.RR` from `+0.5 * T_drive + diff_bias - 0.5*(1-kb)*T_brake`. This intermediate value supports the export/regeneration workflow.
497		Blank line used to separate logical blocks and improve readability.
498	`names = {'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This intermediate value supports the export/regeneration workflow.
499	`diag.names = names;`	Assigns `diag.names` from `names`. This intermediate value supports the export/regeneration workflow.
500	`for i = 1:4`	Starts a Python loop over the specified collection or range.
501	`nm = names{i};`	Assigns `nm` from `names{i}`. This intermediate value supports the export/regeneration workflow.
502	`p_wc_N = [S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];`	Assigns `p_wc_N` from `[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))]`. This intermediate value supports the export/regeneration workflow.
503	`v_wc_N = [S(idx.([nm '_Wc_vx'])); S(idx.([nm '_Wc_vy'])); S(idx.([nm '_Wc_vz']))];`	Assigns `v_wc_N` from `[S(idx.([nm '_Wc_vx'])); S(idx.([nm '_Wc_vy'])); S(idx.([nm '_Wc_vz']))]`. This intermediate value supports the export/regeneration workflow.
504	`head_N = [S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];`	Assigns `head_N` from `[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))]`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
505	`spin_N = [S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];`	Assigns `spin_N` from `[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))]`. This intermediate value supports the export/regeneration workflow.
506	`carrier_omega_N = [S(idx.([nm '_carrier_wx'])); S(idx.([nm '_carrier_wy'])); S(idx.([nm '_carrier_wz']))];`	Assigns `carrier_omega_N` from `[S(idx.([nm '_carrier_wx'])); S(idx.([nm '_carrier_wy'])); S(idx.([nm '_carrier_wz']))]`. This intermediate value supports the export/regeneration workflow.
507	`omega = uv(10+i);`	Assigns `omega` from `uv(10+i)`. This intermediate value supports the export/regeneration workflow.
508	`ck = contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega);`	Assigns `ck` from `contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega)`. This intermediate value supports the export/regeneration workflow.
509	`[F_B, M_B, d] = ribbon_road_tyre_contact_to_body_wrench(qv, par, road, p_wc_N, v_wc_N, head_N, spin_N, omega, carrier_omega_N, ck);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
510	`IN(in.(['FBx' nm])) = F_B(1); IN(in.(['FBy' nm])) = F_B(2); IN(in.(['FBz' nm])) = F_B(3);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
511	`IN(in.(['MBx' nm])) = M_B(1); IN(in.(['MBy' nm])) = M_B(2); IN(in.(['MBz' nm])) = M_B(3);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
512	`IN(in.(['T' nm])) = wheel_torque.(nm);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
513	`diag.Fx(i,1) = d.Fx; diag.Fy(i,1) = d.Fy; diag.Fz(i,1) = d.Fz;`	Assigns `diag.Fx(i,1)` from `d.Fx; diag.Fy(i,1) = d.Fy; diag.Fz(i,1) = d.Fz`. This intermediate value supports the export/regeneration workflow.
514	`diag.kappa(i,1) = d.kappa; diag.alpha(i,1) = d.alpha; diag.compression(i,1) = d.compression;`	Assigns `diag.kappa(i,1)` from `d.kappa; diag.alpha(i,1) = d.alpha; diag.compression(i,1) = d.compression`. This intermediate value supports the export/regeneration workflow.
515	`diag.road_z_at_wheel(i,1) = d.road_patch_z; diag.contact_x(i,1)=d.contact_x; diag.contact_y(i,1)=d.contact_y; diag.contact_z(i,1)=d.contact_z;`	Assigns `diag.road_z_at_wheel(i,1)` from `d.road_patch_z; diag.contact_x(i,1)=d.contact_x; diag.contact_y(i,1)=d.contact_y; diag.contact_z(i,1)=d.contact_z`. This intermediate value supports the export/regeneration workflow.
516	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
517		Blank line used to separate logical blocks and improve readability.
518	`% Aero: zero during drop/settle; smooth placeholder aero during maneuver.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
519	`if t_abs < args.settle_time || args.disable_aero`	Starts a Python conditional branch; the indented block runs only if the condition is true.
520	`IN(in.FdragF) = 0; IN(in.FdragR) = 0; IN(in.FdownF) = 0; IN(in.FdownR) = 0;`	Assigns `IN(in.FdragF)` from `0; IN(in.FdragR) = 0; IN(in.FdownF) = 0; IN(in.FdownR) = 0`. This intermediate value supports the export/regeneration workflow.
521	`else`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
522	`vx = uv(1); qdyn = 0.5*par.rho*vx*vx;`	Assigns `vx` from `uv(1); qdyn = 0.5*par.rho*vx*vx`. This intermediate value supports the export/regeneration workflow.
523	`[p_af_N,~] = body_point_state_num(qv, uv, [par.aero_front_x;0;par.aero_z]);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
524	`[p_ar_N,~] = body_point_state_num(qv, uv, [par.aero_rear_x;0;par.aero_z]);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
525	`[hF,~,~,~] = road_height_normal_at_point(road, p_af_N);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
526	`[hR,~,~,~] = road_height_normal_at_point(road, p_ar_N);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
527	`multF_raw = 1 + par.aero_h_sens_front*(par.aero_h_ref - hF);`	Assigns `multF_raw` from `1 + par.aero_h_sens_front*(par.aero_h_ref - hF)`. This intermediate value supports the export/regeneration workflow.
528	`multR_raw = 1 + par.aero_h_sens_rear*(par.aero_h_ref - hR);`	Assigns `multR_raw` from `1 + par.aero_h_sens_rear*(par.aero_h_ref - hR)`. This intermediate value supports the export/regeneration workflow.
529	`multF = smooth_max(0.20, multF_raw, 1e-3);`	Assigns `multF` from `smooth_max(0.20, multF_raw, 1e-3)`. This intermediate value supports the export/regeneration workflow.
530	`multR = smooth_max(0.20, multR_raw, 1e-3);`	Assigns `multR` from `smooth_max(0.20, multR_raw, 1e-3)`. This intermediate value supports the export/regeneration workflow.
531	`D = 0.5*par.rho*par.CdA*vx*smooth_abs(vx,0.25);`	Assigns `D` from `0.5*par.rho*par.CdA*vx*smooth_abs(vx,0.25)`. This intermediate value supports the export/regeneration workflow.
532	`IN(in.FdragF) = -0.5*D; IN(in.FdragR) = -0.5*D;`	Assigns `IN(in.FdragF)` from `-0.5*D; IN(in.FdragR) = -0.5*D`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
533	`IN(in.FdownF) = -qdyn*par.ClA_front*multF;`	Assigns `IN(in.FdownF)` from `-qdyn*par.ClA_front*multF`. This intermediate value supports the export/regeneration workflow.
534	`IN(in.FdownR) = -qdyn*par.ClA_rear*multR;`	Assigns `IN(in.FdownR)` from `-qdyn*par.ClA_rear*multR`. This intermediate value supports the export/regeneration workflow.
535	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
536	`diag.throttle = throttle; diag.brake = brake; diag.delta = delta; diag.delta_dot = delta_dot;`	Assigns `diag.throttle` from `throttle; diag.brake = brake; diag.delta = delta; diag.delta_dot = delta_dot`. This intermediate value supports the export/regeneration workflow.
537	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
538		Blank line used to separate logical blocks and improve readability.
539	`function ck = contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega)`	Assigns `function ck` from `contact_kinematics_from_pose(par, road, p_wc_N, v_wc_N, head_N, spin_N, carrier_omega_N, omega)`. This intermediate value supports the export/regeneration workflow.
540	`head_N = normalize_num(head_N, [1;0;0]); spin_N = normalize_num(spin_N, [0;1;0]);`	Assigns `head_N` from `normalize_num(head_N, [1;0;0]); spin_N = normalize_num(spin_N, [0;1;0])`. This intermediate value supports the export/regeneration workflow.
541	`geom0 = oriented_tyre_contact_geometry(par, road, p_wc_N, head_N, spin_N);`	Assigns `geom0` from `oriented_tyre_contact_geometry(par, road, p_wc_N, head_N, spin_N)`. This intermediate value supports the export/regeneration workflow.
542	`head_dot_N = cross(carrier_omega_N, head_N);`	Assigns `head_dot_N` from `cross(carrier_omega_N, head_N)`. This intermediate value supports the export/regeneration workflow.
543	`spin_dot_N = cross(carrier_omega_N, spin_N);`	Assigns `spin_dot_N` from `cross(carrier_omega_N, spin_N)`. This intermediate value supports the export/regeneration workflow.
544	`h = 1e-5;`	Assigns `h` from `1e-5`. This intermediate value supports the export/regeneration workflow.
545	`geomp = oriented_tyre_contact_geometry(par, road, p_wc_N + h*v_wc_N, head_N + h*head_dot_N, spin_N + h*spin_dot_N);`	Assigns `geomp` from `oriented_tyre_contact_geometry(par, road, p_wc_N + h*v_wc_N, head_N + h*head_dot_N, spin_N + h*spin_dot_N)`. This intermediate value supports the export/regeneration workflow.
546	`geomm = oriented_tyre_contact_geometry(par, road, p_wc_N - h*v_wc_N, head_N - h*head_dot_N, spin_N - h*spin_dot_N);`	Assigns `geomm` from `oriented_tyre_contact_geometry(par, road, p_wc_N - h*v_wc_N, head_N - h*head_dot_N, spin_N - h*spin_dot_N)`. This intermediate value supports the export/regeneration workflow.
547	`ck.geom = geom0;`	Assigns `ck.geom` from `geom0`. This intermediate value supports the export/regeneration workflow.
548	`ck.contact_point_velocity_geom_N = (geomp.p_contact_N - geomm.p_contact_N)/(2*h);`	Assigns `ck.contact_point_velocity_geom_N` from `(geomp.p_contact_N - geomm.p_contact_N)/(2*h)`. This intermediate value supports the export/regeneration workflow.
549	`ck.contact_unloaded_velocity_geom_N = (geomp.p_contact_unloaded_N - geomm.p_contact_unloaded_N)/(2*h);`	Assigns `ck.contact_unloaded_velocity_geom_N` from `(geomp.p_contact_unloaded_N - geomm.p_contact_unloaded_N)/(2*h)`. This intermediate value supports the export/regeneration workflow.
550	`ck.radial_down_rate_N = (geomp.radial_down_N - geomm.radial_down_N)/(2*h);`	Assigns `ck.radial_down_rate_N` from `(geomp.radial_down_N - geomm.radial_down_N)/(2*h)`. This intermediate value supports the export/regeneration workflow.
551	`ck.road_normal_rate_N = (geomp.n_road_N - geomm.n_road_N)/(2*h);`	Assigns `ck.road_normal_rate_N` from `(geomp.n_road_N - geomm.n_road_N)/(2*h)`. This intermediate value supports the export/regeneration workflow.
552	`ck.t_long_rate_N = (geomp.t_long_N - geomm.t_long_N)/(2*h);`	Assigns `ck.t_long_rate_N` from `(geomp.t_long_N - geomm.t_long_N)/(2*h)`. This intermediate value supports the export/regeneration workflow.
553	`ck.t_lat_rate_N = (geomp.t_lat_N - geomm.t_lat_N)/(2*h);`	Assigns `ck.t_lat_rate_N` from `(geomp.t_lat_N - geomm.t_lat_N)/(2*h)`. This intermediate value supports the export/regeneration workflow.
554	`ck.compression_rate_normal = (geomp.compression_normal - geomm.compression_normal)/(2*h);`	Assigns `ck.compression_rate_normal` from `(geomp.compression_normal - geomm.compression_normal)/(2*h)`. This intermediate value supports the export/regeneration workflow.
555	`ck.compression_rate_radial = (geomp.compression_radial - geomm.compression_radial)/(2*h);`	Assigns `ck.compression_rate_radial` from `(geomp.compression_radial - geomm.compression_radial)/(2*h)`. This intermediate value supports the export/regeneration workflow.
556	`ck.contact_gap_rate_normal = (geomp.contact_gap_normal - geomm.contact_gap_normal)/(2*h);`	Assigns `ck.contact_gap_rate_normal` from `(geomp.contact_gap_normal - geomm.contact_gap_normal)/(2*h)`. This intermediate value supports the export/regeneration workflow.
557	`ck.contact_gap_rate_radial = (geomp.contact_gap_radial - geomm.contact_gap_radial)/(2*h);`	Assigns `ck.contact_gap_rate_radial` from `(geomp.contact_gap_radial - geomm.contact_gap_radial)/(2*h)`. This intermediate value supports the export/regeneration workflow.
558	`ck.road_plane_residual_rate = (geomp.road_plane_residual - geomm.road_plane_residual)/(2*h);`	Assigns `ck.road_plane_residual_rate` from `(geomp.road_plane_residual - geomm.road_plane_residual)/(2*h)`. This intermediate value supports the export/regeneration workflow.
559	`r_wc_to_contact_N = geom0.r_wc_to_contact_N;`	Assigns `r_wc_to_contact_N` from `geom0.r_wc_to_contact_N`. This intermediate value supports the export/regeneration workflow.
560	`spin_axis_N = geom0.spin_axis_N;`	Assigns `spin_axis_N` from `geom0.spin_axis_N`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
561	`ck.contact_patch_velocity_carrier_N = v_wc_N + cross(carrier_omega_N, r_wc_to_contact_N);`	Assigns `ck.contact_patch_velocity_carrier_N` from `v_wc_N + cross(carrier_omega_N, r_wc_to_contact_N)`. This intermediate value supports the export/regeneration workflow.
562	`ck.contact_patch_velocity_spin_N = cross(omega*spin_axis_N, r_wc_to_contact_N);`	Assigns `ck.contact_patch_velocity_spin_N` from `cross(omega*spin_axis_N, r_wc_to_contact_N)`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
563	`ck.contact_patch_velocity_material_N = ck.contact_patch_velocity_carrier_N + ck.contact_patch_velocity_spin_N;`	Assigns `ck.contact_patch_velocity_material_N` from `ck.contact_patch_velocity_carrier_N + ck.contact_patch_velocity_spin_N`. This intermediate value supports the export/regeneration workflow.
564	`ck.carrier_omega_N = carrier_omega_N;`	Assigns `ck.carrier_omega_N` from `carrier_omega_N`. This intermediate value supports the export/regeneration workflow.
565	`ck.road_normal_velocity_residual = dot(ck.contact_point_velocity_geom_N, geom0.n_road_N);`	Assigns `ck.road_normal_velocity_residual` from `dot(ck.contact_point_velocity_geom_N, geom0.n_road_N)`. This intermediate value supports the export/regeneration workflow.
566	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
567		Blank line used to separate logical blocks and improve readability.
568	`function geom = oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`	Assigns `function geom` from `oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`. This intermediate value supports the export/regeneration workflow.
569	`[s,n,p_road_N,e_long_N,e_lat_N,n_road_N] = road_project_point(road, p_wc_N);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
570	`wheel_heading_N = normalize_num(wheel_heading_N, e_long_N);`	Assigns `wheel_heading_N` from `normalize_num(wheel_heading_N, e_long_N)`. This intermediate value supports the export/regeneration workflow.
571	`spin_axis_N = normalize_num(wheel_spin_axis_N, e_lat_N);`	Assigns `spin_axis_N` from `normalize_num(wheel_spin_axis_N, e_lat_N)`. This intermediate value supports the export/regeneration workflow.
572	`t_long_raw = cross(spin_axis_N, n_road_N);`	Assigns `t_long_raw` from `cross(spin_axis_N, n_road_N)`. This intermediate value supports the export/regeneration workflow.
573	`if norm(t_long_raw) < 1e-10`	Starts a Python conditional branch; the indented block runs only if the condition is true.
574	`t_long_raw = wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N;`	Assigns `t_long_raw` from `wheel_heading_N - dot(wheel_heading_N,n_road_N)*n_road_N`. This intermediate value supports the export/regeneration workflow.
575	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
576	`t_long_N = normalize_num(t_long_raw, e_long_N);`	Assigns `t_long_N` from `normalize_num(t_long_raw, e_long_N)`. This intermediate value supports the export/regeneration workflow.
577	`if dot(t_long_N, wheel_heading_N) < 0, t_long_N = -t_long_N; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
578	`t_lat_N = normalize_num(cross(n_road_N, t_long_N), e_lat_N);`	Assigns `t_lat_N` from `normalize_num(cross(n_road_N, t_long_N), e_lat_N)`. This intermediate value supports the export/regeneration workflow.
579	`normal_in_radial_plane = n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N;`	Assigns `normal_in_radial_plane` from `n_road_N - dot(n_road_N, spin_axis_N)*spin_axis_N`. This intermediate value supports the export/regeneration workflow.
580	`gamma = norm(normal_in_radial_plane);`	Assigns `gamma` from `norm(normal_in_radial_plane)`. This intermediate value supports the export/regeneration workflow.
581	`if gamma < 1e-10`	Starts a Python conditional branch; the indented block runs only if the condition is true.
582	`gamma = 1.0; radial_down_N = -n_road_N; degenerate = true;`	Assigns `gamma` from `1.0; radial_down_N = -n_road_N; degenerate = true`. This intermediate value supports the export/regeneration workflow.
583	`else`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
584	`radial_down_N = -normal_in_radial_plane/gamma; degenerate = false;`	Assigns `radial_down_N` from `-normal_in_radial_plane/gamma; degenerate = false`. This intermediate value supports the export/regeneration workflow.
585	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
586	`contact_to_wc_unit_N = -radial_down_N;`	Assigns `contact_to_wc_unit_N` from `-radial_down_N`. This intermediate value supports the export/regeneration workflow.
587	`r_wc_to_unloaded_N = par.rw*radial_down_N;`	Assigns `r_wc_to_unloaded_N` from `par.rw*radial_down_N`. This intermediate value supports the export/regeneration workflow.
588	`p_contact_unloaded_N = p_wc_N + r_wc_to_unloaded_N;`	Assigns `p_contact_unloaded_N` from `p_wc_N + r_wc_to_unloaded_N`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
589	`signed_center_distance = dot(p_wc_N - p_road_N, n_road_N);`	Assigns `signed_center_distance` from `dot(p_wc_N - p_road_N, n_road_N)`. This intermediate value supports the export/regeneration workflow.
590	`radial_depth_normal = par.rw*gamma;`	Assigns `radial_depth_normal` from `par.rw*gamma`. This intermediate value supports the export/regeneration workflow.
591	`compression_normal = radial_depth_normal - signed_center_distance;`	Assigns `compression_normal` from `radial_depth_normal - signed_center_distance`. This intermediate value supports the export/regeneration workflow.
592	`distance_center_to_road_along_radial = signed_center_distance/gamma;`	Assigns `distance_center_to_road_along_radial` from `signed_center_distance/gamma`. This intermediate value supports the export/regeneration workflow.
593	`compression_radial = par.rw - distance_center_to_road_along_radial;`	Assigns `compression_radial` from `par.rw - distance_center_to_road_along_radial`. This intermediate value supports the export/regeneration workflow.
594	`r_wc_to_contact_N = distance_center_to_road_along_radial*radial_down_N;`	Assigns `r_wc_to_contact_N` from `distance_center_to_road_along_radial*radial_down_N`. This intermediate value supports the export/regeneration workflow.
595	`p_contact_N = p_wc_N + r_wc_to_contact_N;`	Assigns `p_contact_N` from `p_wc_N + r_wc_to_contact_N`. This intermediate value supports the export/regeneration workflow.
596	`road_plane_residual = dot(p_contact_N - p_road_N, n_road_N);`	Assigns `road_plane_residual` from `dot(p_contact_N - p_road_N, n_road_N)`. This intermediate value supports the export/regeneration workflow.
597	`geom.s=s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N;`	Assigns `geom.s` from `s; geom.n=n; geom.p_road_N=p_road_N; geom.e_long_N=e_long_N; geom.e_lat_N=e_lat_N; geom.n_road_N=n_road_N`. This intermediate value supports the export/regeneration workflow.
598	`geom.t_long_N=t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N; geom.contact_to_wc_unit_N=contact_to_wc_unit_N;`	Assigns `geom.t_long_N` from `t_long_N; geom.t_lat_N=t_lat_N; geom.spin_axis_N=spin_axis_N; geom.radial_down_N=radial_down_N; geom.contact_to_wc_unit_N=contact_to_wc_unit_N`. This intermediate value supports the export/regeneration workflow.
599	`geom.r_wc_to_contact_N=r_wc_to_contact_N; geom.p_contact_N=p_contact_N; geom.p_contact_road_N=p_contact_N; geom.r_wc_to_unloaded_N=r_wc_to_unloaded_N; geom.p_contact_unloaded_N=p_contact_unloaded_N;`	Assigns `geom.r_wc_to_contact_N` from `r_wc_to_contact_N; geom.p_contact_N=p_contact_N; geom.p_contact_road_N=p_contact_N; geom.r_wc_to_unloaded_N=r_wc_to_unloaded_N; geom.p_contact_unlo...`. This intermediate value supports the export/regeneration workflow.
600	`geom.signed_center_distance=signed_center_distance; geom.radial_projection_norm=gamma; geom.radial_depth=radial_depth_normal; geom.radial_depth_normal=radial_depth_normal;`	Assigns `geom.signed_center_distance` from `signed_center_distance; geom.radial_projection_norm=gamma; geom.radial_depth=radial_depth_normal; geom.radial_depth_normal=radial_depth_normal`. This intermediate value supports the export/regeneration workflow.
601	`geom.distance_center_to_road_along_radial=distance_center_to_road_along_radial; geom.compression=compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=compression_radial;`	Assigns `geom.distance_center_to_road_along_radial` from `distance_center_to_road_along_radial; geom.compression=compression_normal; geom.compression_normal=compression_normal; geom.compression_radial=comp...`. This intermediate value supports the export/regeneration workflow.
602	`geom.contact_gap_normal=-compression_normal; geom.contact_gap_radial=-compression_radial; geom.road_plane_residual=road_plane_residual; geom.degenerate_contact=degenerate;`	Assigns `geom.contact_gap_normal` from `-compression_normal; geom.contact_gap_radial=-compression_radial; geom.road_plane_residual=road_plane_residual; geom.degenerate_contact=degenerate`. This intermediate value supports the export/regeneration workflow.
603	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
604		Blank line used to separate logical blocks and improve readability.
605	`function [F_B, M_B, diag] = ribbon_road_tyre_contact_to_body_wrench(qv, par, road, p_wc_N, v_wc_N, wheel_heading_N, wheel_spin_axis_N, omega, carrier_omega_N, ck)`	Assigns `function [F_B, M_B, diag]` from `ribbon_road_tyre_contact_to_body_wrench(qv, par, road, p_wc_N, v_wc_N, wheel_heading_N, wheel_spin_axis_N, omega, carrier_omega_N, ck)`. This intermediate value supports the export/regeneration workflow.
606	`R_NB = body_rotation_from_q(qv); R_BN = R_NB.';`	Assigns `R_NB` from `body_rotation_from_q(qv); R_BN = R_NB.'`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
607	`geom = oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N);`	Assigns `geom` from `oriented_tyre_contact_geometry(par, road, p_wc_N, wheel_heading_N, wheel_spin_axis_N)`. This intermediate value supports the export/regeneration workflow.
608	`n_road_N = geom.n_road_N; t_long_N = geom.t_long_N; t_lat_N = geom.t_lat_N; spin_axis_N = geom.spin_axis_N; r_wc_to_contact_N = geom.r_wc_to_contact_N;`	Assigns `n_road_N` from `geom.n_road_N; t_long_N = geom.t_long_N; t_lat_N = geom.t_lat_N; spin_axis_N = geom.spin_axis_N; r_wc_to_contact_N = geom.r_wc_to_contact_N`. This intermediate value supports the export/regeneration workflow.
609	`compression = geom.compression;`	Assigns `compression` from `geom.compression`. This intermediate value supports the export/regeneration workflow.
610	`compression_rate = ck.compression_rate_normal;`	Assigns `compression_rate` from `ck.compression_rate_normal`. This intermediate value supports the export/regeneration workflow.
611	`compression_rate_radial = ck.compression_rate_radial;`	Assigns `compression_rate_radial` from `ck.compression_rate_radial`. This intermediate value supports the export/regeneration workflow.
612	`v_contact_geom_N = ck.contact_point_velocity_geom_N;`	Assigns `v_contact_geom_N` from `ck.contact_point_velocity_geom_N`. This intermediate value supports the export/regeneration workflow.
613	`v_contact_carrier_N = ck.contact_patch_velocity_carrier_N;`	Assigns `v_contact_carrier_N` from `ck.contact_patch_velocity_carrier_N`. This intermediate value supports the export/regeneration workflow.
614	`v_spin_contact_N = ck.contact_patch_velocity_spin_N;`	Assigns `v_spin_contact_N` from `ck.contact_patch_velocity_spin_N`. This intermediate value supports the export/regeneration workflow.
615	`v_contact_material_N = ck.contact_patch_velocity_material_N;`	Assigns `v_contact_material_N` from `ck.contact_patch_velocity_material_N`. This intermediate value supports the export/regeneration workflow.
616	`Fz = smoothplus(par.kt*compression + par.ct*compression_rate, par.normal_smooth_eps);`	Assigns `Fz` from `smoothplus(par.kt*compression + par.ct*compression_rate, par.normal_smooth_eps)`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
617	`Vx = dot(v_contact_carrier_N, t_long_N); Vy = dot(v_contact_carrier_N, t_lat_N);`	Assigns `Vx` from `dot(v_contact_carrier_N, t_long_N); Vy = dot(v_contact_carrier_N, t_lat_N)`. This intermediate value supports the export/regeneration workflow.
618	`Vx_material = dot(v_contact_material_N, t_long_N); Vy_material = dot(v_contact_material_N, t_lat_N);`	Assigns `Vx_material` from `dot(v_contact_material_N, t_long_N); Vy_material = dot(v_contact_material_N, t_lat_N)`. This intermediate value supports the export/regeneration workflow.
619	`rolling_speed = -dot(v_spin_contact_N, t_long_N);`	Assigns `rolling_speed` from `-dot(v_spin_contact_N, t_long_N)`. This intermediate value supports the export/regeneration workflow.
620	`Vden = sqrt(Vx*Vx + par.v_eps*par.v_eps);`	Assigns `Vden` from `sqrt(Vx*Vx + par.v_eps*par.v_eps)`. This intermediate value supports the export/regeneration workflow.
621	`kappa = (rolling_speed - Vx)/Vden;`	Assigns `kappa` from `(rolling_speed - Vx)/Vden`. This intermediate value supports the export/regeneration workflow.
622	`alpha = atan2(Vy, Vden);`	Assigns `alpha` from `atan2(Vy, Vden)`. This intermediate value supports the export/regeneration workflow.
623	`Fx0 = par.Ck*kappa; Fy0 = -par.Ca*alpha;`	Assigns `Fx0` from `par.Ck*kappa; Fy0 = -par.Ca*alpha`. This intermediate value supports the export/regeneration workflow.
624	`limit = par.mu*Fz + 1e-9;`	Assigns `limit` from `par.mu*Fz + 1e-9`. This intermediate value supports the export/regeneration workflow.
625	`norm_tan = sqrt(Fx0*Fx0 + Fy0*Fy0 + 1e-12);`	Assigns `norm_tan` from `sqrt(Fx0*Fx0 + Fy0*Fy0 + 1e-12)`. This intermediate value supports the export/regeneration workflow.
626	`scale = (1 + (norm_tan/limit)^4)^(-0.25);`	Assigns `scale` from `(1 + (norm_tan/limit)^4)^(-0.25)`. This intermediate value supports the export/regeneration workflow.
627	`Fx = Fx0*scale; Fy = Fy0*scale;`	Assigns `Fx` from `Fx0*scale; Fy = Fy0*scale`. This intermediate value supports the export/regeneration workflow.
628	`F_contact_N = Fx*t_long_N + Fy*t_lat_N + Fz*n_road_N;`	Assigns `F_contact_N` from `Fx*t_long_N + Fy*t_lat_N + Fz*n_road_N`. This intermediate value supports the export/regeneration workflow.
629	`F_B = R_BN*F_contact_N;`	Assigns `F_B` from `R_BN*F_contact_N`. This intermediate value supports the export/regeneration workflow.
630	`r_contact_B = R_BN*r_wc_to_contact_N;`	Assigns `r_contact_B` from `R_BN*r_wc_to_contact_N`. This intermediate value supports the export/regeneration workflow.
631	`M_B = cross(r_contact_B, F_B);`	Assigns `M_B` from `cross(r_contact_B, F_B)`. This intermediate value supports the export/regeneration workflow.
632	`diag.s=geom.s; diag.n=geom.n; diag.compression=compression; diag.compression_rate=compression_rate; diag.compression_rate_radial=compression_rate_radial;`	Assigns `diag.s` from `geom.s; diag.n=geom.n; diag.compression=compression; diag.compression_rate=compression_rate; diag.compression_rate_radial=compression_rate_radial`. This intermediate value supports the export/regeneration workflow.
633	`diag.Fz=Fz; diag.Fx=Fx; diag.Fy=Fy; diag.kappa=kappa; diag.alpha=alpha; diag.Vx_contact_carrier=Vx; diag.Vy_contact_carrier=Vy; diag.Vx_contact_material=Vx_material; diag.Vy_contact_material=Vy_material; diag.rolling_speed=rolling_speed;`	Assigns `diag.Fz` from `Fz; diag.Fx=Fx; diag.Fy=Fy; diag.kappa=kappa; diag.alpha=alpha; diag.Vx_contact_carrier=Vx; diag.Vy_contact_carrier=Vy; diag.Vx_contact_material=Vx...`. This intermediate value supports the export/regeneration workflow.
634	`diag.contact_x=geom.p_contact_N(1); diag.contact_y=geom.p_contact_N(2); diag.contact_z=geom.p_contact_N(3); diag.road_patch_x=geom.p_road_N(1); diag.road_patch_y=geom.p_road_N(2); diag.road_patch_z=geom.p_road_N(3);`	Assigns `diag.contact_x` from `geom.p_contact_N(1); diag.contact_y=geom.p_contact_N(2); diag.contact_z=geom.p_contact_N(3); diag.road_patch_x=geom.p_road_N(1); diag.road_patch_y=...`. This intermediate value supports the export/regeneration workflow.
635	`diag.road_normal_velocity_residual=ck.road_normal_velocity_residual; diag.road_plane_residual_rate=ck.road_plane_residual_rate; diag.contact_vx=v_contact_geom_N(1); diag.contact_vy=v_contact_geom_N(2); diag.contact_vz=v_contact_geom_N(3);`	Assigns `diag.road_normal_velocity_residual` from `ck.road_normal_velocity_residual; diag.road_plane_residual_rate=ck.road_plane_residual_rate; diag.contact_vx=v_contact_geom_N(1); diag.contact_vy=v...`. This intermediate value supports the export/regeneration workflow.
636	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
637		Blank line used to separate logical blocks and improve readability.
638	`function [h,e_z,s,n] = road_height_normal_at_point(road,p_N)`	Assigns `function [h,e_z,s,n]` from `road_height_normal_at_point(road,p_N)`. This intermediate value supports the export/regeneration workflow.
639	`[s,n,p_road,~,~,e_z] = road_project_point(road,p_N);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
640	`h = dot(p_N-p_road,e_z);`	Assigns `h` from `dot(p_N-p_road,e_z)`. This intermediate value supports the export/regeneration workflow.
641	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
642		Blank line used to separate logical blocks and improve readability.
643	`function [pN,vN] = body_point_state_num(qv, uv, r_B)`	Assigns `function [pN,vN]` from `body_point_state_num(qv, uv, r_B)`. This intermediate value supports the export/regeneration workflow.
644	`R = body_rotation_from_q(qv); pP = qv(1:3); vP = R*uv(1:3); omega = R*uv(4:6); rN = R*r_B;`	Assigns `R` from `body_rotation_from_q(qv); pP = qv(1:3); vP = R*uv(1:3); omega = R*uv(4:6); rN = R*r_B`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
645	`pN = pP + rN; vN = vP + cross(omega,rN);`	Assigns `pN` from `pP + rN; vN = vP + cross(omega,rN)`. This intermediate value supports the export/regeneration workflow.
646	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
647		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
648	`function R = body_rotation_from_q(qv)`	Assigns `function R` from `body_rotation_from_q(qv)`. This intermediate value supports the export/regeneration workflow.
649	`R = rot_z(qv(4))*rot_y(qv(5))*rot_x(qv(6));`	Assigns `R` from `rot_z(qv(4))*rot_y(qv(5))*rot_x(qv(6))`. This intermediate value supports the export/regeneration workflow.
650	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
651	`function R = rot_x(a), ca=cos(a); sa=sin(a); R=[1 0 0; 0 ca -sa; 0 sa ca]; end`	Assigns `function R` from `rot_x(a), ca=cos(a); sa=sin(a); R=[1 0 0; 0 ca -sa; 0 sa ca]; end`. This intermediate value supports the export/regeneration workflow.
652	`function R = rot_y(a), ca=cos(a); sa=sin(a); R=[ca 0 sa; 0 1 0; -sa 0 ca]; end`	Assigns `function R` from `rot_y(a), ca=cos(a); sa=sin(a); R=[ca 0 sa; 0 1 0; -sa 0 ca]; end`. This intermediate value supports the export/regeneration workflow.
653	`function R = rot_z(a), ca=cos(a); sa=sin(a); R=[ca -sa 0; sa ca 0; 0 0 1]; end`	Assigns `function R` from `rot_z(a), ca=cos(a); sa=sin(a); R=[ca -sa 0; sa ca 0; 0 0 1]; end`. This intermediate value supports the export/regeneration workflow.
654	`function y = smoothplus(x,epsv), y = 0.5*(x + sqrt(x*x + epsv*epsv)); end`	Assigns `function y` from `smoothplus(x,epsv), y = 0.5*(x + sqrt(x*x + epsv*epsv)); end`. This intermediate value supports the export/regeneration workflow.
655	`function y = smooth_max(a,b,epsv), d=b-a; y = a + smoothplus(d,epsv); end`	Assigns `function y` from `smooth_max(a,b,epsv), d=b-a; y = a + smoothplus(d,epsv); end`. This intermediate value supports the export/regeneration workflow.
656	`function y = smooth_abs(x,epsv), y = sqrt(x*x + epsv*epsv); end`	Assigns `function y` from `smooth_abs(x,epsv), y = sqrt(x*x + epsv*epsv); end`. This intermediate value supports the export/regeneration workflow.
657	`function y = normalize_num(v,fallback), n=norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end, end`	Assigns `function y` from `normalize_num(v,fallback), n=norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end, end`. This intermediate value supports the export/regeneration workflow.
658		Blank line used to separate logical blocks and improve readability.
659	`function [s,n,p_road,e_long,e_lat,e_z] = road_project_point(road,p)`	Assigns `function [s,n,p_road,e_long,e_lat,e_z]` from `road_project_point(road,p)`. This intermediate value supports the export/regeneration workflow.
660	`s = p(1);`	Assigns `s` from `p(1)`. This intermediate value supports the export/regeneration workflow.
661	`for k=1:max(0,road.projection_iterations)`	Starts a Python loop over the specified collection or range.
662	`[f,~,~,~,~,~,~] = projection_residual_given_s(road,p,s);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
663	`if abs(f) < road.projection_tol, break; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
664	`ds=max(1e-4,1e-6*(1+abs(s)));`	Assigns `ds` from `max(1e-4,1e-6*(1+abs(s)))`. This intermediate value supports the export/regeneration workflow.
665	`fp=projection_residual_given_s(road,p,s+ds); fm=projection_residual_given_s(road,p,s-ds);`	Assigns `fp` from `projection_residual_given_s(road,p,s+ds); fm=projection_residual_given_s(road,p,s-ds)`. This intermediate value supports the export/regeneration workflow.
666	`dfds=(fp-fm)/(2*ds);`	Assigns `dfds` from `(fp-fm)/(2*ds)`. This intermediate value supports the export/regeneration workflow.
667	`if ~isfinite(dfds) || abs(dfds)<1e-12, break; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
668	`step=min(max(f/dfds,-5),5); s=s-step; if abs(step)<road.projection_tol, break; end`	Assigns `step` from `min(max(f/dfds,-5),5); s=s-step; if abs(step)<road.projection_tol, break; end`. This intermediate value supports the export/regeneration workflow.
669	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
670	`[~,n,p_road,e_long,e_lat,e_z,~] = projection_residual_given_s(road,p,s);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
671	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
672		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
673	`function [f,n,p_road,e_long,e_lat,e_z,p_s] = projection_residual_given_s(road,p,s)`	Assigns `function [f,n,p_road,e_long,e_lat,e_z,p_s]` from `projection_residual_given_s(road,p,s)`. This intermediate value supports the export/regeneration workflow.
674	`[C,~,~,e_y,~,~] = centerline_quantities(road,s);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
675	`n = dot(p-C,e_y);`	Assigns `n` from `dot(p-C,e_y)`. This intermediate value supports the export/regeneration workflow.
676	`[p_road,e_long,e_lat,e_z,p_s,~] = road_surface_at(road,s,n);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
677	`r = p-p_road; f=dot(r,p_s);`	Assigns `r` from `p-p_road; f=dot(r,p_s)`. This intermediate value supports the export/regeneration workflow.
678	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
679		Blank line used to separate logical blocks and improve readability.
680	`function [p_road,e_long,e_lat,e_z,p_s,p_n] = road_surface_at(road,s,n)`	Assigns `function [p_road,e_long,e_lat,e_z,p_s,p_n]` from `road_surface_at(road,s,n)`. This intermediate value supports the export/regeneration workflow.
681	`[C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
682	`p_road = C + n*ey; p_s = Cs + n*dey; p_n = ey;`	Assigns `p_road` from `C + n*ey; p_s = Cs + n*dey; p_n = ey`. This intermediate value supports the export/regeneration workflow.
683	`e_long=normalize_num(p_s,ex); e_lat=normalize_num(p_n,ey); e_z=normalize_num(cross(e_long,e_lat),ezc);`	Assigns `e_long` from `normalize_num(p_s,ex); e_lat=normalize_num(p_n,ey); e_z=normalize_num(cross(e_long,e_lat),ezc)`. This intermediate value supports the export/regeneration workflow.
684	`if e_z(3)<0, e_z=-e_z; e_lat=-e_lat; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
685	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
686		Blank line used to separate logical blocks and improve readability.
687	`function [C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s)`	Assigns `function [C,Cs,ex,ey,dey,ezc]` from `centerline_quantities(road,s)`. This intermediate value supports the export/regeneration workflow.
688	`ke=2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength;`	Assigns `ke` from `2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength`. This intermediate value supports the export/regeneration workflow.
689	`zc=road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s);`	Assigns `zc` from `road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s)`. This intermediate value supports the export/regeneration workflow.
690	`bank=road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s);`	Assigns `bank` from `road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s)`. This intermediate value supports the export/regeneration workflow.
691	`C=[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L];`	Assigns `C` from `[s;0;zc]; Cs=[1;0;dz]; L=sqrt(1+dz*dz); ex=[1/L;0;dz/L]; ey0=[0;1;0]; ez0=[-dz/L;0;1/L]`. This intermediate value supports the export/regeneration workflow.
692	`dez=[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num(cross(ex,ey),[0;0;1]);`	Assigns `dez` from `[-d2z/(L^3);0;-dz*d2z/(L^3)]; cb=cos(bank); sb=sin(bank); ey=cb*ey0+sb*ez0; dey=-sb*db*ey0+cb*db*ez0+sb*dez; ezc=normalize_num(cross(ex,ey),[0;0;1])`. This intermediate value supports the export/regeneration workflow.
693	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
694	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
695		Blank line used to separate logical blocks and improve readability.
696	`(OUT/'vehicle14_save_plots_matlab.m').write_text(r'''function vehicle14_save_plots_matlab(t, x, mf_fun, par, args, road, meta, plot_dir)`	Writes generated text content to an output file.
697	`%VEHICLE14_SAVE_PLOTS_MATLAB Generate diagnostic PNG plots from the MATLAB/CasADi solution.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
698	`if nargin < 8 || isempty(plot_dir), plot_dir = args.plot_dir; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
699	`if ~exist(plot_dir, 'dir'), mkdir(plot_dir); end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
700	`N = numel(t);`	Assigns `N` from `numel(t)`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
701	`throttle=zeros(N,1); brake=zeros(N,1); delta=zeros(N,1); Fz=zeros(4,N); Fx=zeros(4,N); Fy=zeros(4,N); comp=zeros(4,N); roadz=zeros(4,N);`	Assigns `throttle` from `zeros(N,1); brake=zeros(N,1); delta=zeros(N,1); Fz=zeros(4,N); Fx=zeros(4,N); Fy=zeros(4,N); comp=zeros(4,N); roadz=zeros(4,N)`. This intermediate value supports the export/regeneration workflow.
702	`for k=1:N`	Starts a Python loop over the specified collection or range.
703	`[~,~,aux] = vehicle14_core_numeric(t(k), x(k,:).', mf_fun, par, args, road, meta);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
704	`throttle(k)=aux.throttle; brake(k)=aux.brake; delta(k)=aux.delta;`	Assigns `throttle(k)` from `aux.throttle; brake(k)=aux.brake; delta(k)=aux.delta`. This intermediate value supports the export/regeneration workflow.
705	`Fz(:,k)=aux.contact.Fz; Fx(:,k)=aux.contact.Fx; Fy(:,k)=aux.contact.Fy; comp(:,k)=aux.contact.compression; roadz(:,k)=aux.contact.road_z_at_wheel;`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
706	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
707	`names={'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This intermediate value supports the export/regeneration workflow.
708	`fig=figure('Visible','off'); plot3(x(:,1),x(:,2),x(:,3),'LineWidth',1.2); grid on; xlabel('X [m]'); ylabel('Y [m]'); zlabel('Z [m]'); title('Chassis CoM trajectory'); saveas(fig, fullfile(plot_dir,'trajectory_3d.png')); close(fig);`	Assigns `fig` from `figure('Visible','off'); plot3(x(:,1),x(:,2),x(:,3),'LineWidth',1.2); grid on; xlabel('X [m]'); ylabel('Y [m]'); zlabel('Z [m]'); title('Chassis Co...`. This intermediate value supports the export/regeneration workflow.
709	`fig=figure('Visible','off'); plot(t, rad2deg(x(:,6)),'DisplayName','roll [deg]'); hold on; plot(t, rad2deg(x(:,5)),'DisplayName','pitch [deg]'); plot(t, x(:,15),'DisplayName','vx [m/s]'); plot(t, x(:,20),'DisplayName','yaw rate r [rad/s]...`	Assigns `fig` from `figure('Visible','off'); plot(t, rad2deg(x(:,6)),'DisplayName','roll [deg]'); hold on; plot(t, rad2deg(x(:,5)),'DisplayName','pitch [deg]'); plot(t...`. This intermediate value supports the export/regeneration workflow.
710	`fig=figure('Visible','off'); plot(t, throttle,'DisplayName','throttle'); hold on; plot(t, brake,'DisplayName','brake'); plot(t, rad2deg(delta),'DisplayName','steering [deg]'); grid on; xlabel('time [s]'); title('Driver commands'); legend...`	Assigns `fig` from `figure('Visible','off'); plot(t, throttle,'DisplayName','throttle'); hold on; plot(t, brake,'DisplayName','brake'); plot(t, rad2deg(delta),'Display...`. This intermediate value supports the export/regeneration workflow.
711	`fig=figure('Visible','off'); hold on; for i=1:4, plot(t,Fz(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('Fz [N]'); title('Wheel normal loads'); legend show; saveas(fig, fullfile(plot_dir,'wheel_normal_loads.png'...`	Assigns `fig` from `figure('Visible','off'); hold on; for i=1:4, plot(t,Fz(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('Fz [N]'); title('Whe...`. This intermediate value supports the export/regeneration workflow.
712	`fig=figure('Visible','off'); hold on; for i=1:4, plot(t,comp(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('compression [m]'); title('Tyre compression'); legend show; saveas(fig, fullfile(plot_dir,'tyre_compressi...`	Assigns `fig` from `figure('Visible','off'); hold on; for i=1:4, plot(t,comp(i,:),'DisplayName',names{i}); end; grid on; xlabel('time [s]'); ylabel('compression [m]');...`. This intermediate value supports the export/regeneration workflow.
713	`fig=figure('Visible','off'); plot(x(:,1),x(:,3),'DisplayName','CoM Z'); hold on; plot(x(:,1),mean(roadz,1),'DisplayName','mean road Z at wheels'); grid on; xlabel('X [m]'); ylabel('Z [m]'); title('3D ribbon road excitation'); legend show...`	Assigns `fig` from `figure('Visible','off'); plot(x(:,1),x(:,3),'DisplayName','CoM Z'); hold on; plot(x(:,1),mean(roadz,1),'DisplayName','mean road Z at wheels'); grid...`. This intermediate value supports the export/regeneration workflow.
714	`fig=figure('Visible','off'); plot(t, x(:,7:10)); grid on; xlabel('time [s]'); ylabel('z_i [m]'); title('Suspension/wheel-centre heave coordinates'); legend(names{:}); saveas(fig, fullfile(plot_dir,'suspension_travels.png')); close(fig);`	Assigns `fig` from `figure('Visible','off'); plot(t, x(:,7:10)); grid on; xlabel('time [s]'); ylabel('z_i [m]'); title('Suspension/wheel-centre heave coordinates'); le...`. This intermediate value supports the export/regeneration workflow.
715	`fprintf('Saved MATLAB diagnostic plots to %s\n', plot_dir);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
716	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
717	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
718		Blank line used to separate logical blocks and improve readability.
719	`(OUT/'run_vehicle14_matlab_casadi_demo.m').write_text(r'''function run_vehicle14_matlab_casadi_demo()`	Writes generated text content to an output file.
720	`%RUN_VEHICLE14_MATLAB_CASADI_DEMO Integrate the uploaded 14-DOF generated dynamics model through MATLAB/CasADi.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
721	`%`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
722	`% Solver policy:`	Starts a Python block; following indented lines belong to this statement.
723	`% 1) Try MATLAB ode15i on the implicit residual M(x)*xdot-F(x)=0 first.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
724	`% 2) If ode15i fails, fall back to ode15s on the explicit RHS xdot=M\F.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
725	`%`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
726	`% This does not implement a hand-written integration scheme; it uses MATLAB solvers`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
727	`% while CasADi supplies the generated EOM/sensor evaluation.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
728		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
729	`% addpath('D:/Applications/casadi-windows-matlabR2016a-v3.6.5'); % edit if CasADi is not already on the MATLAB path`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
730	`import casadi.*`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
731	`[ver_ok, msg] = check_casadi_available();`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
732	`if ~ver_ok, error('%s', msg); end`	Starts a Python conditional branch; the indented block runs only if the condition is true.

Line	Sanitized code line	Explanation
733		Blank line used to separate logical blocks and improve readability.
734	`[mf_fun, meta] = vehicle14_build_casadi_function();`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
735	`par = vehicle14_params_default();`	Assigns `par` from `vehicle14_params_default()`. This intermediate value supports the export/regeneration workflow.
736	`args = vehicle14_demo_options();`	Assigns `args` from `vehicle14_demo_options()`. This intermediate value supports the export/regeneration workflow.
737	`road = vehicle14_make_demo_road(args);`	Assigns `road` from `vehicle14_make_demo_road(args)`. This intermediate value supports the export/regeneration workflow.
738		Blank line used to separate logical blocks and improve readability.
739	`x0 = vehicle14_initial_state(mf_fun, par, args, road, meta);`	Assigns `x0` from `vehicle14_initial_state(mf_fun, par, args, road, meta)`. This intermediate value supports the export/regeneration workflow.
740	`xdot0 = vehicle14_rhs_full_numeric(0, x0, mf_fun, par, args, road, meta);`	Assigns `xdot0` from `vehicle14_rhs_full_numeric(0, x0, mf_fun, par, args, road, meta)`. This intermediate value supports the export/regeneration workflow.
741	`tspan = [0 args.t_final];`	Assigns `tspan` from `[0 args.t_final]`. This intermediate value supports the export/regeneration workflow.
742	`opts = odeset('RelTol', args.rtol, 'AbsTol', args.atol, 'MaxStep', args.max_step);`	Assigns `opts` from `odeset('RelTol', args.rtol, 'AbsTol', args.atol, 'MaxStep', args.max_step)`. This intermediate value supports the export/regeneration workflow.
743	`solver_used = 'ode15i';`	Assigns `solver_used` from `'ode15i'`. This intermediate value supports the export/regeneration workflow.
744	`try`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
745	`fprintf('Trying implicit MATLAB solver ode15i on M(x)*xdot-F(x)=0...\n');`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
746	`[t,x] = ode15i(@(tt,xx,xxp) vehicle14_residual_full_numeric(tt,xx,xxp,mf_fun,par,args,road,meta), tspan, x0, xdot0, opts);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
747	`catch ME`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
748	`warning('ode15i failed: %s\nFalling back to ode15s with explicit RHS M\\F.', ME.message);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
749	`solver_used = 'ode15s-explicit';`	Assigns `solver_used` from `'ode15s-explicit'`. This intermediate value supports the export/regeneration workflow.
750	`[t,x] = ode15s(@(tt,xx) vehicle14_rhs_full_numeric(tt,xx,mf_fun,par,args,road,meta), tspan, x0, opts);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
751	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
752		Blank line used to separate logical blocks and improve readability.
753	`save(args.output_mat, 't', 'x', 'par', 'args', 'road', 'meta', 'solver_used', '-v7.3');`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
754	`fprintf('Saved solution to %s using %s.\n', args.output_mat, solver_used);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
755	`vehicle14_save_plots_matlab(t, x, mf_fun, par, args, road, meta, args.plot_dir);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
756	`try`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
757	`render_vehicle14_matlab_animation(args.output_mat, args.animation_mp4, args.animation_gif);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
758	`catch ME`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
759	`warning('Animation rendering failed: %s', ME.message);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
760	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
761	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
762		Blank line used to separate logical blocks and improve readability.
763	`function [ok,msg] = check_casadi_available()`	Assigns `function [ok,msg]` from `check_casadi_available()`. This intermediate value supports the export/regeneration workflow.
764	`ok = true; msg = '';`	Assigns `ok` from `true; msg = ''`. This intermediate value supports the export/regeneration workflow.
765	`try`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
766	`import casadi.*`	Imports Python modules needed for file handling, symbolic expression processing, code printing, packaging, or metadata.
767	`SX.sym('probe',1,1);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
768	`catch ME`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
769	`ok = false;`	Assigns `ok` from `false`. This intermediate value supports the export/regeneration workflow.
770	`msg = sprintf('CasADi is not on the MATLAB path or failed to initialize: %s', ME.message);`	Assigns `msg` from `sprintf('CasADi is not on the MATLAB path or failed to initialize: %s', ME.message)`. This intermediate value supports the export/regeneration workflow.
771	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
772	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
773	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
774		Blank line used to separate logical blocks and improve readability.
775	`(OUT/'render_vehicle14_matlab_animation.m').write_text(r'''function render_vehicle14_matlab_animation(mat_file, out_mp4, out_gif)`	Writes generated text content to an output file.
776	`%RENDER_VEHICLE14_MATLAB_ANIMATION Simple MATLAB MP4/GIF renderer for the saved solution.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
777	`if nargin < 1 || isempty(mat_file), mat_file = 'vehicle14_matlab_casadi_solution.mat'; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
778	`if nargin < 2 || isempty(out_mp4), out_mp4 = 'vehicle14_matlab_casadi_animation.mp4'; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
779	`if nargin < 3 || isempty(out_gif), out_gif = 'vehicle14_matlab_casadi_animation.gif'; end`	Starts a Python conditional branch; the indented block runs only if the condition is true.
780	`load(mat_file, 't', 'x', 'par', 'args', 'road', 'meta');`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
781	`[mf_fun, meta2] = vehicle14_build_casadi_function(); %#ok<ASGLU>`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
782	`fps = 20; playback_speed = 1.0;`	Assigns `fps` from `20; playback_speed = 1.0`. This intermediate value supports the export/regeneration workflow.
783	`nFrames = min(450, max(2, ceil((t(end)-t(1))*fps/playback_speed)+1));`	Assigns `nFrames` from `min(450, max(2, ceil((t(end)-t(1))*fps/playback_speed)+1))`. This intermediate value supports the export/regeneration workflow.
784	`tFrames = linspace(t(1), t(end), nFrames);`	Assigns `tFrames` from `linspace(t(1), t(end), nFrames)`. This intermediate value supports the export/regeneration workflow.

Line	Sanitized code line	Explanation
785	`xFrames = interp1(t, x, tFrames, 'linear');`	Assigns `xFrames` from `interp1(t, x, tFrames, 'linear')`. This intermediate value supports the export/regeneration workflow.
786	`fig = figure('Color','w','Position',[100 100 950 720]);`	Assigns `fig` from `figure('Color','w','Position',[100 100 950 720])`. This intermediate value supports the export/regeneration workflow.
787	`v = VideoWriter(out_mp4, 'MPEG-4'); v.FrameRate = fps; open(v);`	Assigns `v` from `VideoWriter(out_mp4, 'MPEG-4'); v.FrameRate = fps; open(v)`. This intermediate value supports the export/regeneration workflow.
788	`for k = 1:nFrames`	Starts a Python loop over the specified collection or range.
789	`clf(fig); ax = axes('Parent',fig); hold(ax,'on'); grid(ax,'on'); axis(ax,'equal'); view(ax, 3);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
790	`q = xFrames(k,1:14).'; u = xFrames(k,15:28).';`	Assigns `q` from `xFrames(k,1:14).'; u = xFrames(k,15:28).'`. This intermediate value supports the export/regeneration workflow.
791	`[~,~,delta,delta_dot] = vehicle14_command_inputs(tFrames(k), par, args);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
792	`IN0=zeros(meta.nin,1); IN0(meta.input_index.delta)=delta; IN0(meta.input_index.delta_dot)=delta_dot;`	Assigns `IN0` from `zeros(meta.nin,1); IN0(meta.input_index.delta)=delta; IN0(meta.input_index.delta_dot)=delta_dot`. This intermediate value supports the export/regeneration workflow.
793	`[~,~,Sdm] = mf_fun(q,u,par.P,IN0); S=full(Sdm); idx=meta.sensor_index;`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
794	`draw_local_road(ax, road, q(1), 10.0, road.width/2, 0.8, 1.2);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
795	`draw_chassis(ax,q,par);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
796	`names={'FL','FR','RL','RR'};`	Assigns `names` from `{'FL','FR','RL','RR'}`. This intermediate value supports the export/regeneration workflow.
797	`for i=1:4`	Starts a Python loop over the specified collection or range.
798	`nm=names{i};`	Assigns `nm` from `names{i}`. This intermediate value supports the export/regeneration workflow.
799	`wc=[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))];`	Assigns `wc` from `[S(idx.([nm '_Wc_x'])); S(idx.([nm '_Wc_y'])); S(idx.([nm '_Wc_z']))]`. This intermediate value supports the export/regeneration workflow.
800	`head=[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))];`	Assigns `head` from `[S(idx.([nm '_head_Nx'])); S(idx.([nm '_head_Ny'])); S(idx.([nm '_head_Nz']))]`. This intermediate value supports the export/regeneration workflow.
801	`spin=[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))];`	Assigns `spin` from `[S(idx.([nm '_spin_Nx'])); S(idx.([nm '_spin_Ny'])); S(idx.([nm '_spin_Nz']))]`. This intermediate value supports the export/regeneration workflow.
802	`draw_wheel(ax,wc,head,spin,par.rw);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
803	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
804	`xlabel(ax,'X [m]'); ylabel(ax,'Y [m]'); zlabel(ax,'Z [m]');`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
805	`title(ax,sprintf('vehicle14 MATLAB/CasADi | t = %.2f s | roll %.2f deg | pitch %.2f deg', tFrames(k), rad2deg(q(6)), rad2deg(q(5))));`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
806	`xlim(ax,[q(1)-4 q(1)+8]); ylim(ax,[q(2)-4 q(2)+4]); zlim(ax,[q(3)-1.0 q(3)+1.8]);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
807	`frame = getframe(fig); writeVideo(v,frame);`	Assigns `frame` from `getframe(fig); writeVideo(v,frame)`. This intermediate value supports the export/regeneration workflow.
808	`[A,map] = rgb2ind(frame2im(frame),256);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
809	`if k == 1`	Starts a Python conditional branch; the indented block runs only if the condition is true.
810	`imwrite(A,map,out_gif,'gif','LoopCount',Inf,'DelayTime',1/fps);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
811	`else`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
812	`imwrite(A,map,out_gif,'gif','WriteMode','append','DelayTime',1/fps);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
813	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
814	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
815	`close(v); close(fig);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
816	`fprintf('Saved animation: %s and %s\n', out_mp4, out_gif);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
817	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
818		Blank line used to separate logical blocks and improve readability.
819	`function draw_chassis(ax,q,par)`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
820	`R=body_rotation_from_q(q); P=q(1:3); Lf=par.lf; Lr=par.lr; W=max(par.tf,par.tr)/2; H=0.18;`	Assigns `R` from `body_rotation_from_q(q); P=q(1:3); Lf=par.lf; Lr=par.lr; W=max(par.tf,par.tr)/2; H=0.18`. This intermediate value supports the export/regeneration workflow.
821	`ptsB=[ Lf W H; Lf -W H; -Lr -W H; -Lr W H; Lf W -H; Lf -W -H; -Lr -W -H; -Lr W -H].';`	Assigns `ptsB` from `[ Lf W H; Lf -W H; -Lr -W H; -Lr W H; Lf W -H; Lf -W -H; -Lr -W -H; -Lr W -H].'`. This intermediate value supports the export/regeneration workflow.
822	`pts=(P + R*ptsB).';`	Assigns `pts` from `(P + R*ptsB).'`. This intermediate value supports the export/regeneration workflow.
823	`edges=[1 2;2 3;3 4;4 1;5 6;6 7;7 8;8 5;1 5;2 6;3 7;4 8];`	Assigns `edges` from `[1 2;2 3;3 4;4 1;5 6;6 7;7 8;8 5;1 5;2 6;3 7;4 8]`. This intermediate value supports the export/regeneration workflow.
824	`for e=1:size(edges,1), plot3(ax,pts(edges(e,:),1),pts(edges(e,:),2),pts(edges(e,:),3),'k-','LineWidth',1.6); end`	Starts a Python loop over the specified collection or range.
825	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
826		Blank line used to separate logical blocks and improve readability.
827	`function draw_wheel(ax,wc,head,spin,rw)`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
828	`spin=normalize_num(spin,[0;1;0]); e1=head-dot(head,spin)*spin; e1=normalize_num(e1,[1;0;0]); e3=normalize_num(cross(spin,e1),[0;0;1]);`	Assigns `spin` from `normalize_num(spin,[0;1;0]); e1=head-dot(head,spin)*spin; e1=normalize_num(e1,[1;0;0]); e3=normalize_num(cross(spin,e1),[0;0;1])`. This intermediate value supports the export/regeneration workflow.
829	`th=linspace(0,2*pi,40); C=wc + rw*(e1*cos(th)+e3*sin(th)); plot3(ax,C(1,:),C(2,:),C(3,:),'LineWidth',1.4);`	Assigns `th` from `linspace(0,2*pi,40); C=wc + rw*(e1*cos(th)+e3*sin(th)); plot3(ax,C(1,:),C(2,:),C(3,:),'LineWidth',1.4)`. This intermediate value supports the export/regeneration workflow.
830	`plot3(ax,[wc(1)-0.12*spin(1), wc(1)+0.12*spin(1)], [wc(2)-0.12*spin(2), wc(2)+0.12*spin(2)], [wc(3)-0.12*spin(3), wc(3)+0.12*spin(3)], 'LineWidth',2.0);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
831	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
832		Blank line used to separate logical blocks and improve readability.
833	`function draw_local_road(ax,road,x0,xhalf,nhalf,ds,dn)`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
834	`sVals=(x0-xhalf):ds:(x0+xhalf); nVals=(-nhalf):dn:nhalf; [SS,NN]=meshgrid(sVals,nVals); X=zeros(size(SS));Y=X;Z=X;`	Assigns `sVals` from `(x0-xhalf):ds:(x0+xhalf); nVals=(-nhalf):dn:nhalf; [SS,NN]=meshgrid(sVals,nVals); X=zeros(size(SS));Y=X;Z=X`. This intermediate value supports the export/regeneration workflow.
835	`for i=1:numel(SS), [p,~,~,~]=road_surface_at(road,SS(i),NN(i)); X(i)=p(1);Y(i)=p(2);Z(i)=p(3); end`	Starts a Python loop over the specified collection or range.
836	`surf(ax,X,Y,Z,'FaceAlpha',0.25,'EdgeAlpha',0.25,'FaceColor',[0.8 0.8 0.8],'EdgeColor',[0.4 0.4 0.4]);`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
837	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
838		Blank line used to separate logical blocks and improve readability.
839	`function [p_road,e_long,e_lat,e_z] = road_surface_at(road,s,n)`	Assigns `function [p_road,e_long,e_lat,e_z]` from `road_surface_at(road,s,n)`. This intermediate value supports the export/regeneration workflow.
840	`[C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s); p_road=C+n*ey; ps=Cs+n*dey; pn=ey; e_long=normalize_num(ps,ex); e_lat=normalize_num(pn,ey); e_z=normalize_num(cross(e_long,e_lat),ezc); if e_z(3)<0, e_z=-e_z; e_lat=-e_lat; end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
841	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
842	`function [C,Cs,ex,ey,dey,ezc] = centerline_quantities(road,s)`	Assigns `function [C,Cs,ex,ey,dey,ezc]` from `centerline_quantities(road,s)`. This intermediate value supports the export/regeneration workflow.
843	`ke=2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength; zc=road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos(ke*s); bank=road.bank_amp*sin(kb*s); db=road.bank_amp*kb*cos(kb*s); C=[s;0;zc]; Cs=[1;...`	Assigns `ke` from `2*pi/road.elev_wavelength; kb=2*pi/road.bank_wavelength; zc=road.elev_amp*(1-cos(ke*s)); dz=road.elev_amp*ke*sin(ke*s); d2z=road.elev_amp*ke*ke*cos...`. This intermediate value supports the export/regeneration workflow.
844	`end`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
845	`function R=body_rotation_from_q(q), R=rot_z(q(4))*rot_y(q(5))*rot_x(q(6)); end`	Assigns `function R` from `body_rotation_from_q(q), R=rot_z(q(4))*rot_y(q(5))*rot_x(q(6)); end`. This intermediate value supports the export/regeneration workflow.
846	`function R=rot_x(a), ca=cos(a); sa=sin(a); R=[1 0 0;0 ca -sa;0 sa ca]; end`	Assigns `function R` from `rot_x(a), ca=cos(a); sa=sin(a); R=[1 0 0;0 ca -sa;0 sa ca]; end`. This intermediate value supports the export/regeneration workflow.
847	`function R=rot_y(a), ca=cos(a); sa=sin(a); R=[ca 0 sa;0 1 0;-sa 0 ca]; end`	Assigns `function R` from `rot_y(a), ca=cos(a); sa=sin(a); R=[ca 0 sa;0 1 0;-sa 0 ca]; end`. This intermediate value supports the export/regeneration workflow.
848	`function R=rot_z(a), ca=cos(a); sa=sin(a); R=[ca -sa 0;sa ca 0;0 0 1]; end`	Assigns `function R` from `rot_z(a), ca=cos(a); sa=sin(a); R=[ca -sa 0;sa ca 0;0 0 1]; end`. This intermediate value supports the export/regeneration workflow.
849	`function y=normalize_num(v,fallback), n=norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end, end`	Assigns `function y` from `normalize_num(v,fallback), n=norm(v); if n<1e-10, y=fallback/norm(fallback); else, y=v/n; end, end`. This intermediate value supports the export/regeneration workflow.
850	`''')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
851		Blank line used to separate logical blocks and improve readability.
852	`# Exporter script included for regeneration.`	Comment line. It documents the exporter workflow; Python does not execute it.
853	`shutil.copyfile(Path(__file__), OUT/'export_vehicle14_matlab_casadi.py')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
854	`# Copy uploaded Python files for traceability.`	Comment line. It documents the exporter workflow; Python does not execute it.
855	`shutil.copyfile(SRC, OUT/'vehicle14_source_uploaded.py')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
856	`shutil.copyfile('/mnt/data/simulate_vehicle14_roadfree_drop_then_maneuver_3d_ribbon_full_kc_v10_nicola_wp_checked(16).py', OUT/'python_reference_simulator_uploaded.py')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
857	`shutil.copyfile('/mnt/data/render_vehicle14_roadfree_animation_3d_ribbon_close_full_kc_v10_nicola_wp_checked(16).py', OUT/'python_reference_renderer_uploaded.py')`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
858		Blank line used to separate logical blocks and improve readability.
859	`(OUT/'README_vehicle14_matlab_casadi.md').write_text(r'''# vehicle14 MATLAB/CasADi package`	Writes generated text content to an output file.
860		Blank line used to separate logical blocks and improve readability.
861	`This package was generated from the uploaded symbolic generator 14-DOF road-free wrench-input vehicle model.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
862		Blank line used to separate logical blocks and improve readability.
863	`## What is included`	Comment line. It documents the exporter workflow; Python does not execute it.
864		Blank line used to separate logical blocks and improve readability.
865	`- `vehicle14_generated_dynamics_matrices_casadi.m` - auto-generated CasADi/SX expressions for the full 28-by-28 mass matrix `M`, full forcing vector `F`, and 99 sensor channels.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
866	`- `vehicle14_component_inputs_numeric.m` - MATLAB numeric 3D-ribbon road, tyre/contact, aero, steering, drive/brake layer. It feeds body-frame wheel-centre force/moment inputs into the road-free generated-core skeleton.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
867	`- `run_vehicle14_matlab_casadi_demo.m` - main simulation script.`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
868	- `render_vehicle14_matlab_animation.m` - simple MP4/GIF renderer from the saved MATLAB solution.	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
869	- `vehicle14_source_uploaded.py`, `python_reference_simulator_uploaded.py`, `python_reference_renderer_uploaded.py` - original uploaded Python files copied for traceability.	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
870		Blank line used to separate logical blocks and improve readability.
871	## Solver choice	Comment line. It documents the exporter workflow; Python does not execute it.
872		Blank line used to separate logical blocks and improve readability.
873	The uploaded model provides an explicit ODE through a full residual	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
874		Blank line used to separate logical blocks and improve readability.
875	```matlab	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
876	M(x,input)*xdot - F(x,input) = 0	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
877	```	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
878		Blank line used to separate logical blocks and improve readability.
879	where `x = [q; u]`. The main MATLAB script first tries `ode15i` directly on that residual. If it fails, it falls back to `ode15s` on the explicit RHS `xdot = M\F`. This obeys the requirement not to write a custom integration scheme.	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
880		Blank line used to separate logical blocks and improve readability.
881	CasADi's own `idas` plugin can also be used as a semi-explicit DAE by introducing an algebraic variable `z = xdot` and enforcing `M*z - F = 0`, but this package uses MATLAB `ode15i` first because the tyre/contact layer is numeric and con...	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
882		Blank line used to separate logical blocks and improve readability.
883	## How to run in MATLAB	Comment line. It documents the exporter workflow; Python does not execute it.
884		Blank line used to separate logical blocks and improve readability.
885	1. Put CasADi on the MATLAB path, for example:	Starts a Python block; following indented lines belong to this statement.
886		Blank line used to separate logical blocks and improve readability.
887	```matlab	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
888	addpath('D:/Applications/casadi-windows-matlabR2016a-v3.6.5')	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
889	```	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
890		Blank line used to separate logical blocks and improve readability.
891	or edit the commented `addpath` line in `run_vehicle14_matlab_casadi_demo.m`.	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
892		Blank line used to separate logical blocks and improve readability.
893	2. Run:	Starts a Python block; following indented lines belong to this statement.
894		Blank line used to separate logical blocks and improve readability.
895	```matlab	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
896	cd path/to/vehicle14_matlab_casadi_package	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
897	run_vehicle14_matlab_casadi_demo	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
898	```	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
899		Blank line used to separate logical blocks and improve readability.
900	Expected outputs:	Starts a Python block; following indented lines belong to this statement.
901		Blank line used to separate logical blocks and improve readability.
902	- `vehicle14_matlab_casadi_solution.mat`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
903	- `vehicle14_matlab_casadi_plots/*.png`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
904	- `vehicle14_matlab_casadi_animation.mp4`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
905	- `vehicle14_matlab_casadi_animation.gif`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
906		Blank line used to separate logical blocks and improve readability.
907	## Notes	Comment line. It documents the exporter workflow; Python does not execute it.
908		Blank line used to separate logical blocks and improve readability.
909	- I could not execute MATLAB or CasADi inside this sandbox because neither MATLAB/Octave nor Python CasADi is installed here. The generated MATLAB files are therefore syntax/logic checked by construction against the uploaded Python sourc...	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
910	- The package keeps the same road-free architecture: symbolic generator does not compute road height, tyre compression, slip, or tyre forces internally; the MATLAB contact layer computes those and passes wheel-centre body-frame wrenches ...	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
911	''')	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
912		Blank line used to separate logical blocks and improve readability.

Line	Sanitized code line	Explanation
913	`# A compact run commands file.`	Comment line. It documents the exporter workflow; Python does not execute it.
914	`(OUT/'RUN_COMMANDS.txt').write_text("""MATLAB command:\n\ncd path/to/vehicle14_matlab_casadi_package\nrun_vehicle14_matlab_casadi_demo\n\nOptional animation-only rerun after a solution exists:\nrender_vehicle14_matlab_animation('vehicle1...`	Writes generated text content to an output file.
915		Blank line used to separate logical blocks and improve readability.
916	`# Zip it.`	Comment line. It documents the exporter workflow; Python does not execute it.
917	`zip_path = Path('/mnt/data/vehicle14_matlab_casadi_package.zip')`	Builds a filesystem path used as an input file, output folder, or generated artifact location.
918	`if zip_path.exists(): zip_path.unlink()`	Starts a Python conditional branch; the indented block runs only if the condition is true.
919	`with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:`	Adds generated files to the output ZIP package.
920	`for p in sorted(OUT.rglob('*')):`	Starts a Python loop over the specified collection or range.
921	`zf.write(p, p.relative_to(OUT.parent))`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
922	`print(json.dumps({`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
923	`'out_dir': str(OUT),`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
924	`'zip': str(zip_path),`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.

Line	Sanitized code line	Explanation
925	`'generated_m_file_size': (OUT/'vehicle14_generated_dynamics_matrices_casadi.m').stat().st_size,`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
926	`'nq': len(model.q_names), 'nu': len(model.u_names), 'np': len(model.p_names), 'nin': len(model.input_names), 'ns': len(model.sensor_names),`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
927	`'nonzero_generated_exprs': len(exprs), 'cse_temps': len(repls)`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.
928	`}, indent=2))`	Executes this Python statement in `valid_field` as part of the export/regeneration workflow.